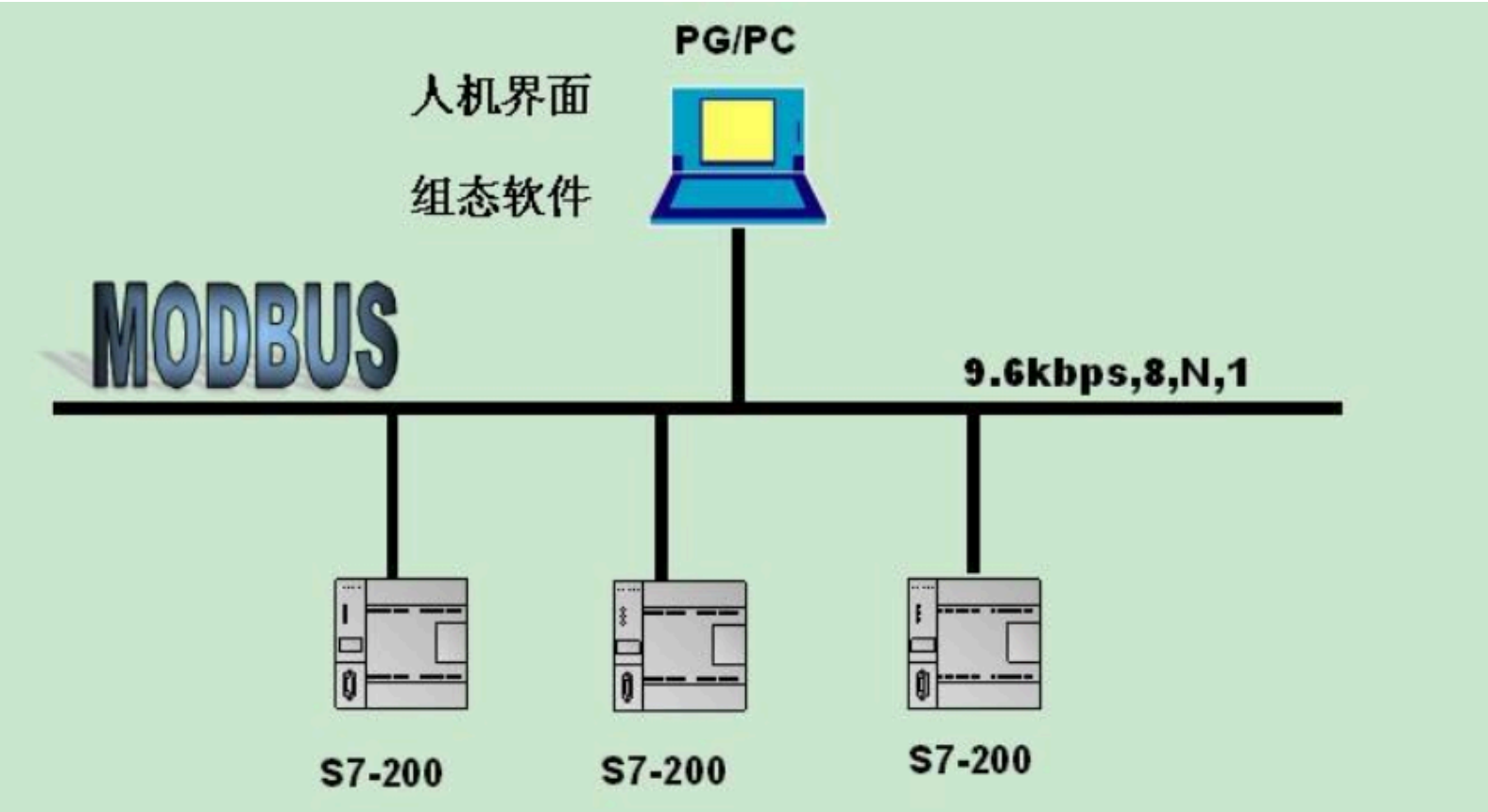


第01章 Modbus协议实战



1.1 MODBUS

1. MODBUS 介绍

Modbus 是工业界常用的一种通讯协议，可以理解为是一种通讯的约定。

Modbus是一种串行通信协议，是Modicon公司（现在的施耐德电气SchneiderElectric）于1979年为使用可编程逻辑控制器（PLC）通信而发表。Modbus已经成为工业领域通信协议的业界标准（Defacto），并且现在是工业电子设备之间常用的连接方式，是工业自动化领域里使用最普遍的协议。随着电子技术计算机和通信技术的不断发，Modbus也从最古老的RS232/Rs485串行总线发展到了 ModbusTCP和透明就绪工业以太网。

从1979年开始，Modbus作为工业串行链路的事实标准，Modbus使成千上万的自动化设备能够通信。目前，对简单而精致的Modbus结构的支持仍在增长。互联网用户能够使用TCP/P栈上的保留系统端口502访问。

ModbusModbus是一个请求/应答协议，并且提供功能码规定的服务。 Modbus功能码是 Modbus请求/应答PDU的元素。

单播模式与广播模式

单播模式：“一对一”通讯。主站只寻找某一确定的从站，从站接收到命令后处理，并返回一个应答报文。主站需要发出一个命令报文并处理从站返回的报文，从站需要接收主站的命令报文并发出一个应答报文。每个子节点必须有唯一的地址（1-247）。

广播模式：“一对所有”。主站向所有从站发送请求，对于主站发送的广播请求没有应答返回，广播请求必须是写命令。所有从站必须接收写功能的广播。地址0用来广播通信。

寄存器（数据的存储和读取）

寄存器种类

寄存器种类	说明	PLC类比	举例
线圈状态	输出端口。可设定端口的输出状态，也可以读取该位的输出状态。可分为两种不同的执行状态，例如保持型或边沿触发型。可读可写。	DO数字量输出	电磁阀输出，MOSFET输出，LED显示等。
离散输入状态	输入端口。通过外部设定改变输入状态，可读但不可写。	DI数字量输入	拨码开关，接近开关等。
保持寄存器	输出参数或保持参数，控制器运行时被设定的某些参数。可读可写。	AO模拟量输出	模拟量输出设定值，PID运行参数，变量阀输出大小，传感器报警上限下限。
输入寄存器	输入参数。控制器运行时从外部设备获得的参数。可读但不可写。	AI模拟量输入	模拟量输入

2. Modbus RTU介绍

Modbus 协议包括 RTU、ASCII、TCP，其中 Modbus-RTU 最常用，比较简单，在单片机上很容易实现。Modbus RTU 协议定义了一个与下层通信层无关的简单协议数据单元（PDU），串行链路上的 Modbus RTU 帧格式如下图所示。

Slave Address	Function Code	Data	CRC
1 byte	1 byte	0 up to 252 byte(s)	2 bytes CRC Low CRC Hi

一般的来说，一个数据帧可以分为：设备码、功能码、数据码、校验码这四个部分。

ModbusRTU是一种主从通讯模式的通讯协议，也就是说，Modbus有一个主机，可以进行通讯的主动要求，其他从机只能对主机进行响应而不能主动发送数据到通讯总线中。这种方法规定了通讯过程中的通讯次序等关系，避免了多个设备同时工作的情况下通讯冲突的产生。

Modbus协议不规定一个字节如何传输，而是规定如何进行一次数据帧的传输。数据帧可以理解为若干个具有特殊功能意义的字节的组合。那么Modbus如何定义一个数据帧？对于Modbus来说，当进行数据传输过程中，出现空闲时间超过3.5个字节持续时间，就认为一次数据帧的结束，之前接收到的字节就是这次数据帧的所有字节。之后再接收到的字节则为下一个数据帧的字节。

例如，在9600bit/s的传输速率下，一个字节传输的时间约为0.8ms，那么当数据传输中，出现约3ms的空闲时间时，设备就认为一帧数据接收完成。

每一帧数据要实现功能，就需要通讯双方采用相同的语法规则，因此对于每个数据帧的构成，Modbus进行了一些规定。

例如一个主机发送的读取功能的数据帧，其组成为：

设备地址	功能码	寄存器起始地址	读取寄存器的个数	CRC校验
1字节	1字节	2字节	2字节	2字节
0x02	0x03	0x00 0x00	0x00 0x01 （n）	0x44 0x3F

从机响应的数据帧为：

设备地址	功能码	返回的字节的个数	对应寄存器的数据	CRC校验
1字节	1字节	1字节	2字节（2*n字节）	2字节
0x02	0x03	0x02（2*n）	0x00 0x01 （...）	0x... 0x...

3. Modbus设备码(从机地址)

就是从机地址。

对于一主多从的通讯架构中，主机的设备码为0，其他各个从机都有独立的设备码，总共分配255个从机设备码，在通讯过程中，主机把数据发送AB总线中，各个从机从AB总线中接收数据，对于不属于该设备码的数据帧，将数据放回至总线，对于属于该设备码的数据帧，设备进行相应的处理后，对主机进行响应，此时设备码的含义就是指定主机的通讯目标设备。

4. Modbus功能码

就是区分是读还是写的。

功能码的含义是表明该通讯帧的功能或目的。

具体来说，例如功能码3表明主机要求读取从机若干个寄存器的数值，从机接收到通讯帧后进行一系列的处理，返回给主机相同的功能码，表明从机对该功能进行了响应。

功能码	功能含义	寄存器地址
01	读单个或多个位状态	00001 - 09999
05	写单个位状态	00001 - 09999
15	写多个位状态	00001 - 09999
03	读单个或多个寄存器	40001 - 49999
06	写单个寄存器	40001 - 49999
16	写多个寄存器	40001 - 49999

0x01：读线圈寄存器
0x02：读离散输入寄存器
0x03：读保持寄存器

0x04：	读输入寄存器
0x05：	写单个线圈寄存器
0x06：	写单个保持寄存器
0x0f：	写多个线圈寄存器
0x10：	写多个保持寄存器

如上所示一共8种功能码。这其中有涉及到线圈、离散输入、保持、输入四种寄存器。这名字也不知道谁起的，让人看了一点不通俗易懂，搞得晕晕乎乎。实际上你要是看清他的本质就很简单了。下面分别解释一下：

- 线圈寄存器，实际上就可以类比为开关量，一个bit都对应一个信号的开关状态。所以一个byte就可以同时控制8路的信号。比如控制外部8路io的高低。线圈寄存器支持读也支持写，写在功能码里面又分为写单个线圈寄存器和写多个线圈寄存器。
 - 对应上面的功能码也就是：0x01 0x05 0x0f
- 离散输入寄存器，如果线圈寄存器理解了这个自然也明白了。离散输入寄存器就相当于线圈寄存器的只读模式，他也是每个bit表示一个开关量，而他的开关量只能读取输入的开关信号，是不能够写的。比如我读取外部按键的按下还是松开。
 - 所以功能码也简单就一个读的 0x02
- 保持寄存器，这个寄存器的单位不再是bit而是两个byte，也就是可以存放具体的数据量的，并且是可读写的。比如我我设置时间年月日，不但可以写也可以读出来现在的时间。写也分为单个写和多个写。
 - 所以功能码有对应的三个：0x03 0x06 0x10
- 输入寄存器，只剩下这最后一个了，这个和保持寄存器类似，但是也是只支持读而不能写。一个寄存器也是占据两个byte的空间。类比我我通过读取输入寄存器获取现在的AD采集值。
 - 对应的功能码也就一个 0x04

5. Modbus数据码

数据码是对功能码的进一步补充和解释，常见的功能码的数据码格式一般在Modbus通用协议中已经做了规范。

例如对于3功能码，后面跟的数据码包括2个字节表示寄存器地址，2个字节表示读取的寄存器个数（寄存器的位数为16位，因此一个寄存器有两个字节的数据）。

返回的3功能码，后接1个字节的返回字节个数（该个数应为上述读取寄存器个数的两倍，因为一个寄存器对应两个字节），和若干个字节的数据。

6. Modbus校验码

Modbus一般采用的16位的CRC校验。

什么是CRC校验呢，简单来说，比如你要发一段数据，最后想在后面加两个字节，这两个字节是通过前面所发的数据通过某种算法计算出来的唯一的两个字节。

如果发送过程中无问题，那么接收端通过相同的算法能够得到同样的最后两位字节，这就表明该帧的所有数据都是正确的。

如果接收端通过同样的算法算出来的两个字节数据和发送过来的最后两个字节不同，那么就表明在发送和接收过程中，有些数据接收或发送错误。通过校验位来判断该帧是否正确发送和接收。

CRC就是其中一种校验方法，其算法是对除数和被除数进行按位异或，然后除数移位，然后两个进行异或，然后除数移位，如此循环，直到除数和被除数位数相同，结束运算。

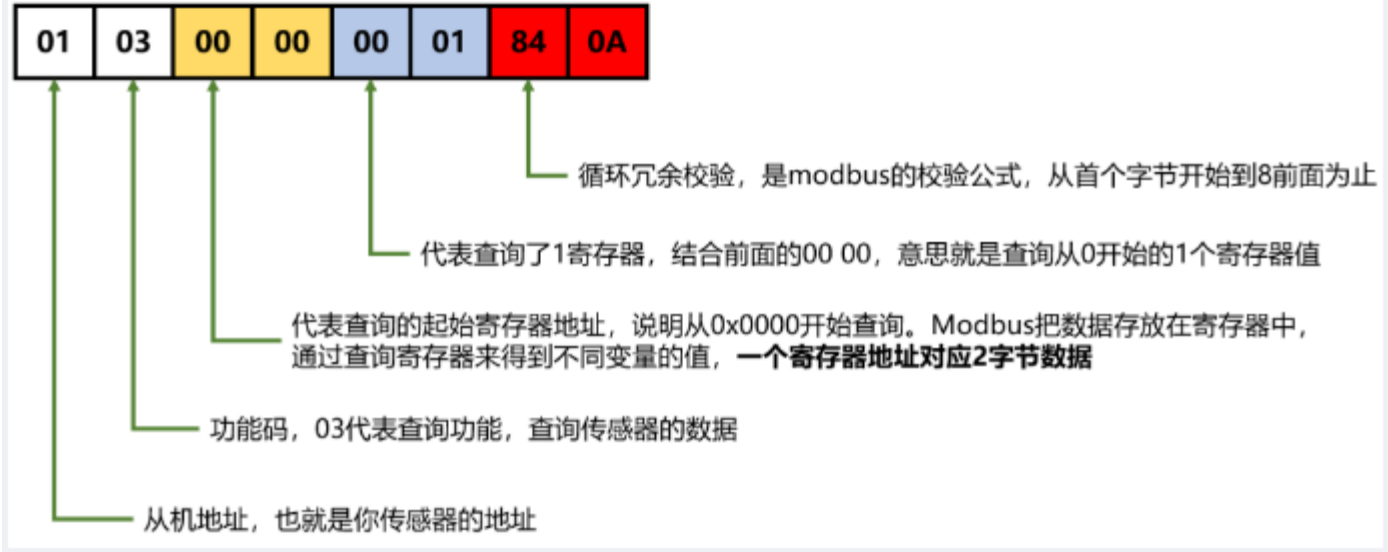
7. 查询功能码

比如我们现在要使用STM32查询某传感器的数据，该传感器的地址为01。

主机发送: 01 03 00 00 00 01 84 0A

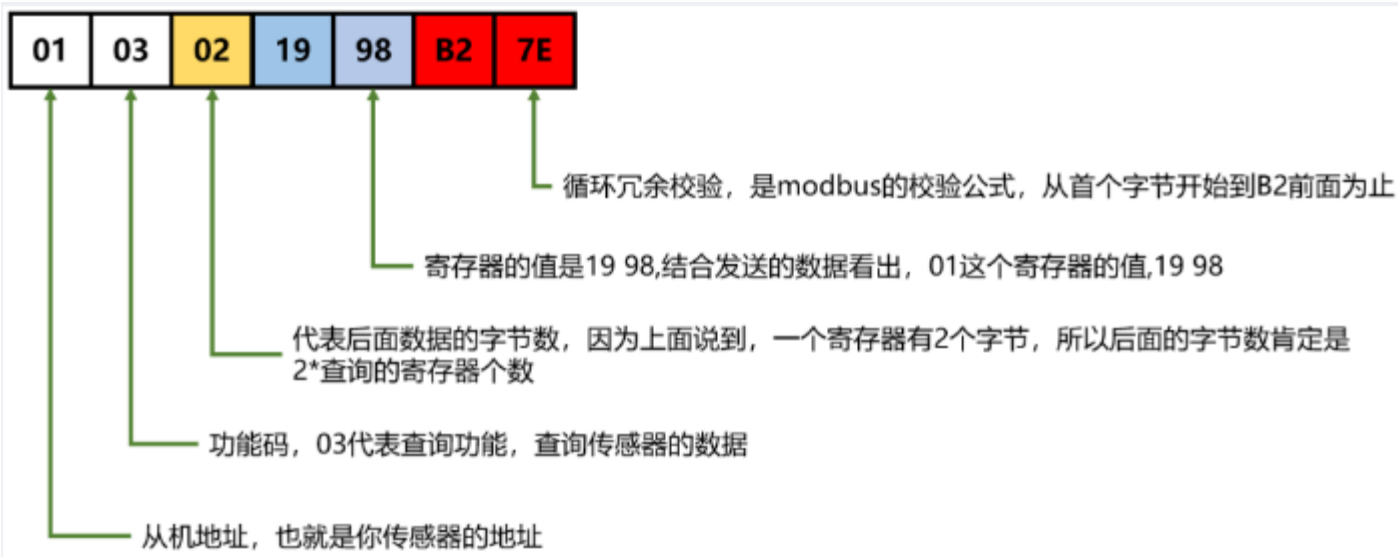
从机回复: 01 03 02 19 98 B2 7E

什么意思？解析如下：



- 01 - 地址，也就是你传感器的地址
- 03 - 功功能码, 03代表查询功能, 查询传感器的数据
- 00 00 - 代表查询的起始寄存器地址.说明从0x0000开始查询。这里需要说明以下，Modbus把数据存放在寄存器中，通过查询寄存器来得到不同变量的值，一个寄存器地址对应2字节数据。
- 00 01 - 代表查询了一个寄存器，结合前面的00 00，意思就是查询从0开始的1个寄存器值。
- 84 0A - 循环冗余校验,是modbus的校验公式,从首个字节开始到84前面为止。

回复数据解析



- 01 - 地址，也就是你传感器的地址
- 03 - 功功能码, 03代表查询功能, 查询传感器的数据。这里要注意的是注意发给从机的功能码是啥，从机就要回复同样的功能码，如果不一样说明这一帧数据有错误
- 02 - 代表后面数据的字节数,因为上面说到,一个寄存器有2个字节,所以后面的字节数肯定是2*查询的寄存器个数;
- 19 98 - 寄存器的值是19 98,结合发送的数据看出,01这个寄存器的值为19 98
- B2 7E - 循环冗余校验

总结就是：

发送：从机的地址+我要干嘛的功能码+我要查的寄存器的地址+我要查的寄存器地址的个数+校验码

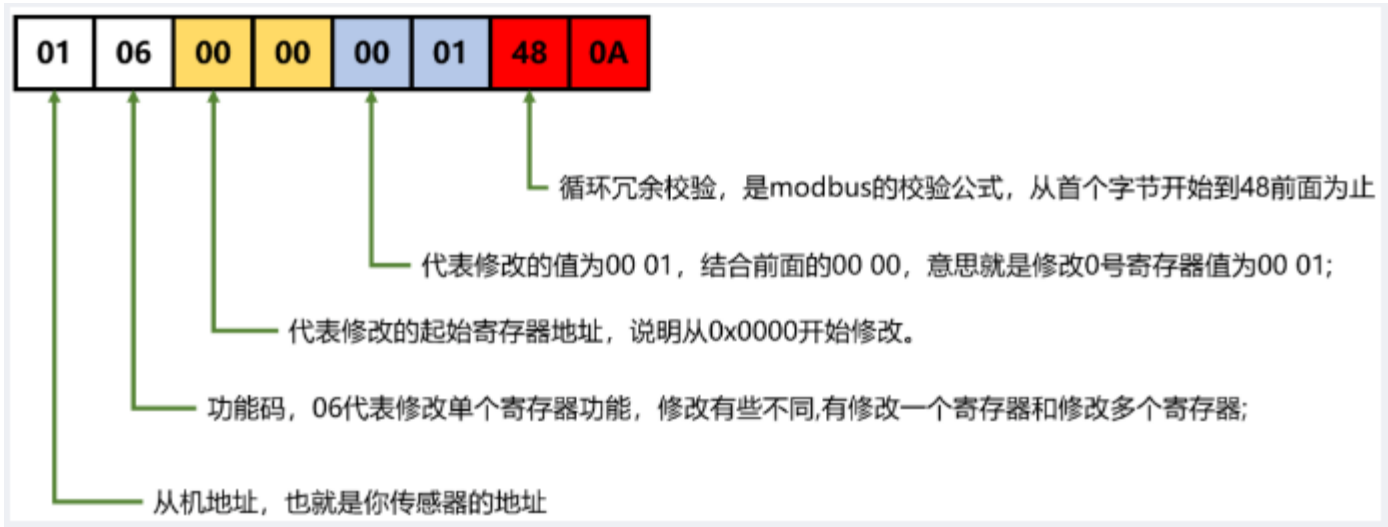
回复：从机的地址+主机发我的功能码+要发送给主机数据的字节数+数据+校验码。

8. 修改功能码

主机发送: 01 06 00 00 00 01 48 0A

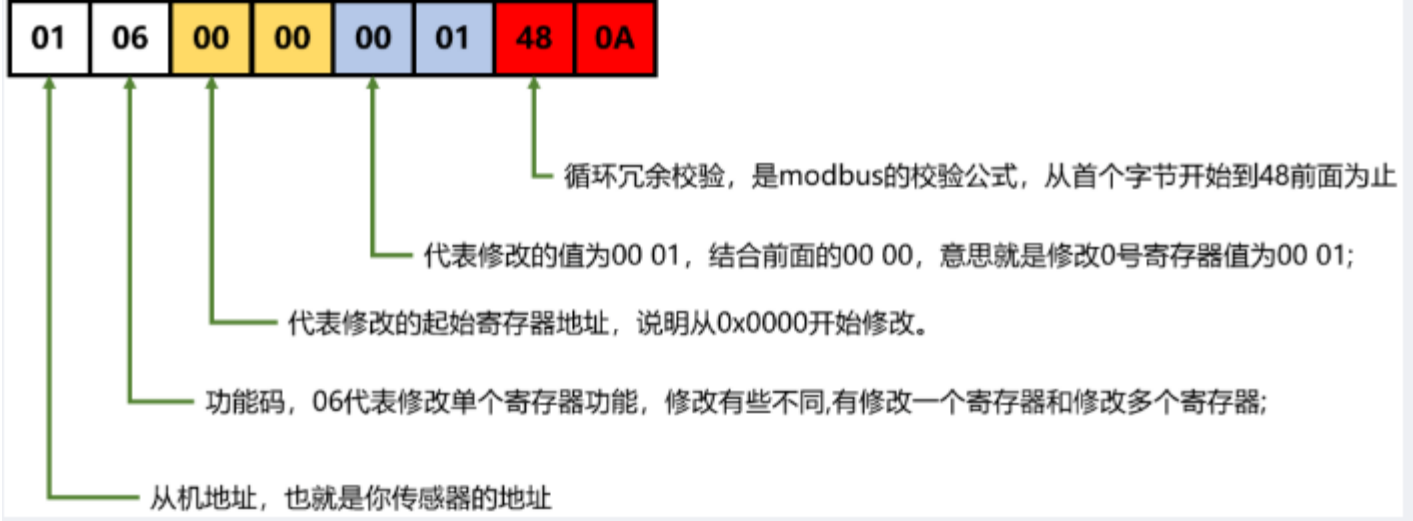
从机回复: 01 06 00 00 00 01 48 0A

发送数据解析



- 01-主机要查询的从机地址
- 06-功能码, 06代表修改单个寄存器功能,修改有些不同,有修改一个寄存器和修改多个寄存器;
- 00 00-代表修改的起始寄存器地址，说明从0x0000开始.
- 00 01-代表修改的值为00 01.结合前面的00 00,意思就是修改0号寄存器值为00 01;
- 48 0A-循环冗余校验,是modbus的校验公式,从首个字节开始到48前面为止。

回复数据解析



01-从机返回给主机自己的地址，说明这就是主机查的从机
06-功能码，代表修改单个寄存器功能，主机发啥功能码，从机就必须回什么功能码；
00 00-代表修改的起始寄存器地址.说明是0x0000.
00 01-代表修改的值为00 01.结合前面的00 00,意思就是修改0号寄存器值为00 01；
48 0A-循环冗余校验,是modbus的校验公式,从首个字节开始到48前面为止；

如果回复的一样，说明这个数据是修改成功的；如果功能码不是06，而是别的，说明从机回复的数据有误，主机可以做相应的处理。

如果我要修改多个寄存器，难道用06发好几次，这样不会太傻了吗？所以Modbus RTU协议包含了修改连续多个寄存器的方法，就是功能码为0x10；这个大家自己去查询，基本和上面的数据格式差不多。

注意，ModBus只是一个软件协议，传输时可以通过串口来进行通信，为了提升传输效率，可以结合DMA使用。

1.2 FreeModbus

1. FreeModbus介绍

FreeModbus 是一个开源的 Modbus 协议栈实现。Modbus 是一种通信协议，用于在工业自动化系统中传输数据。FreeModbus 提供了一套可移植且易于使用的函数接口，使开发人员能够轻松地在嵌入式系统上实现 Modbus 通信。

FreeModbus 支持 Modbus RTU（串行）和 Modbus TCP（以太网）两种传输模式。它可以在各种硬件平台上运行，并且具有高度可定制性，可以根据特定的应用需求进行配置。

使用 FreeModbus，您可以构建自己的 Modbus 主站或从站设备。它提供了读取和写入 Modbus 寄存器的功能，以及处理 Modbus 异常和错误的能力。

FreeModbus 是一个广泛使用的开源项目，已经在许多工业自动化应用中得到验证。它是一个可靠且成熟的解决方案，可以帮助您快速实现 Modbus 通信功能。

2. FreeModbus的获取

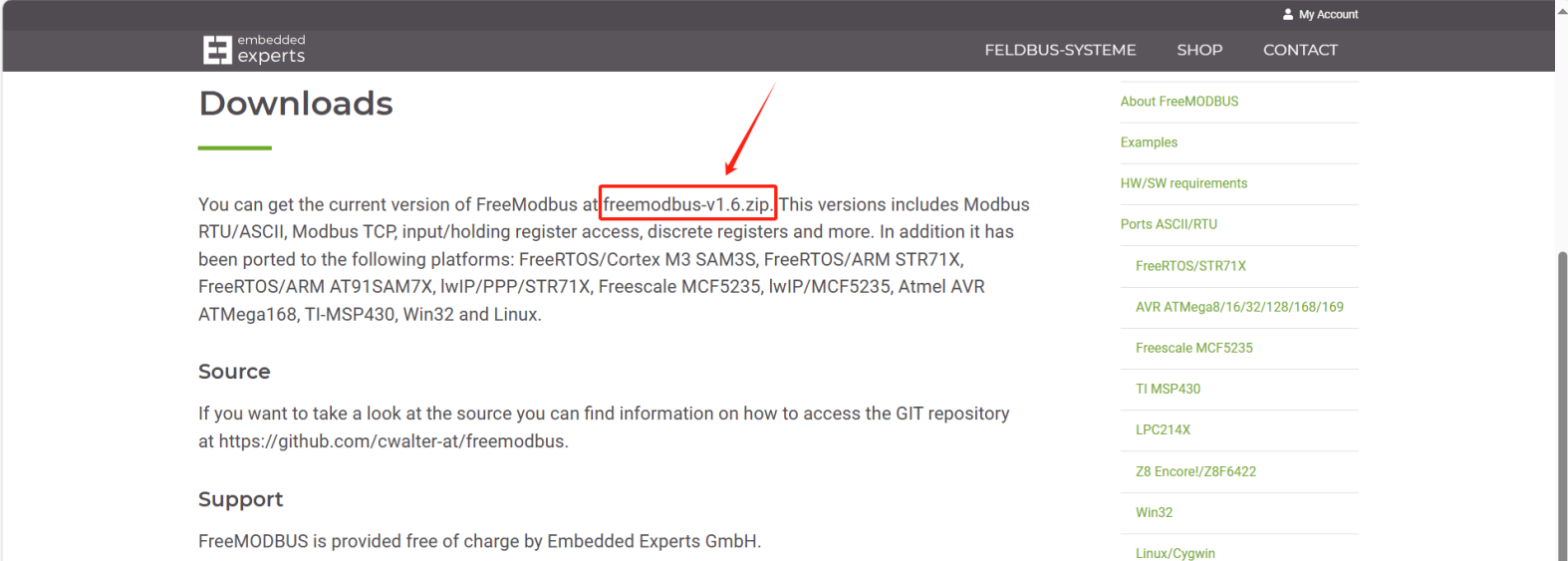
Modbus相关的协议栈有不少，比较有名是 LibModbus 和 FreeModbus。

LibModbus主要用在linux环境，而FreeModbus主要用在mcu环境，资源占用会小一些。

目前，FreeModbus最新版本是V1.6，可以通过官网下载，链接如下：

<https://www.embedded-experts.at/en/freemodbus-downloads/>

点击 freemodbus-v1.6.zip，即可下载。



可以下载压缩包，也可以使用git工具来下载。

3. Modbus文件夹介绍

接下来我们来看看Modbus文件夹里面都有哪些东西。

打开ModBus文件夹，里面有五个文件夹和一个C文件，说明如下：

序号	名称	说明/描述
1	ascii	这个文件夹包含Modbus-ASCII协议的实现代码
2	functions	这个文件夹主要包括一些功能码对应的处理函数
3	include	里面主要是Modbus协议需要使用的一些头文件和配置文件
4	rtu	这个文件夹包含Modbus-RTU协议的实现代码
5	tcp	这个文件夹包含Modbus-TCP协议的实现代码
6	mb.c	这个是MODBUS协议栈的主文件，这个文件夹只是一个框架，与具体的协议无关。因为Modbus有三种具体的协议，分别为RTU、ASCII和TCP，具体的实现在1、4、5文件夹中，而mb.c在初始化的时候会根据使用情况将指针指向具体的处理函数。

4. FreeModbus源码

demo.c

打开FreeModbus文件夹中的demo文件夹，该文件夹下是各个平台下的demo，这里我们选择BARE平台来分析。

打开BARE文件夹下的demo.c文件，main函数代码如下：

```
/* ----- Start implementation ----- */
int
main( void )
{
    eMBCode      eStatus;

    eStatus = eMBInit( MB_RTU, 0x0A, 0, 38400, MB_PAR_EVEN );

    /* Enable the Modbus Protocol Stack. */
    eStatus = eMBEnable( );

    for( ;; )
    {
        ( void )eMBPoll( );

        /* Here we simply count the number of poll cycles. */
        usRegInputBuf[0]++;
    }
}
```

要想使用FreeModbus，这里只要调用三个函数即可，即 eMBInit()、eMBEnable()、eMBPoll() 三个函数，这三个函数的功能如下：

名称	功能
eMBInit()	完成MODBUS的初始化配置
eMBEnable()	使能Modbus协议栈
eMBPoll()	轮询Modbus的数据接收，并进行数据的处理，这个函数需要循环调用

在主函数中调用上面三个函数，即可完成Modbus的使用。是不是很简单。

在系统上电后先初始化协议栈，然后使能协议栈，最后在一个循环中循环调用eMBPoll()函数即可。其他的函数我们暂时不讨论，等到最后移植的时候再来看，我们现在只关注主干部分。

关于这三个函数具体怎么实现的，我们来看一下mb.c文件。

mb.c

打开modbus\mb.c文件，代码如下：

```
/*
 * FreeModbus Library: A portable Modbus implementation for Modbus ASCII/RTU.
 * Copyright (c) 2006-2018 Christian Walter <cwalter@embedded-solutions.at>
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 * notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 * notice, this list of conditions and the following disclaimer in the
 * documentation and/or other materials provided with the distribution.
 * 3. The name of the author may not be used to endorse or promote products
```

```

    *   derived from this software without specific prior written permission.
    *
    * THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS'' AND ANY EXPRESS OR
    * IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,THE IMPLIED WARRANTIES
    * OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
    * IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT,
    * INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT
    * NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
    * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
    * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
    * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF
    * THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
    *
    */

/* ----- System includes -----*/
#include "stdlib.h"
#include "string.h"

/* ----- Platform includes -----*/
#include "port.h"

/* ----- Modbus includes -----*/
#include "mb.h"
#include "mbconfig.h"
#include "mbframe.h"
#include "mbproto.h"
#include "mbfunc.h"

#include "mbport.h"
#if MB_RTU_ENABLED == 1
#include "mbrtu.h"
#endif
#if MB_ASCII_ENABLED == 1
#include "mbascii.h"
#endif
#if MB_TCP_ENABLED == 1
#include "mbtcp.h"
#endif

#ifndef MB_PORT_HAS_CLOSE
#define MB_PORT_HAS_CLOSE 0
#endif

/* ----- Static variables -----*/

static UCHAR    ucMBAAddress;
static eMBMode  eMBCurrentMode;

static enum
{
    STATE_ENABLED,
    STATE_DISABLED,
    STATE_NOT_INITIALIZED
} eMBState = STATE_NOT_INITIALIZED;

/* Functions pointer which are initialized in eMBInit( ). Depending on the
 * mode (RTU or ASCII) the are set to the correct implementations.
 */
static peMBFrameSend peMBFrameSendCur;
static pvMBFrameStart pvMBFrameStartCur;
static pvMBFrameStop pvMBFrameStopCur;
static peMBFrameReceive peMBFrameReceiveCur;
static pvMBFrameClose pvMBFrameCloseCur;

/* Callback functions required by the porting layer. They are called when
 * an external event has happend which includes a timeout or the reception
 * or transmission of a character.
 */
BOOL( *pxMBFrameCBByteReceived ) ( void );
BOOL( *pxMBFrameCBTransmitterEmpty ) ( void );
BOOL( *pxMBPortCBTimerExpired ) ( void );

BOOL( *pxMBFrameCBReceiveFSMCur ) ( void );
BOOL( *pxMBFrameCBTransmitFSMCur ) ( void );

/* An array of Modbus functions handlers which associates Modbus function
 * codes with implementing functions.
 */
static xMBFunctionHandler xFuncHandlers[MB_FUNC_HANDLERS_MAX] = {
#if MB_FUNC_OTHER_REP_SLAVEID_ENABLED > 0
    {MB_FUNC_OTHER_REPORT_SLAVEID, eMBFuncReportsSlaveID},
#endif
#if MB_FUNC_READ_INPUT_ENABLED > 0
    {MB_FUNC_READ_INPUT_REGISTER, eMBFuncReadInputRegister},
#endif
#if MB_FUNC_READ_HOLDING_ENABLED > 0
    {MB_FUNC_READ_HOLDING_REGISTER, eMBFuncReadHoldingRegister},
#endif
#if MB_FUNC_WRITE_MULTIPLE_HOLDING_ENABLED > 0
    {MB_FUNC_WRITE_MULTIPLE_REGISTERS, eMBFuncwriteMultipleHoldingRegister},
#endif
#if MB_FUNC_WRITE_HOLDING_ENABLED > 0
    {MB_FUNC_WRITE_REGISTER, eMBFuncWriteHoldingRegister},
#endif
#if MB_FUNC_READWRITE_HOLDING_ENABLED > 0
    {MB_FUNC_READWRITE_MULTIPLE_REGISTERS, eMBFuncReadwriteMultipleHoldingRegister},
#endif
#if MB_FUNC_READ_COILS_ENABLED > 0
    {MB_FUNC_READ_COILS, eMBFuncReadCoils},
#endif
#if MB_FUNC_WRITE_COIL_ENABLED > 0
    {MB_FUNC_WRITE_SINGLE_COIL, eMBFuncWriteCoil},
#endif
#if MB_FUNC_WRITE_MULTIPLE_COILS_ENABLED > 0
    {MB_FUNC_WRITE_MULTIPLE_COILS, eMBFuncWriteMultipleCoils},
#endif
#if MB_FUNC_READ_DISCRETE_INPUTS_ENABLED > 0
    {MB_FUNC_READ_DISCRETE_INPUTS, eMBFuncReadDiscreteInputs},
#endif
};

/* ----- Start implementation -----*/
eMBErrorCode
eMBInit( eMBMode eMode, UCHAR ucSlaveAddress, UCHAR ucPort, ULONG ulBaudRate, eMBParity eParity )
{
    eMBErrorCode    eStatus = MB_ENOERR;

    /* check preconditions */
    if( ( ucSlaveAddress == MB_ADDRESS_BROADCAST ) ||
        ( ucSlaveAddress < MB_ADDRESS_MIN ) || ( ucSlaveAddress > MB_ADDRESS_MAX ) )
    {
        eStatus = MB_EINVAL;
    }
    else
    {
        ucMBAAddress = ucSlaveAddress;

        switch ( eMode )
        {
        #if MB_RTU_ENABLED > 0
            case MB_RTU:
                pvMBFrameStartCur = eMBRTUStart;
                pvMBFrameStopCur = eMBRTUStop;
                peMBFrameSendCur = eMBRTUSend;
                peMBFrameReceiveCur = eMBRTUReceive;
                pvMBFrameCloseCur = MB_PORT_HAS_CLOSE ? vMBPortClose : NULL;
                pxMBFrameCBByteReceived = xMBRTUReceiveFSM;
                pxMBFrameCBTransmitterEmpty = xMBRTUTransmitFSM;
                pxMBPortCBTimerExpired = xMBRTUTimerT35Expired;

                eStatus = eMBRTUInit( ucMBAAddress, ucPort, ulBaudRate, eParity );
                break;
            #endif
            #if MB_ASCII_ENABLED > 0
            case MB_ASCII:
                pvMBFrameStartCur = eMBASCIIStart;
                pvMBFrameStopCur = eMBASCIIStop;
                peMBFrameSendCur = eMBASCIIISend;
                peMBFrameReceiveCur = eMBASCIIReceive;
                pvMBFrameCloseCur = MB_PORT_HAS_CLOSE ? vMBPortClose : NULL;
                pxMBFrameCBByteReceived = xMBASCIIReceiveFSM;
                pxMBFrameCBTransmitterEmpty = xMBASCIITransmitFSM;
                pxMBPortCBTimerExpired = xMBASCIITimerT15Expired;

                eStatus = eMBASCIIInit( ucMBAAddress, ucPort, ulBaudRate, eParity );
                break;
            #endif
        }
    }
}
```

```

        default:
            eStatus = MB_EINVAL;
        }

        if( eStatus == MB_ENOERR )
        {
            if( !xMBPortEventInit( ) )
            {
                /* port dependent event module initialization failed. */
                eStatus = MB_EPORTERR;
            }
            else
            {
                eMBCurrentMode = eMode;
                eMBState = STATE_DISABLED;
            }
        }
    }
    return eStatus;
}

#if MB_TCP_ENABLED > 0
eMBErrorCode
eMBTCPInit( USHORT ucTCPPort )
{
    eMBErrorCode    eStatus = MB_ENOERR;

    if( ( eStatus = eMBTCPDoInit( ucTCPPort ) ) != MB_ENOERR )
    {
        eMBState = STATE_DISABLED;
    }
    else if( !xMBPortEventInit( ) )
    {
        /* Port dependent event module initialization failed. */
        eStatus = MB_EPORTERR;
    }
    else
    {
        pvMBFrameStartCur = eMBTCPStart;
        pvMBFrameStopCur = eMBTCPStop;
        peMBFrameReceiveCur = eMBTCPReceive;
        peMBFrameSendCur = eMBTCPSend;
        pvMBFrameCloseCur = MB_PORT_HAS_CLOSE ? vMBTCPPortClose : NULL;
        ucMBAddress = MB_TCP_PSEUDO_ADDRESS;
        eMBCurrentMode = MB_TCP;
        eMBState = STATE_DISABLED;
    }
    return eStatus;
}
#endif

eMBErrorCode
eMBRegisterCB( UCHAR ucFunctionCode, pxMBFunctionHandler pxHandler )
{
    int            i;
    eMBErrorCode    eStatus;

    if( ( 0 < ucFunctionCode ) && ( ucFunctionCode <= 127 ) )
    {
        ENTER_CRITICAL_SECTION( );
        if( pxHandler != NULL )
        {
            for( i = 0; i < MB_FUNC_HANDLERS_MAX; i++ )
            {
                if( ( xFuncHandlers[i].pxHandler == NULL ) ||
                    ( xFuncHandlers[i].pxHandler == pxHandler ) )
                {
                    xFuncHandlers[i].ucFunctionCode = ucFunctionCode;
                    xFuncHandlers[i].pxHandler = pxHandler;
                    break;
                }
            }
            eStatus = ( i != MB_FUNC_HANDLERS_MAX ) ? MB_ENOERR : MB_ENORES;
        }
        else
        {
            for( i = 0; i < MB_FUNC_HANDLERS_MAX; i++ )
            {
                if( xFuncHandlers[i].ucFunctionCode == ucFunctionCode )
                {
                    xFuncHandlers[i].ucFunctionCode = 0;
                    xFuncHandlers[i].pxHandler = NULL;
                    break;
                }
            }
            /* Remove can't fail. */
            eStatus = MB_ENOERR;
        }
        EXIT_CRITICAL_SECTION( );
    }
    else
    {
        eStatus = MB_EINVAL;
    }
    return eStatus;
}

eMBErrorCode
eMBClose( void )
{
    eMBErrorCode    eStatus = MB_ENOERR;

    if( eMBState == STATE_DISABLED )
    {
        if( pvMBFrameCloseCur != NULL )
        {
            pvMBFrameCloseCur( );
        }
    }
    else
    {
        eStatus = MB_EILLSTATE;
    }
    return eStatus;
}

eMBErrorCode
eMBEnable( void )
{
    eMBErrorCode    eStatus = MB_ENOERR;

    if( eMBState == STATE_DISABLED )
    {
        /* Activate the protocol stack. */
        pvMBFrameStartCur( );
        eMBState = STATE_ENABLED;
    }
    else
    {
        eStatus = MB_EILLSTATE;
    }
    return eStatus;
}

eMBErrorCode
eMBDisable( void )
{
    eMBErrorCode    eStatus;

    if( eMBState == STATE_ENABLED )
    {
        pvMBFrameStopCur( );
        eMBState = STATE_DISABLED;
        eStatus = MB_ENOERR;
    }
    else if( eMBState == STATE_DISABLED )
    {
        eStatus = MB_ENOERR;
    }
    else
    {
        eStatus = MB_EILLSTATE;
    }
    return eStatus;
}

eMBErrorCode

```

```
eMBPoll( void )
{
    static UCHAR    *ucMBFrame;
    static UCHAR    ucRcvAddress;
    static UCHAR    ucFunctionCode;
    static USHORT    usLength;
    static eMBException eException;

    int            i;
    eMBCurrentMode eStatus = MB_ENOERR;
    eMBEventType   eEvent;

    /* Check if the protocol stack is ready. */
    if( eMBState != STATE_ENABLED )
    {
        return MB_EILLSTATE;
    }

    /* Check if there is a event available. If not return control to caller.
     * Otherwise we will handle the event. */
    if( xMBPortEventGet( &eEvent ) == TRUE )
    {
        switch ( eEvent )
        {
            case EV_READY:
                break;

            case EV_FRAME_RECEIVED:
                eStatus = peMBFrameReceiveCur( &ucRcvAddress, &ucMBFrame, &usLength );
                if( eStatus == MB_ENOERR )
                {
                    /* Check if the frame is for us. If not ignore the frame. */
                    if( ( ucRcvAddress == ucMBAddress ) || ( ucRcvAddress == MB_ADDRESS_BROADCAST ) )
                    {
                        ( void )xMBPortEventPost( EV_EXECUTE );
                    }
                }
                break;

            case EV_EXECUTE:
                ucFunctionCode = ucMBFrame[MB_PDU_FUNC_OFF];
                eException = MB_EX_ILLEGAL_FUNCTION;
                for( i = 0; i < MB_FUNC_HANDLERS_MAX; i++ )
                {
                    /* No more function handlers registered. Abort. */
                    if( xFuncHandlers[i].ucFunctionCode == 0 )
                    {
                        break;
                    }
                    else if( xFuncHandlers[i].ucFunctionCode == ucFunctionCode )
                    {
                        eException = xFuncHandlers[i].pxHandler( ucMBFrame, &usLength );
                        break;
                    }
                }

                /* If the request was not sent to the broadcast address we
                 * return a reply. */
                if( ucRcvAddress != MB_ADDRESS_BROADCAST )
                {
                    if( eException != MB_EX_NONE )
                    {
                        /* An exception occurred. Build an error frame. */
                        usLength = 0;
                        ucMBFrame[usLength++] = ( UCHAR )( ucFunctionCode | MB_FUNC_ERROR );
                        ucMBFrame[usLength++] = eException;
                    }
                    if( ( eMBCurrentMode == MB_ASCII ) && MB_ASCII_TIMEOUT_WAIT_BEFORE_SEND_MS )
                    {
                        vMBPortTimersDelay( MB_ASCII_TIMEOUT_WAIT_BEFORE_SEND_MS );
                    }
                    eStatus = peMBFrameSendCur( ucMBAddress, ucMBFrame, usLength );
                }
                break;

            case EV_FRAME_SENT:
                break;
        }
    }
    return MB_ENOERR;
}
```

我们详细地分析一下这个文件。

从机地址

```
static UCHAR ucMBAddress
```

ucMBAddress变量存储了modbus的从机地址，该地址用一个无符号字符型的变量来表示，可以表示的数据范围为0~255；

从机模式

```
static eMBMode eMBCurrentMode
```

eMBCurrentMode用来表示当前modbus协议栈的类型，modbus协议栈的类型有三种，由如下枚举类型定义，该定义在mb.h文件中。

```
typedef enum
{
    MB_RTU,                /*!< RTU transmission mode. */
    MB_ASCII,              /*!< ASCII transmission mode. */
    MB_TCP,                /*!< TCP mode. */
} eMBMode;
```

从上面的定义，我们可以看出，FreeModbus协议共支持三种类型的modbus协议，分别是MODBUS-RTU、MODBUS-ASCII和MODBUS-TCP。

modbus协议函数

mb.c主要实现了7个函数，先用表格简单描述一下这几个函数的功能。

函数名	功能
eMBInit()	主要实现modbus协议栈的初始化，这里主要初始化MODBUS-RTU和MODBUS-ASCII，不包括MODBUS-TCP
eMBTCPInit()	主要完成MODBUS-TCP的初始化
eMBRegisterCB()	注册新的功能码和相应的处理函数到功能码数组中，便于我们扩展
eMBClose()	关闭modbus协议栈
eMBEnable()	使能modbus协议栈
eMBDisable()	禁止modbus协议栈
eMBPoll()	modbus轮询函数，主要完成事件的查询和相关处理函数的调用

eMBInit

该函数的接收参数为modbus的工作模式、从机地址、端口号、波特率、奇偶校验设置。

广播地址为：0xFF。

函数进来之后首先检查设置的地址合法性，如果设置的地址为广播地址或者不在最小地址和最大地址范围之内，则返回故障。

如果地址正确，则将地址设置到Modbus的地址当中。然后根据Modbus的设置模式，将Modbus的处理函数和RTU或者ASCII的函数关联起来。然后初始化串口控制器和事件控制器。

eMBTCPInit

eMBTCPInit() 函数和 eMBInit() 函数类似，一个是初始化RTU和ASCII协议，一个是初始化TCP协议。这里 eMBTCPInit() 函数初始化TCP协议栈。过程和RTU与ASCII相同，只是传递的参数不同。

eMBRegisterCB

这个函数主要是注册或者取消注册Modbus协议的功能码和相应的处理函数的。当传入的函数指针为NULL的时候，注销功能码和它的处理函数。

eMBClose

该函数主要是关闭串口传输，当不再使用协议的时候，可以关闭。

eMBEnable

该函数主要是使能UART的接收中断和开启定时器。接收中断用来接收主栈发送过来的数据，定时器用来进行超时检测。

eMBDisable

该函数和 eMBEnable() 函数功能相反，用来关闭UART接收中断和发送中断，关闭定时器。

eMBPoll

这是modbus协议的最主要的函数，该函数对事件进行轮询，当接收到数据的时候进行处理。看一下这个函数。

函数首先判断Modbus协议是否使能，如果没有使能，则返回故障。接下来查询Modbus的事件。

当系统识别到接收完成事件的时候，进行数据的接收。接受完数据之后，判断是否需要处理（地址是自己的地址或者是广播地址），如果需要处理，则发送一个执行事件，否则就不做处理。系统获取到处理事件的时候，进入数据处理过程。

首先从接收到的数据中获取到功能码，然后查找功能码表（上面说到的xFuncHandlers数组），然后调用相应功能码的处理函数进行数据处理。数据处理完之后，判断是否需要发送返回帧，如果不是广播地址就需要返回，如果错误，返回的功能码最高位置1，没有错误，则调用发送函数，将返回帧发送出去。

5. RTU代码

mb.c只是实现modbus协议的一个框架，具体的RTU、ASCII和TCP相关的代码再具体的文件夹中。这里我们以使用比较多的RTU为例，来分析一下RTU的实现。

RTU文件夹的文件

rtu文件夹下有四个文件，其说明如下：

文件名称	说明
mbcrc.c	这个文件只包含一个函数，就是标准的CRC16校验函数
mbcrc.h	包含CRC校验函数的函数声明
mbrtu.c	实现RTU协议的具体函数，rtu协议相关的实现函数都在这个文件中
mbrtu.h	头文件，包含rtu函数的声明

这里我们主要分析mbrtu.c文件，其他文件都比较简单。

mbrtu.c

```
/*
 * FreeModbus Library: A portable Modbus implementation for Modbus ASCII/RTU.
 * Copyright (c) 2006-2018 Christian Walter <cwalter@embedded-solutions.at>
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 * 3. The name of the author may not be used to endorse or promote products
 *    derived from this software without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS'' AND ANY EXPRESS OR
 * IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
 * OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
 * IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT,
 * INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT
 * NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
 * DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
 * THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
 * (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF
 * THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
 */

/* ----- System includes -----*/
#include "stdlib.h"
#include "string.h"

/* ----- Platform includes -----*/
#include "port.h"

/* ----- Modbus includes -----*/
#include "mb.h"
#include "mbrtu.h"
#include "mbframe.h"

#include "mbcrc.h"
#include "mbport.h"

/* ----- Defines -----*/
#define MB_SER_PDU_SIZE_MIN      4      /*!< Minimum size of a Modbus RTU frame. */
#define MB_SER_PDU_SIZE_MAX     256    /*!< Maximum size of a Modbus RTU frame. */
#define MB_SER_PDU_SIZE_CRC     2      /*!< Size of CRC field in PDU. */
#define MB_SER_PDU_ADDR_OFF     0      /*!< Offset of slave address in Ser-PDU. */
#define MB_SER_PDU_PDU_OFF      1      /*!< Offset of Modbus-PDU in Ser-PDU. */

/* ----- Type definitions -----*/
typedef enum
{
    STATE_RX_INIT,           /*!< Receiver is in initial state. */
    STATE_RX_IDLE,           /*!< Receiver is in idle state. */
    STATE_RX_RCV,            /*!< Frame is beeing received. */
    STATE_RX_ERROR           /*!< If the frame is invalid. */
} eMBRcvState;

typedef enum
{
    STATE_TX_IDLE,           /*!< Transmitter is in idle state. */
    STATE_TX_XMIT            /*!< Transmitter is in transfer state. */
} eMBSndState;

/* ----- Static variables -----*/
static volatile eMBSndState eSndState;
static volatile eMBRcvState eRcvState;

volatile UCHAR  ucRTUBuf[MB_SER_PDU_SIZE_MAX];

static volatile UCHAR *pucSndBufferCur;
static volatile USHORT usSndBufferCount;

static volatile USHORT usRcvBufferPos;

/* ----- Start implementation -----*/
eMBErrorCode
eMBRTUInit( UCHAR ucSlaveAddress, UCHAR ucPort, ULONG ulBaudRate, eMBParity eParity )
{
    eMBErrorCode    eStatus = MB_ENOERR;
    ULONG           usTimerT35_50us;

    ( void )ucSlaveAddress;
    ENTER_CRITICAL_SECTION( );

    /* Modbus RTU uses 8 Databits. */
    if( xMBPortSerialInit( ucPort, ulBaudRate, 8, eParity ) != TRUE )
    {
        eStatus = MB_EPORTERR;
    }
    else
    {
        /* If baudrate > 19200 then we should use the fixed timer values
         * t35 = 1750us. Otherwise t35 must be 3.5 times the character time.
         */
        if( ulBaudRate > 19200 )
        {
            usTimerT35_50us = 35;          /* 1800us. */
        }
        else
        {
            /* The timer reload value for a character is given by:
             *
             * ChTimevalue = Ticks_per_1s / ( Baudrate / 11 )
             *              = 11 * Ticks_per_1s / Baudrate
             *              = 220000 / Baudrate
             * The reload for t3.5 is 1.5 times this value and similiary
             * for t3.5.
             */
            usTimerT35_50us = ( 7UL * 220000UL ) / ( 2UL * ulBaudRate );
        }
        if( xMBPortTimersInit( ( USHORT ) usTimerT35_50us ) != TRUE )
        {
            eStatus = MB_EPORTERR;
        }
    }
    EXIT_CRITICAL_SECTION( );
}
```

```

        return eStatus;
    }

void
eMBRTUStart( void )
{
    ENTER_CRITICAL_SECTION( );
    /* Initially the receiver is in the state STATE_RX_INIT. we start
     * the timer and if no character is received within t3.5 we change
     * to STATE_RX_IDLE. This makes sure that we delay startup of the
     * modbus protocol stack until the bus is free.
     */
    eRcvState = STATE_RX_INIT;
    vMBPortSerialEnable( TRUE, FALSE );
    vMBPortTimersEnable( );

    EXIT_CRITICAL_SECTION( );
}

void
eMBRTUStop( void )
{
    ENTER_CRITICAL_SECTION( );
    vMBPortSerialEnable( FALSE, FALSE );
    vMBPortTimersDisable( );
    EXIT_CRITICAL_SECTION( );
}

eMBErrorCode
eMBRTUReceive( UCHAR * pucRcvAddress, UCHAR ** pucFrame, USHORT * pusLength )
{
    BOOL            xFrameReceived = FALSE;
    eMBErrorCode    eStatus = MB_ENOERR;

    ENTER_CRITICAL_SECTION( );
    assert( usRcvBufferPos < MB_SER_PDU_SIZE_MAX );

    /* Length and CRC check */
    if( ( usRcvBufferPos >= MB_SER_PDU_SIZE_MIN )
        && ( usMBCRC16( ( UCHAR * ) ucRTUBuf, usRcvBufferPos ) == 0 ) )
    {
        /* Save the address field. All frames are passed to the upper layer
         * and the decision if a frame is used is done there.
         */
        *pucRcvAddress = ucRTUBuf[MB_SER_PDU_ADDR_OFF];

        /* Total length of Modbus-PDU is Modbus-Serial-Line-PDU minus
         * size of address field and CRC checksum.
         */
        *pusLength = ( USHORT )( usRcvBufferPos - MB_SER_PDU_PDU_OFF - MB_SER_PDU_SIZE_CRC );

        /* Return the start of the Modbus PDU to the caller. */
        *pucFrame = ( UCHAR * ) & ucRTUBuf[MB_SER_PDU_PDU_OFF];
        xFrameReceived = TRUE;
    }
    else
    {
        eStatus = MB_EIO;
    }

    EXIT_CRITICAL_SECTION( );
    return eStatus;
}

eMBErrorCode
eMBRTUSend( UCHAR ucSlaveAddress, const UCHAR * pucFrame, USHORT usLength )
{
    eMBErrorCode    eStatus = MB_ENOERR;
    USHORT          usCRC16;

    ENTER_CRITICAL_SECTION( );

    /* Check if the receiver is still in idle state. If not we have to
     * slow with processing the received frame and the master sent another
     * frame on the network. We have to abort sending the frame.
     */
    if( eRcvState == STATE_RX_IDLE )
    {
        /* First byte before the Modbus-PDU is the slave address. */
        pucSndBufferCur = ( UCHAR * ) pucFrame - 1;
        usSndBufferCount = 1;

        /* Now copy the Modbus-PDU into the Modbus-Serial-Line-PDU. */
        pucSndBufferCur[MB_SER_PDU_ADDR_OFF] = ucSlaveAddress;
        usSndBufferCount += usLength;

        /* Calculate CRC16 checksum for Modbus-Serial-Line-PDU. */
        usCRC16 = usMBCRC16( ( UCHAR * ) pucSndBufferCur, usSndBufferCount );
        ucRTUBuf[usSndBufferCount++] = ( UCHAR )( usCRC16 & 0xFF );
        ucRTUBuf[usSndBufferCount++] = ( UCHAR )( usCRC16 >> 8 );

        /* Activate the transmitter. */
        eSndState = STATE_TX_XMIT;
        vMBPortSerialEnable( FALSE, TRUE );
    }
    else
    {
        eStatus = MB_EIO;
    }

    EXIT_CRITICAL_SECTION( );
    return eStatus;
}

BOOL
xMBRTUReceiveFSM( void )
{
    BOOL            xTaskNeedSwitch = FALSE;
    UCHAR          ucByte;

    assert( eSndState == STATE_TX_IDLE );

    /* Always read the character. */
    ( void )xMBPortSerialGetByte( ( CHAR * ) & ucByte );

    switch ( eRcvState )
    {
        /* If we have received a character in the init state we have to
         * wait until the frame is finished.
         */
        case STATE_RX_INIT:
            vMBPortTimersEnable( );
            break;

        /* In the error state we wait until all characters in the
         * damaged frame are transmitted.
         */
        case STATE_RX_ERROR:
            vMBPortTimersEnable( );
            break;

        /* In the idle state we wait for a new character. If a character
         * is received the t1.5 and t3.5 timers are started and the
         * receiver is in the state STATE_RX_RECEIVE.
         */
        case STATE_RX_IDLE:
            usRcvBufferPos = 0;
            ucRTUBuf[usRcvBufferPos++] = ucByte;
            eRcvState = STATE_RX_RCV;

            /* Enable t3.5 timers. */
            vMBPortTimersEnable( );
            break;

        /* We are currently receiving a frame. Reset the timer after
         * every character received. If more than the maximum possible
         * number of bytes in a modbus frame is received the frame is
         * ignored.
         */
        case STATE_RX_RCV:
            if( usRcvBufferPos < MB_SER_PDU_SIZE_MAX )
            {
                ucRTUBuf[usRcvBufferPos++] = ucByte;
            }
            else
            {
                eRcvState = STATE_RX_ERROR;
            }
    }
}

```

```

    vMBPortTimersEnable( );
    break;
}
return xTaskNeedSwitch;
}

BOOL
xMBRTUTransmitFSM( void )
{
    BOOL          xNeedPoll = FALSE;

    assert( eRcvState == STATE_RX_IDLE );

    switch ( eSndState )
    {
        /* we should not get a transmitter event if the transmitter is in
        * idle state. */
        case STATE_TX_IDLE:
            /* enable receiver/disable transmitter. */
            vMBPortSerialEnable( TRUE, FALSE );
            break;

        case STATE_TX_XMIT:
            /* check if we are finished. */
            if( usSndBufferCount != 0 )
            {
                xMBPortSerialPutByte( ( CHAR )*pucSndBufferCur );
                pucSndBufferCur++; /* next byte in sendbuffer. */
                usSndBufferCount--;
            }
            else
            {
                xNeedPoll = xMBPortEventPost( EV_FRAME_SENT );
                /* Disable transmitter. This prevents another transmit buffer
                * empty interrupt. */
                vMBPortSerialEnable( TRUE, FALSE );
                eSndState = STATE_TX_IDLE;
            }
            break;
    }

    return xNeedPoll;
}

BOOL
xMBRTUTimerT35Expired( void )
{
    BOOL          xNeedPoll = FALSE;

    switch ( eRcvState )
    {
        /* Timer t35 expired. Startup phase is finished. */
        case STATE_RX_INIT:
            xNeedPoll = xMBPortEventPost( EV_READY );
            break;

        /* A frame was received and t35 expired. Notify the listener that
        * a new frame was received. */
        case STATE_RX_RCV:
            xNeedPoll = xMBPortEventPost( EV_FRAME_RECEIVED );
            break;

        /* An error occured while receiving the frame. */
        case STATE_RX_ERROR:
            break;

        /* Function called in an illegal state. */
        default:
            assert( ( eRcvState == STATE_RX_INIT ) ||
                    ( eRcvState == STATE_RX_RCV ) || ( eRcvState == STATE_RX_ERROR ) );
    }

    vMBPortTimersDisable( );
    eRcvState = STATE_RX_IDLE;

    return xNeedPoll;
}

```

数据类型说明

该文件主要实现了数据的接收和发送的相关函数。首先，用枚举定义了接收器和发送器的状态，列表说明如下：
接收器状态说明：

接收器状态	说明
STATE_RX_INIT	接收器已初始化
STATE_RX_IDLE	接收器空闲
STATE_RX_RCV	接收器正在接收
STATE_RX_ERROR	接收器错误

发送器状态说明：

发送器状态	说明
STATE_TX_IDLE	发送器空闲
STATE_TX_XMIT	正在发送

接着定义了一个数据缓存区、发送缓存指针、发送缓存长度、数据接收长度。

eMBRTUInit()

该函数实现了RTU涉及到的UART和定时器的初始化。这里也只是提供一个函数，并未对函数进行实现。由于不同的处理器的UART和TIMER的初始化配置方式不同，所以这部分代码需要我们在移植的时候进行实现。具体等到移植的时候再来分析，这里我们重点看一下这个定时器时间是怎么配置的。

我们知道Modbus没有一帧结束的标记符，我们一般是通过超时时间来判断一帧是不是结束。超时时间一般设置为3.5个字符传送的时间。由于波特率不同对应的字符传送时间也不同，因此我们需要针对不同的波特率进行配置。详细说明如下：

根据eMBRTUInit()函数我们可以看出，FreeModbus将波特率大于19200的超时时间固定为1750us，其他的按照3.5个字符的传送时间来设置。这里我们定时器配置为每50us中断一次，因此我们只需要计算不同波特率下的定时器的中断次数即可。

- baudrate>19200时
此种情况下次数T=1750/50=35；所以从代码中可以看到波特率大于19200的时候，次数固定为35。
- baudrates≤19200时
我们知道串口一般发送的格式为一个起始位、8或者9位数位、一位停止位、一般无校验或者一位校验位。加起来一帧大概有11个二进制位（不同的配置有所差别，这里取11个比较合适）。所以传送一个字符的时间就是11/baudrate（单位：s），传送3.5个字符的时间就是（7/2）（11/baudrate）（单位：s），由于我们定时器50us中断一次，1s=20000个50us，所以对应的50us的次数就是(7/2)（11/baudrate)20000=(7220000)/(2*baudrate);

eMBRTUStart()

该函数比较简单，将接收器的状态设置为初始化状态，然后使能UART接收中断，禁止发送中断，使能定时器中断。

eMBRTUStop()

该函数和 eMBRTUstart() 函数功能相反，禁止UART接收和发送中断，禁止定时器中断，不再响应UART发送过来的数据。

eMBRTUReceive()

这个函数就是接收函数，从上篇文章中我们知道，当数据接收完成之后，轮询函数会获取一个接收完成事件，然后调用数据接收函数来接收数据，mb.c中的接收函数就是绑定的这个函数。由这个函数来做具体的数据接收的活。

eMBRTUSend()

这个函数就是发送函数，从上篇文章中我们知道，当数据处理完成之后，轮询函数会根据接收到的地址是不是广播帧来判断是否需要发送回帧，然后调用数据发送函数来发送数据，mb.c中的发送函数就是绑定的这个函数。由这个函数来做具体的数据发送的活。

该函数主要工作就是在发送PDU前面填充从机地址，后面填充CRC校验值，更新发送数据的长度，使能UART发送中断。

xMBRTUReceiveFSM()

这个函数是UART串口中断接收处理函数和2.7中的发送函数都是在串口中断函数中调用的。其功能就是将串口接收到的数据放在ucRTUBuf数据缓冲区当中。

xMBRTUTransmitFSM()

这个函数是串口发送中断处理函数，函数的主要功能就是将缓冲区中的数据发送出去。

xMBRTUTimerT35Expired()

这个是超时处理函数，当接收超时时，说明接收完成，发送一个接收完成事件，然后关闭超时中断。

6. FreeModbus移植 - 实战1

在实例31的基础上新建新的工程，并在新的工程内添加modbus源文件到工程内。

在工程中创建一个目录FreeModbus。

名称	修改日期	类型	大小
Core	2024/9/11 14:28	文件夹	
Drivers	2024/9/11 14:29	文件夹	
FreeModbus	2024/9/11 14:34	文件夹	
MDK-ARM	2024/9/11 15:32	文件夹	
.mxproject	2024/9/11 15:28	MXPROJECT 文件	9 KB
47-modbus.ioc	2024/9/11 15:28	STM32CubeMX	9 KB

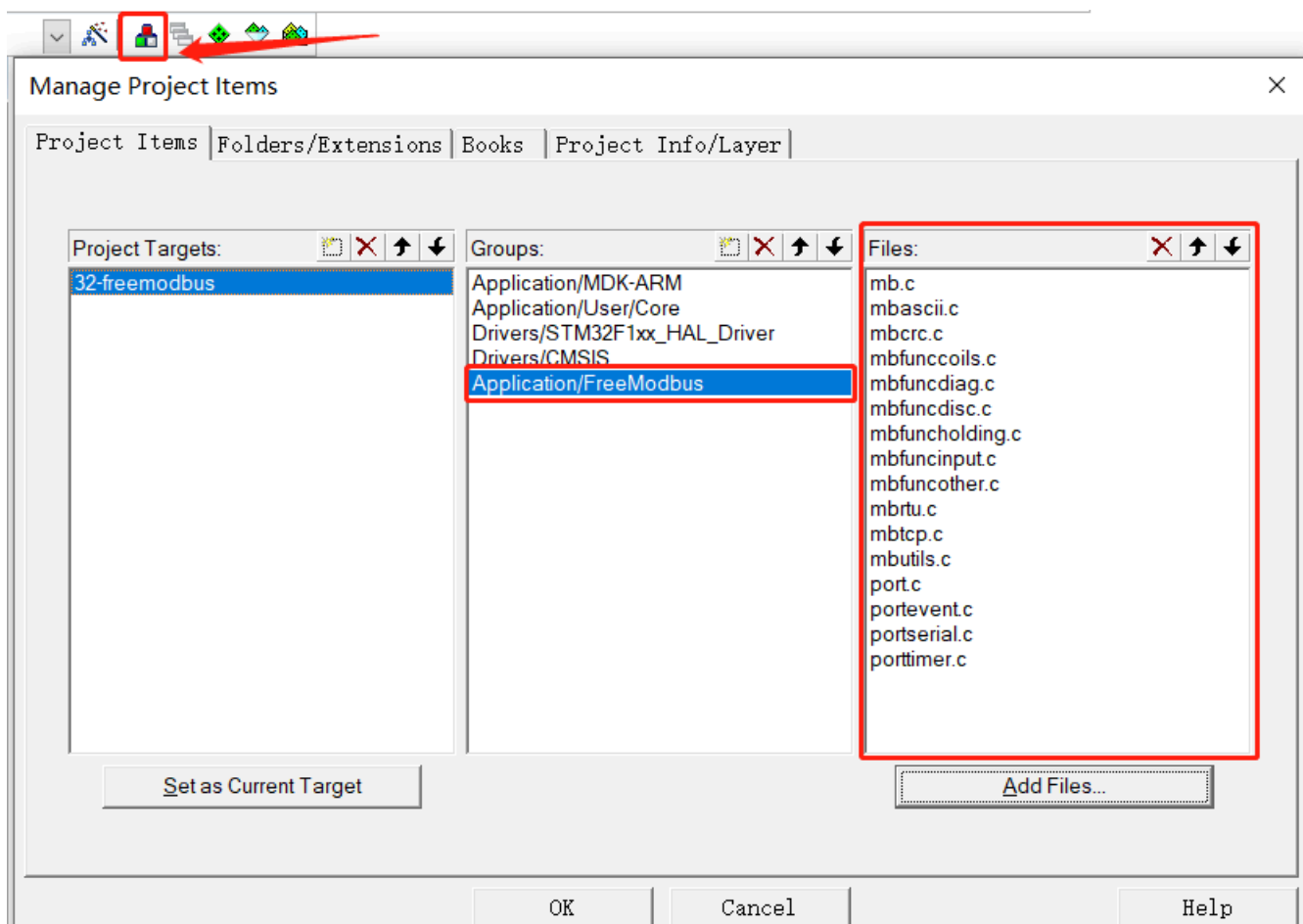
复制 STM32F103开发板资料V9.0\08-移植所需文件\freemodbus-v1.6\demo\BARE\port 目录内的文件到FreeModbus目录内。

为了移植时在keil添加源文件和头文件方便，就把modbus所有的头文件和源文件放到了一个文件夹下，并创建一个 `port.c` 文件，用于编写modbus所必需的回调处理函数：

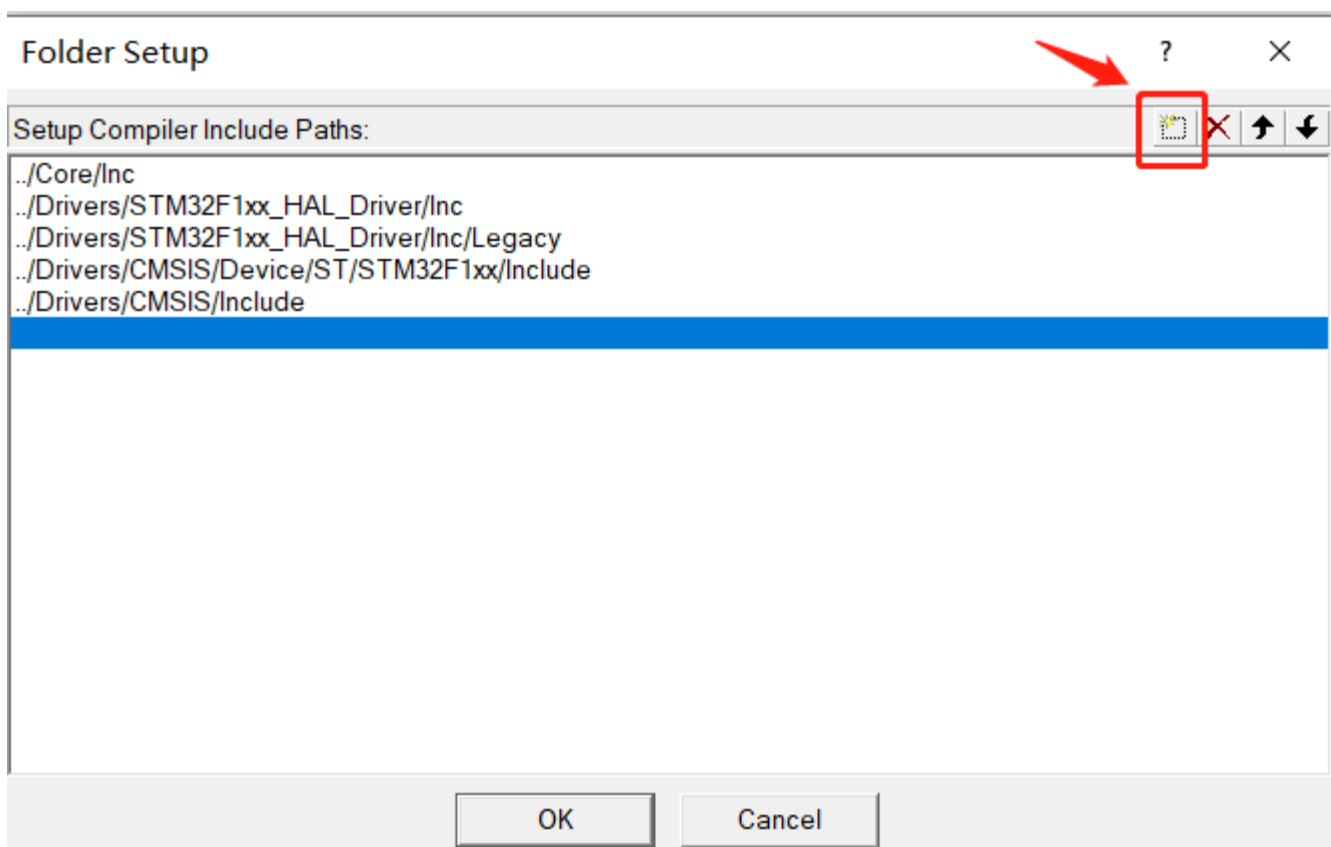
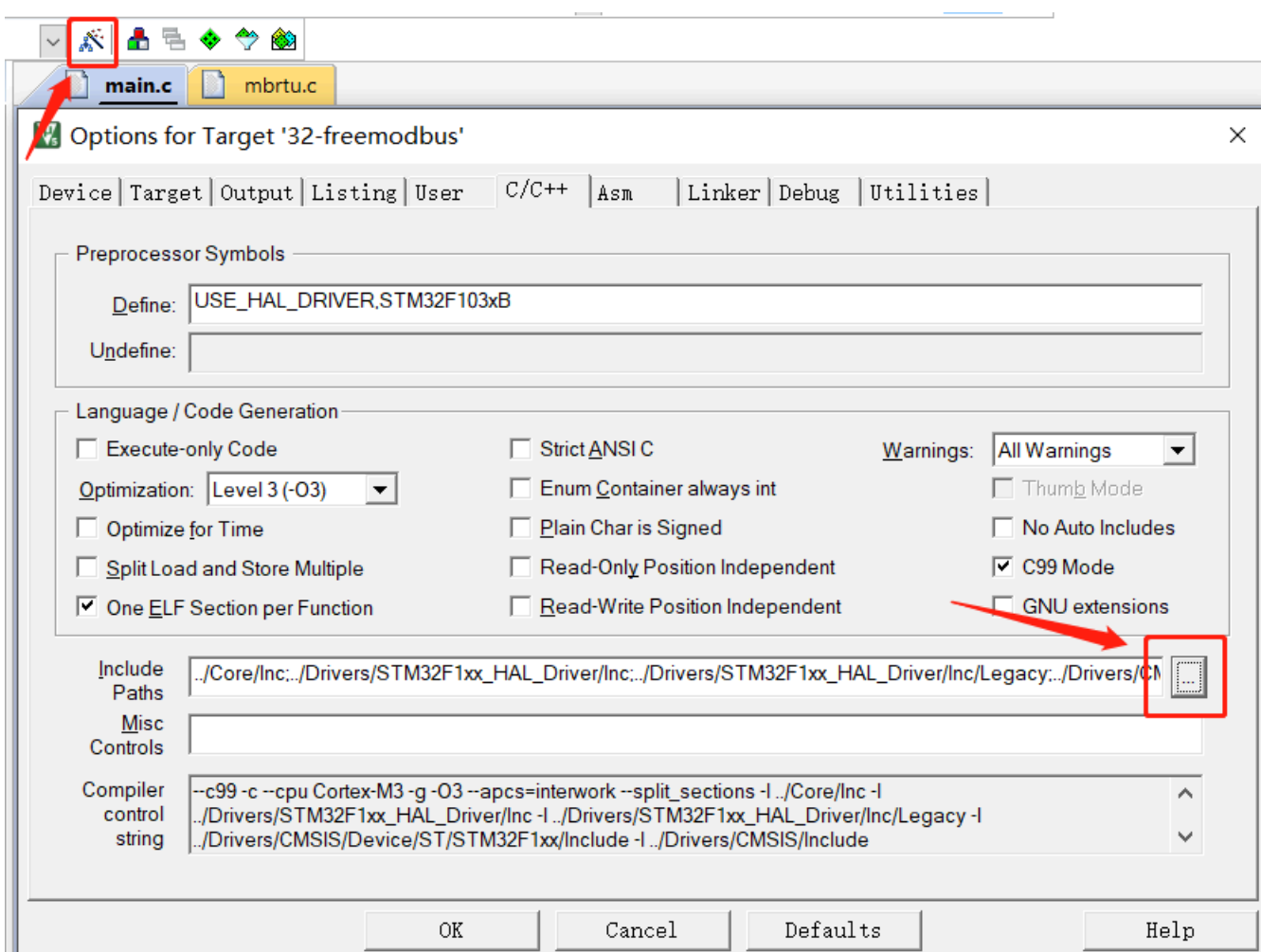
再复制 freemodbus-v1.6\modbus 目录下所有的文件到工程的FreeModbus目录内，子目录下的文件也复制到FreeModbus内。

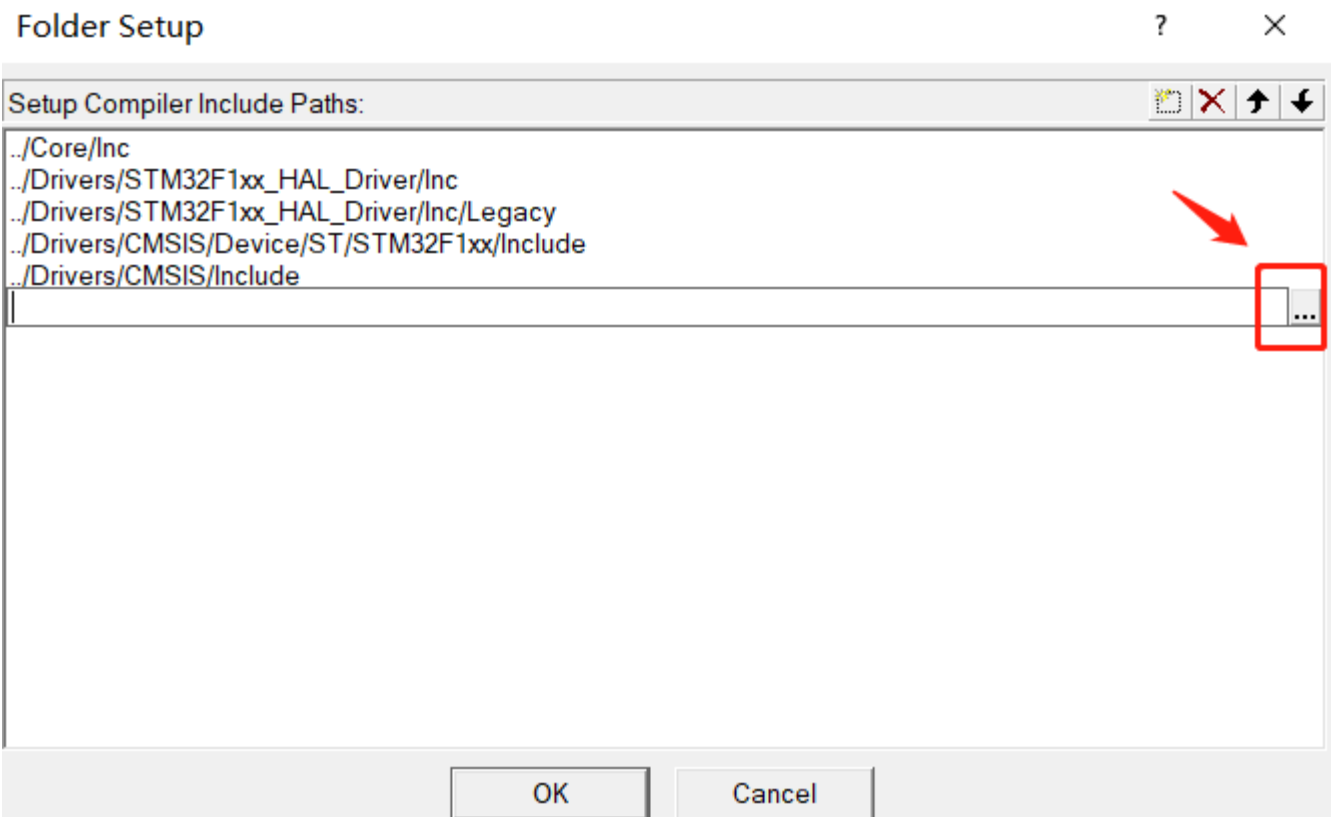
32-freemodbus > FreeModbus				
名称	修改日期	类型	大小	
 mb.c	2018/9/14 4:02	C 源文件	13 KB	
 mb.h	2018/9/14 4:02	C Header 源文件	19 KB	
 mbascii.c	2018/9/14 4:02	C 源文件	15 KB	
 mbascii.h	2018/9/14 4:02	C Header 源文件	3 KB	
 mbconfig.h	2018/9/14 4:02	C Header 源文件	6 KB	
 mbcrc.c	2018/9/14 4:02	C 源文件	6 KB	
 mbcrc.h	2018/9/14 4:02	C Header 源文件	2 KB	
 mbframe.h	2018/9/14 4:02	C Header 源文件	4 KB	
 mbfunc.h	2018/9/14 4:02	C Header 源文件	3 KB	
 mbfunccoils.c	2018/9/14 4:02	C 源文件	10 KB	
 mbfuncdiag.c	2018/9/14 4:02	C 源文件	2 KB	
 mbfuncdisc.c	2018/9/14 4:02	C 源文件	6 KB	
 mbfuncholding.c	2018/9/14 4:02	C 源文件	12 KB	
 mbfuncinput.c	2018/9/14 4:02	C 源文件	5 KB	
 mbfuncother.c	2018/9/14 4:02	C 源文件	4 KB	
 mbport.h	2018/9/14 4:02	C Header 源文件	5 KB	
 mbproto.h	2018/9/14 4:02	C Header 源文件	4 KB	
 mbrtu.c	2018/9/14 4:02	C 源文件	12 KB	
 mbrtu.h	2018/9/14 4:02	C Header 源文件	3 KB	
 mbtcp.c	2018/9/14 4:02	C 源文件	6 KB	
 mbtcp.h	2018/9/14 4:02	C Header 源文件	3 KB	
 mbutils.c	2018/9/14 4:02	C 源文件	5 KB	
 mbutils.h	2018/9/14 4:02	C Header 源文件	4 KB	
 port.c	2023/9/16 18:03	C 源文件	5 KB	
 port.h	2018/9/14 4:02	C Header 源文件	2 KB	
 portevent.c	2018/9/14 4:02	C 源文件	2 KB	
 portserial.c	2023/9/16 18:33	C 源文件	5 KB	
 porttimer.c	2023/9/16 18:13	C 源文件	4 KB	

添加源文件到工程内。

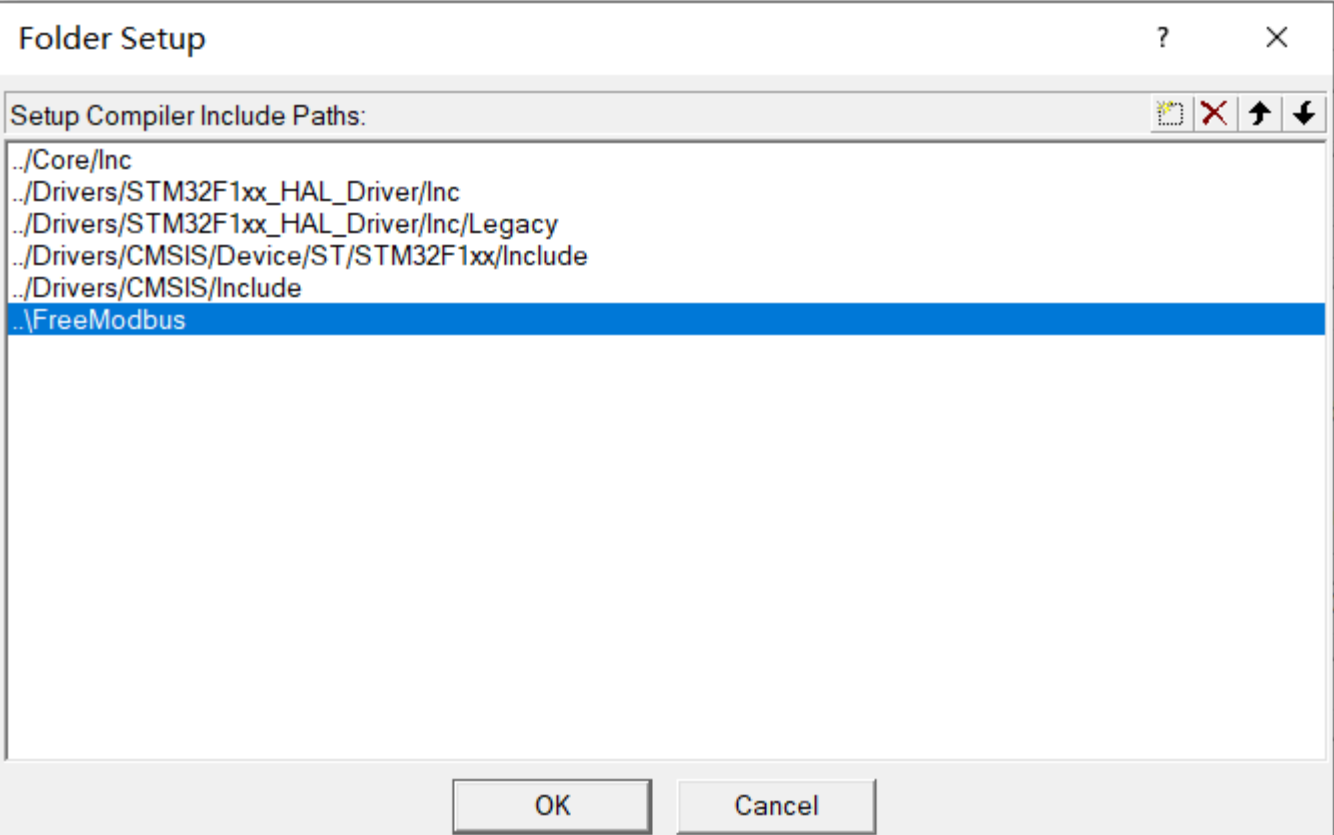
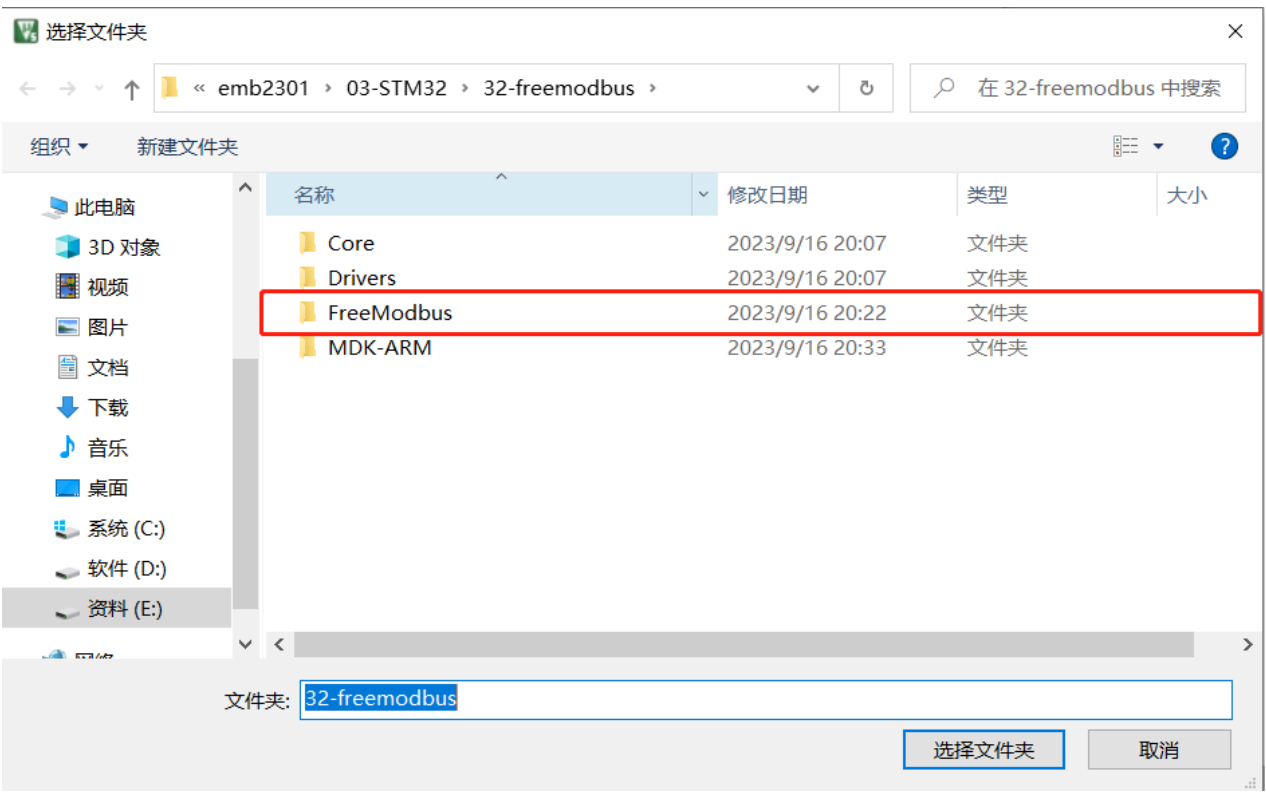


添加头文件包含目录





选择FreeModbus目录，点击打开。

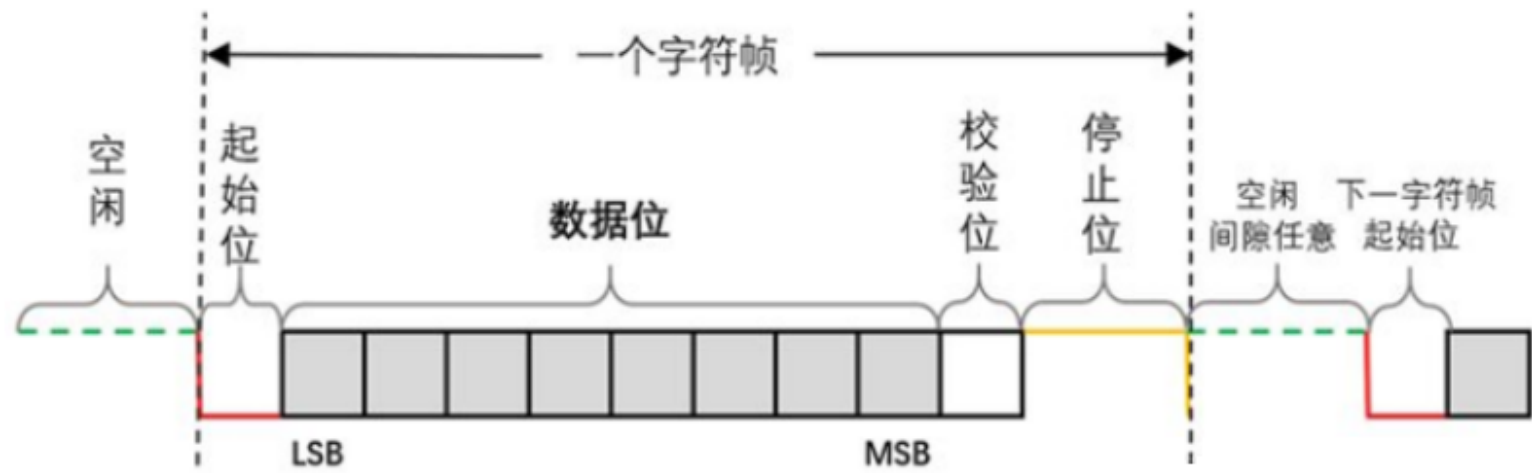


点击确定完成头文件的添加。

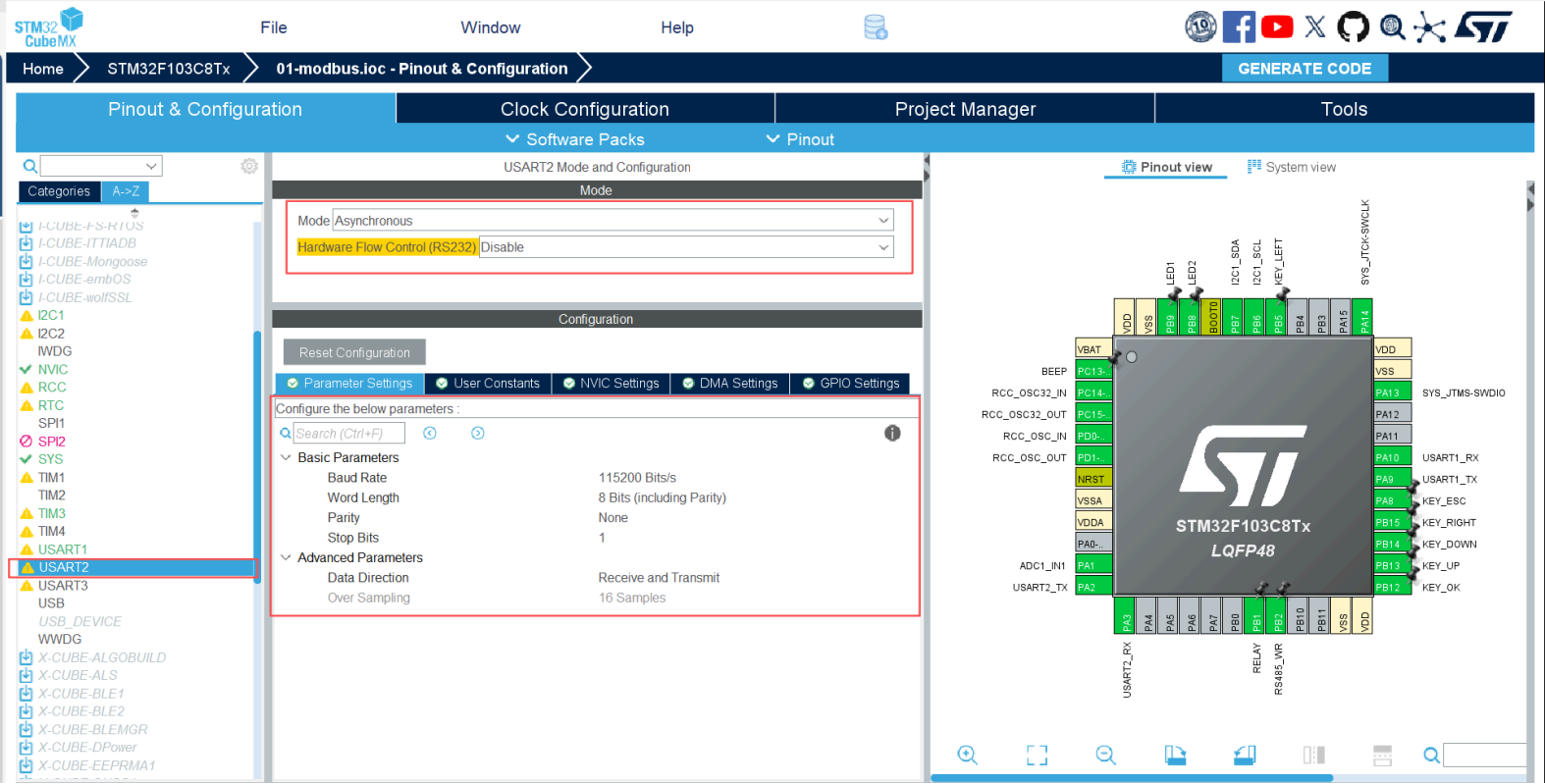
串口配置

本次设计使用的是modbus RTU模式，数据传输依靠串口来实现，所以需要将freemodbus库文件的数据的输入和输出定位在单片机的串口中，串口的移植在 portserial.c 中实现，在文件中主要实现了串口的初始化，串口的中断接收和发送的功能。

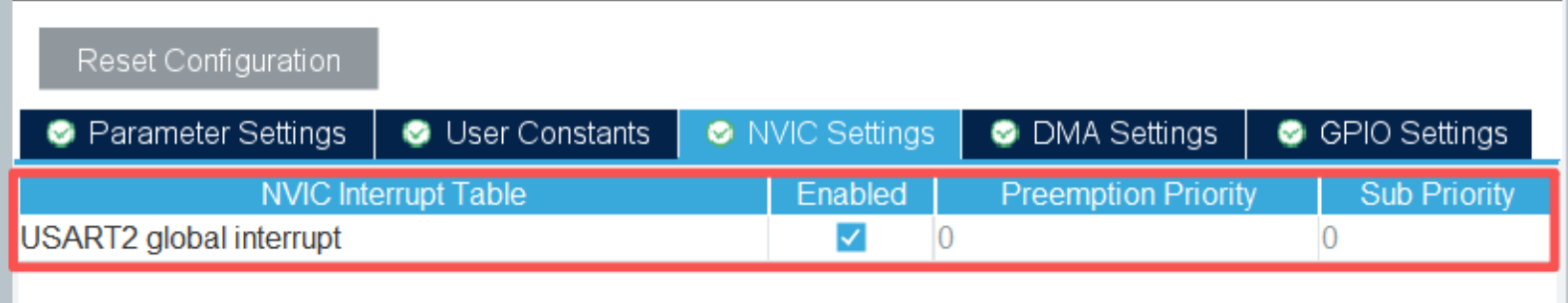
串口通讯的数据包由发送设备通过自身的TXD接口传输到接收设备得RXD接口，在协议层中规定了数据包的内容，具体包括起始位、主体数据（8位或9位）、校验位以及停止位，通讯的双方必须将数据包的格式约定一致才能正常收发数据。通讯帧格式如图所示。



启用UART2作为Modbus协议的底层通信口。



启用Uart2中断



定时器配置

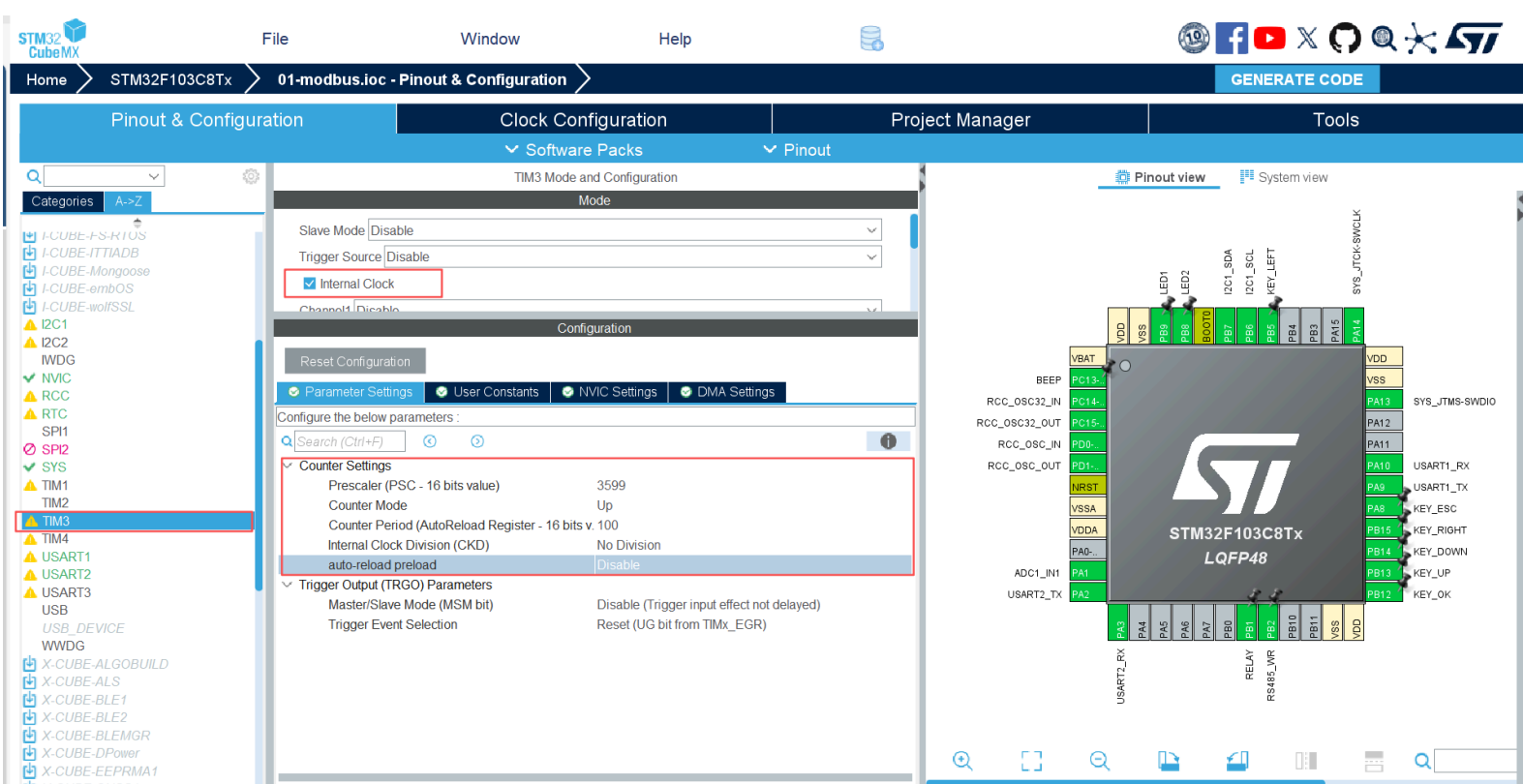
Modbus协议规定，在RTU模式中，消息的发送和接收至少以3.5个字符时间的停顿间隔为标志。实际使用中，网络设备不断侦测网络总线，计算字符间的停顿间隔时间，判断消息帧的起始点。

当接收到第一个域（地址域）时，每个设备都进行解码以判断是否是发给自己的，在最后一个传输字符结束之后，一个至少3.5个字符时间的停顿标志标定了消息的结束，而一个新的消息可在此停顿后开始。

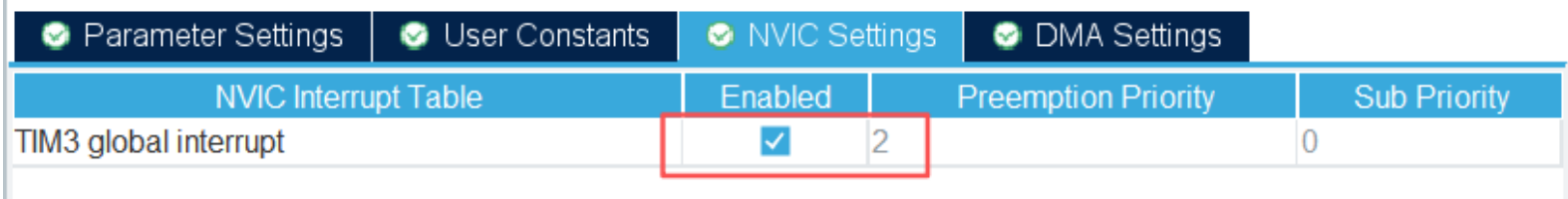
Modbus还规定当串口的波特率小于9600bps时，两个数据帧之间至少有3.5个字符的时间间隔，当波特率大于等于19200bps时，两个数据帧的时间间隔以波特为19200bps时的3.5个字符长度为判断依据。

这里配置不重要， 后序还需要重新初始化。

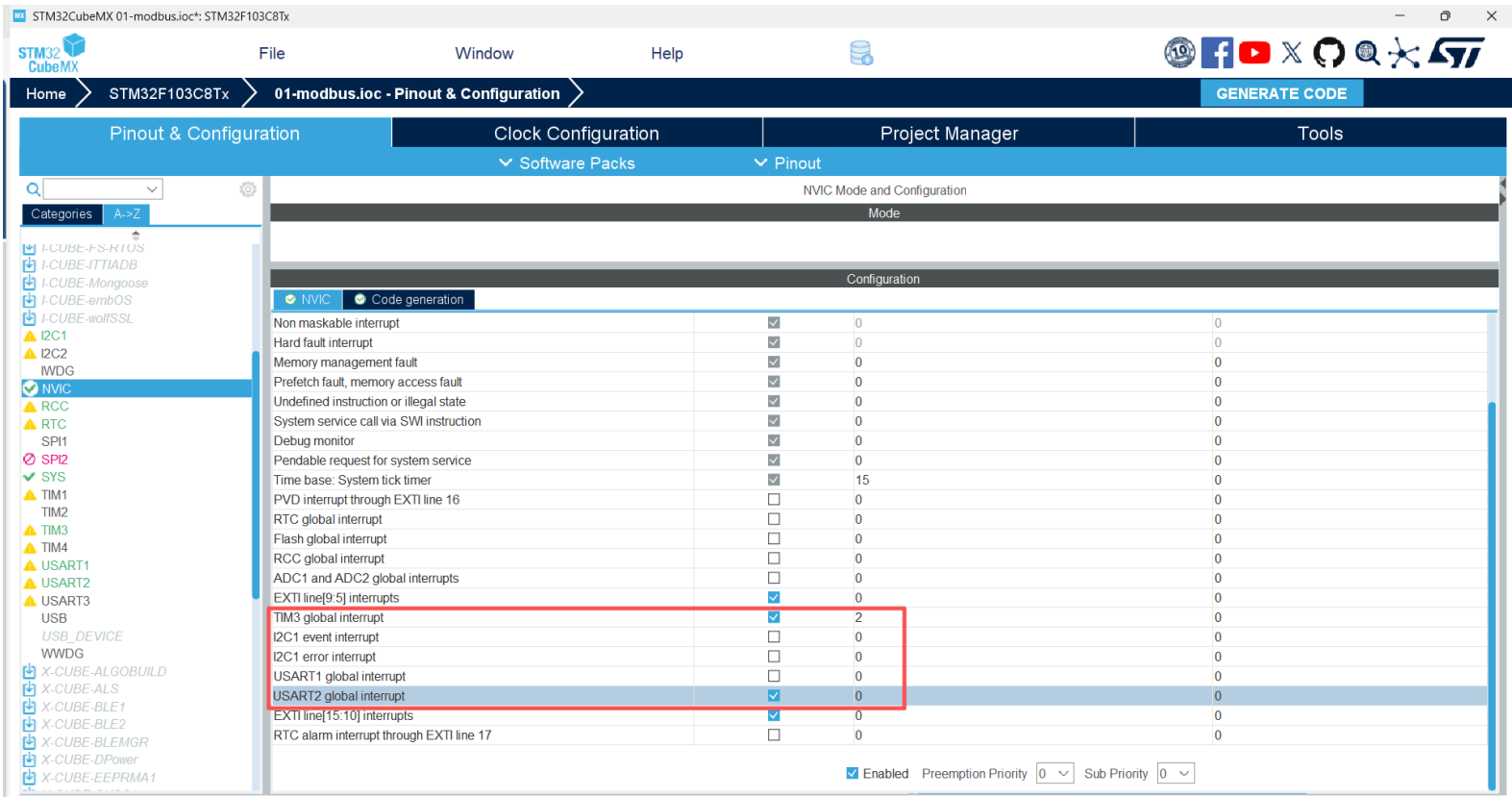
设置TIM3作为Modbus协议的定时器, 是为wifi和4G使用了TIM4， 这里我们使用TIM3。



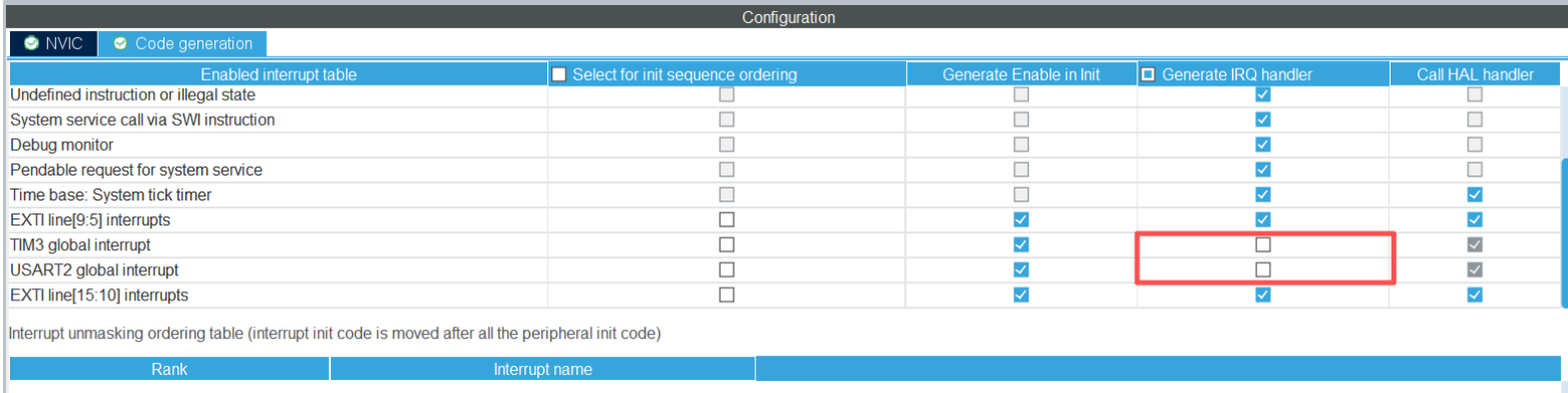
开启TIM3定时器中断



NVIC中设置中断优先级，这里注意，串口的优先级是要比定时器优先级高的。



取消掉自动生成中断服务程序，在移植过程中我们要自己编写串口和定时器的中断服务程序



port.c

```
#include "mb.h"
#include "mbport.h"

// 十路线圈 可读可写的 位控制
#define REG_COILS_SIZE 10
uint8_t REG_COILS_BUF[REG_COILS_SIZE] = {1, 1, 1, 1, 0, 0, 0, 0, 1, 1};
// 低位在前， 高位再后
// 1, 1, 1, 1, 0, 0, 0, 0, | 1, 1
// 0 1 2 3 4 5 6 7 | 8 9
// 0f | 03
// 对应的十六进制数为： 0x0f 0x03

// 十路离散量 只读的位控制
#define REG_DISC_SIZE 10
uint8_t REG_DISC_BUF[REG_DISC_SIZE] = {1,1,1,1,0,0,0,0,1,1};
// 只读的线圈

// 十路保持寄存器 可读可写的2字节寄存器
#define REG_HOLD_SIZE 10
uint16_t REG_HOLD_BUF[REG_HOLD_SIZE];

// 十路输入寄存器 只读的2字节寄存器
#define REG_INPUT_SIZE 10
uint16_t REG_INPUT_BUF[REG_INPUT_SIZE];
// 只读保持寄存器

/// CMD4命令处理回调函数
eMBCError eMBRegInputCB( UCHAR * pucRegBuffer, USHORT usAddress, USHORT usNRegs )
{
    USHORT usRegIndex = usAddress - 1;

    // 非法检测
    if((usRegIndex + usNRegs) > REG_INPUT_SIZE)
    {
        return MB_ENOREG;
    }

    // 循环读取
    while( usNRegs > 0 )
    {
        *pucRegBuffer++ = ( unsigned char )( REG_INPUT_BUF[usRegIndex] >> 8 );
        *pucRegBuffer++ = ( unsigned char )( REG_INPUT_BUF[usRegIndex] & 0xFF );
        usRegIndex++;
        usNRegs--;
    }

    // 模拟输入寄存器被改变
    for(usRegIndex = 0; usRegIndex < REG_INPUT_SIZE; usRegIndex++)
    {
        REG_INPUT_BUF[usRegIndex]++;
    }

    return MB_ENOERR;
}

/// CMD6、3、16命令处理回调函数
eMBCError eMBRegHoldingCB( UCHAR * pucRegBuffer, USHORT usAddress, USHORT usNRegs, eMBRegisterMode eMode )
{
    USHORT usRegIndex = usAddress - 1;

    // 非法检测
    if((usRegIndex + usNRegs) > REG_HOLD_SIZE)
    {
        return MB_ENOREG;
    }

    // 写寄存器
    if(eMode == MB_REG_WRITE)
    {
        while( usNRegs > 0 )
        {
            REG_HOLD_BUF[usRegIndex] = (pucRegBuffer[0] << 8) | pucRegBuffer[1];
            pucRegBuffer += 2;
            usRegIndex++;
            usNRegs--;
        }
    }

    // 读寄存器
    else
```

```

{
    while( usNRegs > 0 )
    {
        *pucRegBuffer++ = ( unsigned char )( REG_HOLD_BUF[usRegIndex] >> 8 );
        *pucRegBuffer++ = ( unsigned char )( REG_HOLD_BUF[usRegIndex] & 0xFF );
        usRegIndex++;
        usNRegs--;
    }
}

return MB_ENOERR;
}

/// CMD1、5、15命令处理回调函数
emBErrorCode eMBRegCoilsCB( UCHAR * pucRegBuffer, USHORT usAddress, USHORT usNCoils, emBRegisterMode eMode )
{
    USHORT usRegIndex    = usAddress - 1;
    UCHAR  ucBits         = 0;
    UCHAR  ucState        = 0;
    UCHAR  ucLoops        = 0;

    // 非法检测
    if((usRegIndex + usNCoils) > REG_COILS_SIZE)
    {
        return MB_ENOREG;
    }

    if(eMode == MB_REG_WRITE)
    {
        ucLoops = (usNCoils - 1) / 8 + 1;
        while(ucLoops != 0)
        {
            ucState = *pucRegBuffer++;
            ucBits  = 0;
            while(usNCoils != 0 && ucBits < 8)
            {
                REG_COILS_BUF[usRegIndex++] = (ucState >> ucBits) & 0x01;
                usNCoils--;
                ucBits++;
            }
            ucLoops--;
        }
    }
    else
    {
        ucLoops = (usNCoils - 1) / 8 + 1;
        while(ucLoops != 0)
        {
            ucState = 0;
            ucBits  = 0;
            while(usNCoils != 0 && ucBits < 8)
            {
                if(REG_COILS_BUF[usRegIndex])
                {
                    ucState |= (1 << ucBits);
                }
                usNCoils--;
                usRegIndex++;
                ucBits++;
            }
            *pucRegBuffer++ = ucState;
            ucLoops--;
        }
    }

    return MB_ENOERR;
}

/// CMD2命令处理回调函数
emBErrorCode eMBRegDiscreteCB( UCHAR * pucRegBuffer, USHORT usAddress, USHORT usNDiscrete )
{
    USHORT usRegIndex    = usAddress - 1;
    UCHAR  ucBits         = 0;
    UCHAR  ucState        = 0;
    UCHAR  ucLoops        = 0;

    // 非法检测
    if((usRegIndex + usNDiscrete) > REG_DISC_SIZE)
    {
        return MB_ENOREG;
    }

    ucLoops = (usNDiscrete - 1) / 8 + 1;
    while(ucLoops != 0)
    {
        ucState = 0;
        ucBits  = 0;
        while(usNDiscrete != 0 && ucBits < 8)
        {
            if(REG_DISC_BUF[usRegIndex])
            {
                ucState |= (1 << ucBits);
            }
            usNDiscrete--;
            usRegIndex++;
            ucBits++;
        }
        *pucRegBuffer++ = ucState;
        ucLoops--;
    }

    // 模拟离散量输入被改变
    for(usRegIndex = 0; usRegIndex < REG_DISC_SIZE; usRegIndex++)
    {
        REG_DISC_BUF[usRegIndex] = !REG_DISC_BUF[usRegIndex];
    }

    return MB_ENOERR;
}

```

porttimer.c

```

/*
 * FreeModbus Library: BARE Port
 * Copyright (C) 2006 Christian Walter <woltti@sil.at>
 *
 * This library is free software; you can redistribute it and/or
 * modify it under the terms of the GNU Lesser General Public
 * License as published by the Free Software Foundation; either
 * version 2.1 of the License, or (at your option) any later version.
 *
 * This library is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
 * Lesser General Public License for more details.
 *
 * You should have received a copy of the GNU Lesser General Public
 * License along with this library; if not, write to the Free Software
 * Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA
 *
 * File: $Id$
 */

/* ----- Platform includes -----*/
#include "port.h"
#include "tim.h"
#include <stdio.h>

/* ----- Modbus includes -----*/
#include "mb.h"
#include "mbport.h"

/* ----- static functions -----*/
static void prvVTIMERExpiredISR( void );

/* ----- Start implementation -----*/
BOOL xMBPortTimersInit( USHORT usTim1Timerout50us )
{
    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    htim3.Instance = TIM3;
    // 72M/ (7199+1) = 10KHz
    // 定时器工作一次的时间为 100us

    // 72M/ (3599+1) = 20KHz

```

```
// 定时器工作一次的时间为 50us

htim3.Init.Prescaler = 3599; // 50us 记一次数
// htim3.Init.Prescaler = 7199; // 100us记一次数
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
htim3.Init.Period = usTim1Timerout50us - 1; // usTim1Timerout50us * 50即为定时器溢出时间
htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim3) != HAL_OK)
{
    return FALSE;
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim3, &sClockSourceConfig) != HAL_OK)
{
    return FALSE;
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
{
    return FALSE;
}

__HAL_TIM_CLEAR_FLAG(&htim3, TIM_FLAG_UPDATE); // 先清除一下定时器的中断标记,防止使能中断后直接触发中断
__HAL_TIM_ENABLE_IT(&htim3, TIM_IT_UPDATE); // 使能定时器更新中断
return TRUE;
}

inline void VMBPortTimersEnable( )
{
    __HAL_TIM_SET_COUNTER(&htim3, 0); // 清空计数器
    __HAL_TIM_ENABLE(&htim3); // 使能定时器
}

inline void VMBPortTimersDisable( )
{
    __HAL_TIM_DISABLE(&htim3); // 禁能定时器
}

/* Create an ISR which is called whenever the timer has expired. This function
 * must then call pxMBPortCBTimerExpired( ) to notify the protocol stack that
 * the timer has expired.
 */
static void prvvtIMERExpiredISR( void )
{
    ( void )pxMBPortCBTimerExpired( );
}

// 定时器3中断服务程序
void TIM3_IRQHandler(void)
{
    if(__HAL_TIM_GET_FLAG(&htim3, TIM_FLAG_UPDATE)) // 更新中断标记被置位
    {
        __HAL_TIM_CLEAR_FLAG(&htim3, TIM_FLAG_UPDATE); // 清除中断标记
        prvvtIMERExpiredISR(); // 通知modbus3.5个字符等待时间到
    }
}
```

portserial.c

```
/*
 * FreeModbus Library: BARE Port
 * Copyright (C) 2006 Christian Walter <woltti@sil.at>
 *
 * This library is free software; you can redistribute it and/or
 * modify it under the terms of the GNU Lesser General Public
 * License as published by the Free Software Foundation; either
 * version 2.1 of the License, or (at your option) any later version.
 *
 * This library is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
 * Lesser General Public License for more details.
 *
 * You should have received a copy of the GNU Lesser General Public
 * License along with this library; if not, write to the Free Software
 * Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA
 *
 * File: $Id$
 */

#include "port.h"
#include "usart.h"

/* ----- Modbus includes -----*/
#include "mb.h"
#include "mbport.h"

/* ----- static functions -----*/
static void prvvUARTTxReadyISR( void );
static void prvvUARTRxISR( void );

/* ----- Start implementation -----*/
void VMBPortSerialEnable( BOOL xRxEnable, BOOL xTxEnable )
{
    if(xRxEnable)
    {
        //
        // 如果使用了485控制芯片,那么在此处将485设置为接收模式
        //
        // 低电平 默认为接收模式
        HAL_GPIO_WritePin(RS485_WR_GPIO_Port, RS485_WR_Pin, GPIO_PIN_RESET);
        __HAL_UART_ENABLE_IT(&huart2, UART_IT_RXNE); // 使能接收非空中断
    }
    else
    {
        __HAL_UART_DISABLE_IT(&huart2, UART_IT_RXNE); // 禁能接收非空中断
    }

    if(xTxEnable)
    {
        // 如果使用了485控制芯片,那么在此处将485设置为发送模式
        // RS485_WR ENABLE 高电平 发送模式
        HAL_GPIO_WritePin(RS485_WR_GPIO_Port, RS485_WR_Pin, GPIO_PIN_SET);

        __HAL_UART_ENABLE_IT(&huart2, UART_IT_TXE); // 使能发送为空中断
    }
    else
    {
        __HAL_UART_DISABLE_IT(&huart2, UART_IT_TXE); // 禁能发送为空中断
    }
}

BOOL xMBPortSerialInit( UCHAR ucPORT, ULONG ulBaudRate, UCHAR ucDataBits, eMBParity eParity )
{
    huart2.Instance = USART2;
    huart2.Init.BaudRate = ulBaudRate;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.Oversampling = UART_OVERSAMPLING_16;

    switch(eParity)
    {
        // 奇校验
        case MB_PAR_ODD:
            huart2.Init.Parity = UART_PARITY_ODD;
            huart2.Init.WordLength = UART_WORDLENGTH_9B; // 带奇偶校验数据位为9bits
            break;

        // 偶校验
        case MB_PAR_EVEN:
            huart2.Init.Parity = UART_PARITY_EVEN;
            huart2.Init.WordLength = UART_WORDLENGTH_9B; // 带奇偶校验数据位为9bits
            break;

        // 无校验
        default:
            huart2.Init.Parity = UART_PARITY_NONE;
            huart2.Init.WordLength = UART_WORDLENGTH_8B; // 无奇偶校验数据位为8bits
            break;
    }
    return HAL_UART_Init(&huart2) == HAL_OK ? TRUE : FALSE;
}
```

```

BOOL xMBPortSerialPutByte( CHAR ucByte )
{
    USART2->DR = ucByte;
    return TRUE;
}

BOOL xMBPortSerialGetByte( CHAR * pucByte )
{
    *pucByte = (USART2->DR & (uint16_t)0x00FF);
    return TRUE;
}

/* Create an interrupt handler for the transmit buffer empty interrupt
 * (or an equivalent) for your target processor. This function should then
 * call pxMBFrameCBTransmitterEmpty( ) which tells the protocol stack that
 * a new character can be sent. The protocol stack will then call
 * xMBPortSerialPutByte( ) to send the character.
 */
static void prvvUARTTxReadyISR( void )
{
    pxMBFrameCBTransmitterEmpty( );
}

/* Create an interrupt handler for the receive interrupt for your target
 * processor. This function should then call pxMBFrameCBByteReceived( ). The
 * protocol stack will then call xMBPortSerialGetByte( ) to retrieve the
 * character.
 */
static void prvvUARTRxISR( void )
{
    pxMBFrameCBByteReceived( );
}

void USART2_IRQHandler(void)
{
    if(__HAL_UART_GET_FLAG(&huart2, UART_FLAG_RXNE)           // 接收非空中断标记被置位
    {
        __HAL_UART_CLEAR_FLAG(&huart2, UART_FLAG_RXNE);      // 清除中断标记
        prvvUARTRxISR();                                       // 通知modbus有数据到达
    }

    if(__HAL_UART_GET_FLAG(&huart2, UART_FLAG_TXE) &&
        __HAL_UART_GET_FLAG(&huart2, UART_FLAG_TC)) // 发送为空中断标记被置位
    {
        __HAL_UART_CLEAR_FLAG(&huart2, UART_FLAG_TXE);        // 清除中断标记
        prvvUARTTxReadyISR();                                   // 通知modbus数据可以发送
    }
}

```

main.c

```

/* USER CODE BEGIN Header */
/**
     * @file           : main.c
     * @brief          : Main program body
     *
     * @attention
     *
     * Copyright (c) 2025 STMicroelectronics.
     * All rights reserved.
     *
     * This software is licensed under terms that can be found in the LICENSE file
     * in the root directory of this software component.
     * If no LICENSE file comes with this software, it is provided AS-IS.
     *
     */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include "adc.h"
#include "i2c.h"
#include "rtc.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "oled.h"
#include "aht20.h"
#include "ina226.h"
#include "mb.h"
/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
     * @brief The application entry point.
     * @retval int
     */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    MX_I2C1_Init();
    MX_RTC_Init();
    MX_ADC1_Init();
    MX_TIM3_Init();
    MX_USART2_UART_Init();
    /* USER CODE BEGIN 2 */
    printf("Uart1 init is ok\n");

```

```
printf("Modbus Test\n");
//EEPROM_Test();
OLED_Init();
OLED_CLS(); // 清除屏幕显示
OLED_ShowStr(16,0, (unsigned char*)"Modbus Control", 2);
AHT20_Init();
INA226_Init();

eMBInit(MB_RTU, 0x01, 0, 115200, MB_PAR_NONE); // 初始化modbus为RTU方式，波特率115200，无校验
eMBEnable();
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    eMBPoll();// 轮询查询
    }
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE|RCC_OSCILLATORTYPE_LSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.LSEState = RCC_LSE_ON;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
     */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
    PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_RTC|RCC_PERIPHCLK_ADC;
    PeriphClkInit.RTCClockSelection = RCC_RTCCCLKSOURCE_LSE;
    PeriphClkInit.AdcClockSelection = RCC_ADCPCLK2_DIV6;
    if (HAL_RCCEX_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
    {
        Error_Handler();
    }
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

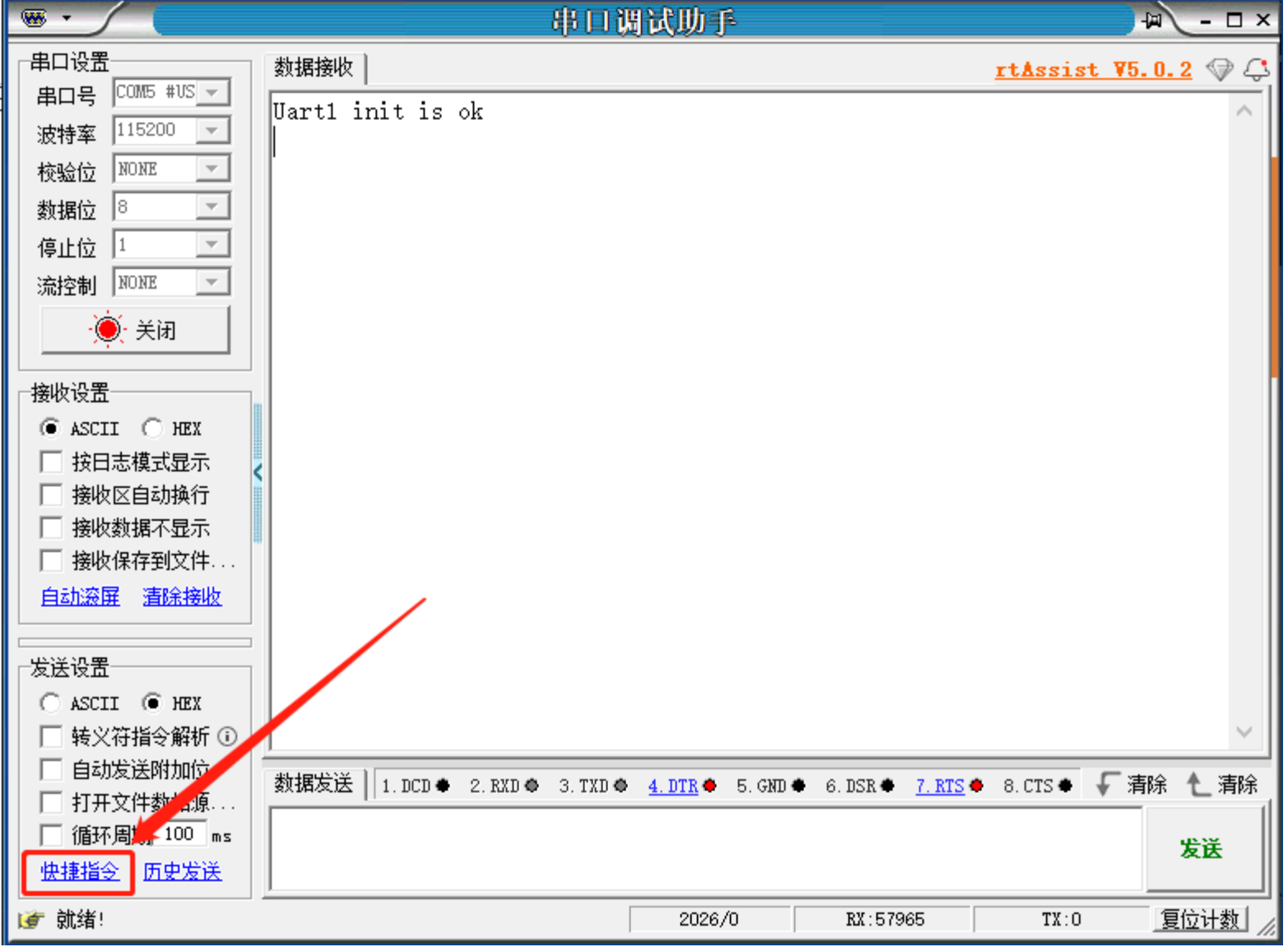
7. 测试FreeModbus功能

编译程序并下载程序，复位系统后。

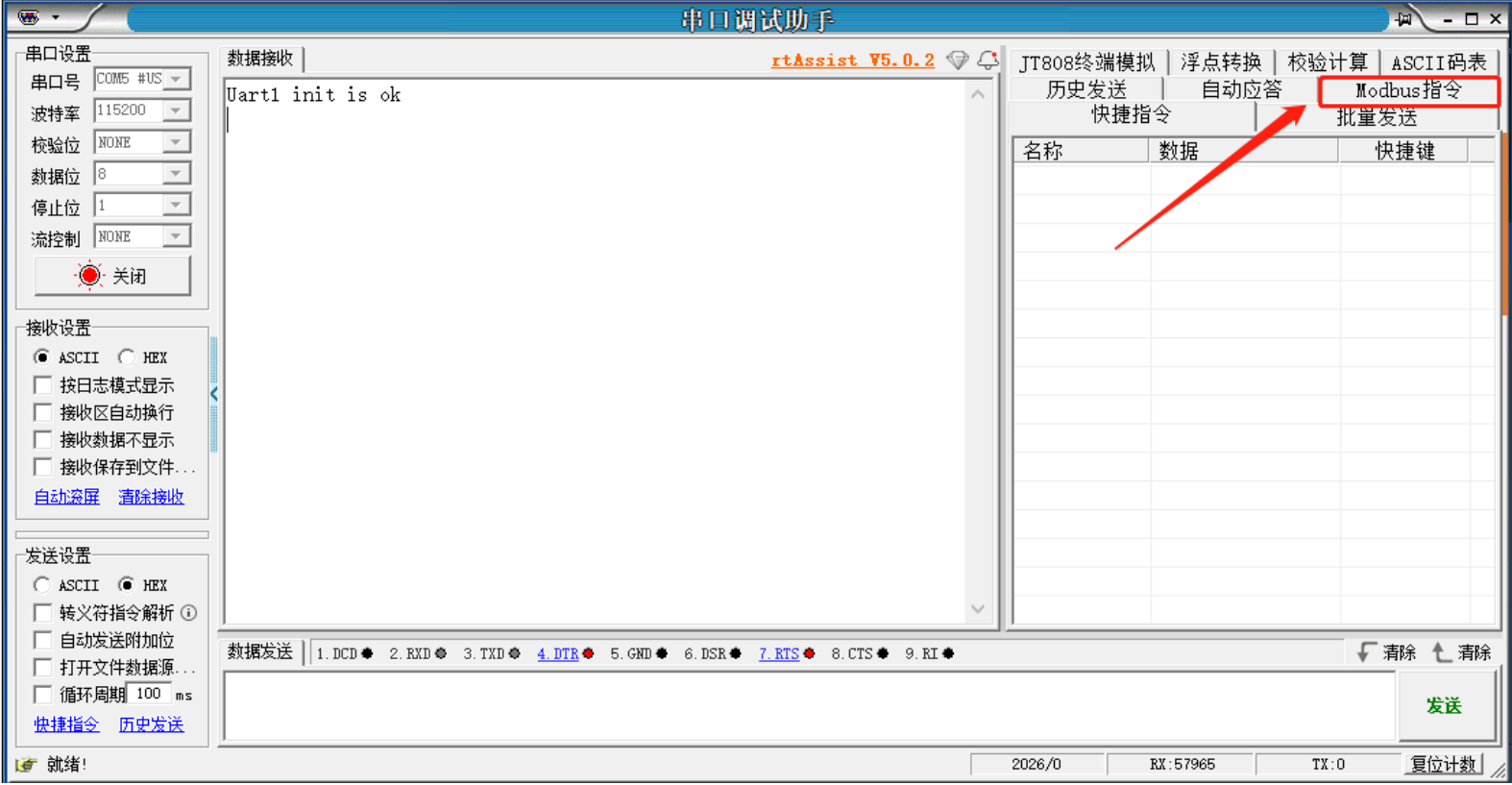


打开串口调试助手的modbus功能部分。

点击快捷指令， 打开辅助界面。



点击Modbus指令，打开Modbus调试辅助功能。

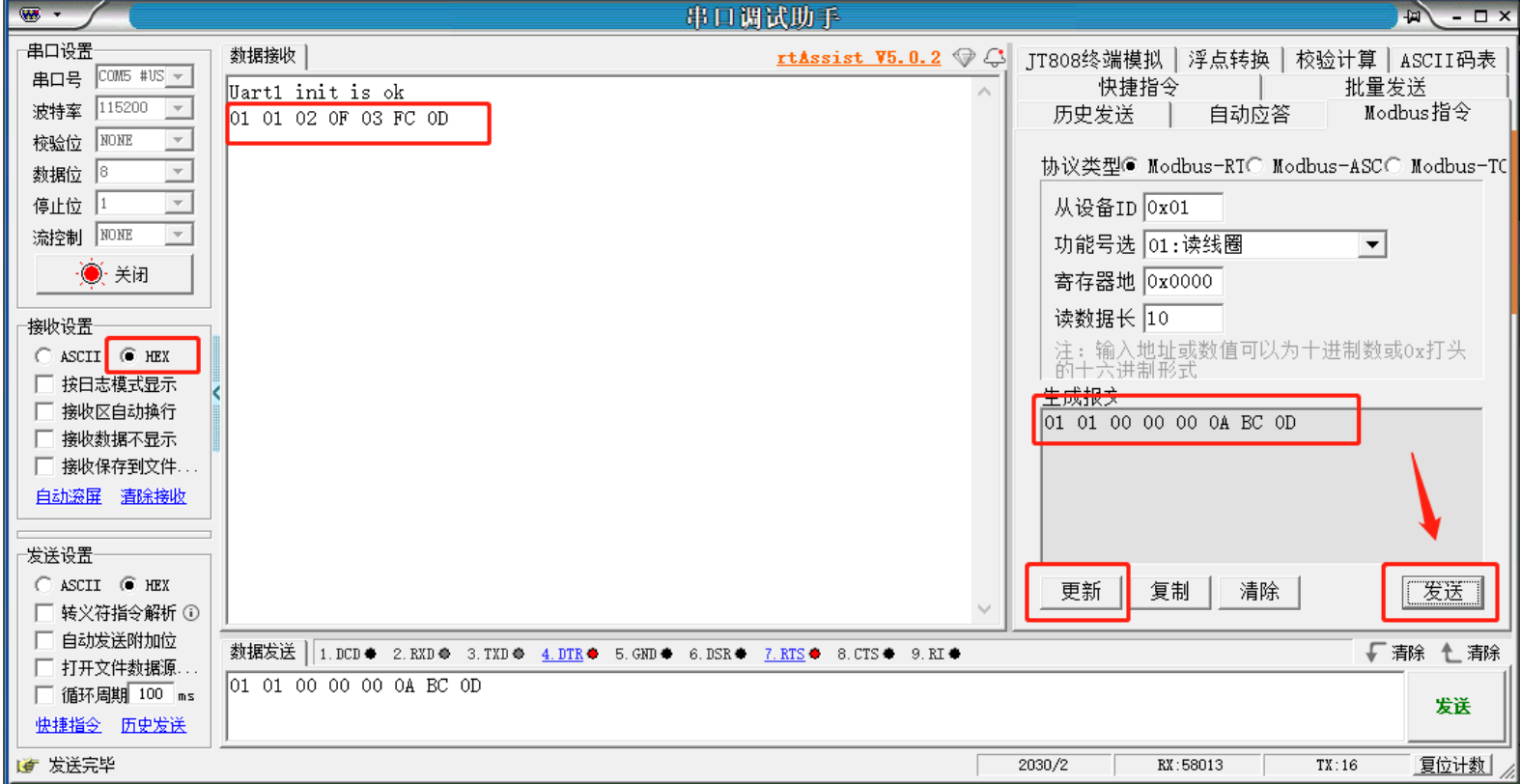


测试Modbus的功能模块。



读10路线圈

选择HEX进制显示接收部分，然后点击更新，去生成modbus指令，最后点击小按钮的发送，进行命令的发送。

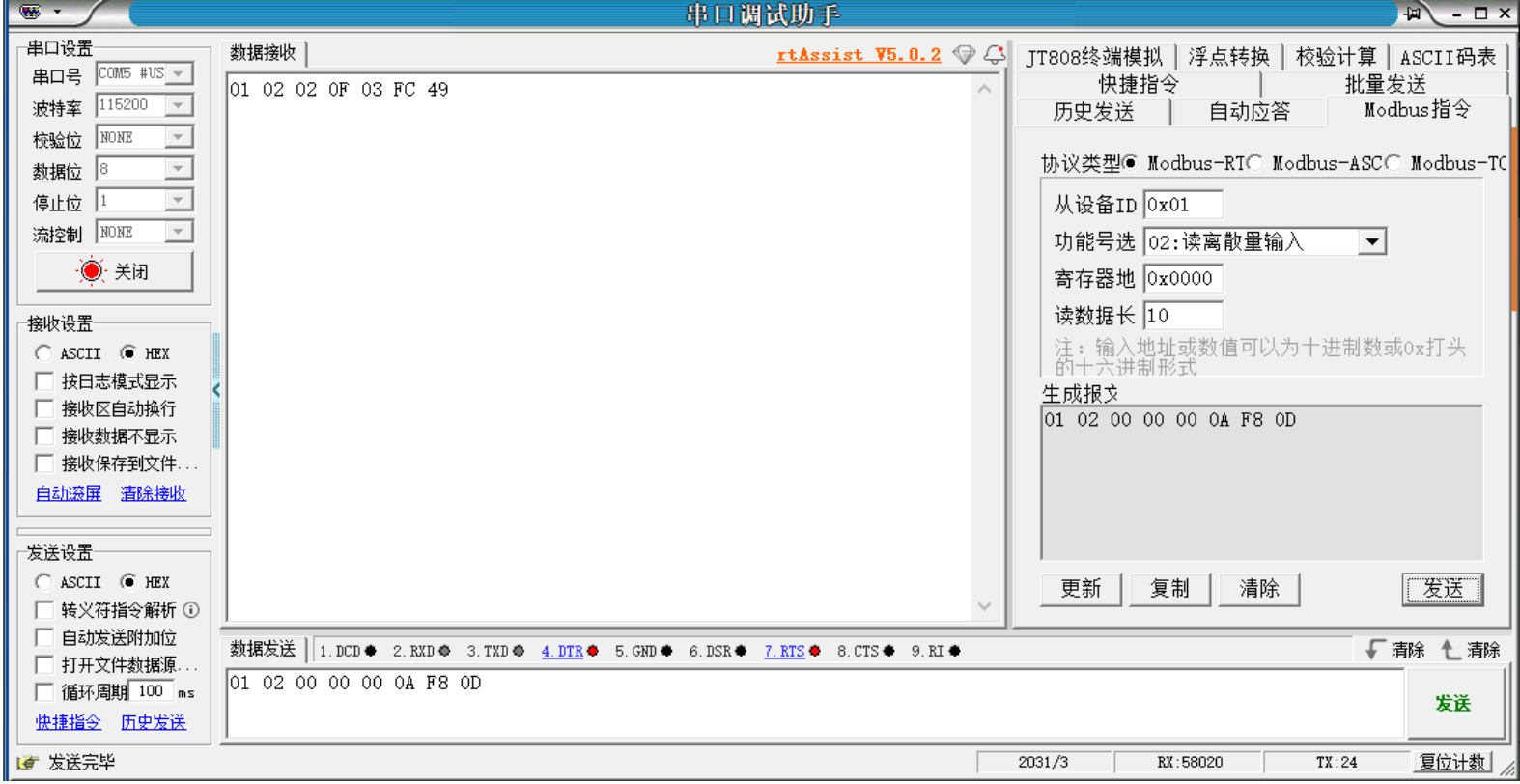


对应的代码为。

```
port.c

// 十路线圈 可读可写的 位控制
#define REG_COILS_SIZE 10
uint8_t REG_COILS_BUF[REG_COILS_SIZE] = {1, 1, 1, 1, 0, 0, 0, 0, 1, 1};
// 低位在前， 高位再后
// 1, 1, 1, 1, 0, 0, 0, 0, 1, 1
// 0 1 2 3 4 5 6 7 | 8 9
// 0f | 03
// 对应的十六进制数为： 0x0f 0x03
```

读10路离散

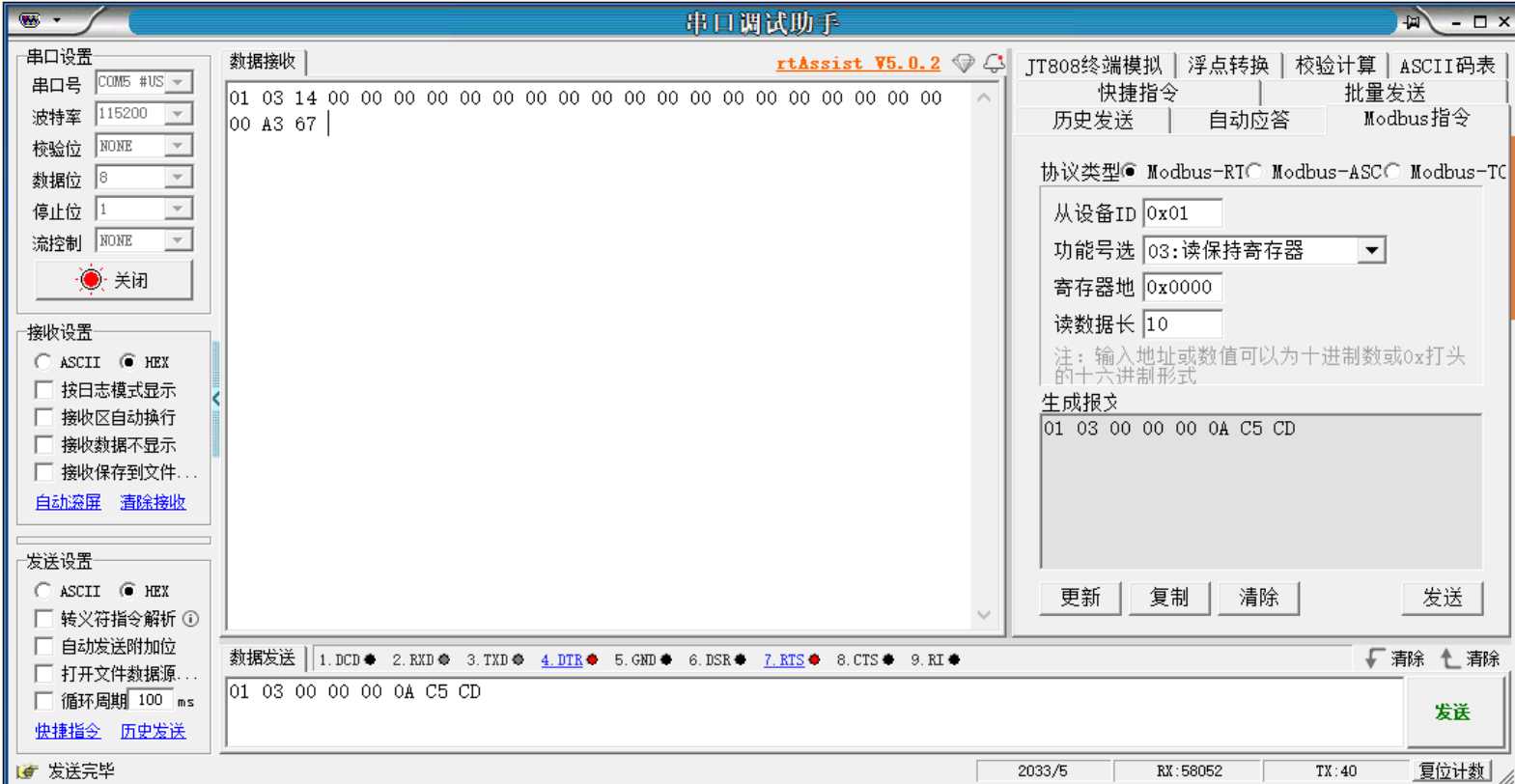


对应的源码

```
port.c

// 十路离散量 只读的位控制
#define REG_DISC_SIZE 10
uint8_t REG_DISC_BUF[REG_DISC_SIZE] = {1,1,1,1,0,0,0,0,1,1};
// 只读的线圈
```

读10路保持寄存器



因为保持寄存器的大小是2个字节， 因此读10个寄存器需要返回20个字节的数据。

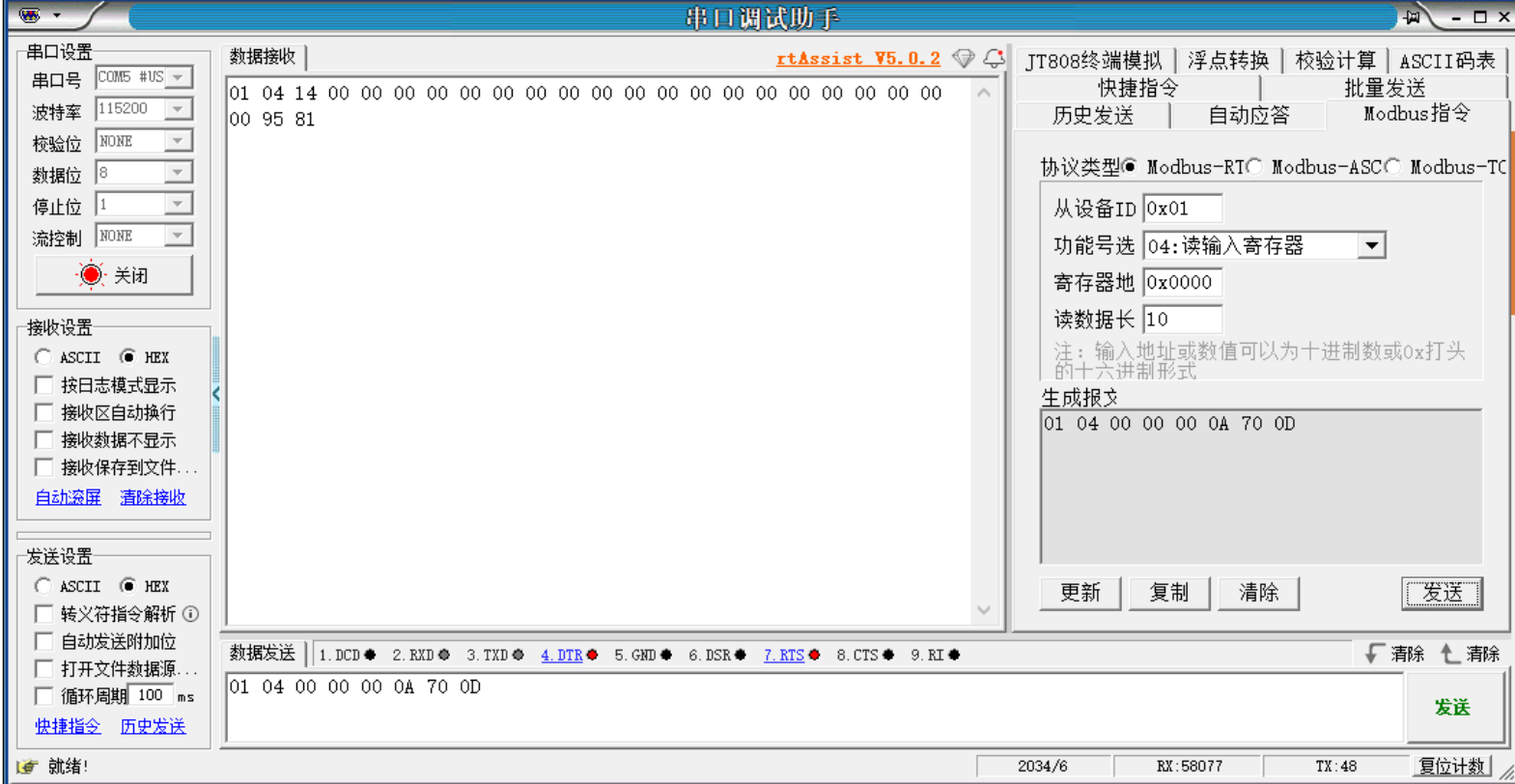
对应的代码

port.c

```
// 十路保持寄存器 可读可写的2字节寄存器
#define REG_HOLD_SIZE 10
uint16_t REG_HOLD_BUF[REG_HOLD_SIZE];
```

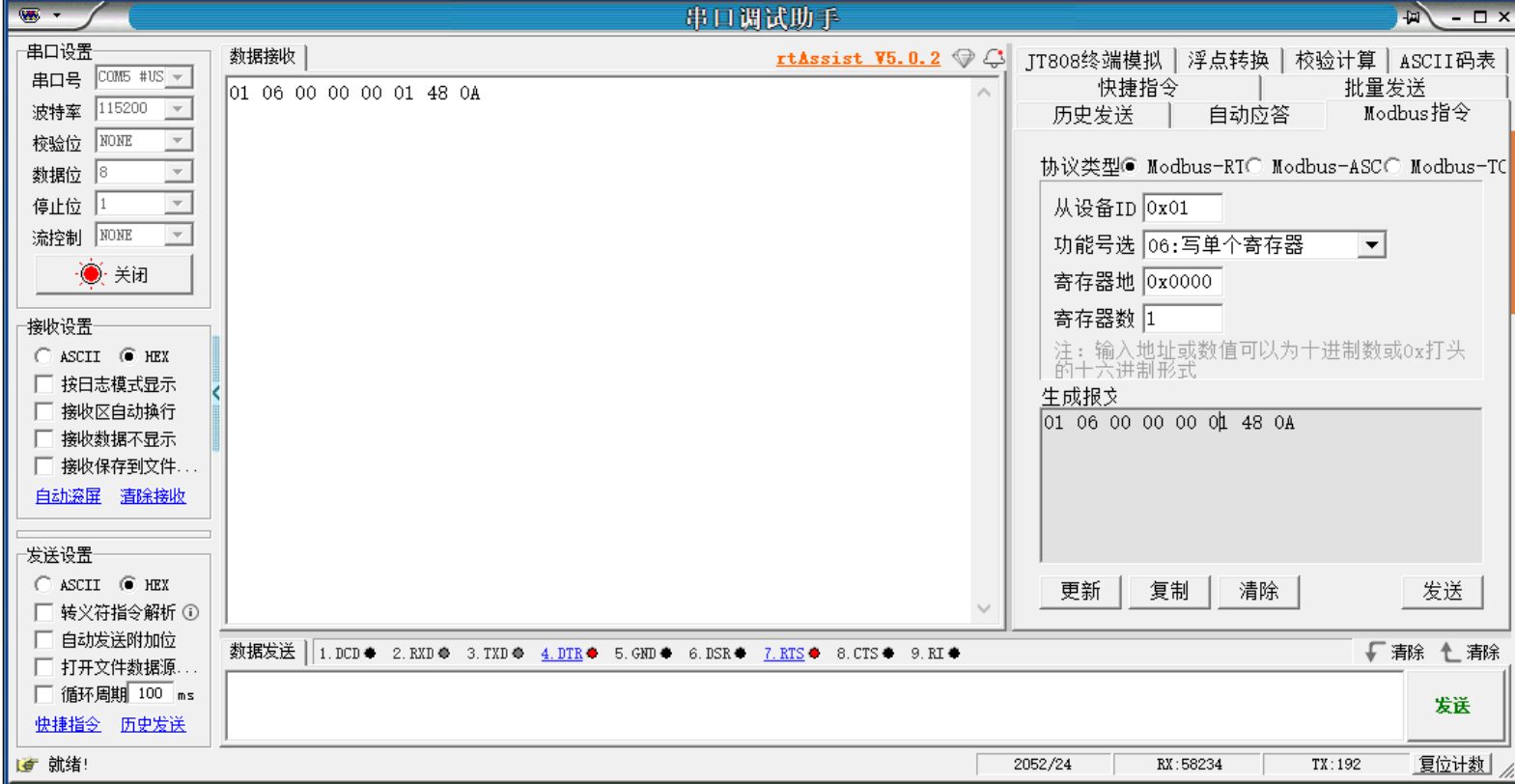
读10路输入寄存器

就是只读的保持寄存器

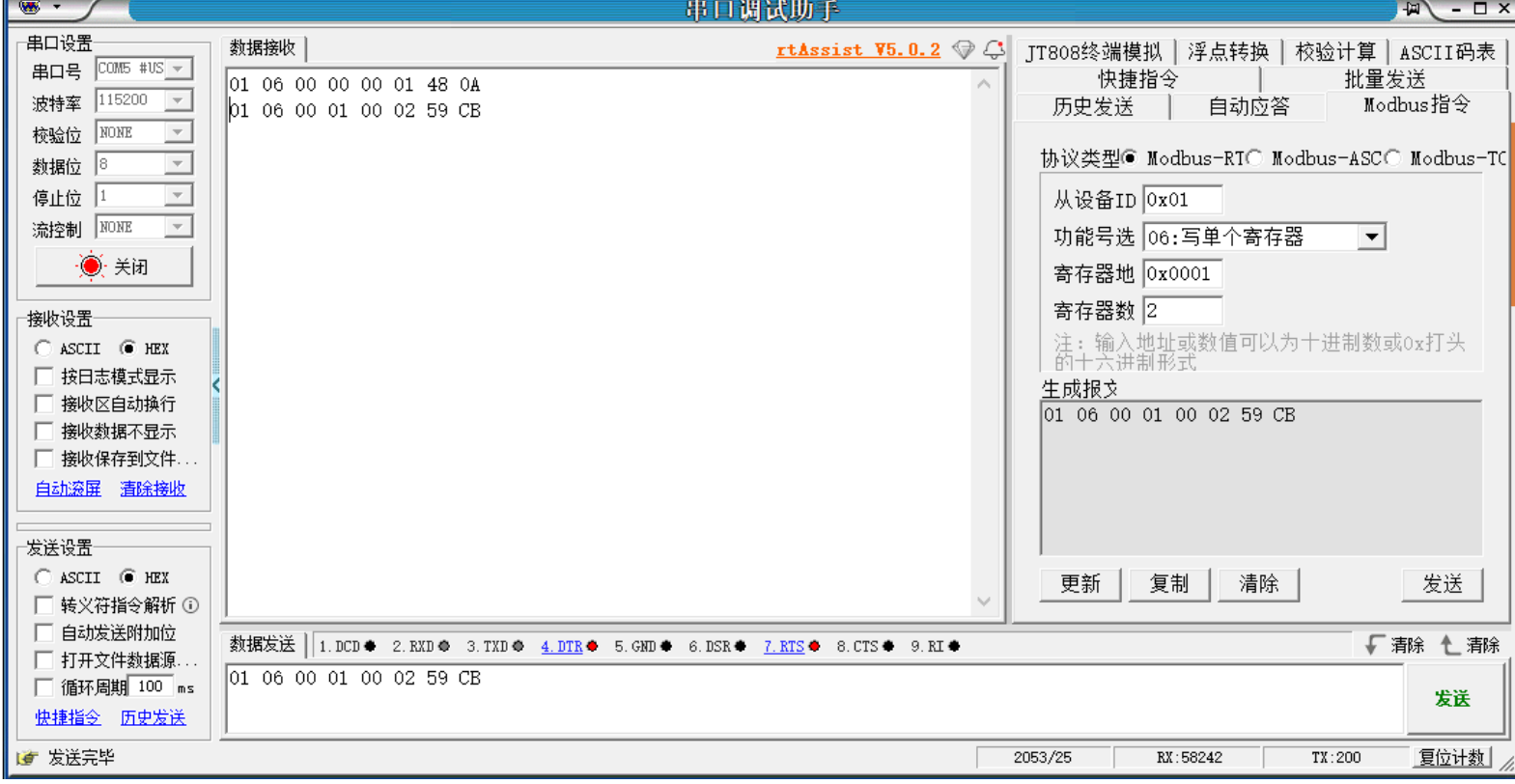


写10路保持寄存器并读取

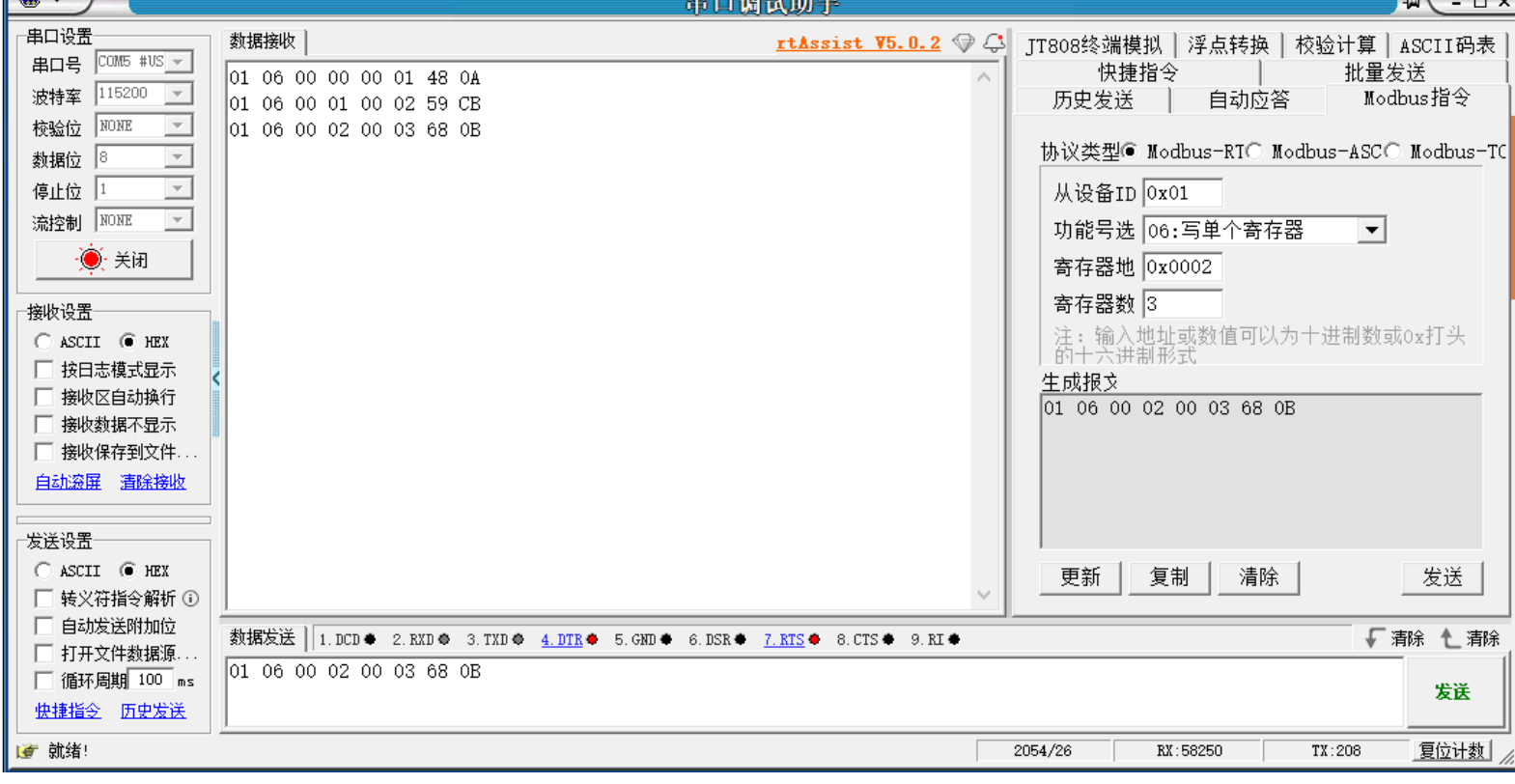
在地址0中写入1



在地址1中写入2



地址2中写入3



读地址0-2的值



1.3 设计自己的Modbus通信协议

1. 从机地址

从机地址：0x01(默认值)

广播地址：0x00

2. 写单个保持寄存器指令

功能码	设备名称	设备寄存器地址	控制的位	位说明
06	LED1	0x0000	0b0000 0001	为1开，为0关
06	LED2	0x0000	0b0000 0010	为1开，为0关
06	BEEP	0x0000	0b0000 0100	为1开，为0关
06	RELAY	0x0000	0b0000 1000	为1开，为0关

例如

```
// 发送指令
01 06 00 00 00 07 c8 08

// 接收指令
01 06 00 00 00 07 c8 08
```

3. 读单个保持寄存器指令

功能码	设备名称	保持寄存器地址	字节数	说明
03	温度 tp	0x0001	2个字节	
03	湿度 hm	0x0002	2个字节	
03	电压 vl	0x0003	2个字节	
03	电流 cu	0x0004	2个字节	
03	功率 pw	0x0005	2个字节	
03	电位器电压 vr	0x0006	2个字节	
03	CPU温度 cpu	0x0007	2个字节	

例如，读温度

```
// 发送数据
01 03 00 01 00 01 D5 CA

// 接收数据
01 03 02 0A F4 BF 63
```

4. 读多个保持寄存器指令

```
// 读7个寄存器的值
01 03 00 01 00 07 55 c8
// 返回值
01 03 0E 0A F8 17 83 00 D6 01 D3 03 ED 01 49 0E 39 05 0C
```

5. 修改从机地址的方法

我们使用保持寄存器的最后地址作为从机的新地址，默认Freemodbus的广播地址为0。

功能码	设备名称	保持寄存器地址	位说明
06	从机新地址	0x0009	从机新地址

```
// 发送指令 ， 设置从机地址位5
01 06 00 09 00 05 99 CB

// 返回值
01 06 00 09 00 05 99 CB
```

1.4 实战项目1 - Modbus程序实现

1. 程序设计思路

- 设置定时器TIM1，每隔500MS执行一次中断。
- 在中断处理函数中，检查保持寄存器0x0000的值，来解析去开关控制LED1、LED2、BEEP、RELAY的状态。
- 在中断处理函数中，读温湿度、电压、电流、功率传感器、读电位器电压、CPU温度值给保持寄存器即可。
- 在遇到小数的时候，可以选择放大100倍，来避免出现小数。

2. 增加修改从机地址的功能

工程中添加modbus.c 和 modbus.h，并增加从机地址修改功能

- 支持修改从机地址
- 从机地址保存在EEPROM内。
- 初始化时，从EEPROM中读取从机地址。
- 支持实时设置从机地址。
- 修改从机的广播地址为255。

3. 修改从机的广播地址

FreeModbus\mbproto.h

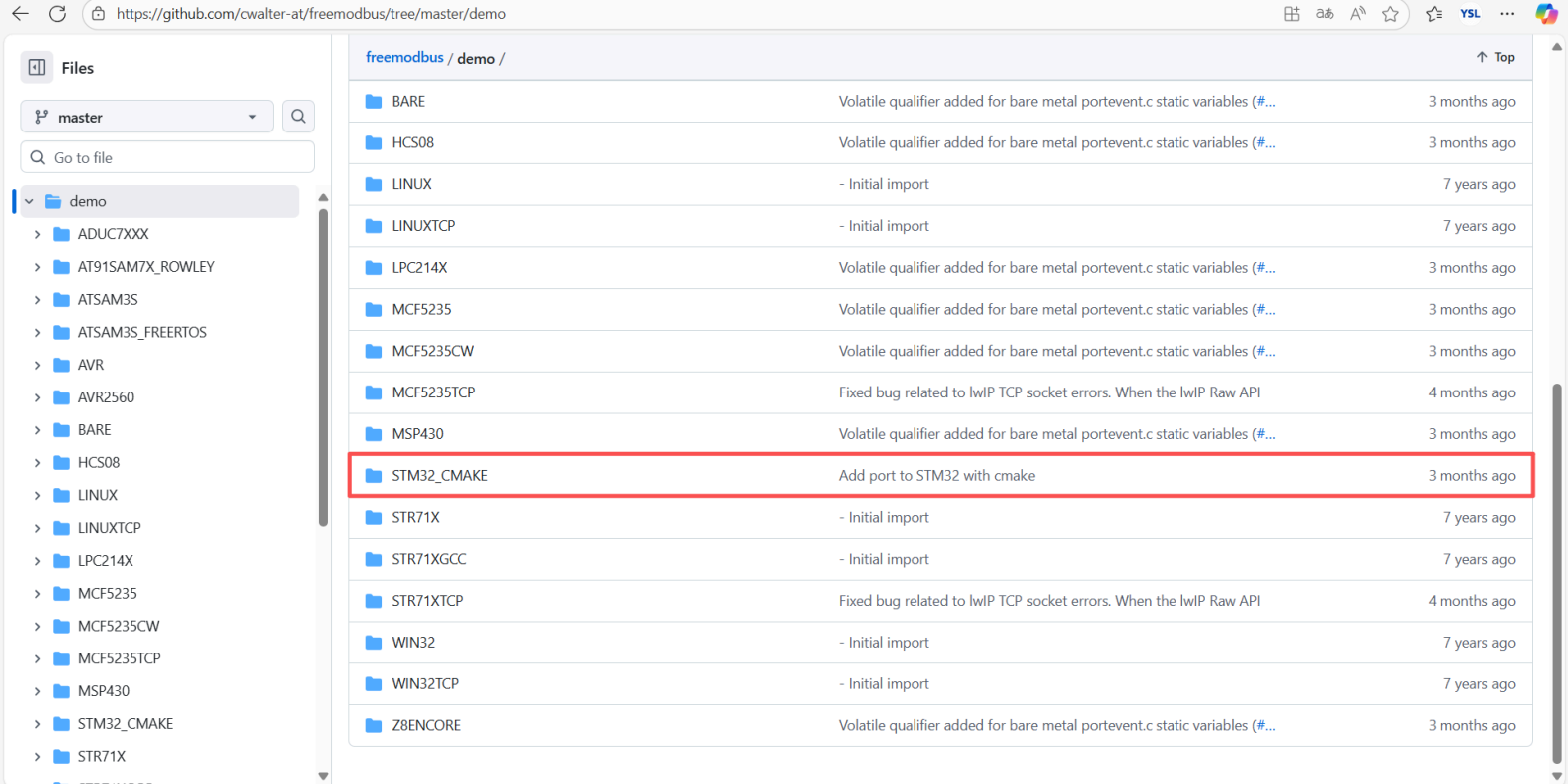
```
#define MB_ADDRESS_BROADCAST ( 0 ) /*! Modbus broadcast address. */
#define MB_ADDRESS_MIN ( 1 ) /*! Smallest possible slave address. */
#define MB_ADDRESS_MAX ( 247 ) /*! Biggest possible slave address. */
```

修改为：

```
#define MB_ADDRESS_BROADCAST ( 255 ) /*! Modbus broadcast address. */
```

4. 官网的最新的更新

官网针对STM32做了一版更新， 有兴趣的小伙伴可以去移植新的版本，这里我们就不移植了。



<https://github.com/cwalter-at/freemodbus/tree/master>

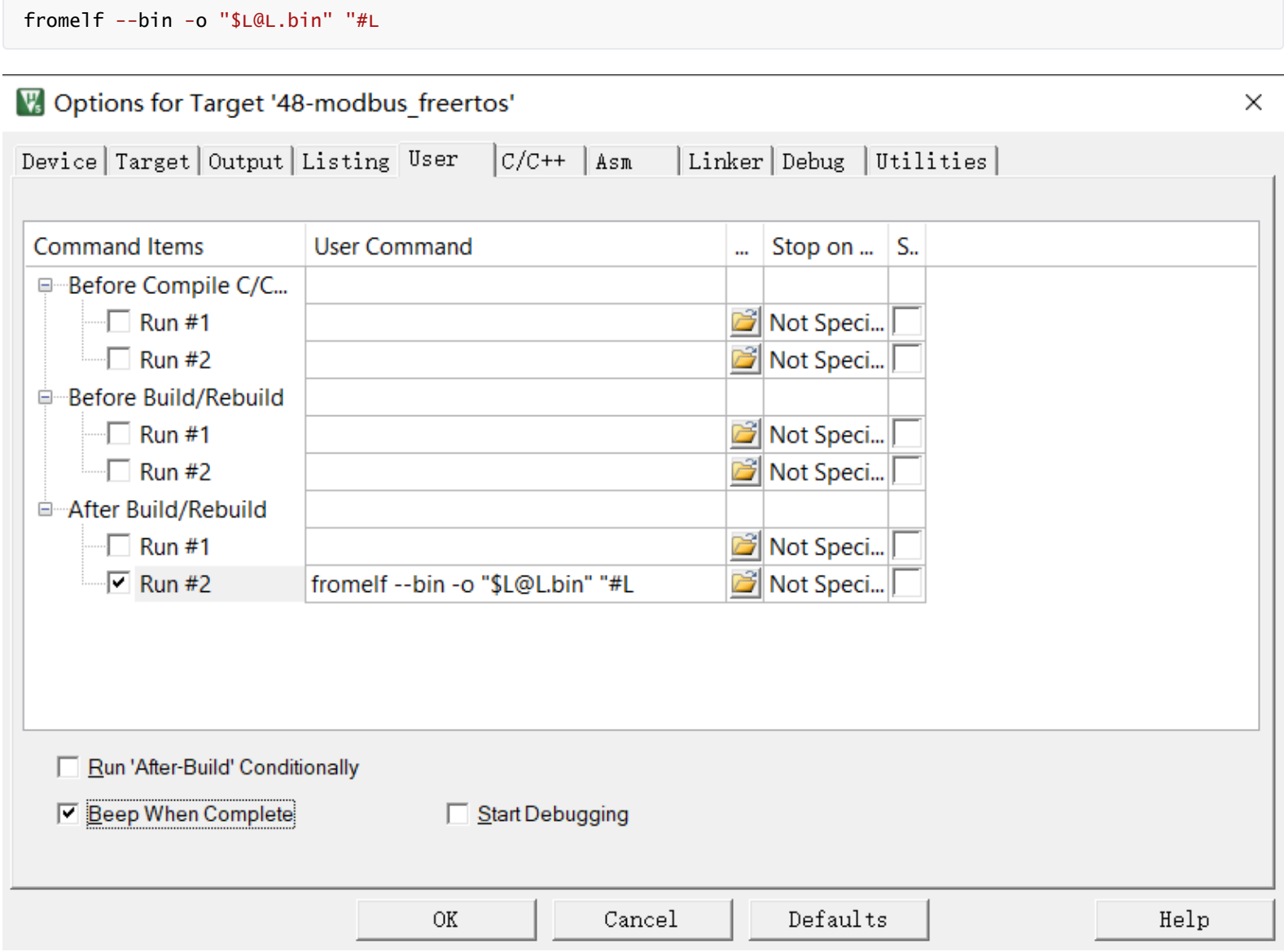
官网的代码我单独存放在 00-Tools 目录内了，需要的自己去下载使用

1.5. 实战项目2 - Modbus+Freertos

1. 程序要求

- 在实战1的基础上完成移植
- 添加FreeRTOS的支持，在FreeRTOS中添加一个任务负责处理Modbus相关的功能。

生成二进制的命令



第02章 MQTT协议实战

2.1 MQTT协议介绍

1. 网络传输协议对比

网络传输协议对比

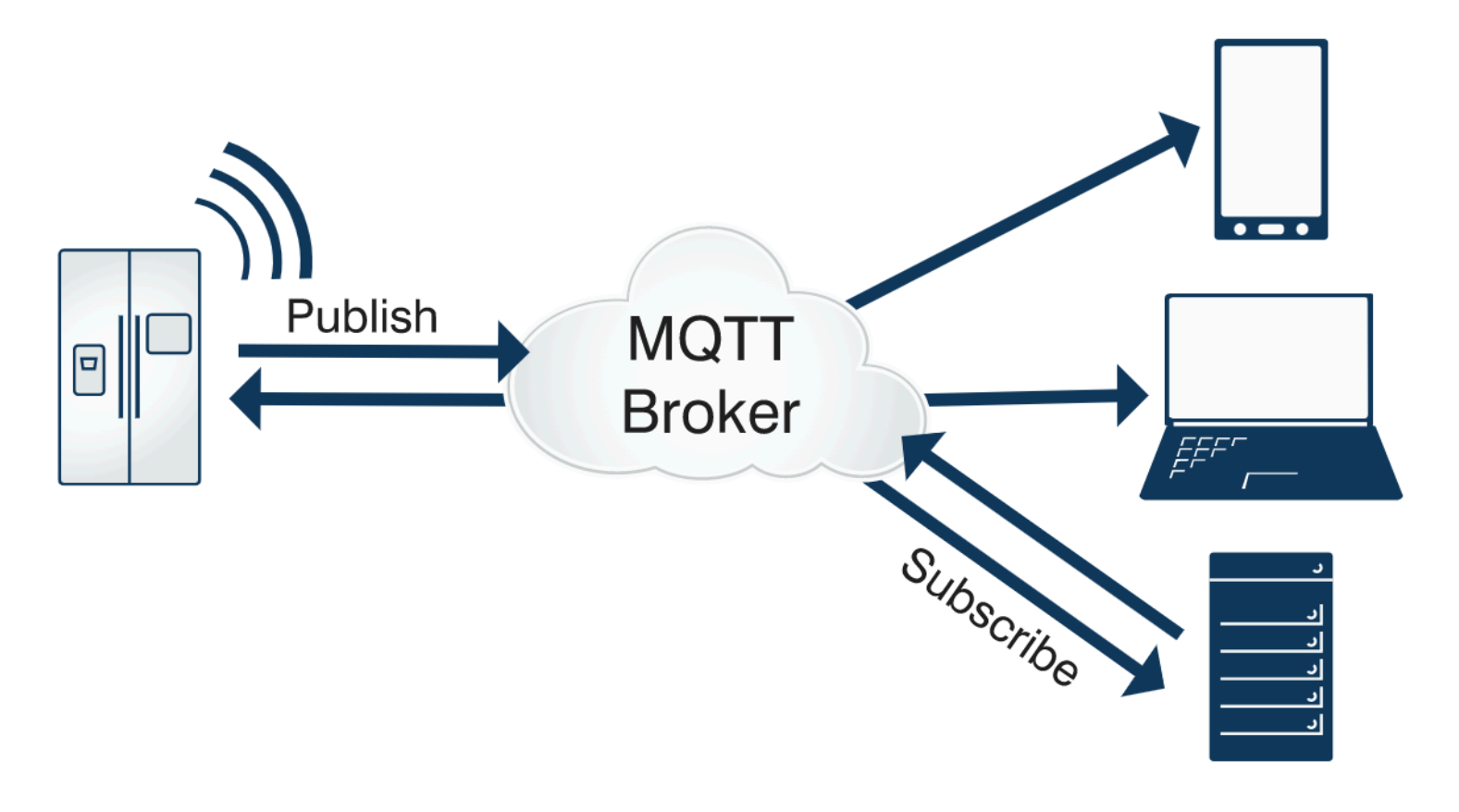
网络分层	协议	
应用层	MQTT	HTTP
传输层	UDP	TCP
网络层	IP	
数据链路与物理层	以太网、WIFI、4G	

传输协议对比	协议
UDP	比较底层，需要自定义协议，开发难度大
TCP	比较底层，需要自定义协议，开发难度大
HTTP	数据吞吐量大，低带宽和不稳定的网络中效果较差
MQTT	数据吞吐量小，低带宽和不稳定的网络中效果较好，主要用于物联网和移动应用

MQTT是为低带宽和不稳定的网络环境设计的轻量级协议

2. MQTT简述

MQTT（Message Queuing Telemetry Transport，消息队列遥测传输协议），是一种基于发布/订阅（publish/subscribe）模式的"轻量级"通讯协议，该协议构建于TCP/IP协议上，由IBM在1999年发布。MQTT最大优点在于，可以以极少的代码和有限的带宽，为连接远程设备提供实时可靠的消息服务。作为一种低开销、低带宽占用的即时通讯协议，使其在物联网、小型设备、移动应用等方面有较广泛的应用。



主要的概念有5个：

- Broker 代理：MQTT 服务器
- Publish 发布者：客户端，公众号的运营者。
- Subscribe 订阅者：客户端，可订阅多个 topic
- Topic 主题：消息的类型，订阅主题之后就可以收到该 topic 的消息内容即 payload
- Payload 消息内容：具体的内容

简述MQTT协议中的订阅、主题、负载的理解

负载：
消息订阅者所具体接收的内容。
形象的比喻：微信公众号文章内容。

订阅：
订阅是MQTT中实现消息通信的重要机制之一，它允许客户端通过订阅感兴趣的主题来接收消息，从而实现了发布-订阅模式的通信方式。
形象的比喻：微信的公众号

主题：
MQTT 协议中的主题（Topics）就像是不同的频道或者话题。它们是消息的标签，用来区分不同类型或者不同主题的消息。
形象的比喻：微信公众号文章标题。

2. 设计规范

MQTT物联网协议的设计原则

- 动态创建主题**
- 零运维成本**
- 高效率**
通信网络把传输量降到最低以提高传输效率
- 精简**
没有可有可无的功能
- 发布/订阅（Pub/Sub）模式**
方便消息在传感器之间传递

由于物联网的环境是非常特别的，所以MQTT遵循以下设计原则：

- （1）精简，不添加可有可无的功能；
- （2）发布/订阅（Pub/Sub）模式，方便消息在传感器之间传递；
- （3）允许用户动态创建主题，零运维成本；
- （4）把传输量降到最低以提高传输效率；

- (5) 把低带宽、高延迟、不稳定的网络等因素考虑在内；
- (6) 支持连续的会话控制；
- (7) 理解客户端计算能力可能很低；
- (8) 提供服务质量管理；
- (9) 假设数据不可知，不强求传输数据的类型与格式，保持灵活性。

3. 主要特性

MQTT协议工作在低带宽、不可靠的网络的远程传感器和控制设备通讯而设计的协议，它具有以下主要的几项特性：

- (1) 使用发布/订阅消息模式，提供一对多的消息发布，类似公众号。
- (2) 对负载内容屏蔽的消息传输。
- (3) 使用TCP/IP提供网络连接。

主流的MQTT是基于TCP连接进行数据推送的，但是同样有基于UDP的版本，叫做MQTT-SN。这两种版本由于基于不同的连接方式，优缺点自然也就各有不同了。



- (4) 有三种消息发布服务质量：

"至多一次"，消息发布完全依赖底层TCP/IP网络。会发生消息丢失或重复。这一级别可用于如下情况，环境传感器数据，丢失一次读记录无所谓，因为不久后还会有第二次发送。这一种方式主要普通APP的推送，倘若你的智能设备在消息推送时未联网，推送过去没收到，再次联网也就收不到了。

"至少一次"，确保消息到达，但消息重复可能会发生。

"只有一次"，确保消息到达一次。在一些要求比较严格的计费系统中，可以使用此级别。在计费系统中，消息重复或丢失会导致不正确的结果。这种最高质量的消息发布服务还可以用于即时通讯类的APP的推送，确保用户收到且只会收到一次。
- (5) 小型传输，开销很小（固定长度的头部是2字节），协议交换最小化，以降低网络流量。

这就是为什么在介绍里说它非常适合"在物联网领域，传感器与服务器的通信，信息的收集"，要知道嵌入式设备的运算能力和带宽都相对薄弱，使用这种协议来传递消息再适合不过了。

4. MQTT协议原理

MQTT协议实现方式

实现MQTT协议需要客户端和服务端通讯完成，在通讯过程中，MQTT协议中有三种身份：发布者（Publish）、代理（Broker）（服务器）、订阅者（Subscribe）。其中，消息的发布者和订阅者都是客户端，消息代理是服务器，消息发布者可以同时是订阅者。

MQTT传输的消息分为：主题（Topic）和负载（payload）两部分：

- (1) Topic，可以理解为消息的类型，订阅者订阅（Subscribe）后，就会收到该主题的消息内容（payload）；
- (2) payload，可以理解为消息的内容，是指订阅者具体要使用的内容。

网络传输与应用消息

MQTT会构建底层网络传输：它将建立客户端到服务器的连接，提供两者之间的一个有序的、无损的、基于字节流的双向传输。

当应用数据通过MQTT网络发送时，MQTT会把与之相关的服务质量（QoS）和主题名（Topic）相关连。

MQTT客户端

一个使用MQTT协议的应用程序或者设备，它总是建立到服务器的网络连接。客户端可以：

- (1) 发布其他客户端可能会订阅的信息；
- (2) 订阅其它客户端发布的消息；
- (3) 退订或删除应用程序的消息；
- (4) 断开与服务器连接。

MQTT服务器

MQTT服务器以称为"消息代理"（Broker），可以是一个应用程序或一台设备。它是位于消息发布者和订阅者之间，它可以：

- (1) 接受来自客户的网络连接；
- (2) 接受客户发布的应用信息；
- (3) 处理来自客户端的订阅和退订请求；
- (4) 向订阅的客户转发应用程序消息。

MQTT协议中的订阅、主题、会话

订阅（Subscription）

订阅包含主题筛选器（Topic Filter）和最大服务质量（QoS）。订阅会与一个会话（Session）关联。一个会话可以包含多个订阅。每一个会话中的每个订阅都有一个不同的主题筛选器。

会话（Session）

每个客户端与服务器建立连接后就是一个会话，客户端和服务端之间有状态交互。会话存在于一个网络之间，也可能在客户端和服务端之间跨越多个连续的网络连接。

主题名（Topic Name）

连接到一个应用程序消息的标签，该标签与服务器的订阅相匹配。服务器会将消息发送给订阅所匹配标签的每个客户端。

主题筛选器（Topic Filter）

一个对主题名通配符筛选器，在订阅表达式中使用，表示订阅所匹配到的多个主题。

负载（Payload）

消息订阅者所具体接收的内容。

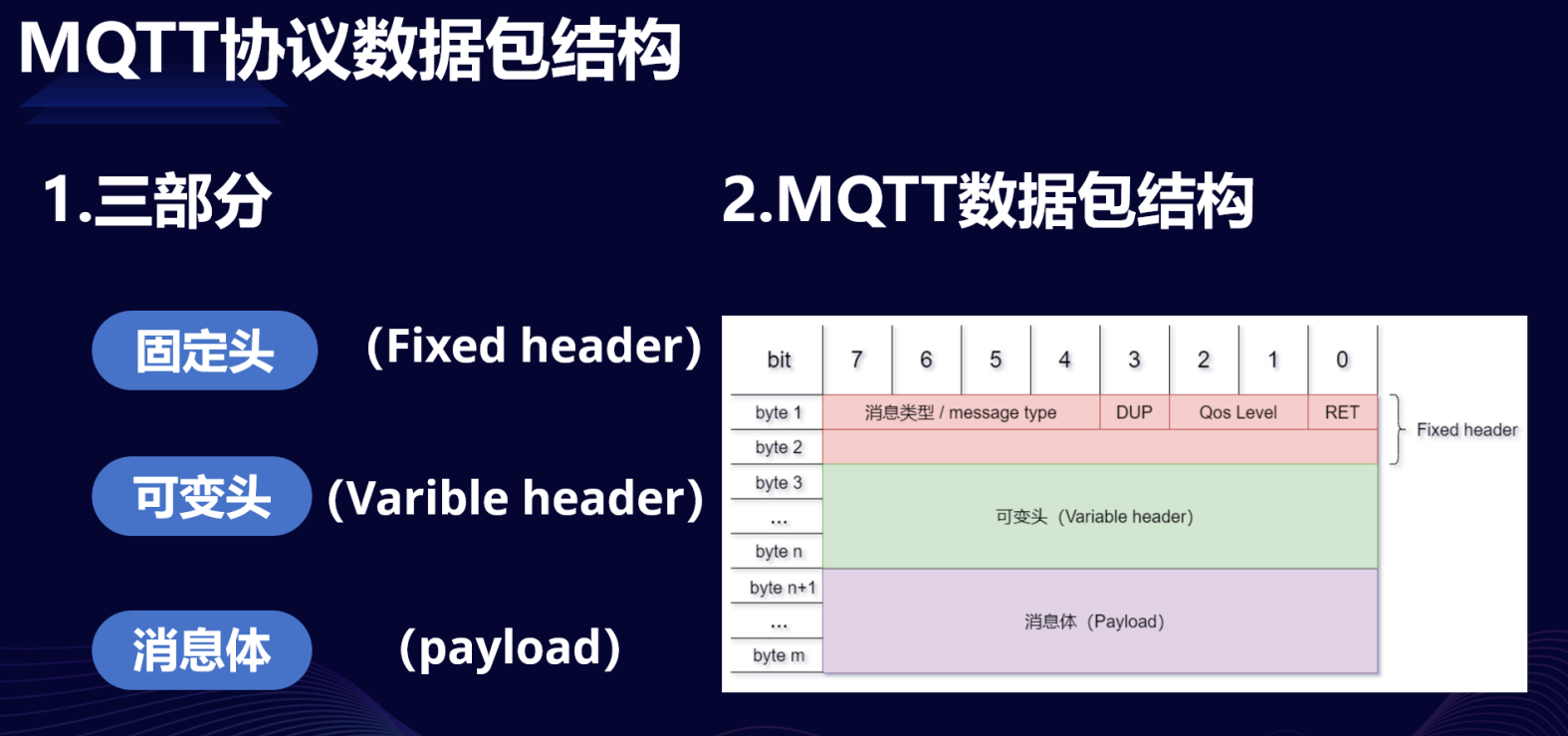
MQTT协议中的方法

MQTT协议中定义了一些方法（也被称为动作），来于表示对确定资源所进行操作。这个资源可以代表预先存在的数据或动态生成数据，这取决于服务器的实现。通常来说，资源指服务器上的文件或输出。主要方法有：

- (1) Connect。等待与服务器建立连接。
- (2) Disconnect。等待MQTT客户端完成所做的工作，并与服务器断开TCP/IP会话。
- (3) Subscribe。等待完成订阅。
- (4) UnSubscribe。等待服务器取消客户端的一个或多个topics订阅。
- (5) Publish。MQTT客户端发送消息请求，发送完成后返回应用程序线程。

5. MQTT协议数据包结构

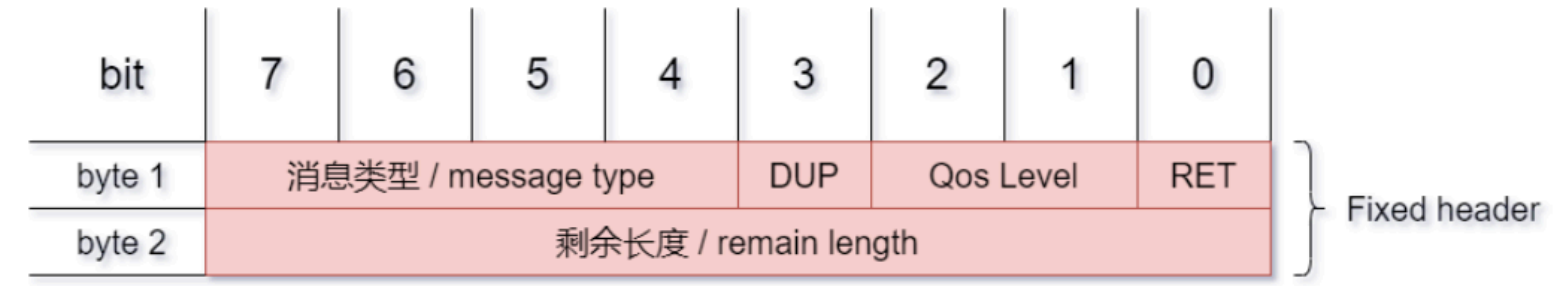
在MQTT协议中，一个MQTT数据包由：固定头（Fixed header）、可变头（Variable header）、消息体（payload）三部分构成。MQTT数据包结构如下：



- (1) 固定头（Fixed header）。存在于所有MQTT数据包中，表示数据包类型及数据包的分组类标识。
- (2) 可变头（Variable header）。存在于部分MQTT数据包中，数据包类型决定了可变头是否存在及其具体内容。
- (3) 消息体（Payload）。存在于部分MQTT数据包中，表示客户端收到的具体内容。

MQTT固定头

固定头存在于所有MQTT数据包中，其结构如下：



MQTT数据包类型

位置：Byte 1中bits 7-4。

相于一个4位的无符号值，类型、取值及描述如下：

名称	值	流方向	描述
Reserved	0	不可用	保留位
CONNECT	1	客户端到服务器	客户端请求连接到服务器
CONNACK	2	服务器到客户端	连接确认
PUBLISH	3	双向	发布消息
PUBACK	4	双向	发布确认
PUBREC	5	双向	发布收到（保证第1部分到达）
PUBREL	6	双向	发布释放（保证第2部分到达）
PUBCOMP	7	双向	发布完成（保证第3部分到达）
SUBSCRIBE	8	客户端到服务器	客户端请求订阅
SUBACK	9	服务器到客户端	订阅确认
UNSUBSCRIBE	10	客户端到服务器	请求取消订阅
UNSUBACK	11	服务器到客户端	取消订阅确认
PINGREQ	12	客户端到服务器	PING请求
PINGRESP	13	服务器到客户端	PING应答
DISCONNECT	14	客户端到服务器	中断连接
Reserved	15	不可用	保留位

标识位

位置：Byte 1中bits 3-0。

在不使用标识位的消息类型中，标识位被作为保留位。如果收到无效的标志时，接收端必须关闭网络连接：

- （1）DUP：发布消息的副本。用来在保证消息的可靠传输，如果设置为1，则在下面的变长中增加MessageId，并且需要回复确认，以保证消息传输完成，但不能用于检测消息重复发送。

（2）QoS：发布消息的服务质量，即：保证消息传递的次数

00：最多一次，即：<=1

01：至少一次，即：>=1

10：一次，即：=1

11：预留

- （3）RETAIN：发布保留标识，表示服务器要保留这次推送的信息，如果有新的订阅者出现，就把这消息推送给它，如果设有那么推送至当前订阅者后释放。

剩余长度（Remaining Length）

地址：Byte 2。

固定头的第二字节用来保存变长头部和消息体的总大小的，但不是直接保存的。这一字节是可以扩展，其保存机制，前7位用于保存长度，后一部用做标识。当最后一位为1时，表示长度不足，需要使用二个字节继续保存。例如：计算出后面的大小为0

MQTT可变头

MQTT数据包中包含一个可变头，它驻位于固定的头和负载之间。可变头的内容因数据包类型而不同，较常的应用是作为包的标识：

很多类型数据包中都包括一个2字节的数据包标识字段，这些类型的包有：PUBLISH (QoS > 0)、PUBACK、PUBREC、PUBREL、PUBCOMP、SUBSCRIBE、SUBACK、UNSUBSCRIBE、UNSUBACK。

Payload消息体

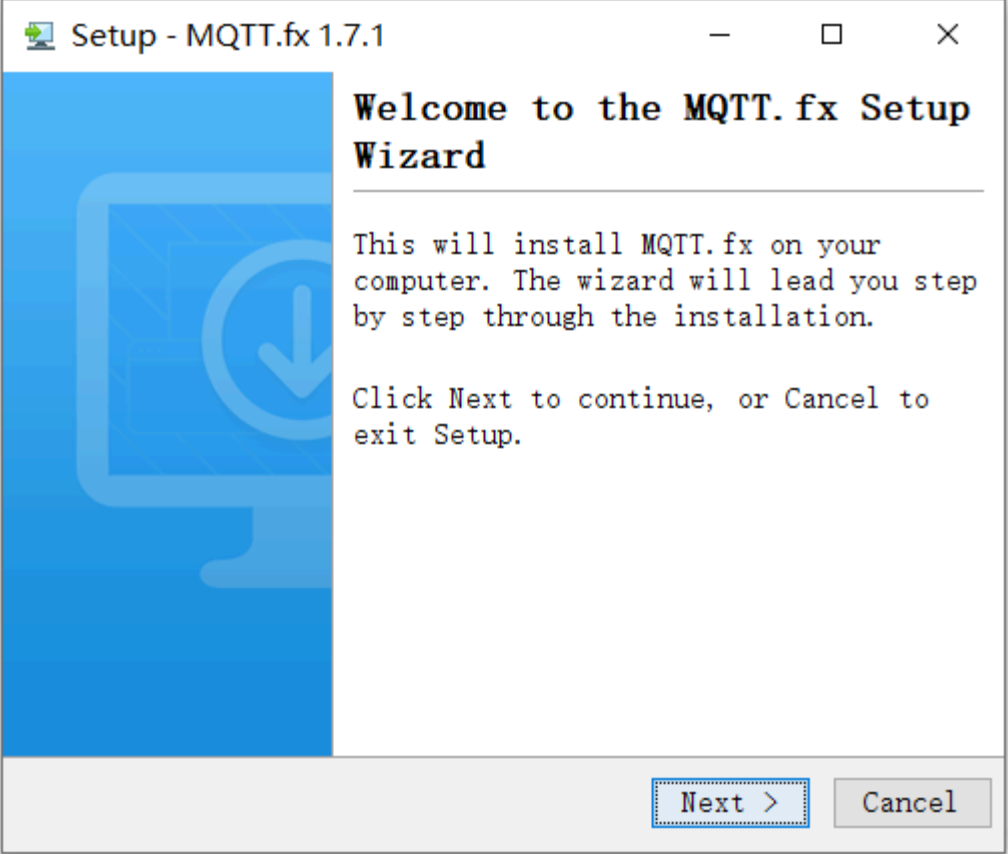
Payload消息体位MQTT数据包的第三部分，包含CONNECT、SUBSCRIBE、SUBACK、UNSUBSCRIBE四种类型的消息：

- （1）CONNECT，消息体内容主要是：客户端的ClientID、订阅的Topic、Message以及用户名和密码。
- （2）SUBSCRIBE，消息体内容是一系列的要订阅的主题以及QoS。
- （3）SUBACK，消息体内容是服务器对于SUBSCRIBE所申请的主题及QoS进行确认和回复。
- （4）UNSUBSCRIBE，消息体内容是要订阅的主题。

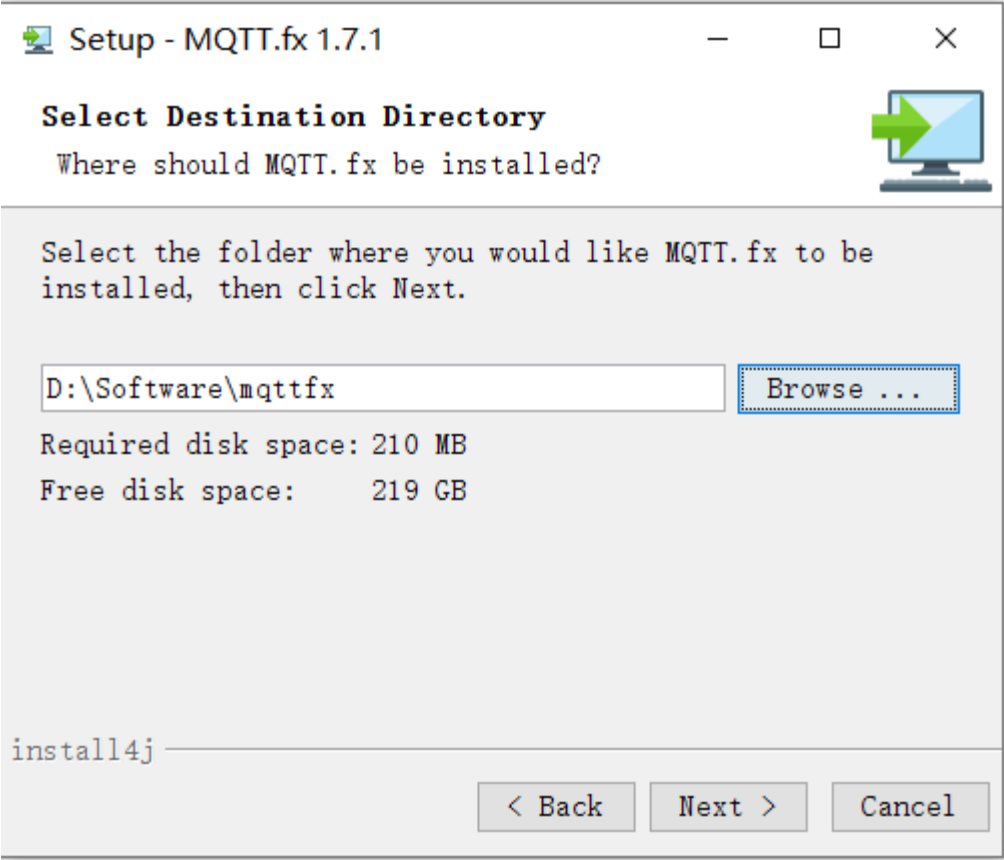
2.2 MQTT通信验证

1. mqttfx工具安装

双击 STM32F103开发板资料V9.0\01-工具软件\mqttfx-1.7.1.exe 安装包， 安装MQTT调试工具。

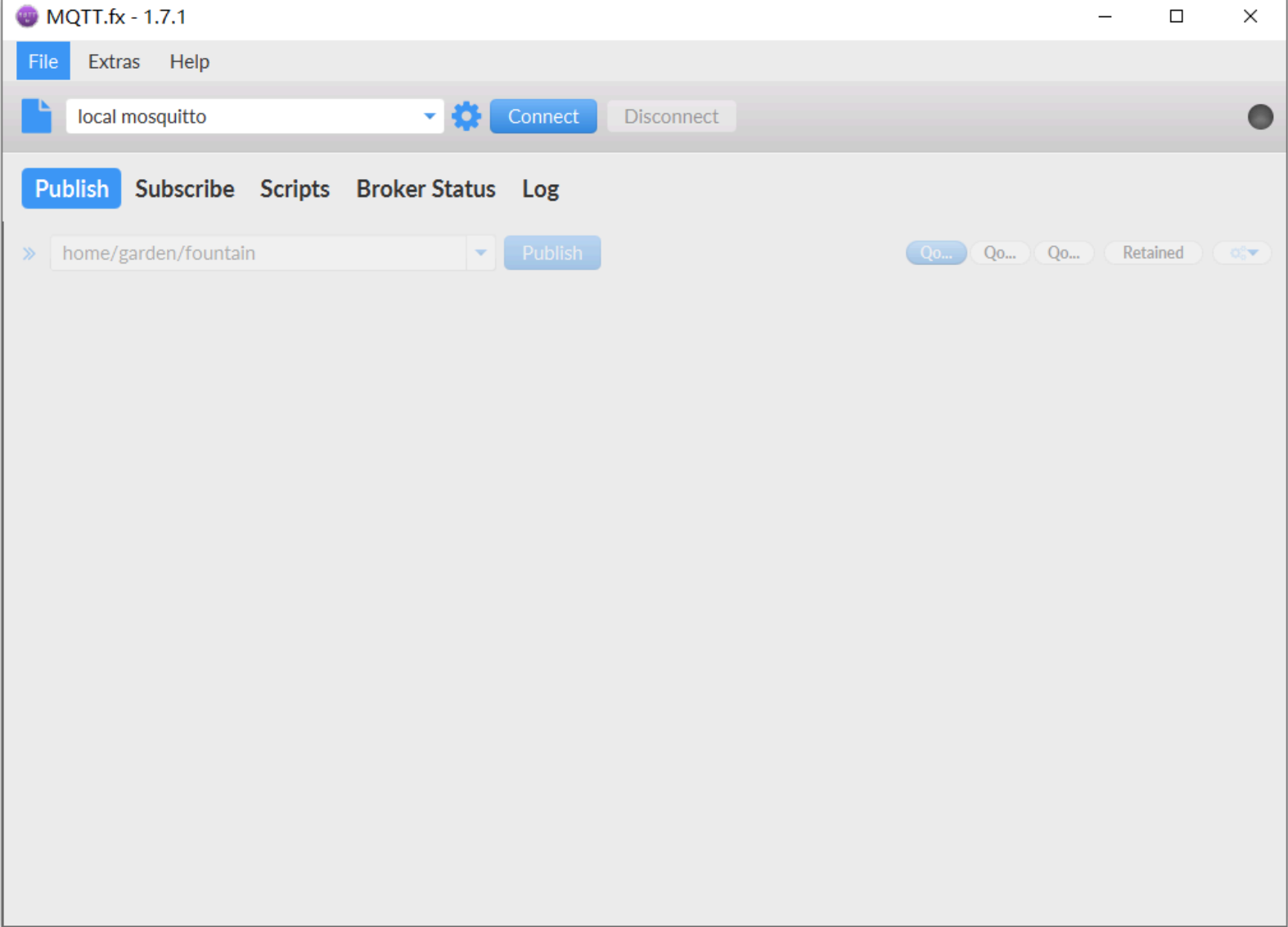


安装到系统即可。



点击下一步， 直到安装完成。

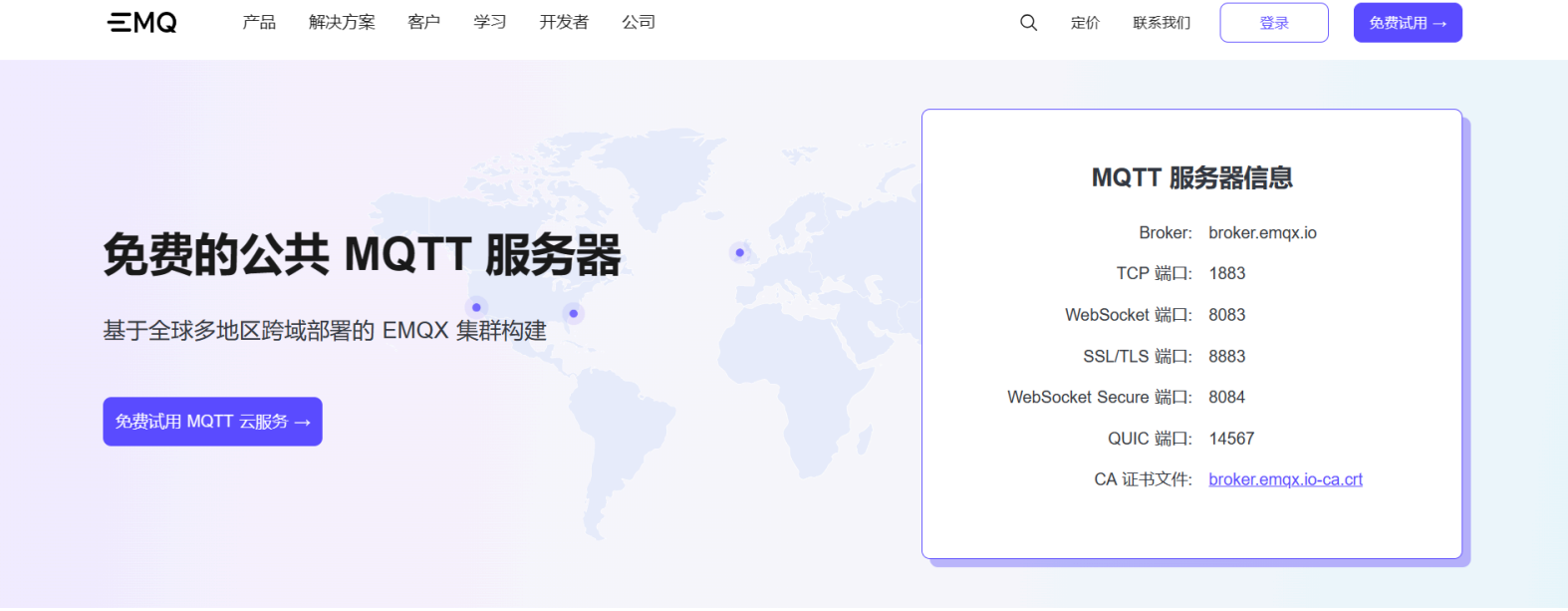
安装完成后，运行程序。



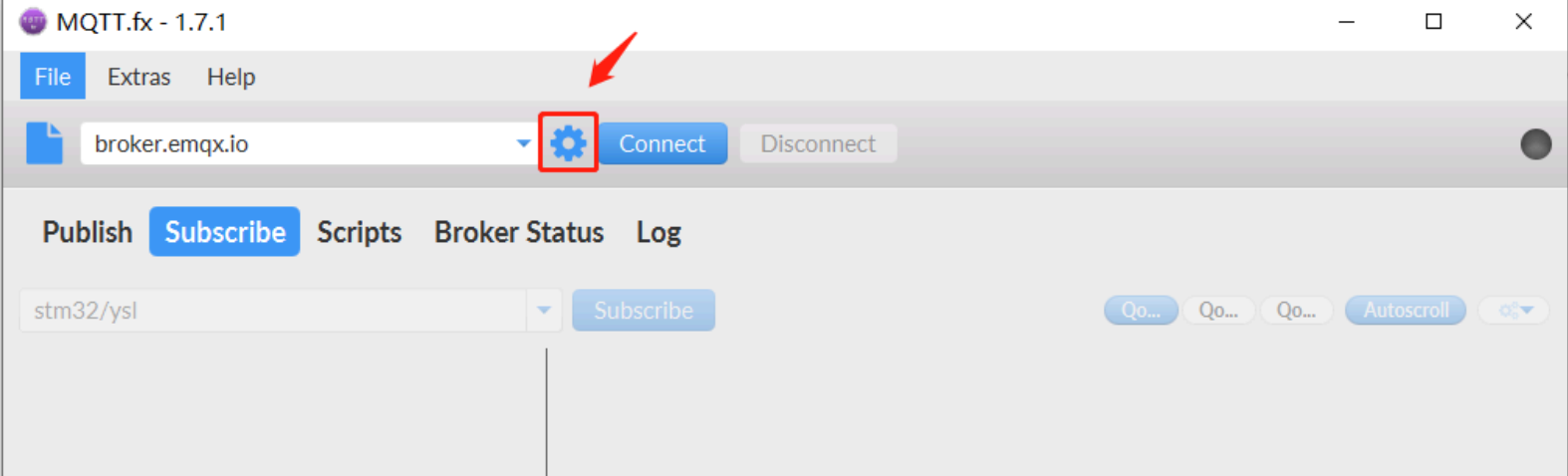
2. emqx服务器通信

连接服务器

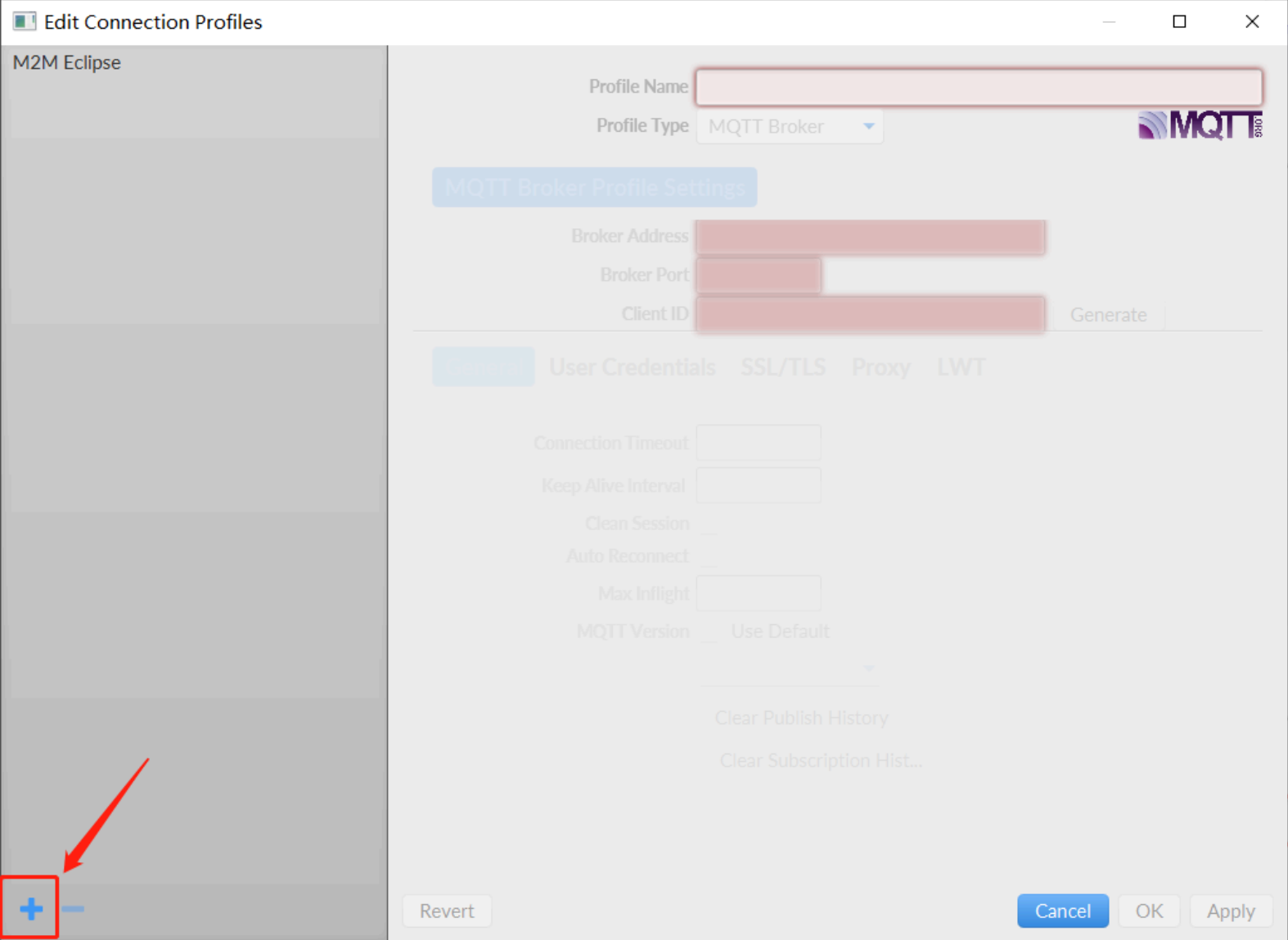
EMQX服务器：broker.emqx.io



新建服务器连接



新建配置



配置界面中，修改自己的名称。

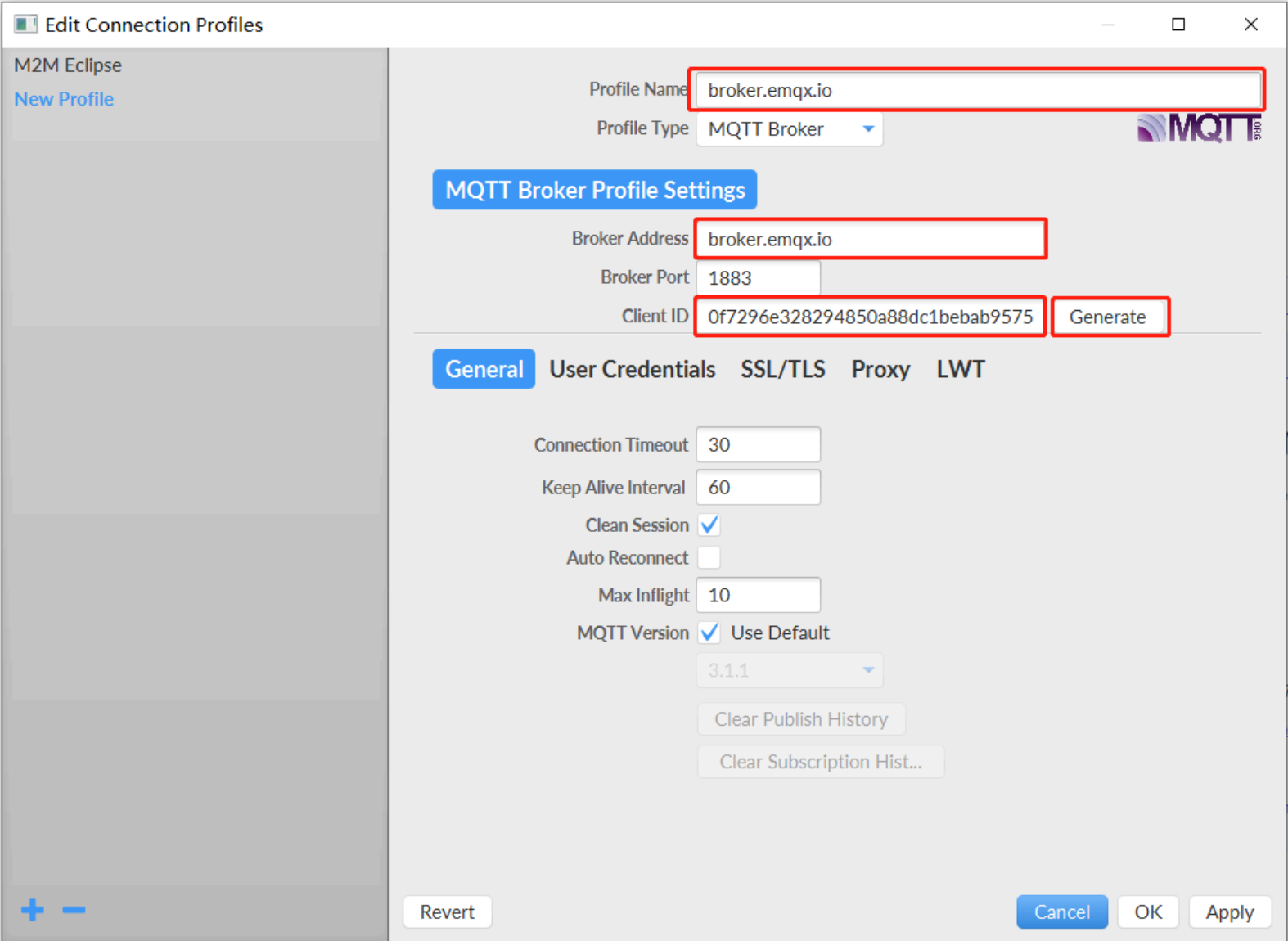
输入服务器IP地址

域名名为： broker.emqx.io

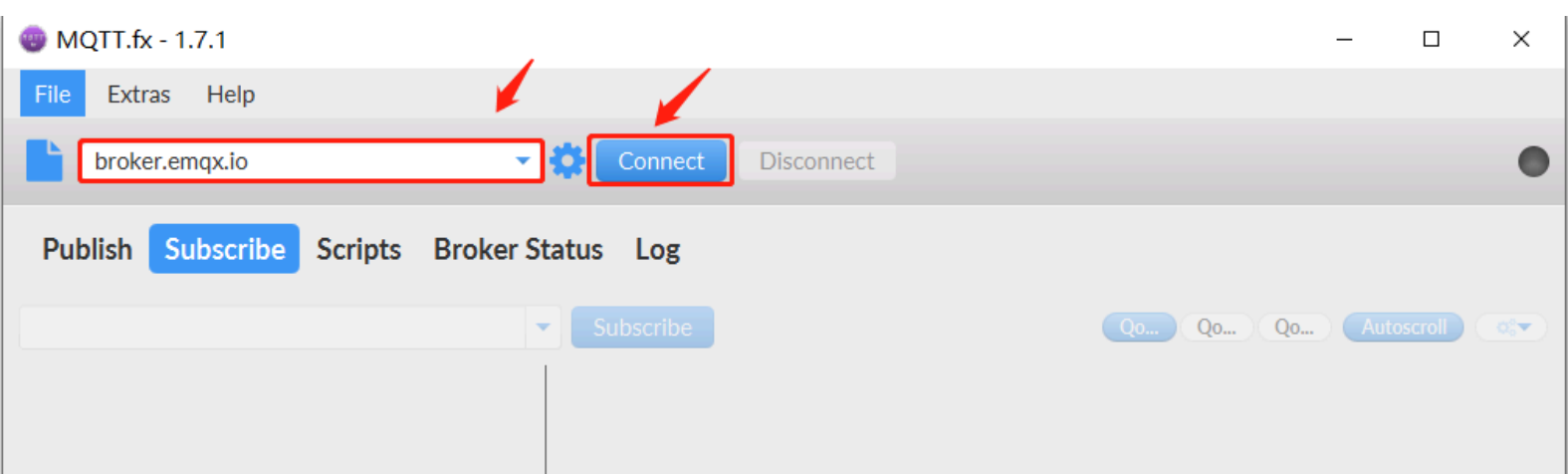
端口：1883

注意：Client ID 这里要注意，要点击Generate 去随机生成一个ID号，防止重名，重名后服务器会自动断开连接。

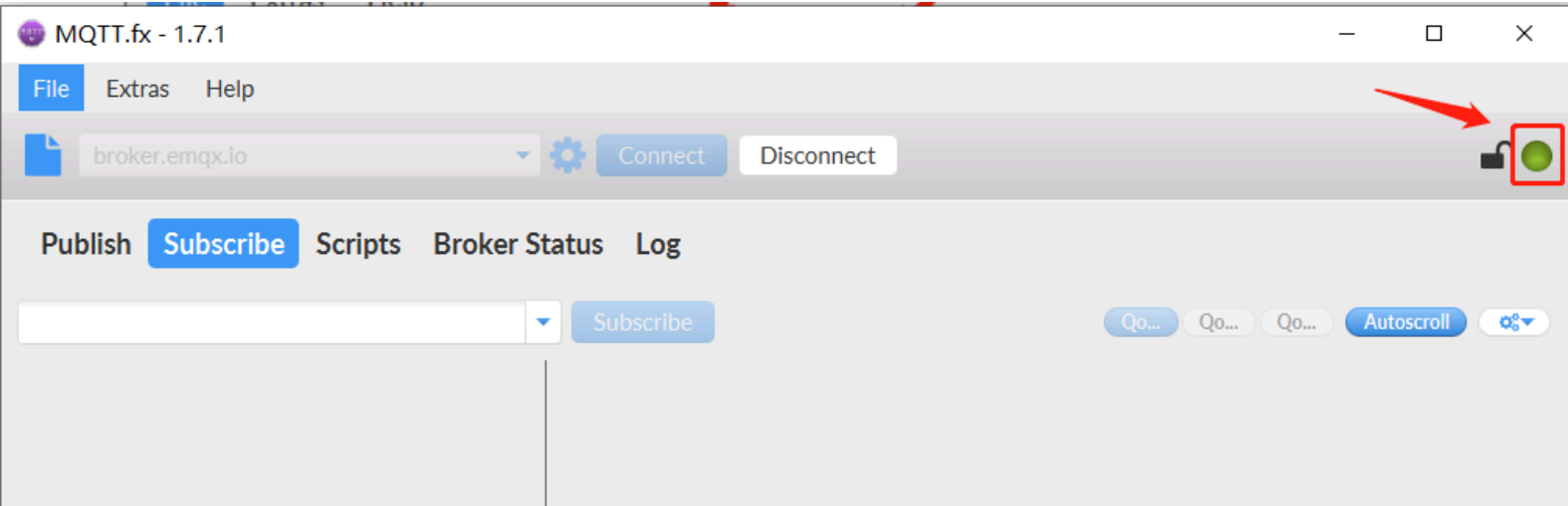
点击 Apply，让配置生效，之后点击关闭窗口。



下拉列表中选择服务器， 点击Connect，开始连接服务器

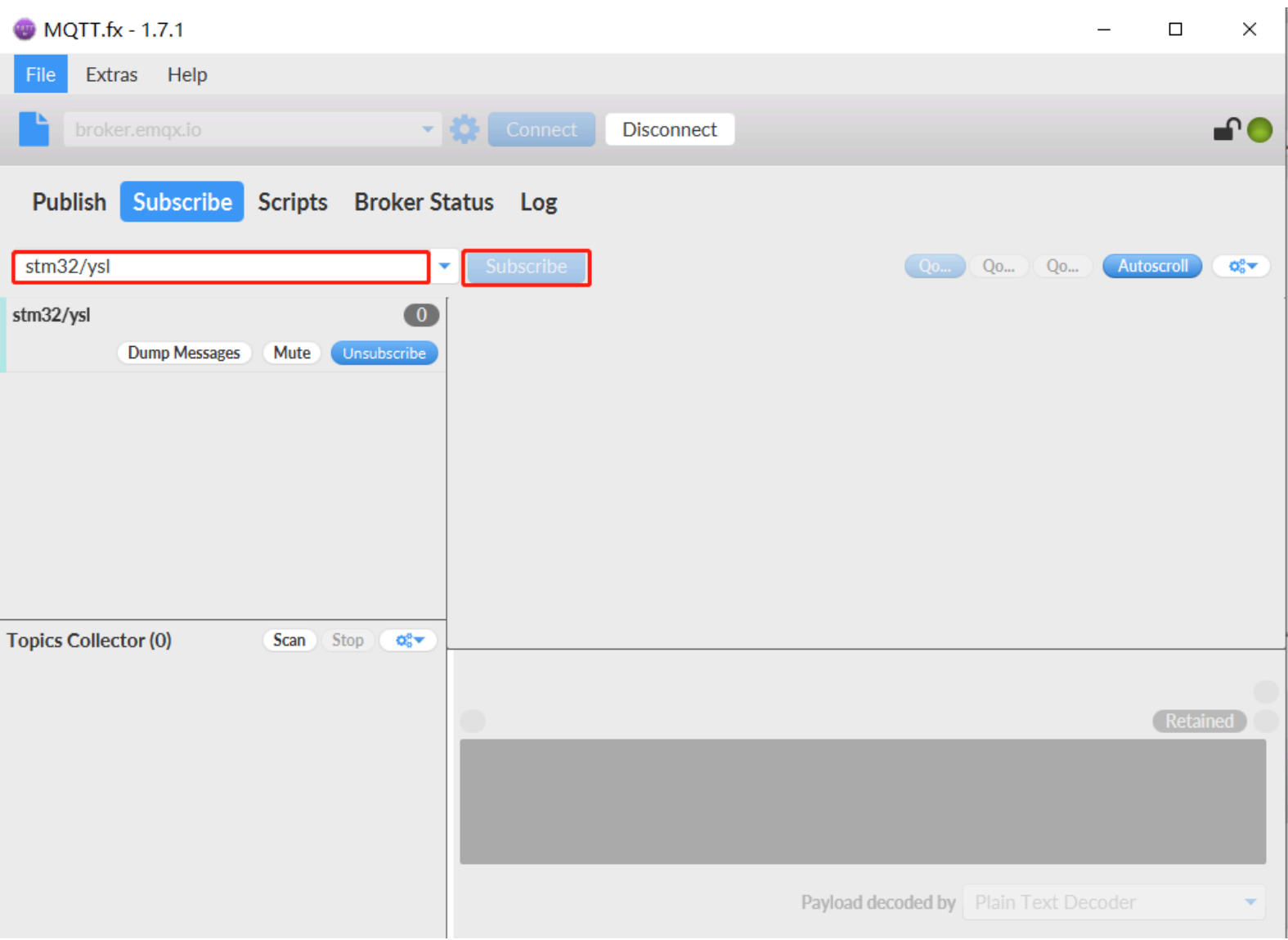


连接成功后，指示灯亮起。

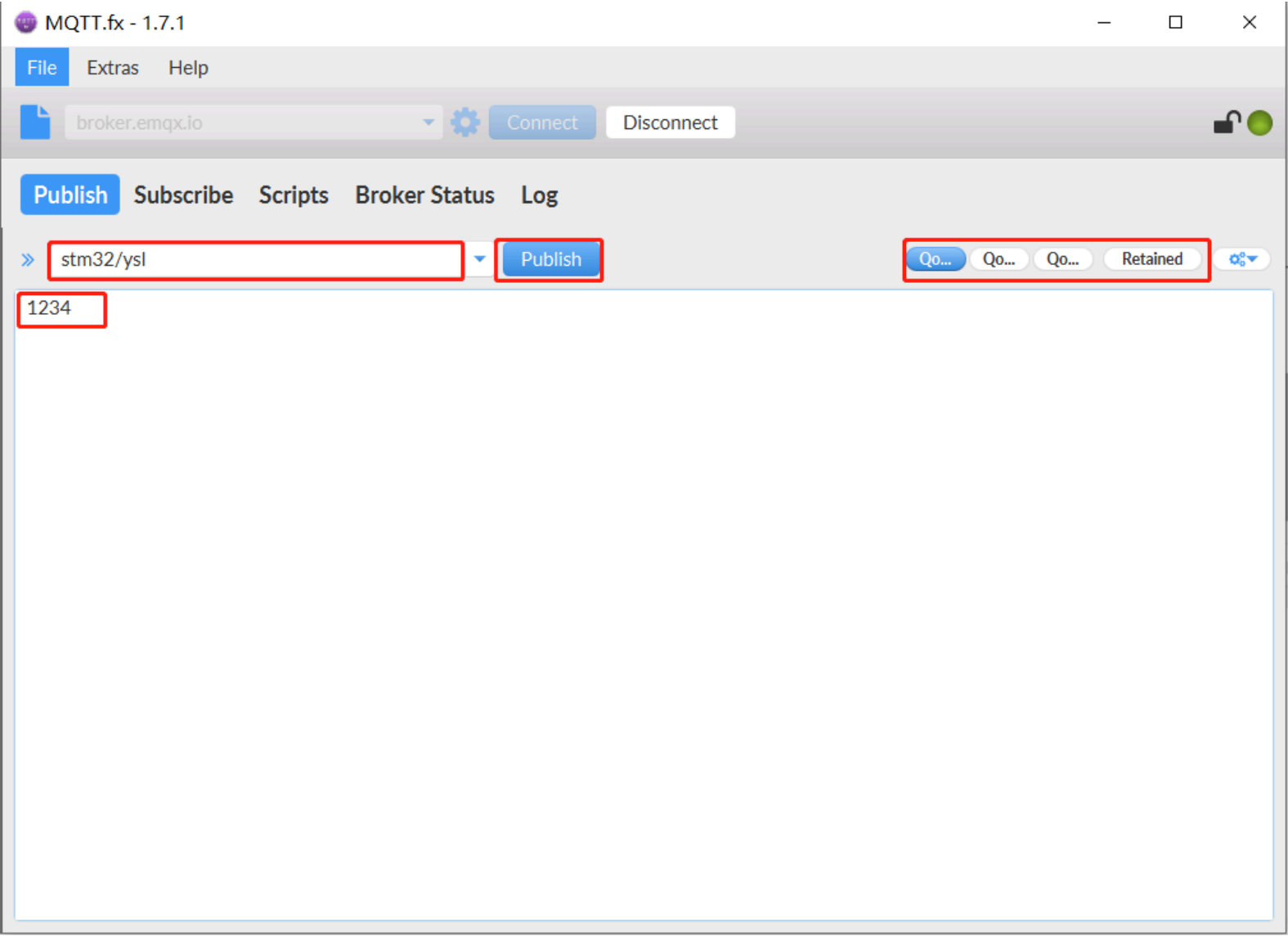


订阅与发布

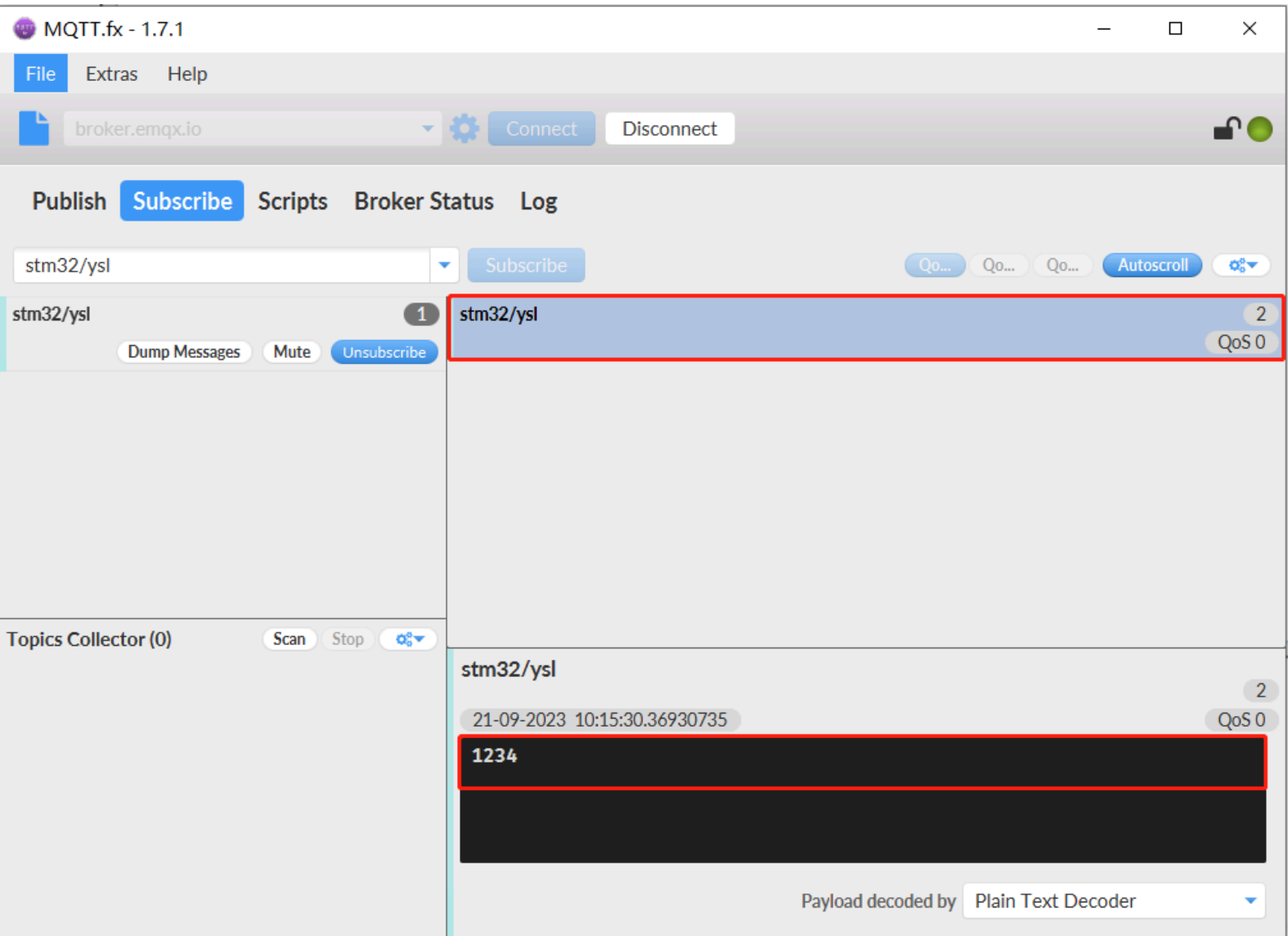
订阅主题



发布主题消息



查看接收的消息

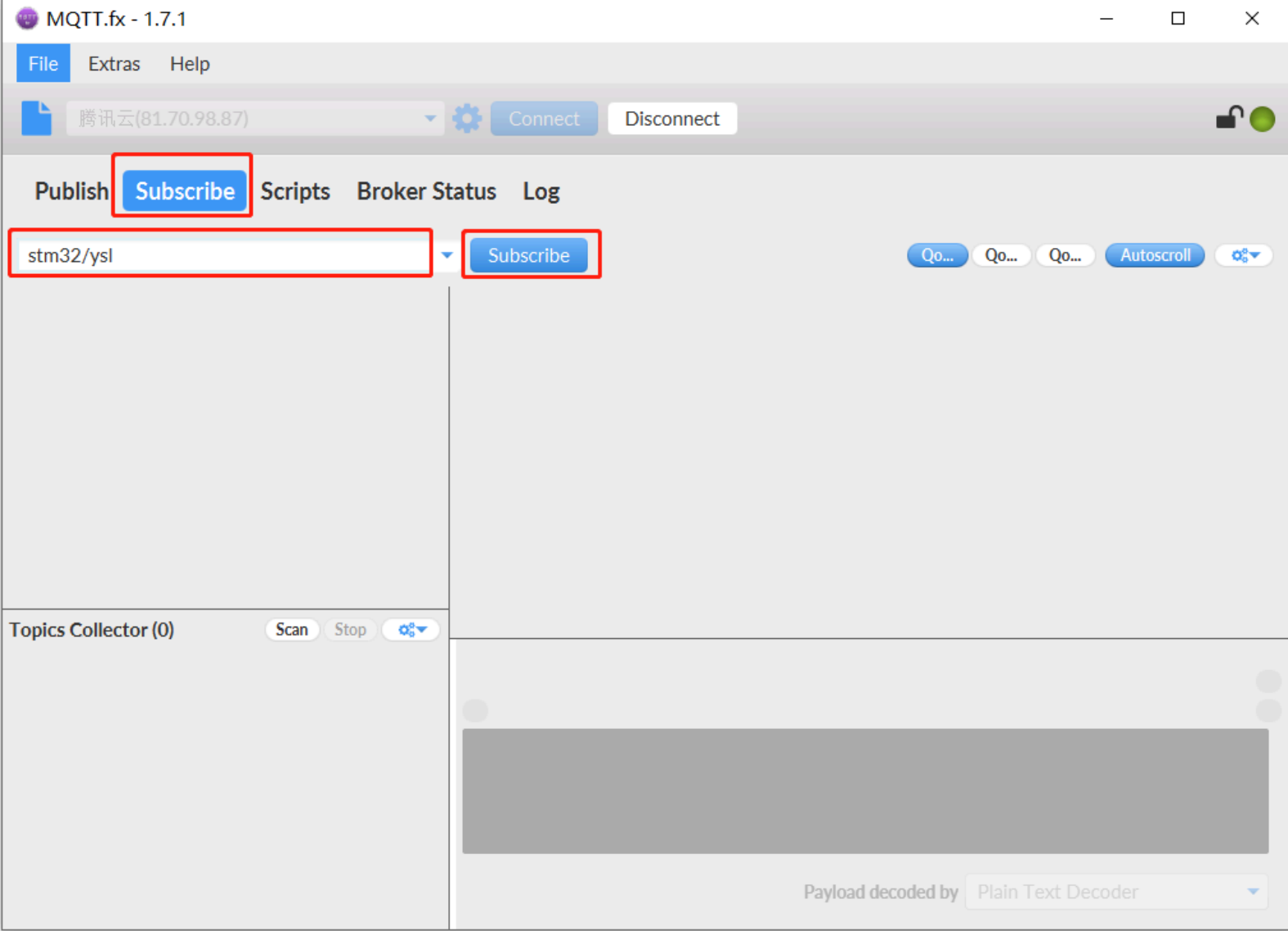


通信测试结束。

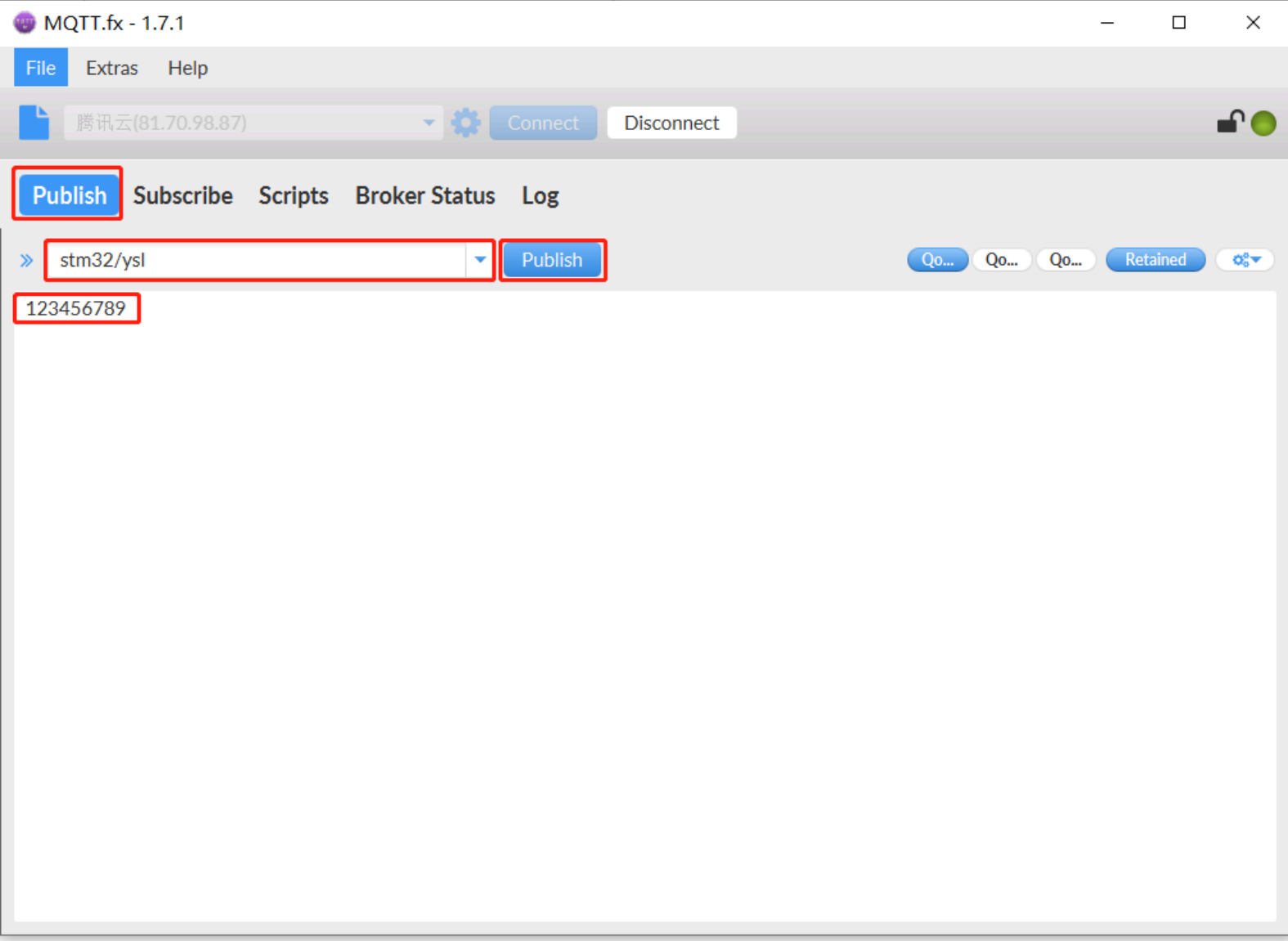
订阅与发布

订阅自己的主题

使用stm32 加上自己的名字即可。



发布消息



2.3 ESP8266的MQTT指令集

1. MQTT指令介绍

[AT 命令集 — ESP-AT 用户指南 文档 \(espressif.com\)](#)

- AT+MQTTUSERCFG: 设置 MQTT 用户属性
- AT+MQTTLONGCLIENTID: 设置 MQTT 客户端 ID
- AT+MQTTLONGUSERNAME: 设置 MQTT 登陆用户名
- AT+MQTTLONGPASSWORD: 设置 MQTT 登陆密码
- AT+MQTTCONNCFG: 设置 MQTT 连接属性
- AT+MQTTCONN: 连接 MQTT Broker
- AT+MQTTPUB: 发布 MQTT 消息（字符串）
- AT+MQTTPUBRAW: 发布 MQTT 消息（二进制）
- AT+MQTTSUB: 订阅 MQTT Topic
- AT+MQTTUNSUB: 取消订阅 MQTT Topic
- AT+MQTTCLEAN: 断开 MQTT 连接
- MQTT AT 错误码
- MQTT AT 说明

2. 连接MQTT服务器

给板子下载编程练习35的代码， 进行转发测试。

配置STA模式

配置为无线网卡模式

```
AT+CWMODE=1
```



连接路由器

```
AT+CWJAP="EMBAI303", "EMBAI2021"
```

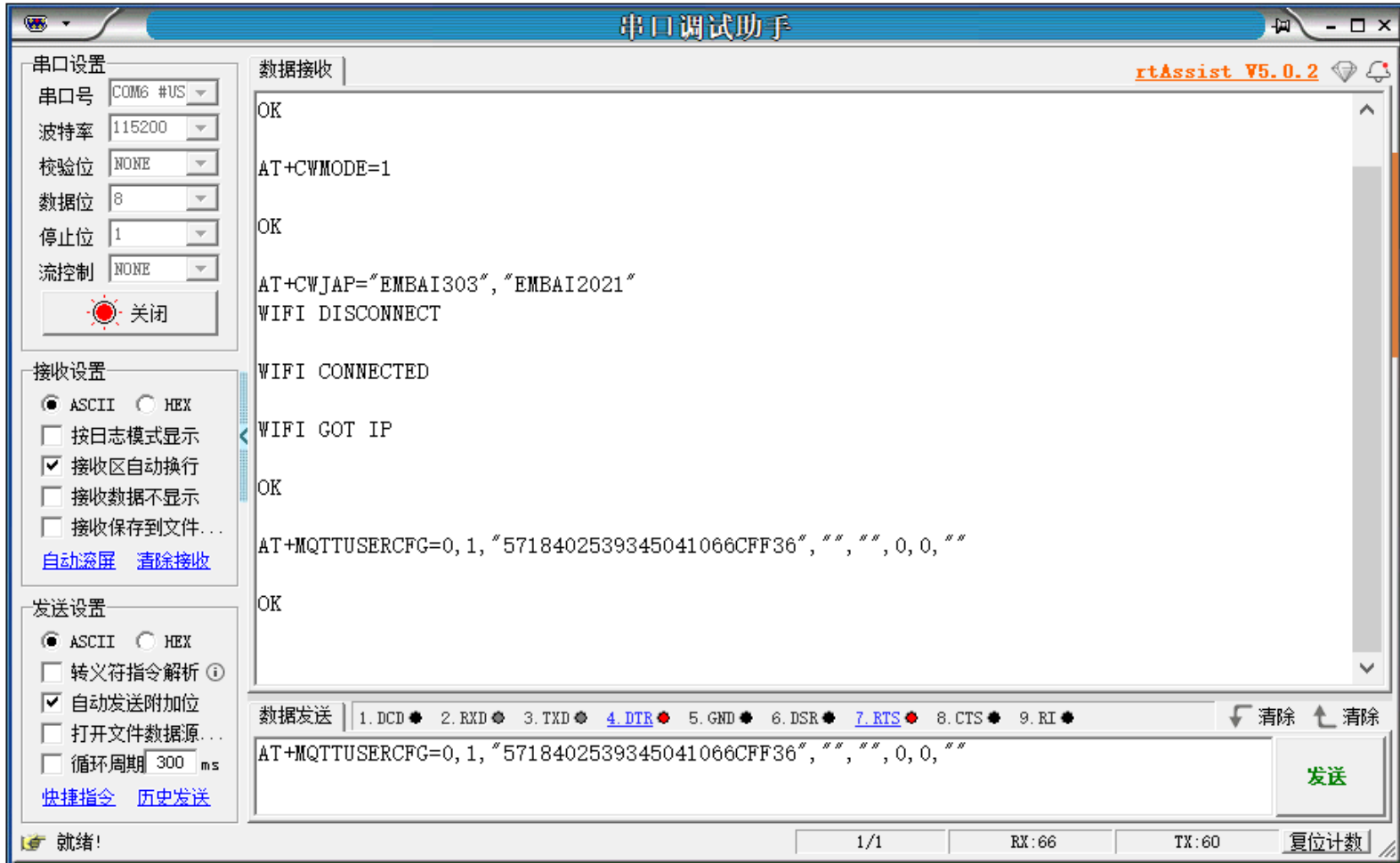


配置 MQTT 属性

注意：client ID 名字一定要长，服务器中发生重名后，服务器会自动断开连接。

请使用自己修改后的id，否在会可能重名，重名后服务器会自动断开连接。

```
AT+MQTTUSERCFG=0,1,"5718402539345041066CFF36","","",0,0,""
```



连接MQTT服务器

```
AT+MQTTCONN=0,"broker.emqx.io",1883,0
```



查询服务器连接状态

```
AT+MQTTCONN?
```



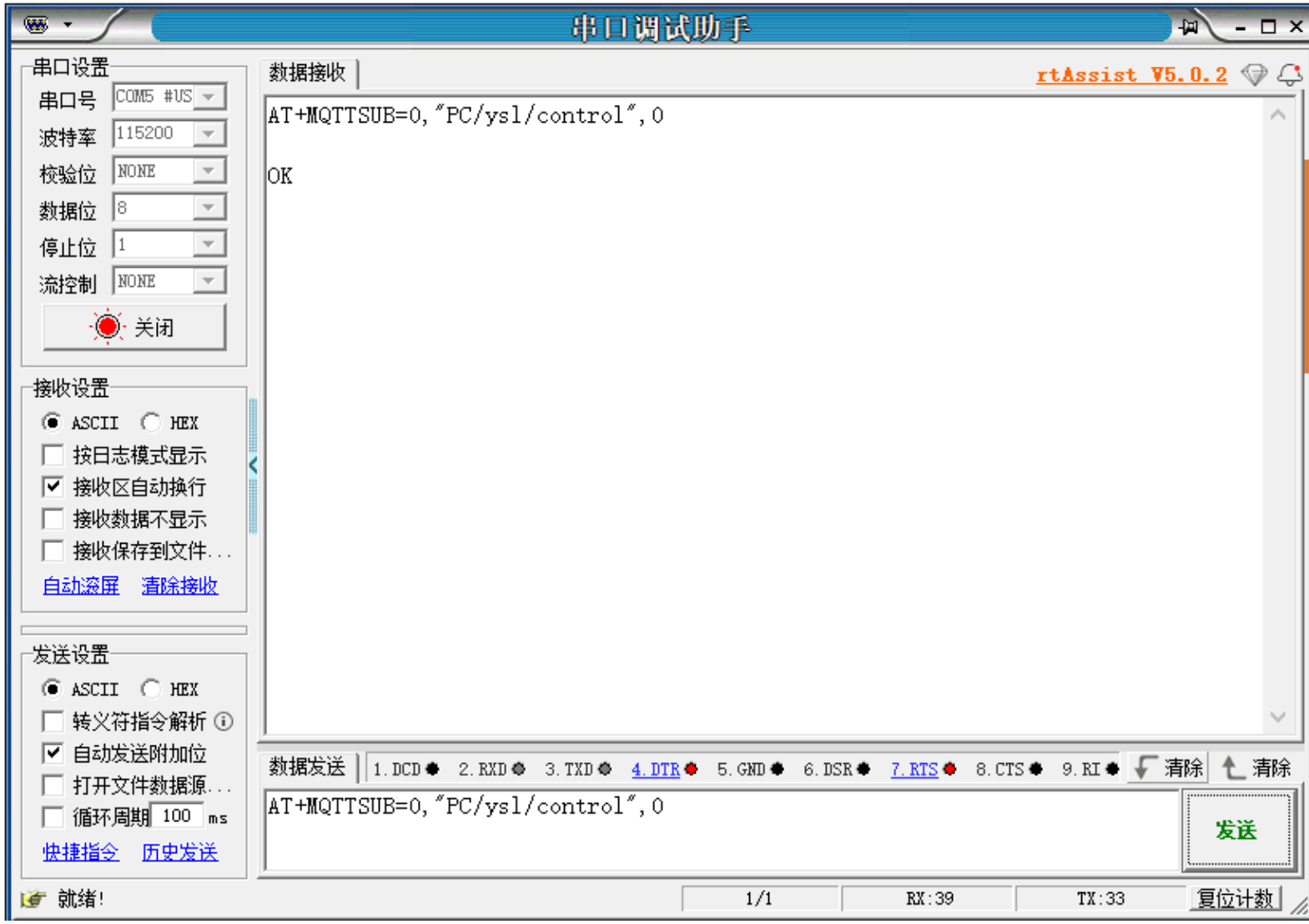
发布主题消息

```
AT+MQTTPUB=0,"ESP8266/ys1/sensor","123456",0,0
```



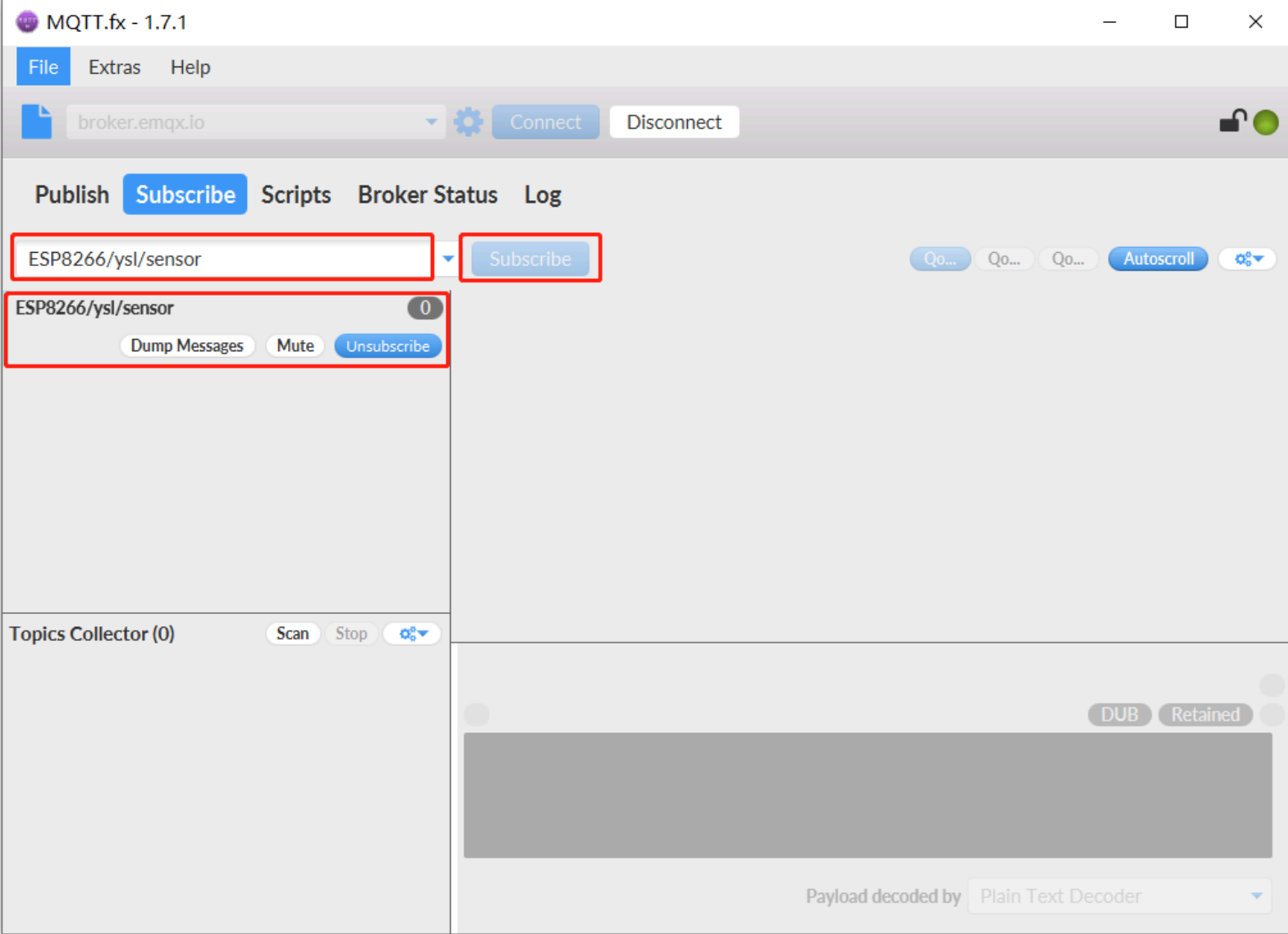
订阅主题消息

```
AT+MQTTSUB=0,"ESP8266/ysl/control",0
```



3. mqttfx接收主题消息

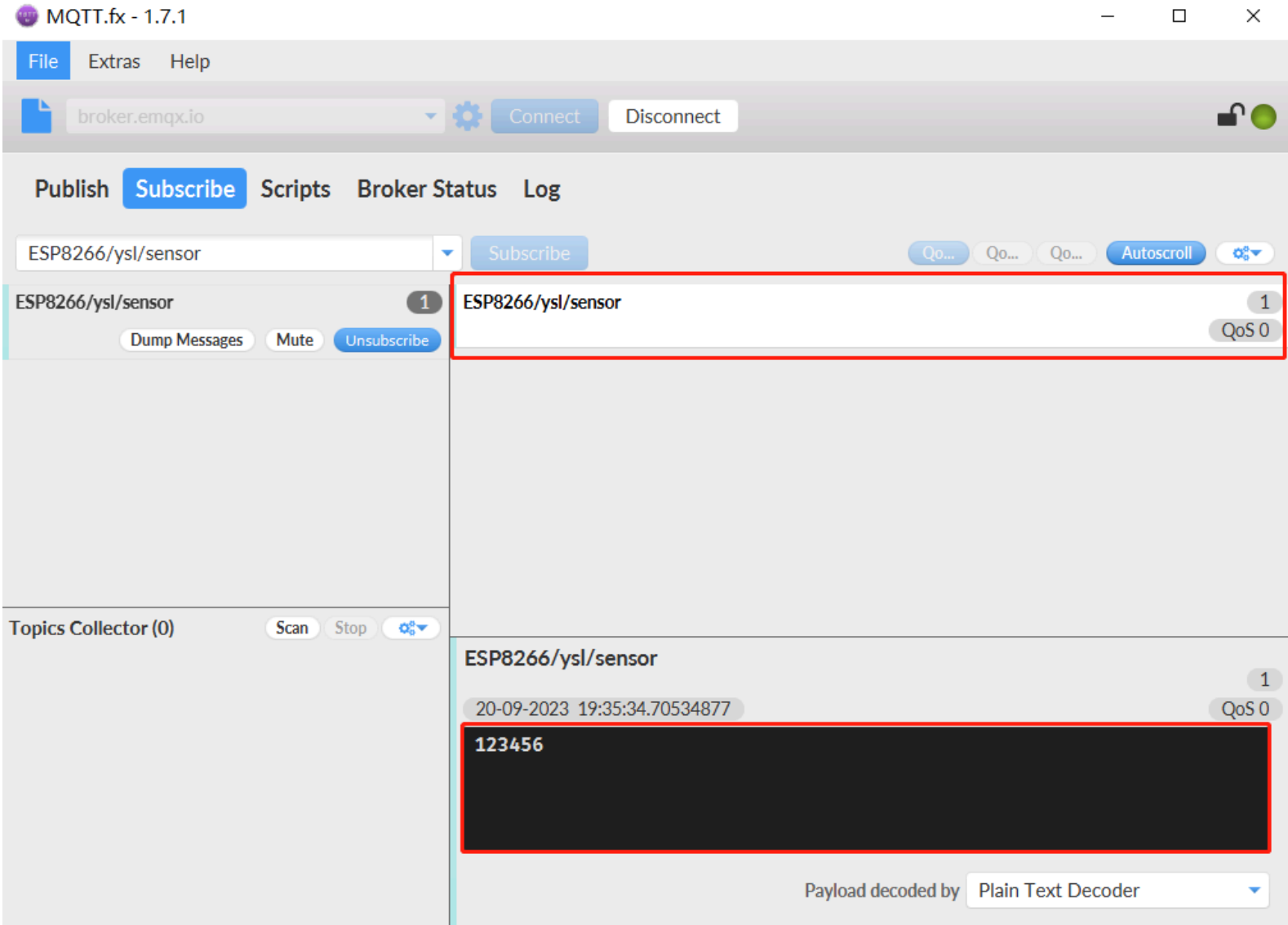
订阅ESP8266的消息



通过ESP8266 发送消息



在mqttfx中查看接收的数据

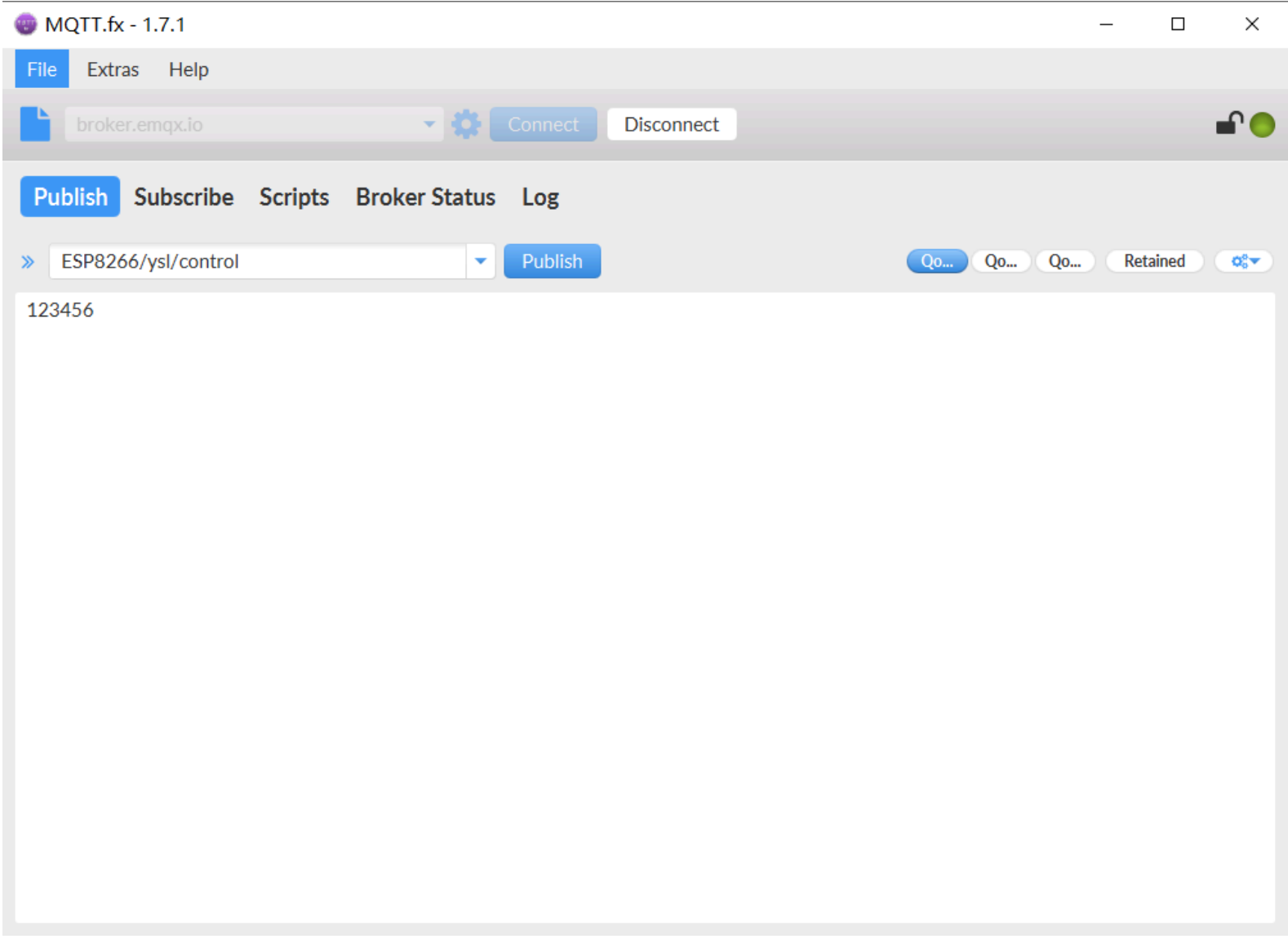


4. mqttfx发送主题消息

esp8266订阅主题消息， 订阅后可以接收这个主题的消息。



mqttfx向主题发送消息



esp8266接收主题消息



2.4 JSON字符串

1. JSON与cJSON

JSON -- 轻量级的数据格式

JSON 全称 JavaScript Object Notation，即 JS 对象简谱，是一种轻量级的数据格式。

它采用完全独立于编程语言的文本格式来存储和表示数据，语法简洁、层次结构清晰，易于人阅读和编写，同时也易于机器解析和生成，有效的提升了网络传输效率。

2. JSON语法规则

JSON对象是一个无序的"名称/值"键值对的集合：

- 以"{"开始，以"}"结束，允许嵌套使用；
- 每个名称和值成对出现，名称和值之间使用":"分隔；
- 键值对之间用","分隔
- 在这些字符前后允许存在无意义的空白符；

对于键值，可以有如下值：

- 一个新的json对象
- 数组：使用"["和"]"表示
- 数字：直接表示，可以是整数，也可以是浮点数

- 字符串：使用引号"表示
- 字面值：false、null、true中的一个(必须是小写)

示例如下：

```
{
    "name": "mculover666",
    "age": 22,
    "weight": 55.5,
    "address":
    {
        "country": "China",
        "zip-code": 111111
    },
    "skill": ["c", "Java", "Python"],
    "student": false
}
```

3. cJSON

cJSON是一个使用C语言编写的JSON数据解析器，具有超轻便，可移植，单文件的特点，使用MIT开源协议。

cJSON项目托管在Github上，仓库地址如下：<https://github.com/DaveGamble/cJSON>

其中cJSON的源码文件只有两个：

- cJSON.h
- cJSON.c

使用的时候，只需要将这两个文件复制到工程目录，然后包含头文件cJSON.h即可，如下：

```
/* The cJSON structure: */
typedef struct cJSON
{
    /* next/prev allow you to walk array/object chains. Alternatively, use
    GetArraySize/GetArrayItem/GetObjectItem */
    struct cJSON *next;
    struct cJSON *prev;
    /* An array or object item will have a child pointer pointing to a chain of the items in the
    array/object. */
    struct cJSON *child;

    /* The type of the item, as above. */
    int type;

    /* The item's string, if type==cJSON_String and type == cJSON_Raw */
    char *valstring;
    /* writing to valueint is DEPRECATED, use cJSON_SetNumberValue instead */
    int valueint;
    /* The item's number, if type==cJSON_Number */
    double valuedouble;

    /* The item's name string, if this item is the child of, or is in the list of subitems of an object. */
    char *string;
} cJSON;
```

cJSON的设计很巧妙。

首先，它不是将一整段JSON数据抽象出来，而是将其中的一条JSON数据抽象出来，也就是一个键值对，用上面的结构体 struct cJSON 来表示，其中用来存放值的成员列表如下：

- String：用于表示该键值对的名称；
- type：用于表示该键值对中值的类型；
- valstring：如果键值类型(type)是字符串，则将该指针指向键值；
- valueint：如果键值类型(type)是整数，则将该指针指向键值；
- valuedouble：如果键值类型(type)是浮点数，则将该指针指向键值；

其次，一段完整的JSON数据中由很多键值对组成，并且涉及到键值对的查找、删除、添加，所以使用链表来存储整段JSON数据，如上面的代码所示：

- next指针：指向下一个键值对
- prev指针：指向上一个键值对

最后，因为JSON数据支持嵌套，所以一个键值对的值会是一个新的JSON数据对象（一条新的链表），也有可能是一个数组，方便起见，在cJSON中，数组也表示为一个数组对象，用链表存储，所以：

在键值对结构体中，当该键值对的值是一个嵌套的JSON数据或者一个数组时，由child指针指向该条新链表。

4. JSON数据封装

封装方法

封装JSON数据的过程，其实就是创建链表和向链表中添加节点的过程。

首先来讲述一下链表中的一些术语：

- 头指针：指向链表头结点的指针；
- 头结点：不存放有效数据，方便链表操作；
- 首节点：第一个存放有效数据的节点；
- 尾节点：最后一个存放有效数据的节点；

明白了这几个概念之后，我们开始讲述创建一段完整的JSON数据，即如何创建一条完整的链表。

创建头指针：

```
cJSON* cJSON_test = NULL;
```

创建头结点，并将头指针指向头结点：

```
cjson_test = cJSON_CreateObject();
```

尽情的向链表中添加节点：

```
cJSON_AddNullToObject(cJSON * const object, const char * const name);

cJSON_AddTrueToObject(cJSON * const object, const char * const name);

cJSON_AddFalseToObject(cJSON * const object, const char * const name);

cJSON_AddBoolToObject(cJSON * const object, const char * const name, const cJSON_bool boolean);

cJSON_AddNumberToObject(cJSON * const object, const char * const name, const double number);

cJSON_AddStringToObject(cJSON * const object, const char * const name, const char * const string);

cJSON_AddRawToObject(cJSON * const object, const char * const name, const char * const raw);

cJSON_AddObjectToObject(cJSON * const object, const char * const name);

cJSON_AddArrayToObject(cJSON * const object, const char * const name);
```

输出JSON数据

上面讲述，一段完整的JSON数据就是一条长长的链表，那么，如何打印出这段JSON数据呢？

cJSON提供了一个API，可以将整条链表中存放的JSON信息输出到一个字符串中：

```
(char *) cJSON_Print(const cJSON *item);
```

使用的时候，只需要接收该函数返回的指针地址即可。

封装数据和打印数据示例

单纯的讲述方法还不够，下面用一个例子来说明，封装出开头给出的那段JSON数据：

json字符串

```
{
    "name": "mculover666",
    "age": 22,
    "weight": 55.5,
    "address":
    {
        "country": "China",
        "zip-code": 111111
    },
    "skill": ["c", "Java", "Python"],
    "student": false
}
```

源码

```
#include <stdio.h>
```

```
#include "cJSON.h"
```

```
int main(void)
{
    cJSON* cJSON_test = NULL;
    cJSON* cJSON_address = NULL;
    cJSON* cJSON_skill = NULL;
    char* str = NULL;

    /* 创建一个JSON数据对象(链表头结点) */
    cJSON_test = cJSON_CreateObject();

    /* 添加一条字符串类型的JSON数据(添加一个链表节点) */
    cJSON_AddStringToObject(cJSON_test, "name", "mculover666");

    /* 添加一条整数类型的JSON数据(添加一个链表节点) */
    cJSON_AddNumberToObject(cJSON_test, "age", 22);

    /* 添加一条浮点类型的JSON数据(添加一个链表节点) */
    cJSON_AddNumberToObject(cJSON_test, "weight", 55.5);

    /* 添加一个嵌套的JSON数据（添加一个链表节点） */
    cJSON_address = cJSON_CreateObject();
    cJSON_AddStringToObject(cJSON_address, "country", "China");
    cJSON_AddNumberToObject(cJSON_address, "zip-code", 111111);
    cJSON_AddItemToObject(cJSON_test, "address", cJSON_address);

    /* 添加一个数组类型的JSON数据(添加一个链表节点) */
    cJSON_skill = cJSON_CreateArray();
    cJSON_AddItemToArray(cJSON_skill, cJSON_CreateString( "C" ));
    cJSON_AddItemToArray(cJSON_skill, cJSON_CreateString( "Java" ));
    cJSON_AddItemToArray(cJSON_skill, cJSON_CreateString( "Python" ));
    cJSON_AddItemToObject(cJSON_test, "skill", cJSON_skill);

    /* 添加一个值为 False 的布尔类型的JSON数据(添加一个链表节点) */
    cJSON_AddFalseToObject(cJSON_test, "student");

    /* 打印JSON对象(整条链表)的所有数据 */
    str = cJSON_Print(cJSON_test);
    printf("%s\n", str);

    return 0;
}
```

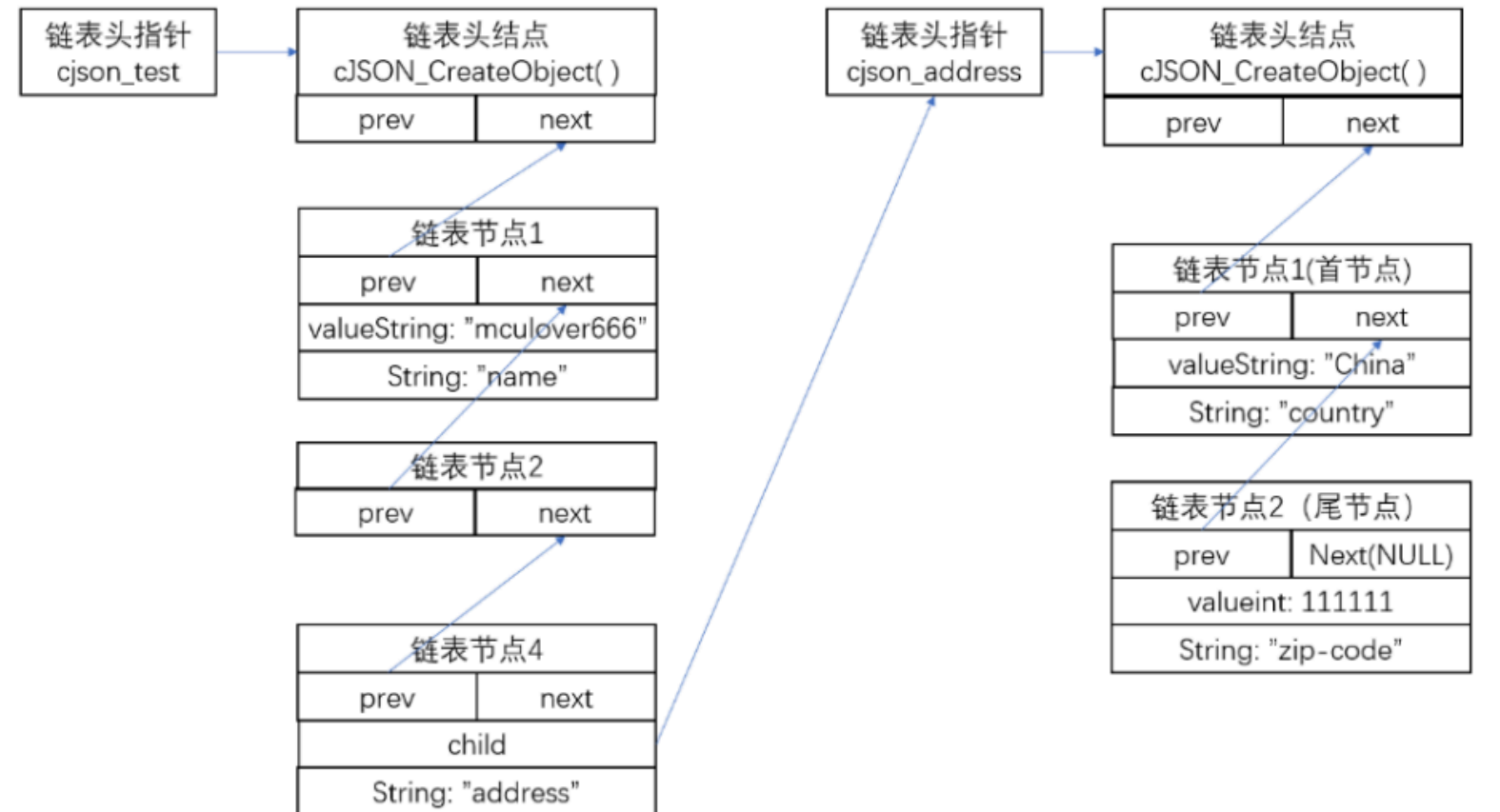
编译运行：

```
gcc cJSON_test.c cJSON.c -o test
```

实验结果如图：

```
PS E:\YShengli\02-ClassNotes\emb2407\cjson> gcc .\cjson_test.c .\cJSON.c -o test
PS E:\YShengli\02-ClassNotes\emb2407\cjson> .\test.exe
{
    "name": "mculover666",
    "age": 22,
    "weight": 55.5,
    "address": {
        "country": "China",
        "zip-code": 111111
    },
    "skill": ["C", "Java", "Python"],
    "student": false
}
PS E:\YShengli\02-ClassNotes\emb2407\cjson>
```

该JSON数据链表的关系如图：



5. cJSON数据解析

解析方法

解析JSON数据的过程，其实就是剥离一个一个链表节点(键值对)的过程。

解析方法如下：

- 创建链表头指针：

```
cJSON* cJSON_test = NULL;
```

- 解析整段JSON数据，并将链表头结点地址返回，赋值给头指针。解析整段数据使用的API只有一个：

```
(cJSON *) cJSON_Parse(const char *value);
```

- 根据键值对的名称从链表中取出对应的值，返回该键值对(链表节点)的地址

```
(cJSON *) cJSON_GetObjectItem(const cJSON * const object, const char * const string);
```

- 如果JSON数据的值是数组，使用下面的两个API提取数据：

```
(int) cJSON_GetArraySize(const cJSON *array);
(cJSON *) cJSON_GetArrayItem(const cJSON *array, int index);
```

解析示例

下面用一个例子来说明如何解析出开头给出的那段JSON数据：

```
#include <stdio.h>
#include "cJSON.h"

char *message =
"{
    \"name\": \"mculover666\",
    \"age\": 22,
    \"weight\": 55.5,
    \"address\": {
        \"country\": \"China\",
        \"zip-code\": 111111
    },
    \"skill\": [\"C\", \"Java\", \"Python\"],
    \"student\": false
}";

int main(void)
{
    cJSON* cJSON_test = NULL;
    cJSON* cJSON_name = NULL;
    cJSON* cJSON_age = NULL;
    cJSON* cJSON_weight = NULL;
    cJSON* cJSON_address = NULL;
    cJSON* cJSON_address_country = NULL;
    cJSON* cJSON_address_zipcode = NULL;
    cJSON* cJSON_skill = NULL;
    cJSON* cJSON_student = NULL;
    int skill_array_size = 0, i = 0;
    cJSON* cJSON_skill_item = NULL;

    /* 解析整段JSON数据 */
    cJSON_test = cJSON_Parse(message);
    if(cJSON_test == NULL)
    {
```

```
printf("parse fail.\n");
    return -1;
}

/* 依次根据名称提取JSON数据（键值对） */
cjson_name = cJSON_GetObjectItem(cjson_test, "name");
cjson_age = cJSON_GetObjectItem(cjson_test, "age");
cjson_weight = cJSON_GetObjectItem(cjson_test, "weight");

printf("name: %s\n", cjson_name->valuestring);
printf("age: %d\n", cjson_age->valueint);
printf("weight: %.1f\n", cjson_weight->valuedouble);

/* 解析嵌套json数据 */
cjson_address = cJSON_GetObjectItem(cjson_test, "address");
cjson_address_country = cJSON_GetObjectItem(cjson_address, "country");
cjson_address_zipcode = cJSON_GetObjectItem(cjson_address, "zip-code");
printf("address-country: %s\naddress-zipcode: %d\n", cjson_address_country->valuestring,
    cjson_address_zipcode->valueint);

/* 解析数组 */
cjson_skill = cJSON_GetObjectItem(cjson_test, "skill");
skill_array_size = cJSON_GetArraySize(cjson_skill);
printf("skill: [");
for(i = 0; i < skill_array_size; i++)
{
    cjson_skill_item = cJSON_GetArrayItem(cjson_skill, i);
    printf("%s,", cjson_skill_item->valuestring);
}
printf("\b]\n");

/* 解析布尔型数据 */
cjson_student = cJSON_GetObjectItem(cjson_test, "student");
if(cjson_student->valueint == 0)
{
    printf("student: false\n");
}
else
{
    printf("student: error\n");
}

return 0;
}
```

编译：

```
gcc .\cjson_test2.c .\cJSON.c -o test2
```

运行结果如图：

```
PS E:\YShengli\02-ClassNotes\emb2407\cjson> gcc .\cjson_test2.c .\cJSON.c -o test2
PS E:\YShengli\02-ClassNotes\emb2407\cjson> .\test2.exe
name: mculover666
age:22
weight:55.5
address-country:China
address-zipcode:111111
skill:[c,Java,Python]
student: false
PS E:\YShengli\02-ClassNotes\emb2407\cjson>
```

注意事项

在本示例中，因为我提前知道数据的类型，比如字符型或者浮点型，所以我直接使用指针指向对应的数据域提取，在实际使用时，如果提前不确定数据类型，应该先判断type的值，确定数据类型，再从对应的数据域中提取数据。

6. cJSON使用过程中的内存问题

内存及时释放

cJSON的所有操作都是基于链表的，所以cJSON在使用过程中大量的使用malloc从堆中分配动态内存的，所以在使用完之后，应当及时调用下面的函数，清空cJSON指针所指向的内存，该函数也可用于删除某一条数据：

```
(void) cJSON_Delete(cJSON *item);
```

注意：该函数删除一条JSON数据时，如果有嵌套，会连带删除。

内存钩子

cJSON在支持自定义malloc函数和free函数，方法如下：

- 使用cJSON_Hooks来连接自定义malloc函数和free函数：

```
typedef struct cJSON_Hooks
{
    /* malloc/free are CDECL on windows regardless of the default calling convention of the compiler,
    so ensure the hooks allow passing those functions directly. */
    void *(cJSON_CDECL *malloc_fn)(size_t sz);
    void (cJSON_CDECL *free_fn)(void *ptr);
} cJSON_Hooks;
```

初始化钩子cJSON_Hooks

```
(void) cJSON_InitHooks(cJSON_Hooks* hooks);
```

2.5 设计自己的通信协议

1. 控制设备的json字符串

控制设备开

```
"device":    // 可以带 可以不带
{
    "LED1":true,
    "LED2":true,
    "BEEP":true,
    "RELAY":true
}
```

控制设备关

```
"device":
{
    "LED1":false,
    "LED2":false,
    "BEEP":false,
    "RELAY":false
}
```

2. 传感器数据上传的json字符串

```
{
    "TP":25.5,    // °C
    "RH":45.5,    // %
    "CU":201,     // ma
    "VL":4.85,    // V
    "PW":809,     // mW
    "VR":3.0,     // V
    "CPU":33.2    // °C
}

// TP   :   温度           类型为 实数
// RH   :   湿度           类型为 实数
// CU   :   电流           类型为 整数
// VL   :   电压           类型为 实数
// PW   :   功率           类型为 整数
// VR   :   电位器电压     类型为 实数
// CPU  :   CPU温度       类型为 实数
```

3. 主题设置

STM32V9 的发送或上行主题为

```
STM32V9/UPLoad/cpuid
// 例如
STM32V9/UPLoad/5718402539345041066CFF36
```

STM32V9 的接收或下行送主题为

```
STM32V9/DownLoad/cpuid
// 例如
STM32V9/DownLoad/5718402539345041066CFF36
```

设备全部工作

```
{

    "LED1":true,
    "LED2":true,
    "BEEP":true,
    "RELAY":true

}
```

设备全部不工作

```
{

    "LED1":false,
    "LED2":false,
    "BEEP":false,
    "RELAY":false

}
```

单独控制LED1工作

```
{

    "LED1":true

}
```

单独控制RELAY不工作

```
{

    "RELAY":false

}
```

2.6 项目实战3 - WIFI实现MQTT通信

1. 第一阶段：链接MQTT服务器

- 程序在实战项目1的基础上完成后序的开发。
- 开机后自动连接MQTT服务器并订阅主题，OLED显示WIFI状态，MQTT服务器状态。
- 在USART3空闲中断处理函数中开启定时器TIM4，设定超时时间为10ms，定时器时间到开始处理队列接收的数据，把队列接收的数据导出给一个数组，在数组中查找OK，并给出指令的返回状态，可以快速的判断出指令返回的状态，可以快速执行下一条指令。

mqtt.c

```
uint8_t *Get_CPUID(void)
{
    static uint8_t cpuid[32] = {0};
    // 每个芯片都有唯一的 96_bit unique ID
    uint32_t CPU_ID[3];
    CPU_ID[0] = *(__IO uint32_t *) (0X1FFFF7F0); // 高字节
    CPU_ID[1] = *(__IO uint32_t *) (0X1FFFF7EC); //
    CPU_ID[2] = *(__IO uint32_t *) (0X1FFFF7E8); // 低字节

    /* 芯片的唯一ID */
    sprintf((char *)cpuid, "%08x%08x%08x", CPU_ID[0], CPU_ID[1], CPU_ID[2]);
    return cpuid;
}

uint8_t MQTT_WIFI_Connect_Server(void)
{
    uint8_t ret;
    uint8_t buf[128]={0};
    WIFI4G_CMD_Status = WIFI4G_NOT ;      // 初始化标志位
    strcpy((char *)Parse_Substr,"OK\r\n");
    printf("cpuid:%s\n", Get_CPUID());
    // 设置用户信息
    OLED_ShowStr(0,4,(unsigned char *)"AT+MQTTUSERCFG",2);          //测试8*16字符

    sprintf((char *)buf,"AT+MQTTUSERCFG=0,1,\"%s\\",\\"\\",\\"\\",0,0,\"\\\"\\r\\n",Get_CPUID()); // 设置客户端ID
    HAL_UART_Transmit(&huart3,buf,strlen((char *)buf),1000);

    if(Test_WIFI4G_CMD_Status(1000) == WIFI4G_ERROR) {
        OLED_ShowStr(0,4,(unsigned char *)"MQTTServer... --",2);    //测试8*16字符
        return RESET;
    }
    // 连接mqtt服务器
    WIFI4G_CMD_Status = WIFI4G_NOT ;      // 初始化标志位
    OLED_ShowStr(0,4,(unsigned char *)"AT+MQTTCONN",2);          //测试8*16字符
    strcpy((char *)buf,"AT+MQTTCONN=0,\"broker.emqx.io\\",1883,1\\r\\n");
    HAL_UART_Transmit(&huart3,buf,strlen((char *)buf),1000);

    if(Test_WIFI4G_CMD_Status(5*1000) == WIFI4G_ERROR) {
        OLED_ShowStr(0,4,(unsigned char *)"MQTTServer... --",2);    //测试8*16字符
        return RESET ;
    }

    // 订阅主题消息 ， 订阅一个下行数据的主题
    WIFI4G_CMD_Status = WIFI4G_NOT ;      // 初始化标志位
    OLED_ShowStr(0,4,(unsigned char *)"AT+MQTTSUB",2);          //测试8*16字符
    sprintf((char *)buf,"AT+MQTTSUB=0,\"STM32V9/DownLoad/%s\\",0\\r\\n",Get_CPUID());
    HAL_UART_Transmit(&huart3,buf,strlen((char *)buf),1000);

    ret = Test_WIFI4G_CMD_Status(5*1000) ;
    if (ret == WIFI4G_OK)
    {
        OLED_ShowStr(0,4,(unsigned char *)"MQTTServer... OK",2);    //测试8*16字符
    }
    else
    {
        OLED_ShowStr(0,4,(unsigned char *)"MQTTServer... --",2);    //测试8*16字符
        return RESET;
    }
    return SET;
}

}
```

2. 第二阶段：上传传感器数据

- 设置5秒标志位， 进行周期性的上传数据。
- 把传感器的输出采集并传到MQTT服务器上，并用MQTTfx软件查看。

```
uint8_t MQTT_SendData(void)
{

    static uint8_t sendbuf[128] = {0};
    static uint8_t buf[256] = {0};

    sprintf((char *)sendbuf,"\"{\\\\"TP\\\\"":%d\\,\\\\"RH\\\\"":%d\\,\\\\"VO\\\\"":%d\\,\\\\"CU\\\\"":%d\\,\\\\"PW\\\\"":%d\\,\\\\"VR\\\\"":%d\\,\\\\"CPU\\\\"":%d}\\\"",
        REG_HOLD_BUF[1],REG_HOLD_BUF[2],
        REG_HOLD_BUF[3],REG_HOLD_BUF[4],REG_HOLD_BUF[5],REG_HOLD_BUF[6],REG_HOLD_BUF[7]);

    //printf("%s",sendbuf);
    // 发送数据到mqtt服务器
    sprintf((char *)buf,"AT+MQTTPUB=0,\"STM32V9/UPLoad/%s\\",%s,0,0\\r\\n",Get_CPUID(),sendbuf);
    //printf("%s",buf);

    HAL_UART_Transmit(&huart3,(uint8_t*)buf,strlen((char *)buf),1000);

    return SET;
}
```

3. 第三阶段：接收命令并解析命令

- 移植cjson.c和cjson.h文件到工程内。
- 解析USART3的接收到的数据，使用strstr函数进行查找并分割， 找个完整的json字符串。
- 把字符串交给json函数进行解析并分析数据对相应的设备进行控制。
- 注意把系统的堆调大一倍， 防止cjson解析内存不够， 我在这里犯了一个错误。

解析Json字符串的核心代码

wifi4g.c的WIFI4G_Parse_Queue函数中增加核心代码

```
// 使用 "{" 对接收的数据进行分割， 分割后可以提取出json字符串
if(MQTT_Download_Flag) // 连接到服务器后开始解析数据
{
```

```
if( memcmp(RecvBuf, "AT+MQTTPUB", strlen("AT+MQTTPUB")) != 0) // 数据回显，防止数据错误解析
{
    char * leftp  = strstr((char *)RecvBuf, "{"); // 找到左花括号
    if(leftp != NULL)
    {
        //printf("leftp=%s\n", leftp);
        MQTT_Parse_JsonData((uint8_t*)leftp); // 解析json字符串并控制设备
        //printf("Json_Buf=%s\n", Json_Buf);
    }
}

}
```

mqtt.c

```
uint8_t MQTT_Parse_JsonData(uint8_t *json)
{

    cJSON *cjson_device = NULL;
    cJSON *cjson_led1 = NULL;
    cJSON *cjson_led2 = NULL;
    cJSON *cjson_bEEP = NULL;
    cJSON *cjson_relay = NULL;

    /* 解析整段JSON数据 */
    cjson_device = cJSON_Parse((char *)json);
    if (cjson_device == NULL)
    {
        printf("parse fail.\n");
        return RESET;
    }
    else
    {
        // printf("json->%s\n", cJSON_Print(cjson_device));
    }
    /* 依次根据名称提取JSON数据（键值对） */
    cjson_led1  = cJSON_GetObjectItem(cjson_device, "LED1");
    cjson_led2  = cJSON_GetObjectItem(cjson_device, "LED2");
    cjson_bEEP  = cJSON_GetObjectItem(cjson_device, "BEEP");
    cjson_relay = cJSON_GetObjectItem(cjson_device, "RELAY");
    // printf("LED1:%d\n", cjson_led1->valueint);
    // printf("LED2:%d\n", cjson_led2->valueint);
    // printf("BEEP:%d\n", cjson_bEEP->valueint);
    // printf("RELAY:%d\n", cjson_relay->valueint);
    // LED1 控制
    if (cjson_led1->valueint == 0)
    {
        LED_Control(LED1, OFF);
    }
    else
    {
        LED_Control(LED1, ON);
    }

    // LED2 控制
    if (cjson_led2->valueint == 0)
    {
        LED_Control(LED2, OFF);
    }
    else
    {
        LED_Control(LED2, ON);
    }

    // BEEP 控制
    if (cjson_bEEP->valueint == 0)
    {
        BEEP_Control(OFF);
    }
    else
    {
        BEEP_Control(ON);
    }

    // RELAY 控制
    if (cjson_relay->valueint == 0)
    {
        RELAY_Control(OFF);
    }
    else
    {
        RELAY_Control(ON);
    }

    cJSON_Delete(cjson_device); // 一定要释放内存
    return SET;
}
```

2.7 实战项目4- 4G实现MQTT通信

1. 程序设计要求

- 重新设计连接MQTT服务器代码，让代码兼容4G模组链接MQTT服务器。
- 上传数据代码兼容4G模组。

mqtt.h

```
// 宏的值为1 ，使能WIFI
// 宏的值为0 ，使能4G
#define MQTT_WIFI_4G_ENABLE 0
```

mqtt.c

```
uint8_t ML307_Connect_MQTTServer(void)
{

    uint8_t buf[64] = {0};
    OLED_ShowStr(0,4,(unsigned char *)"MQTTServer... --",2);    //测试8*16字符

    printf("cpuid:%s\n",Get_CPUID());

    strcpy((char *)Parse_Substr, "OK\r\n");
    // AT+DTUTASK="1", "20"
    WIFI4G_CMD_Status = WIFI4G_NOT;    // 初始化标志位
    sprintf((char *)buf, "AT+DTUTASK=\"%1\", \"20\" \r\n");
    HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
    if (Test_WIFI4G_CMD_Status(1000) == WIFI4G_ERROR)
    {
        return RESET;    // 等待OK返回
    }

    WIFI4G_CMD_Status = WIFI4G_NOT;    // 初始化标志位
    sprintf((char *)buf, "AT+MQTT=\"%s\" \r\n",Get_CPUID()); // 单链接
    HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
    if (Test_WIFI4G_CMD_Status(1000) == WIFI4G_ERROR)
    {
        return RESET;    // 等待OK返回
    }

    // 设置mqtt服务器
    WIFI4G_CMD_Status = WIFI4G_NOT ;    // 初始化标志位
    strcpy((char *)buf, "AT+MQTTIP=\"broker.emqx.io\", \"1883\" \r\n");
    HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
    if(Test_WIFI4G_CMD_Status(5*1000) == WIFI4G_ERROR) {
        return RESET ;
    }

    // 订阅主题消息 ，订阅一个下行数据的主题
    WIFI4G_CMD_Status = WIFI4G_NOT ;    // 初始化标志位
    sprintf((char *)buf, "AT+MQTTSUB=0, \"STM32V9/DownLoad/%s\", 0\r\n",Get_CPUID());
    HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
    if(Test_WIFI4G_CMD_Status(1000) == WIFI4G_ERROR) {
        return RESET ;
    }

    // 订阅主题消息 ，订阅一个下行数据的主题
    WIFI4G_CMD_Status = WIFI4G_NOT ;    // 初始化标志位
    sprintf((char *)buf, "AT+MQSUB=\"%1\", \"1\", \"4\", \"STM32V9/DownLoad/%s\" \r\n",Get_CPUID());
    HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
    if(Test_WIFI4G_CMD_Status(1000) == WIFI4G_ERROR) {
        return RESET ;
    }

    // 订阅主题消息 ，设置一个上行数据的主题
    WIFI4G_CMD_Status = WIFI4G_NOT ;    // 初始化标志位
    // AT+MQPUB=0,1,0,4, "STM32V9/UPLoad/6703401957694950066AFF51"
    //AT+MQPUB="1", "1", "4", "/a12o68BvvH0/myDevice/user/mypub"
```

```
        sprintf((char *)buf, "AT+MQPUB=\"1\\", \"1\\", \"4\\", \"STM32V9/UPLoad/%s\\\"\\r\\n", Get_CPUID());
        printf("buf=%s\\n", buf);
        HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
        if (Test_WIFI4G_CMD_Status(1000) == WIFI4G_ERROR)    {
            return RESET ;
        }

        //  设置单通道通信 ，  1个上行  1个下行
        WIFI4G_CMD_Status = WIFI4G_NOT ;           //  初始化标志位
        sprintf((char *)buf, "AT+MQMULTEN=0\\r\\n"); //
        HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
        if (Test_WIFI4G_CMD_Status(1000) == WIFI4G_ERROR)    {
            return RESET ;
        }

        //  AT+REST  重启后模块配置生效

        //  订阅主题消息 ，  订阅一个下行数据的主题

        WIFI4G_CMD_Status = WIFI4G_NOT ;           //  初始化标志位
        sprintf((char *)buf, "AT+REST\\r\\n");
        strcpy((char *)Parse_Substr, "MQTT_CONNECT:"); //  服务器连接成功的返回值
        HAL_UART_Transmit(&huart3, buf, strlen((char *)buf), 1000);
        uint8_t ret = Test_WIFI4G_CMD_Status(180*1000) ;
        if (ret == WIFI4G_OK)
        {
            OLED_ShowStr(0,4,(unsigned char *) "MQTTServer... OK", 2);      //测试8*16字符
            return SET;
        }
        else
        {
            OLED_ShowStr(0,4,(unsigned char *) "MQTTServer... --", 2);      //测试8*16字符
            return RESET;
        }
    }

}

uint8_t MQTT_Connect_Server(void)
{
    #if MQTT_WIFI_4G_ENABLE
    //  连接wifi
    ESP8266_Connect_WIFI();
    //  连接mqtt服务器
    if( ESP8266_Connect_MQTTServer() == SET)
    {
        MQTT_Download_Flag = 1 ; //  开启解析  服务器数据的标志位
        printf("MQTT  服务器连接成功  \\n");

    }

    #else
    //  连接mqtt服务器
    if( ML307_Connect_MQTTServer() == SET)
    {
        MQTT_Download_Flag = 1 ; //  开启解析  服务器数据的标志位
        printf("MQTT  服务器连接成功  \\n");

    }

    #endif
    return SET;
}

uint8_t MQTT_SendData(void)
{
    static uint8_t sendbuf[128] = {0};
    static uint8_t buf[256] = {0};
    #if MQTT_WIFI_4G_ENABLE
        sprintf((char *)sendbuf, "\\
{\\\\"TP\\\\"":%d\\,\\\\"RH\\\\"":%d\\,\\\\"VO\\\\"":%d\\,\\\\"CU\\\\"":%d\\,\\\\"PW\\\\"":%d\\,\\\\"VR\\\\"":%d\\,\\\\"CPU\\\\"":%d}\\",
        REG_HOLD_BUF[1], REG_HOLD_BUF[2],
        REG_HOLD_BUF[3], REG_HOLD_BUF[4], REG_HOLD_BUF[5], REG_HOLD_BUF[6], REG_HOLD_BUF[7]);

        //printf("%s", sendbuf);
        //  发送数据到mqtt服务器
        sprintf((char *)buf, "AT+MQTTPUB=0,\\\"STM32V9/UPLoad/%s\\\",%s,0,0\\r\\n", Get_CPUID(), sendbuf);
        //printf("%s", buf);
    #else
        //  MQPUB,1,PUB2,123
        sprintf((char *)sendbuf, "{\\"TP\\\\"":%d,\\"RH\\\\"":%d,\\"VO\\\\"":%d,\\"CU\\\\"":%d,\\"PW\\\\"":%d,\\"VR\\\\"":%d,\\"CPU\\\\"":%d}",
        REG_HOLD_BUF[1], REG_HOLD_BUF[2], REG_HOLD_BUF[3], REG_HOLD_BUF[4],
        REG_HOLD_BUF[5], REG_HOLD_BUF[6], REG_HOLD_BUF[7]);

        //printf("%s", sendbuf);
        //  发送数据到mqtt服务器
        sprintf((char *)buf, "%s", sendbuf);
        printf("%s", buf);

    #endif
    HAL_UART_Transmit(&huart3, (uint8_t*)buf, strlen((char *)buf), 1000);
    return SET;
}

}
```

2.8 实战项目5- MQTT+FreeRTOS

1. 程序设计要求：

- 程序支持Modbus通信，同样也支持MQTT协议通信。
- 任务1用来处理modbus协议数据
- 任务2用来处理mqtt协议数据。
- TIM4用来处理MQTT返回数据，TIM3用来处理Modbus接收的数据。
- 实时查看生成可执行程序的大小，防止程序超过64K。

第03章 CANOpen协议实战

3.1 CAN概述

1. 什么是CAN

Controller Area Network，控制器局域网，是国际上应用最广泛的现场总线之一。

20世纪80年代初德国Bosch公司为解决现代汽车中众多控制单元、测试仪器之间的实时数据交换而开发的一种串行通信协议。

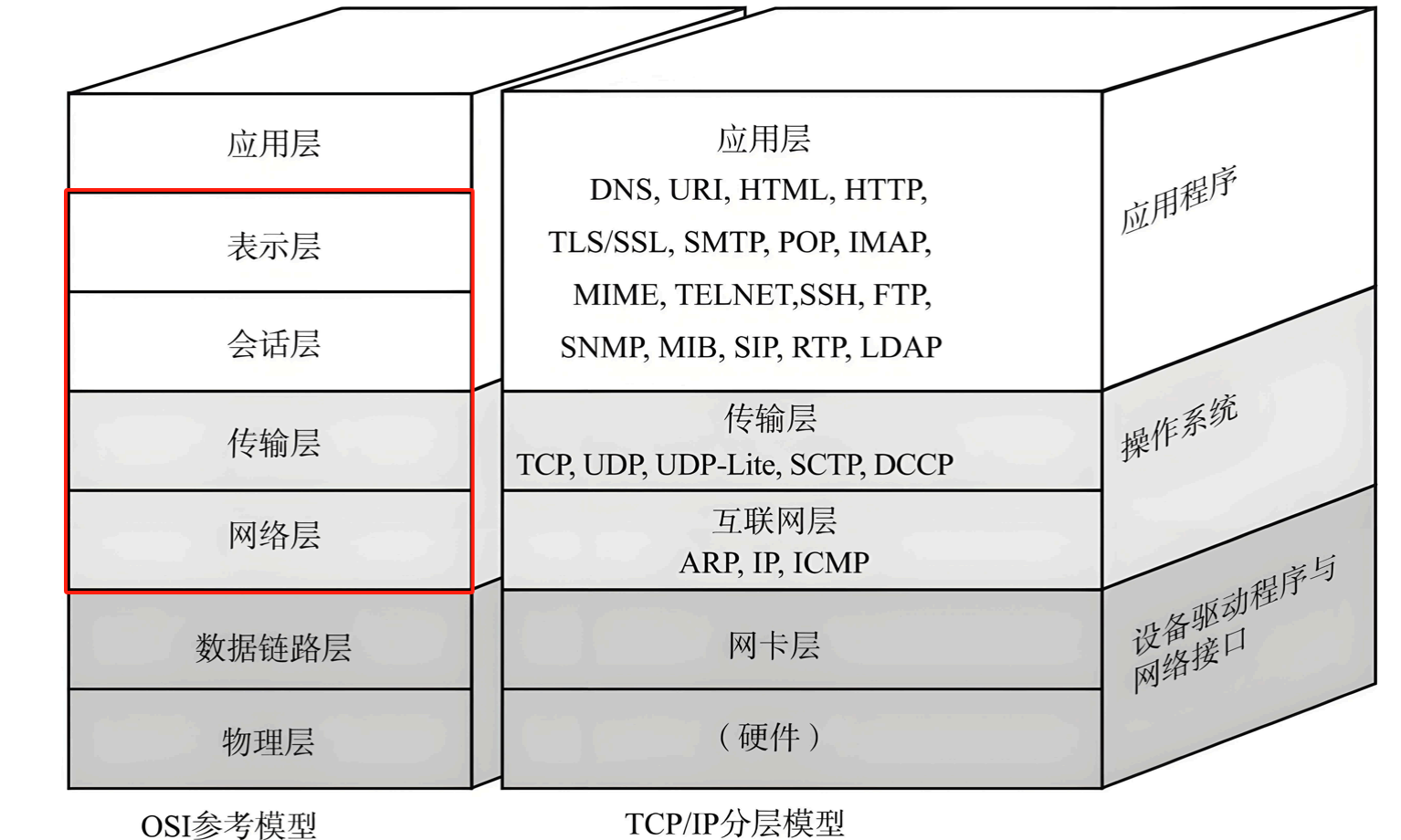
2. CAN的历史

- 1983年，Bosch开始研究车上网络技术。
- 1986年，Bosch在SAE(美国汽车工程师协会)大会公布CAN协议。
- 1987年，Intel和Philips先后推出CAN控制器芯片。
- 1991年，Bosch颁布CAN 2.0技术规范，CAN2.0包括A和B两个部分。
- 1991年，CAN总线最先在Benz S系列轿车上实现。
- 1993年，ISO颁布CAN国际标准ISO-11898。
- 1994年，SAE颁布基于CAN的J1939标准。
- 2003年，Maybach发布带76个ECU的新车型（CAN，LIN，MOST）。
- 2003年，VW发布带35个ECU的新型Golf。

3. CiA组织

1992年，一些公司创建了非盈利性组织CAN in Automation（CiA），以提供与CAN相关的技术、产品和市场营销方面的信息。此举旨在提升CAN的形象，并且为CAN协议未来的发展铺平道路。到2020年年初，已有670家公司加入CiA组织。

4. CAN的OSI模型



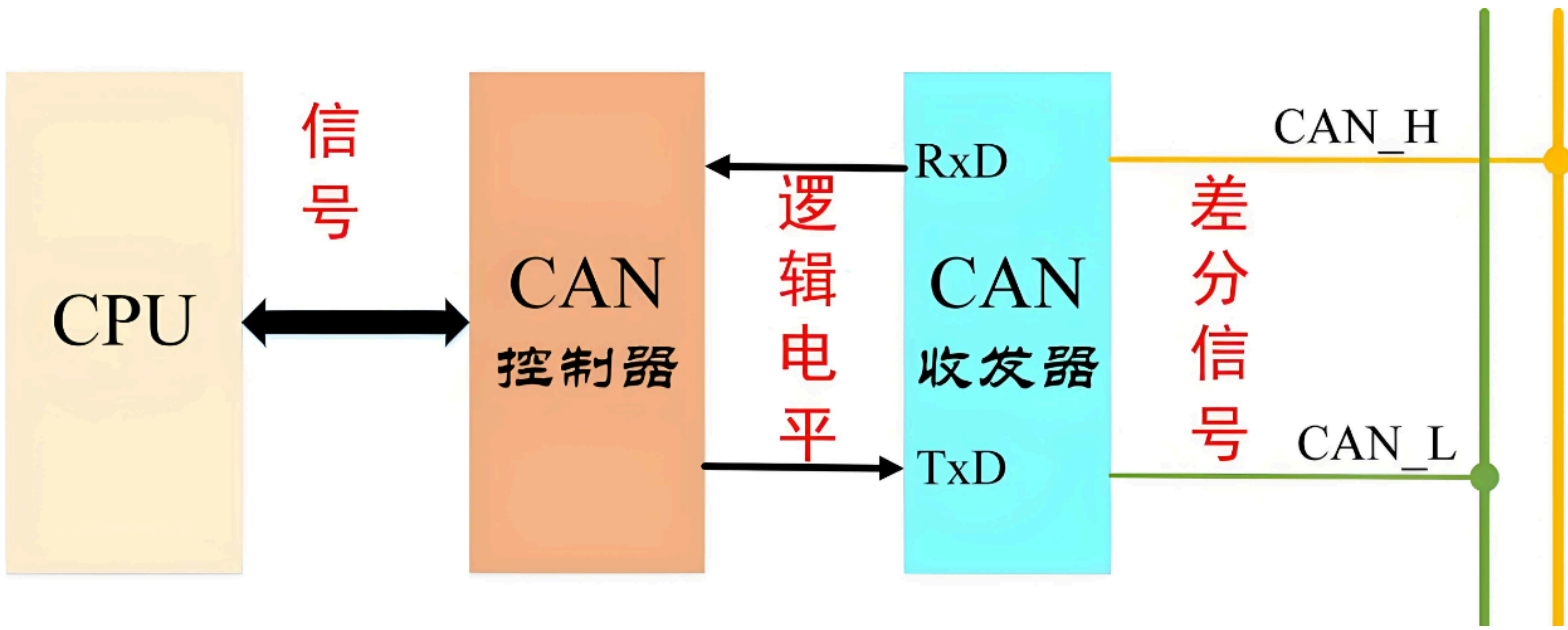
参考模型是国际标准化组织（ISO）制定的一个用于计算机或通信系统间互联的标准体系，一般称为OSI参考模型或七层模型。

从OSI网络模型的角度来看同，现场总线网络一般只实现了第1层（物理层）、第2层（数据链路层）、第7层（应用层）。

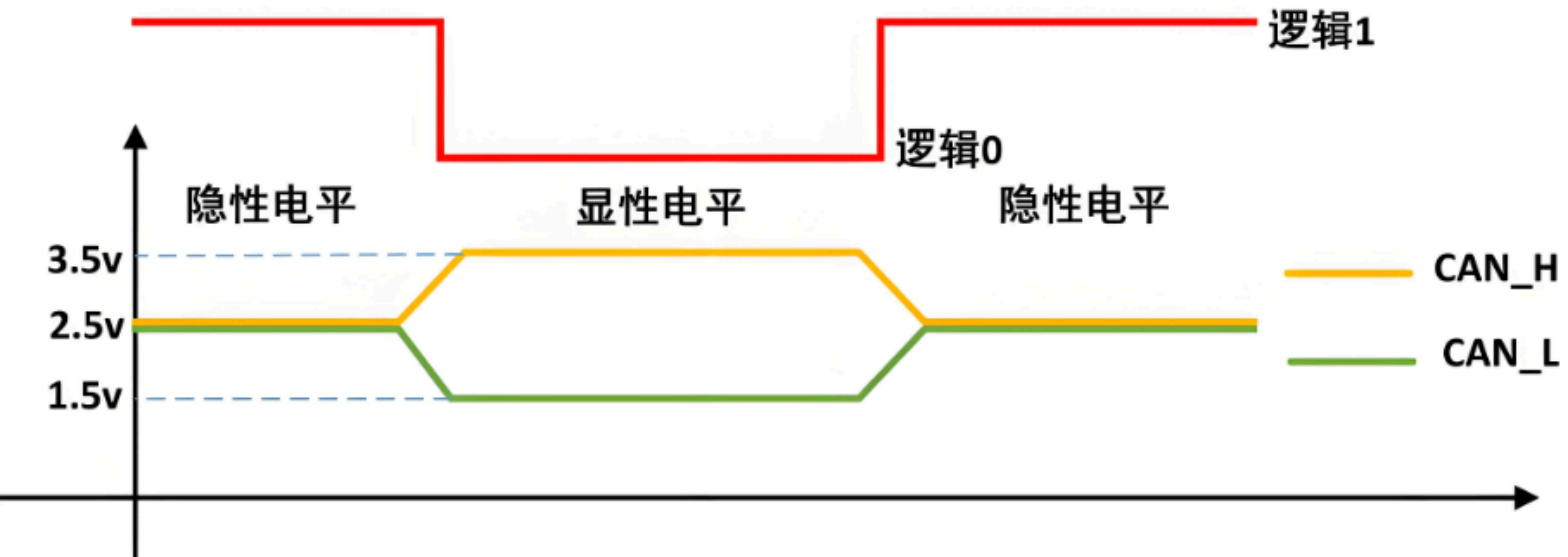
因为现场总线通常只包括一个网段，因此不需要第3层（传输层）、第4层（网络层）、第5层（会话层）、第6层（描述层）的作用。

CAN只定义物理层和数据链路层，没有规定应用层。

5. CAN的差分传输



采用双线差分信号，两根线构成总线，CAN High与CAN Low，这两根线之间的电位差可以对应两个不同的逻辑状态进行编码。



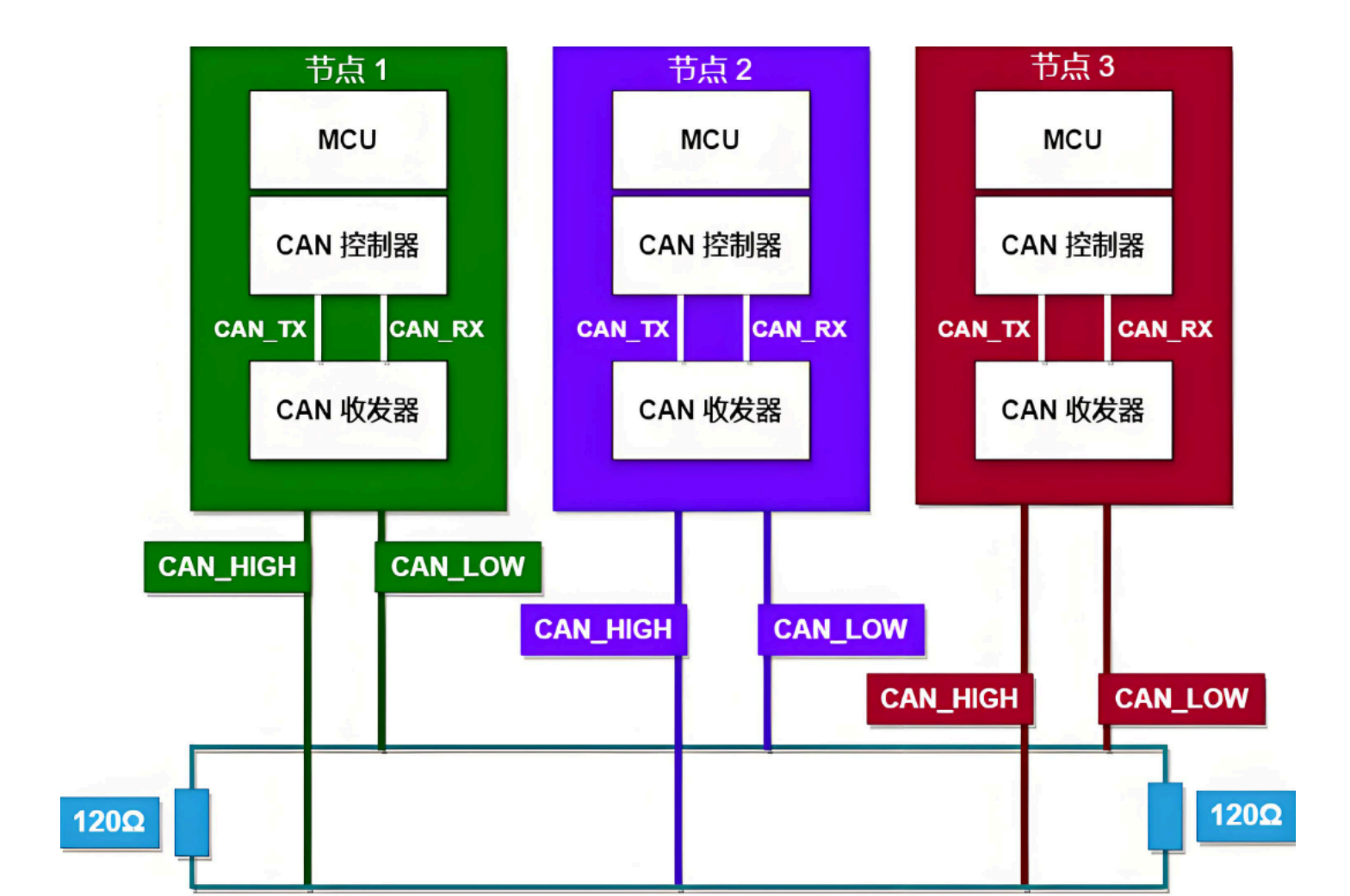
当CAN_H和CAN_L均是2.5V左右，此时CAN_H和CAN_L的压差为0V，此时状态表示为逻辑1，称为隐性位。

当CAN_H=3.5V和CAN_L=1.5V，此时CAN_H和CAN_L的压差为2V，此时状态表示为逻辑0，称为显性位，竞争时显性位优先。

6. CAN的传输距离

波特率	总线长度
1 Mbit/s	25m
500 kbit/s	100m
250 kbit/s	250m
125 kbit/s	500m
50 kbit/s	1.0km
20 kbit/s	2.5km
10 kbit/s	5.0km
5 kbit/s	13km

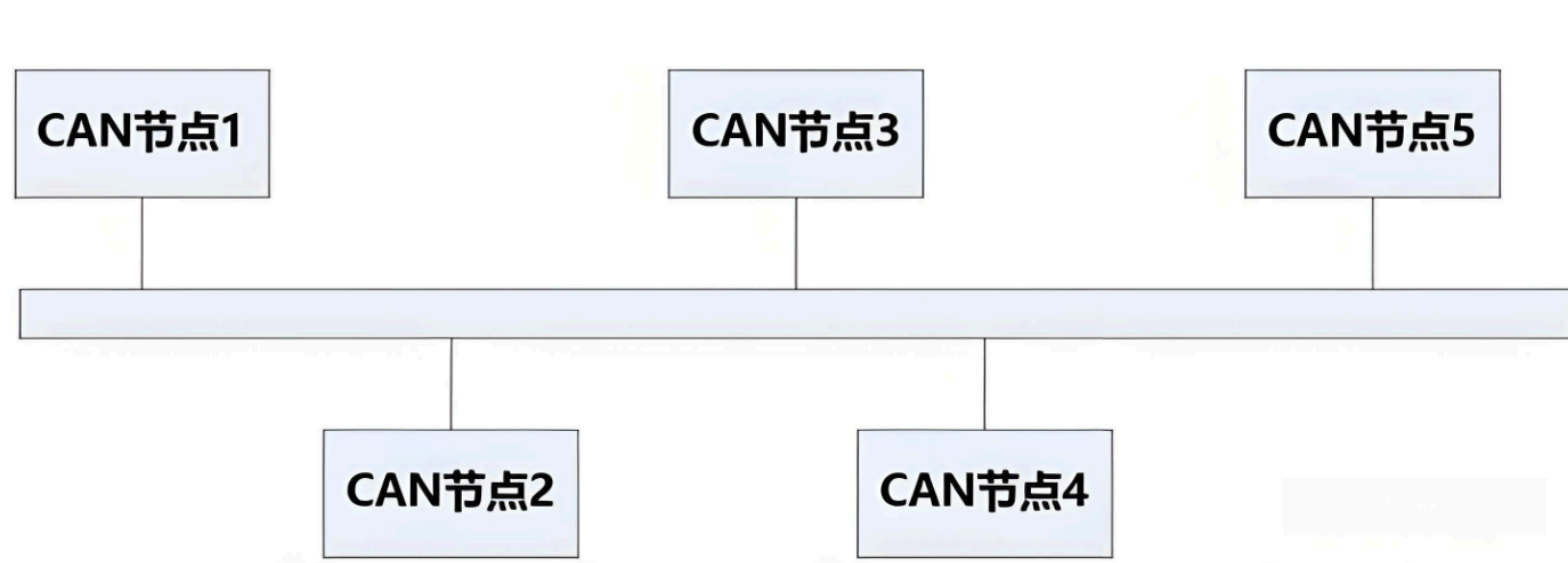
7. CAN总线组网方式



8. CAN总线节点数量

协议本身对节点的数量没有限制，实际应用中节点数目受网络硬件的电气特性所限制。

总线上节点的数量可以动态改变，发送广播报文，报文可以被所有节点同时接收。



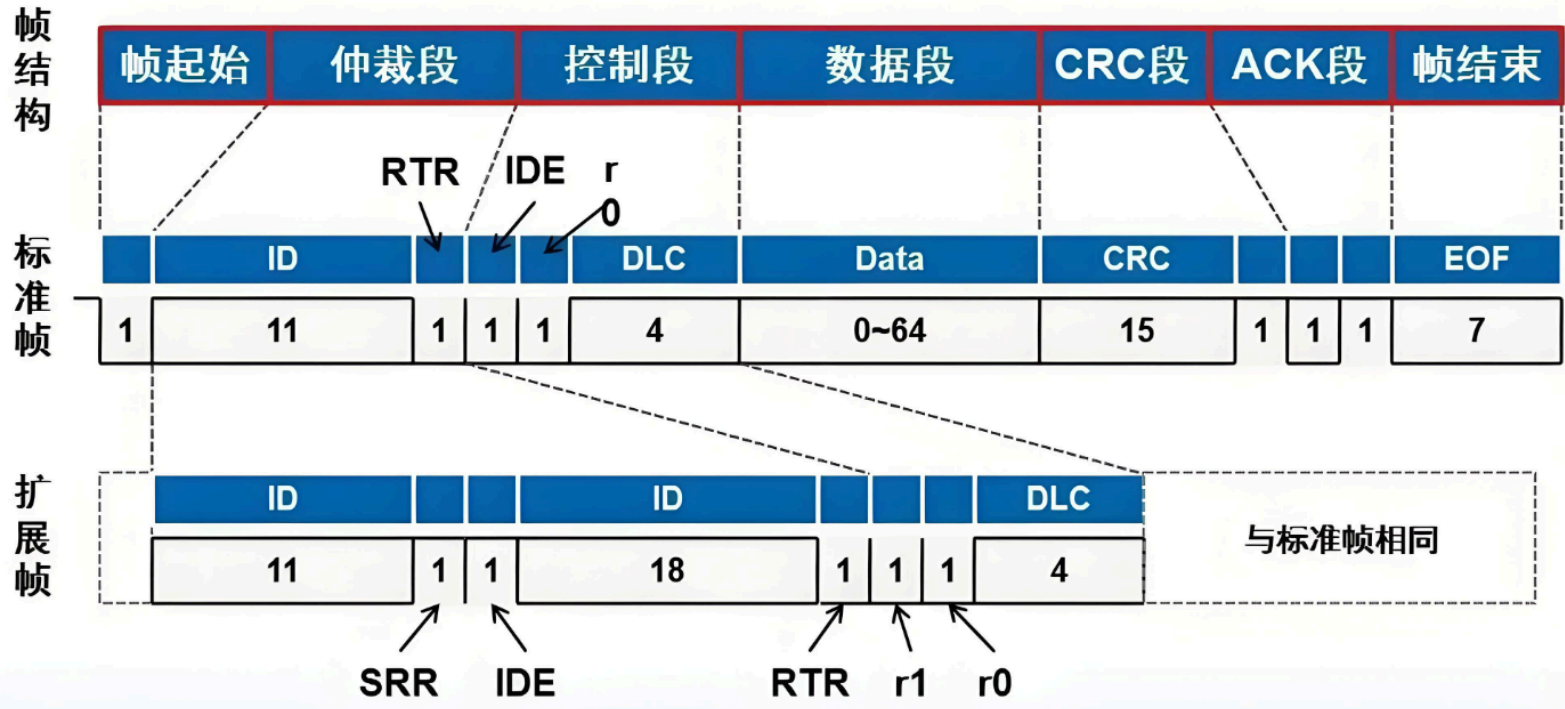
3.2 CAN总线数据格式

CAN总线节点上的节点发送数据是以报文的形式广播给网络中所有节点。收发器接收到数据就把数据传送给控制器，再由控制器检查判断是不是所需数据。不是则忽略。

- 网络上任何一个节点在任何时候都可以发送数据。
- 多个节点发送数据，优先级低主动退出发送。
- 短帧结构，每帧数据信息为0~8字节（具体用户定义），对数据编码而不是地址编码。
- CAN每帧都有CRC校验和其他检验措施，严重错误的情况下具有自动关闭输出的功能。

1. 数据帧

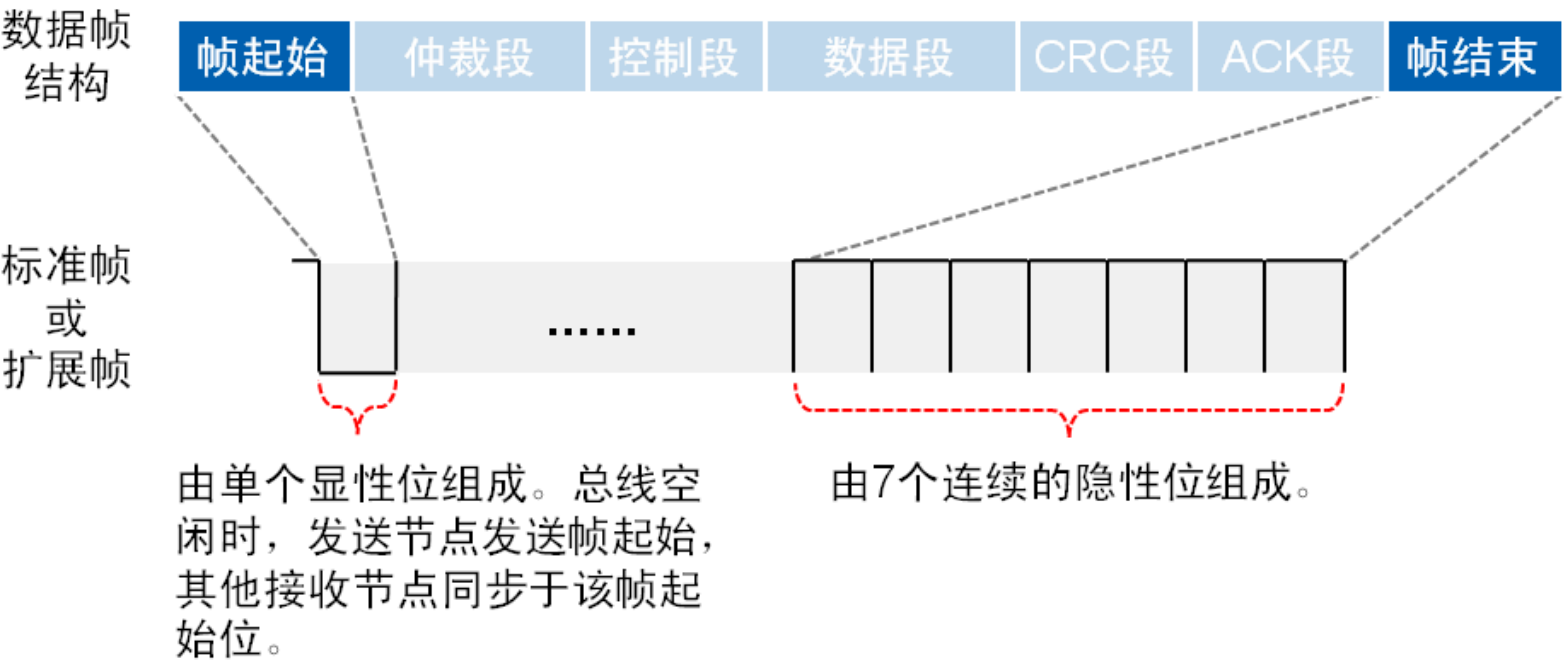
结构上由7个段组成，其中根据仲裁段ID码长度的不同，分为标准帧（CAN2.0A）和扩展帧（CAN2.0B）。



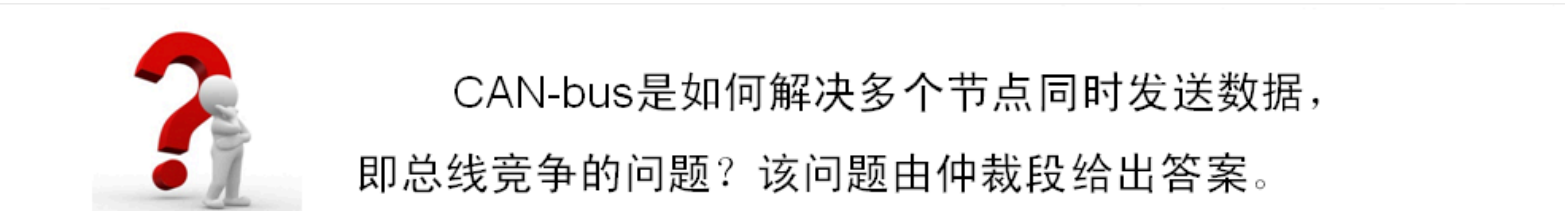
2. 帧起始、帧结束

- 1个位的低电平表示帧的开始
- 7个位的高电平表示帧的结束

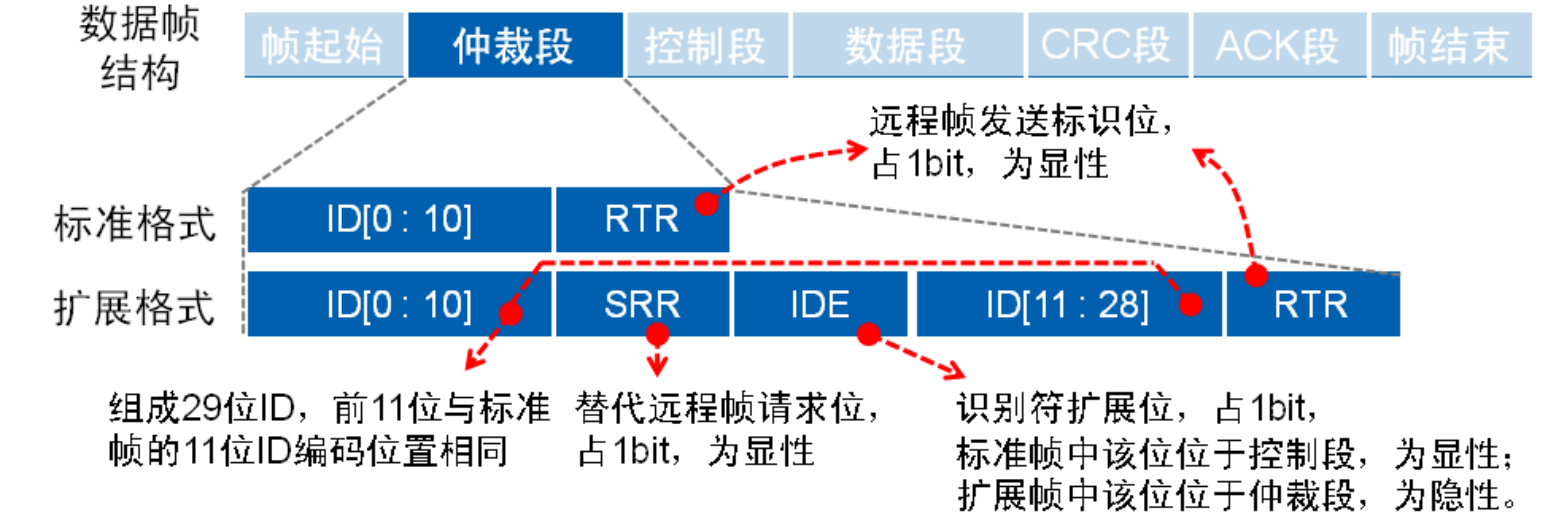
帧起始和帧结束用于界定一个数据帧，无论是标准数据帧或扩展数据帧都包含这两个段。



3. 仲裁段

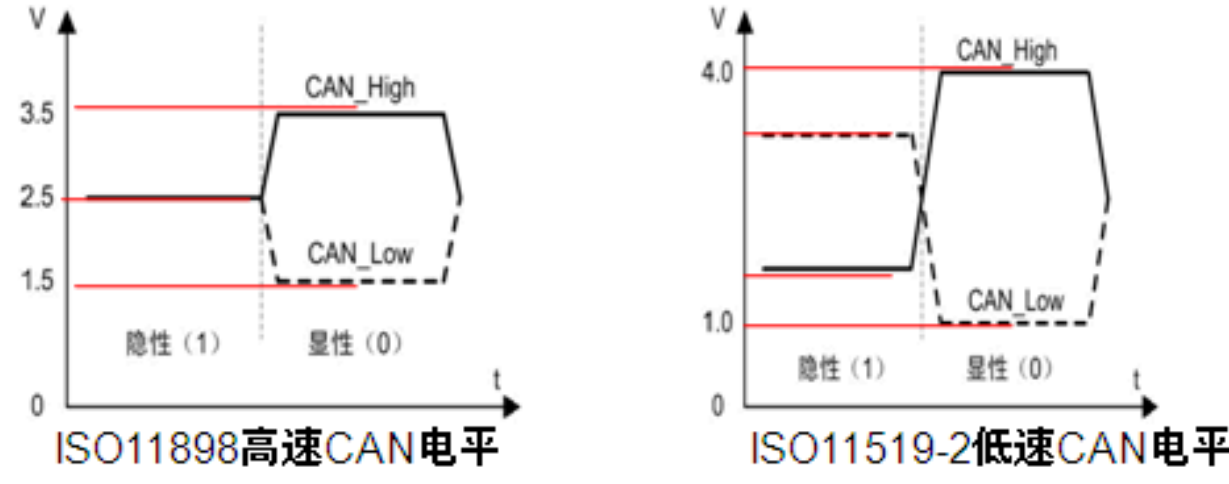


CAN-bus并没有规定节点的优先级，但通过仲裁段帧ID规定了数据帧的优先级。根据CAN2.0标准版本不同，帧ID分为11位和29位两种。



4. 显性隐性

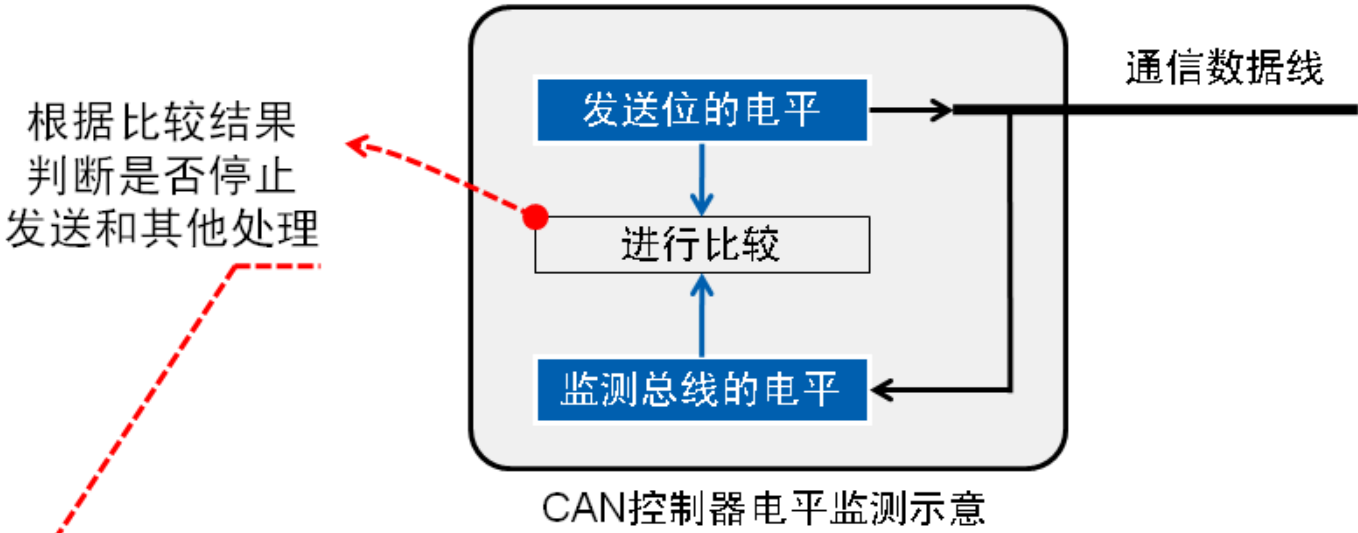
CAN-bus发布了ISO11898和ISO11519两个通信标准，此两个标准中差分电平的特性不相同。



物理层	ISO11898		ISO11519-2	
电平	显性	隐性	显性	隐性
CAH_H/V	3.50	3.00	4.00	1.75
CAN_H/V	1.50	3.00	1.00	3.25
电位差/V	2.00	0	3.00	-1.50

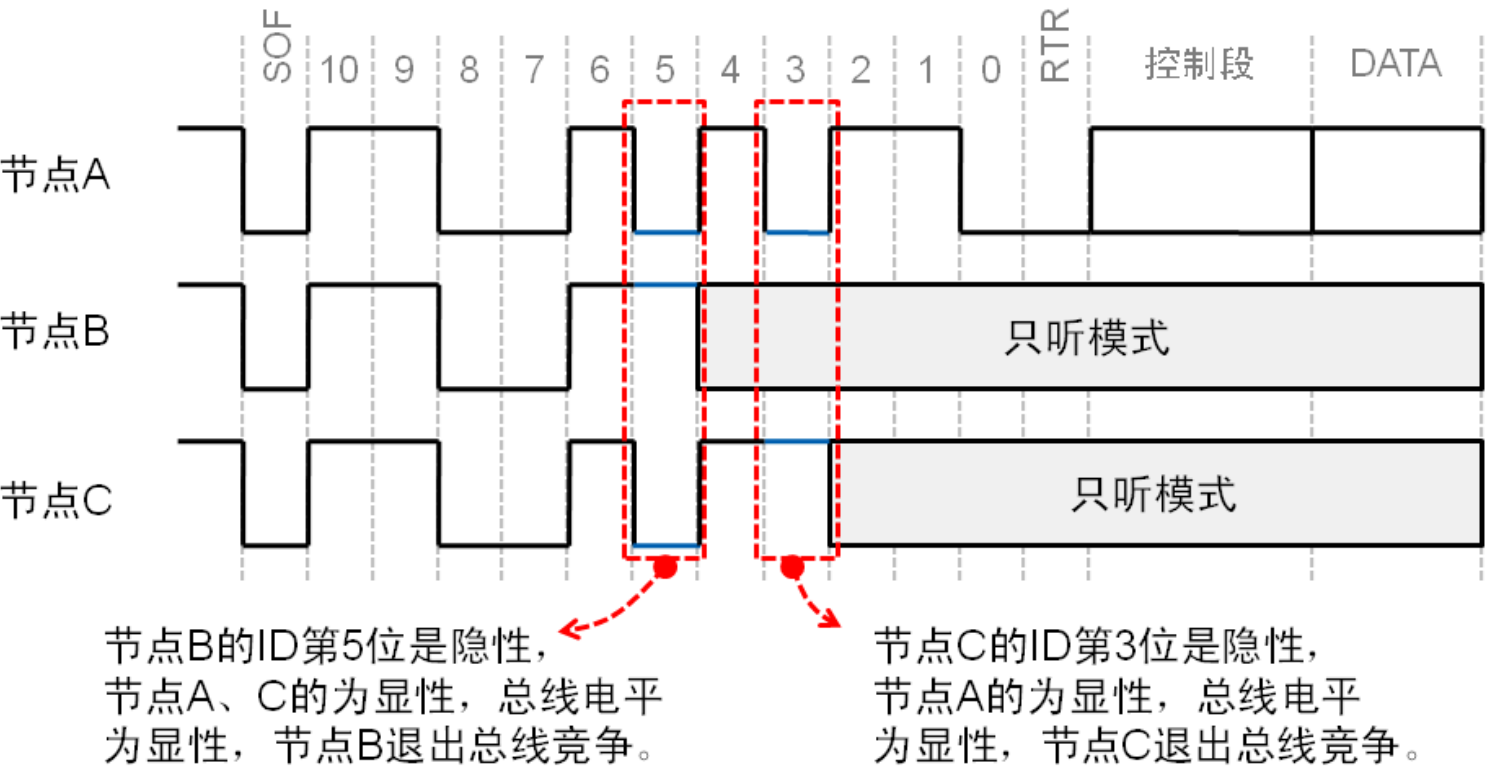
5. 总线仲裁

CAN控制器在发送数据的同时监测数据线的电平是否与发送数据对应电平相同，如果不同，则停止发送并做其他处理。



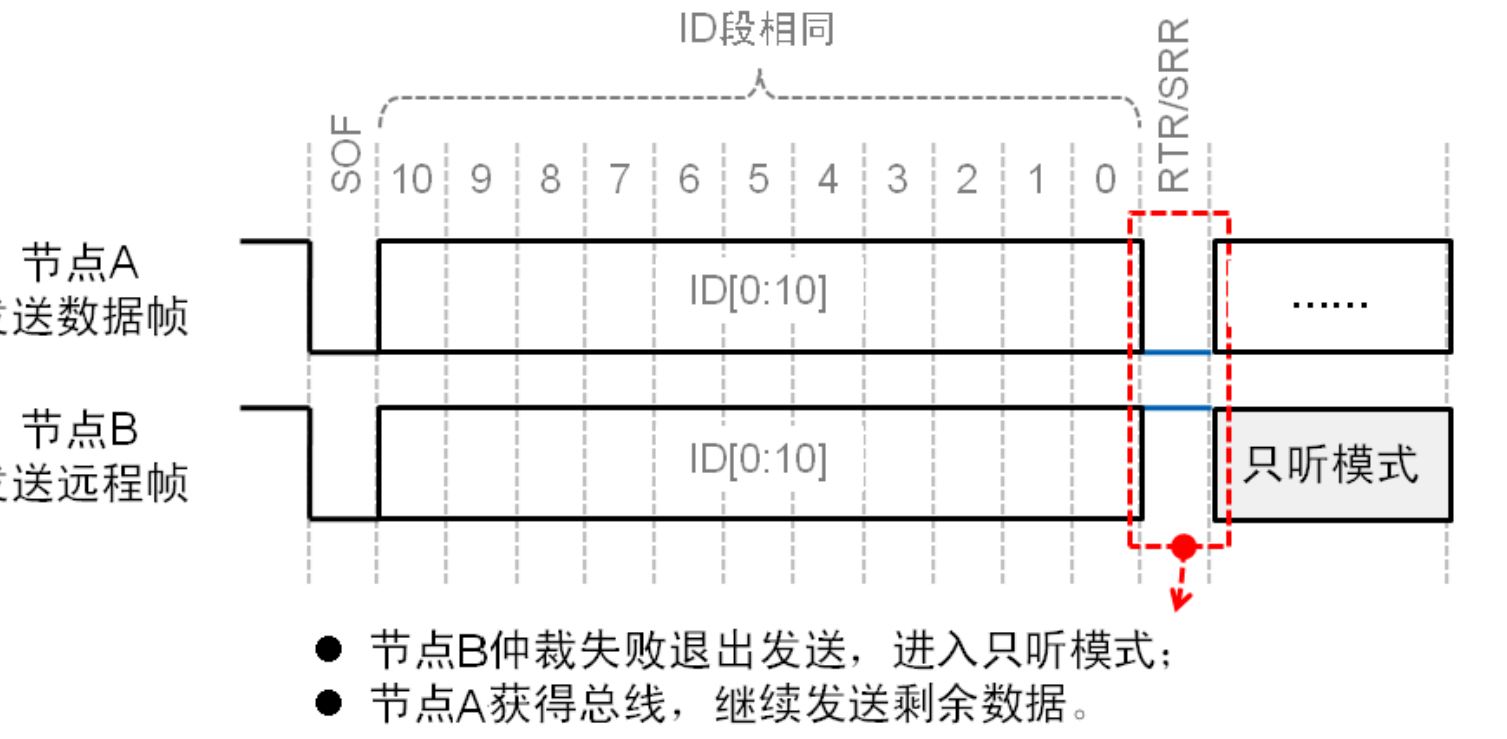
- 如果该位属于仲裁段，则退出总线竞争；
- 如果处于其他段，则产生错误事件（帧ACK时间段或被动错误标志传输期间除外）。

假设节点A、B和C都发送相同格式相同类型的帧，如标准格式数据帧，它们竞争总线的过程是：

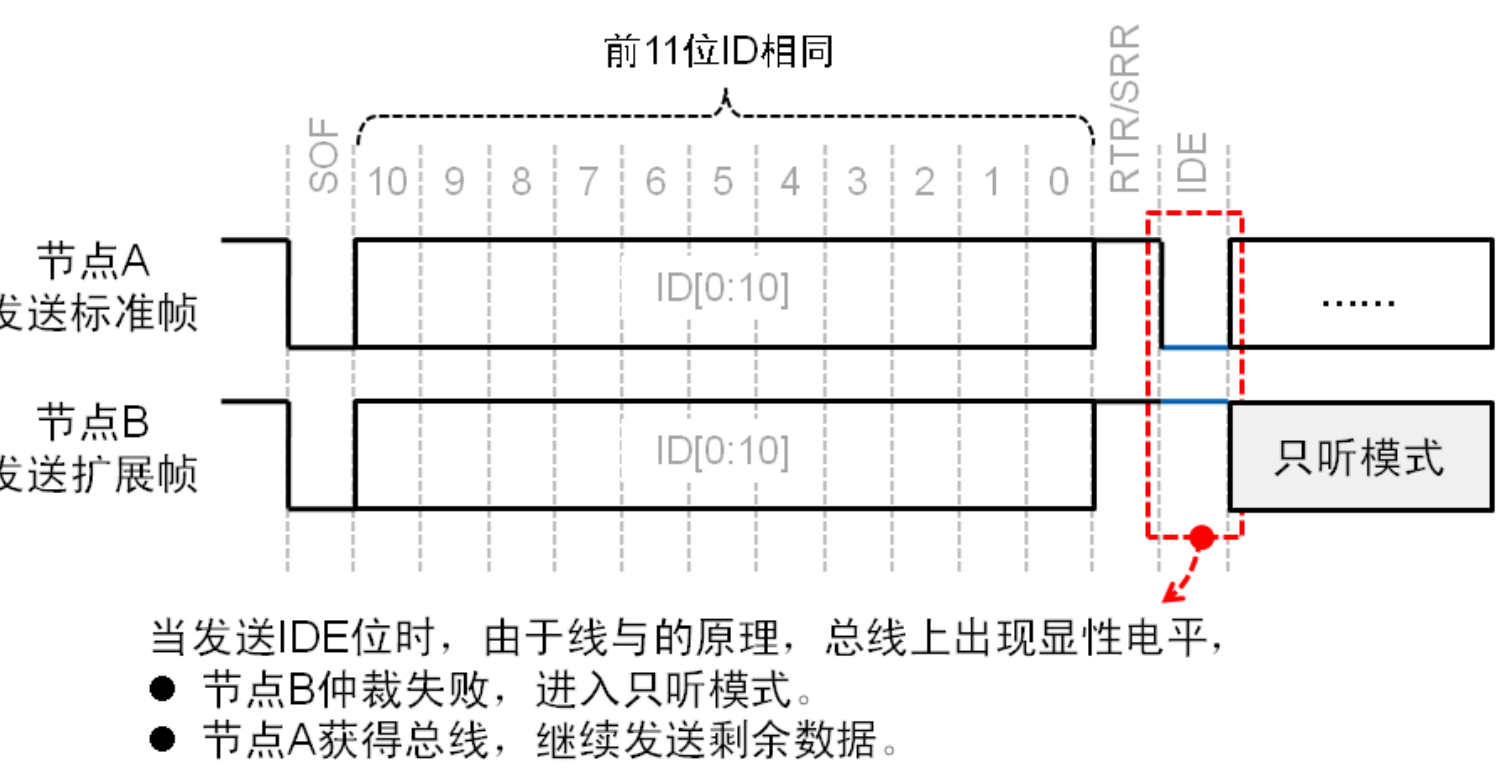


- 从该分析过程得出结论是：帧ID值越小，优先级越高
- 对于同为扩展格式数据帧、标准格式远程帧和扩展格式远程帧的情况同理。

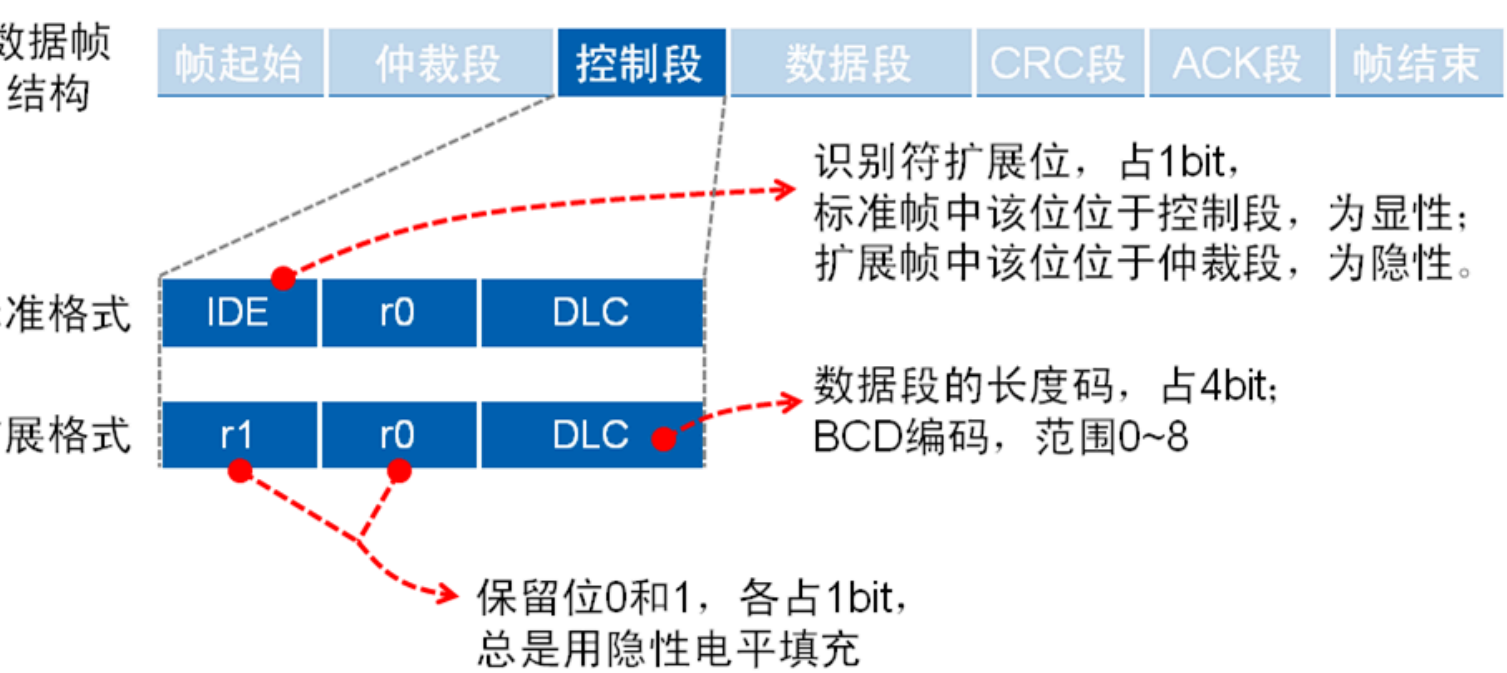
由于数据帧的RTR位为显性电平，远程帧的RTR位为隐性电平，所以帧格式和帧ID都相同情况下，数据帧的优先级比远程帧优先级高：



由于标准帧的IDE位为显性电平，扩展帧的IDE位为隐性电平，对于前11位ID相同的标准帧和扩展帧，标准帧优先级比扩展帧高。

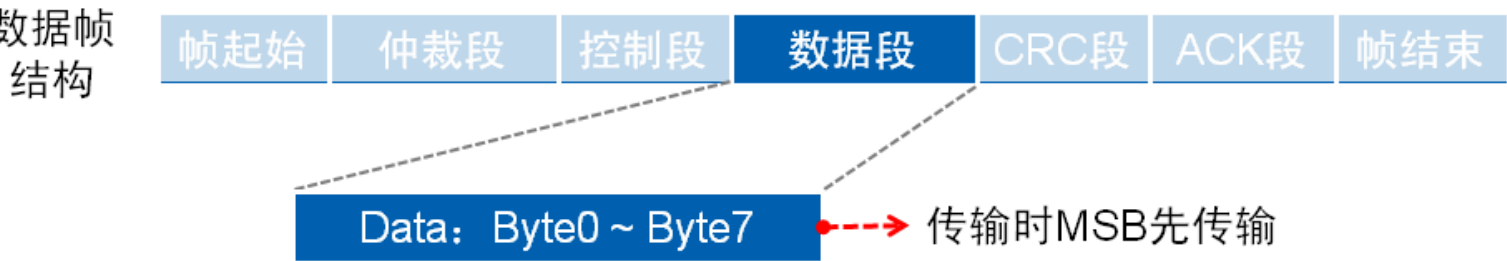


控制段共6位，标准帧的控制段由扩展帧标志位IDE、保留位r0和数据长度代码DLC组成；扩展帧控制段则由IDE、r1、r0和DLC组成。



6. 数据段

一个数据帧传输的数据量为0~8个字节，这种短帧结构使得CAN-bus实时性很高，非常适合汽车和工控应用场合。

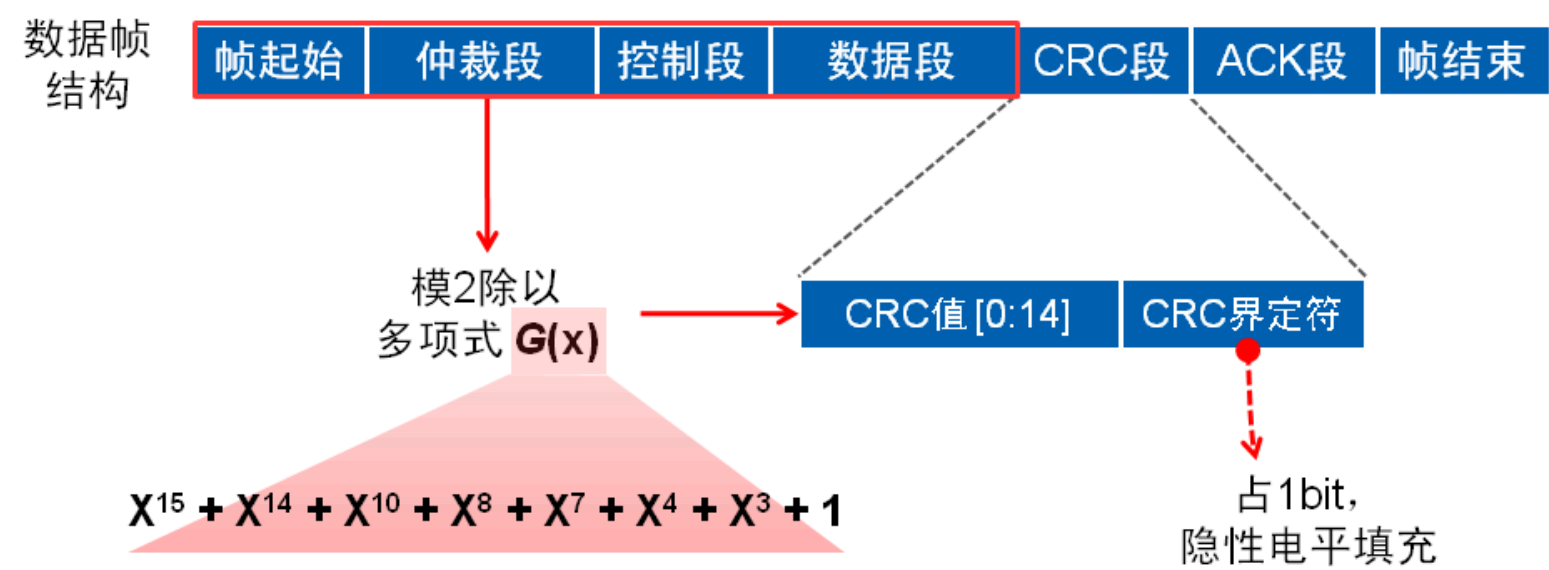


与其他总线协议相比，CAN-bus的短帧结构具有以下优势：

- 优势 1：** 数据量小，发送和接收时间短，实时性高。
- 优势 2：** 数据量小，被干扰的概率小，抗干扰能力强。

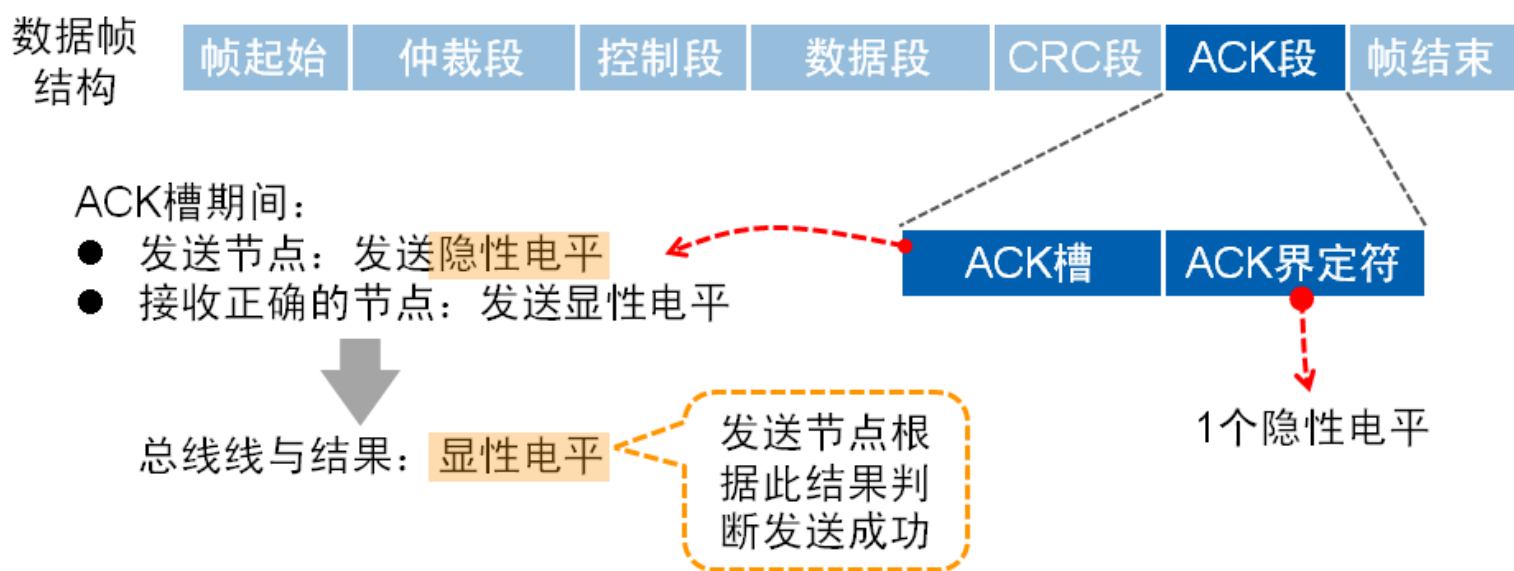
7. CRC段

CAN-bus使用CRC校验进行数据检错，CRC校验值存放于CRC段。CRC校验段由15位CRC值和1位CRC界定符构成。



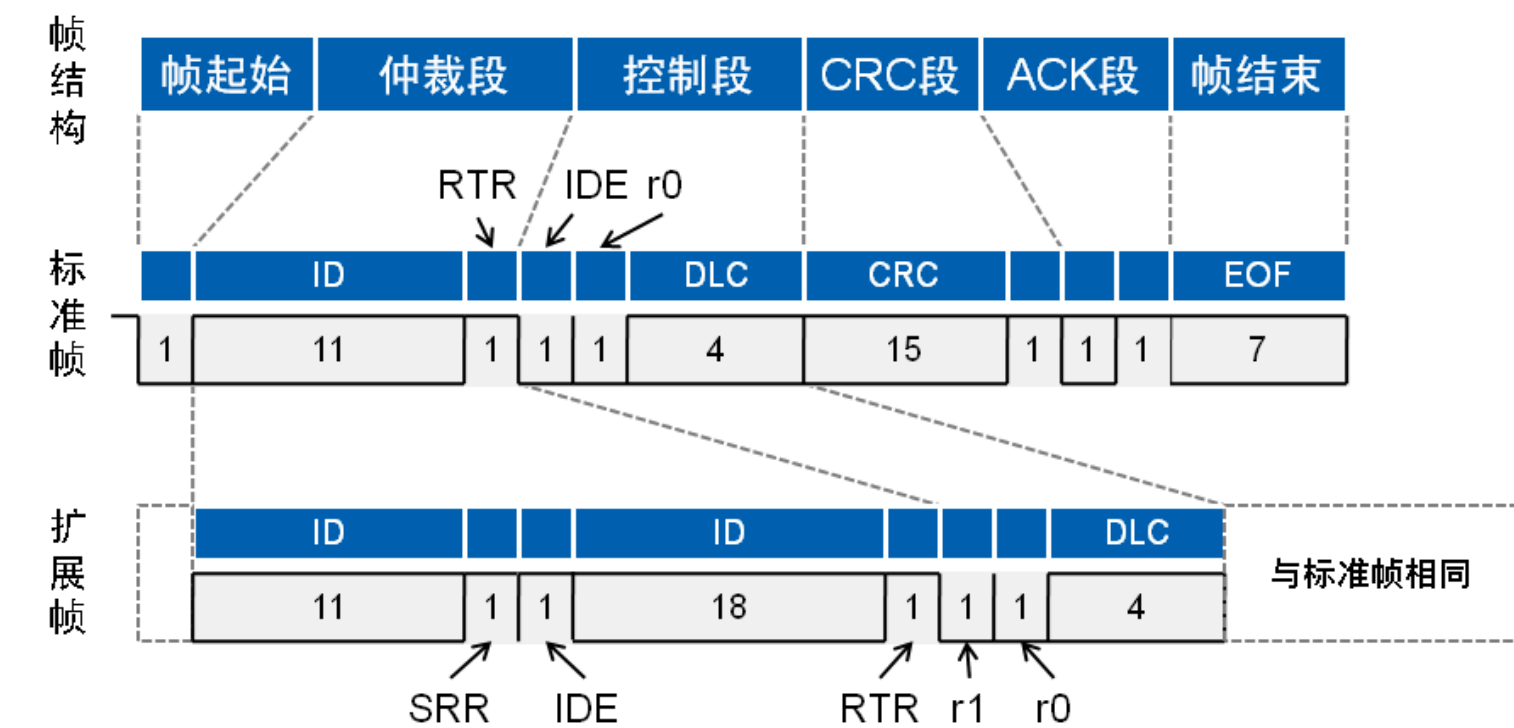
8. ACK段

当一个接收节点接收的帧起始到CRC段之间的内容没发生错误时，它将在ACK段发送一个显性电平。



9. 远程帧

与数据帧相比，远程帧结构上无数据段，由6个段组成，同理分为标准格式和扩展格式，且RTR位为1（隐性电平）。

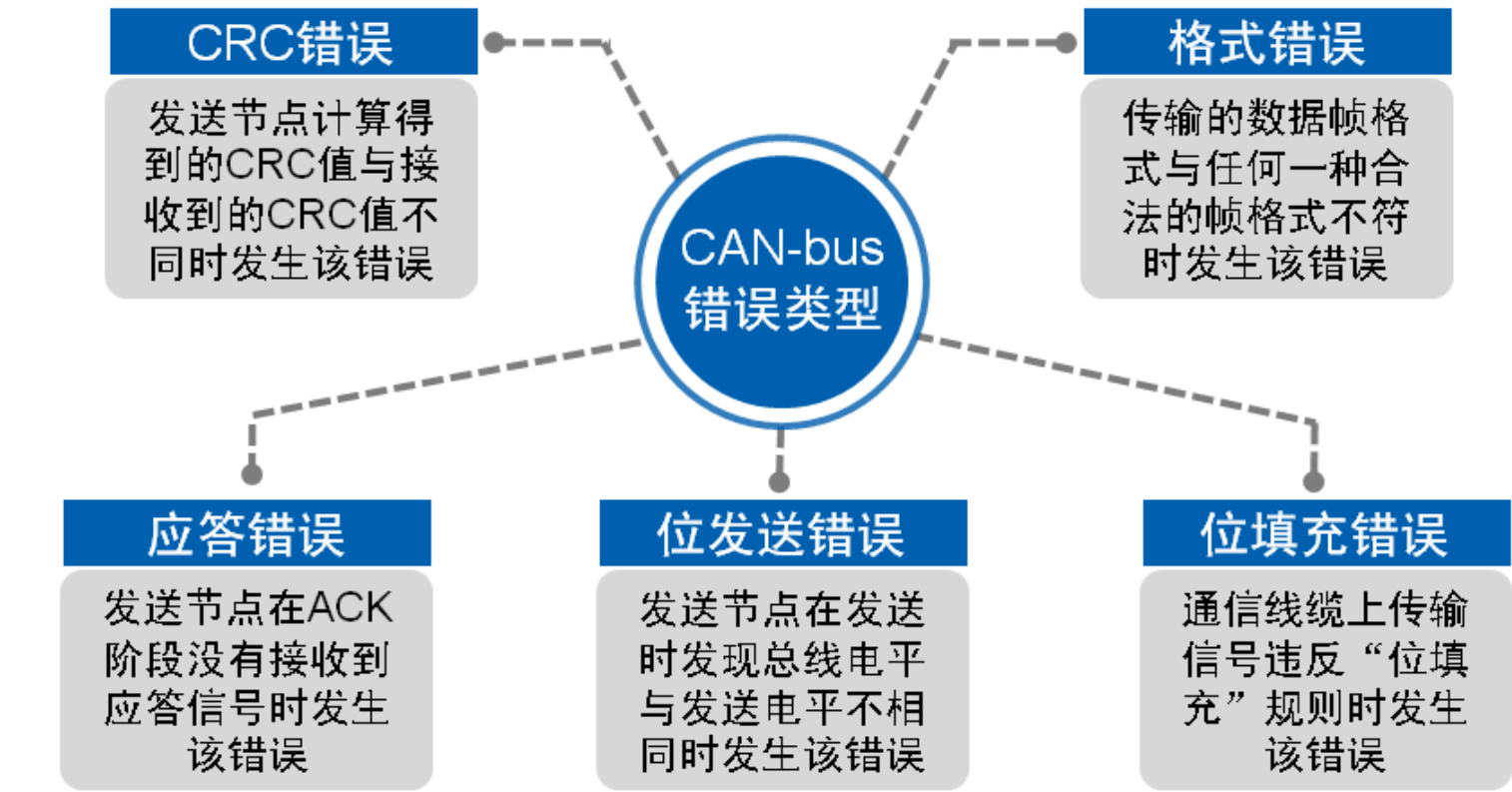


10. 数据帧与远程帧的区别

比较内容	数据帧	远程帧
ID	发送节点的ID	被请求发送节点的ID
SRR	0（显性电平）	1（隐性电平）
RTR	0（显性电平）	1（隐性电平）
DLC	发送数据长度	请求的数据长度
是否有数据段	是	否
CRC校验范围	帧起始 + 仲裁段 + 控制段 + 数据段	帧起始 + 仲裁段 + 控制段

11. CAN-bus 错误类型

尽管CAN-bus是可靠性很高的总线，但依然可能出现错误；CAN-bus的错误类型共有5种。



名称	波特率	规格	适用领域
SAE J1939-11	250k	双线式、屏蔽双绞线	卡车、大客车
SAE J1939-12	250k	双线式、屏蔽双绞线、12V 供电	农用机械
SAE J2284	500k	双线式、双绞线（非屏蔽）	汽车 （高速：动力、传动系统）
SAE J24111	33.3k、83.3k	单线式	汽车 （低速：车身系统）
NMEA-2000	62.5k、125k、250k、500k、1M	双线式、屏蔽双绞线 供电	船舶
DeviceNet	125k、250k、500k	双线式、屏蔽双绞线 24V 供电	工业设备
CANopen	10k、20k、50k、125k、250k、500k、800k、1M	双线式、双绞线 可选（屏蔽、供电）	工业设备
SDS	125k、250k、500k、1M	双线式、屏蔽双绞线 可选（供电）	工业设备

1. 什么是CANopen

OSI vs TCP/IP				
OSI七层网络模型	TCP/IP四层概念模型	对应网络协议	对应的典型设备	区域
应用层（Application）	应用层	TFTP、FTP、NFS、WAIS	应用程序，如FTP，SMTP，HTTP	计算机
表示层(Presentation)		Telnet、Rlogin、SNMP、Gopher	编码方式，图像编解码、URL字段传输编码	
会话层(Session)		SMTP、DNS	建立会话，SESSION认证、断点续传	
传输层(Transport)	传输层	TCP、UDP	进程和端口	网络
网络层（Network）	网际层	IP、ICMP、ARP、RARP、AKP、UUCP	路由器，防火墙、多层交换机	
数据链路层（Data Link）	网络接口	FDDI、Ethernet、Arpanet、PDN、SLIP、PPP	网卡，网桥，交换机	
物理层（Physical）		IEEE 802. 1A、IEEE802. 2到IEEE802. 11	中继器，集线器、网线	

CAN

CAN（Controllor Area Network）现场总线仅仅定义了OSI 7 层网络模型的第 1 层（物理层，见 ISO11898-2 标准）、第 2 层（数据链路层，见 ISO11898-1 标准）。

CiA

CiA（CAN in AUTOMATION）协会是1992年，在德国 Erlangen注册的有关推广CAN和CAN协议的非营利组织。

CAL

CAL（CAN Application Layer）协议是目前基于CAN的高层通讯协议中的一种，最早由飞利浦(Philips)公司医疗设备部门制定。

现在CAL由CiA 协会负责管理、 发展和推广。

CANopen

CANopen协议，是20 世纪 90 年代末，由CiA 协会（<http://www.can-cia.org>）在 CAL（CAN Application Layer）的基础上制定的一种架构在控制器局域网络（Controller Area Network, CAN）上的高层通讯协议标准。

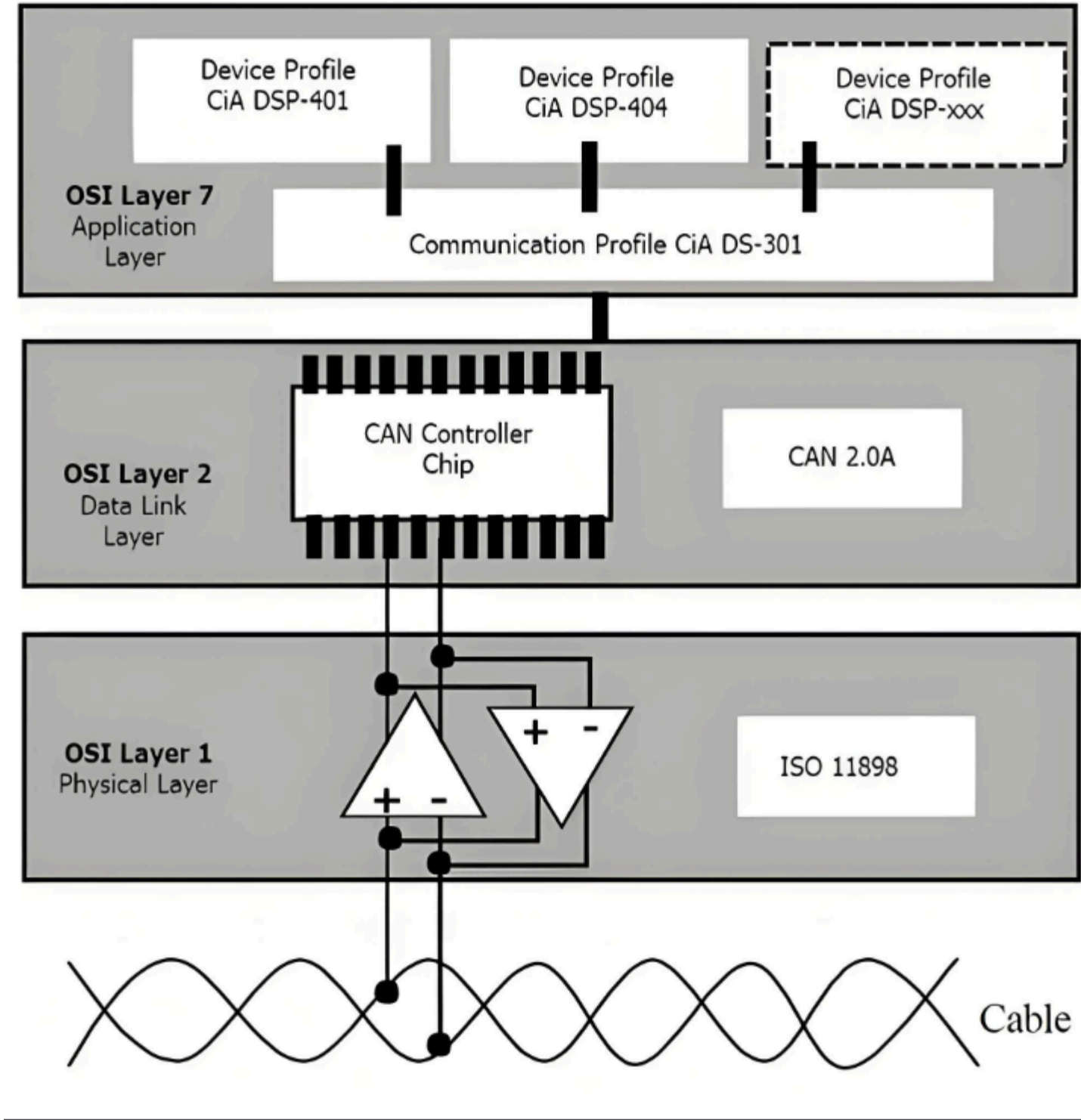
CANopen协议制定了相当于 OSI 模型 中第五层（会话层）、第六层（表示层）和第七层（应用层）的技术规范。

CAL提供了所有的网络管理服务和报文传送协议，但并没有定义通讯对象的内容或者正在通讯的对象的类型（它只定义了how，没有定义what）。而这正是CANopen切入点。

CANopen是在CAL基础上开发的，使用CAL通讯和服务协议子集，提供了分布式控制系统的一种实现方案。

CANopen协议是免许可证的，任何组织和个人都可以开发支持CANopen协议的设备而不用支付版税。

2. CANopen网络模型



3. CANopen常见协议规范

CANopen协议是CiA定义的标准之一，在欧洲，CANopen协议被认为是在基于CAN的工业系统中占领导地位的标准。

CANopen 是一种成熟的工业通信协议，拥有多种规范和标准来指导其在实际应用中的实现。

以下是一些常见的 CANopen 规范：

- **CiA 301: Application Layer and Communication Profile**，这是 **CANopen** 的核心规范，定义了应用层和通信概况，涵盖了：
 - **网络管理（NMT）**：启动、停止、和重置网络节点。
 - **同步（SYNC）**：周期性同步消息，用于时间同步。
 - **过程数据对象（PDO）**：用于实时数据交换。
 - **服务数据对象（SDO）**：用于访问和配置对象字典中的数据。
 - **对象字典（OD）**：定义设备的所有功能和参数。
 - **心跳和节点 guarding**：用于监控节点的状态和健康。

- CiA 302: Motion Control，针对驱动器和运动控制设备的规范。

- CiA 303: Generic I/O Modules，针对通用 I/O 模块的规范

- CiA 304: Encoders，针对编码器的规范。

- CiA 305: Safety，针对安全相关应用的规范。

- CiA 306: Integrated Safety 针对驱动器和安全功能的规范。

- CiA 307: Inverter Drives，针对逆变器驱动器的规范。

- CiA 308:Electric Drives，针对电动驱动设备的规范，

- CiA 401 series，针对通用 I/O 模块的规范

- CiA 402 series，针对运动设备（包括伺服驱动器、逆变器、步进电机驱动器等）

- CiA 421 series: Train vehicle control system 列车车辆控制系统

- CiA 423 series: Diesel engine control system 柴油机控制系统

- CiA 424 series: Door control system 门控制系统

- CiA 426 series: Exterior light control system 外部灯控制系统

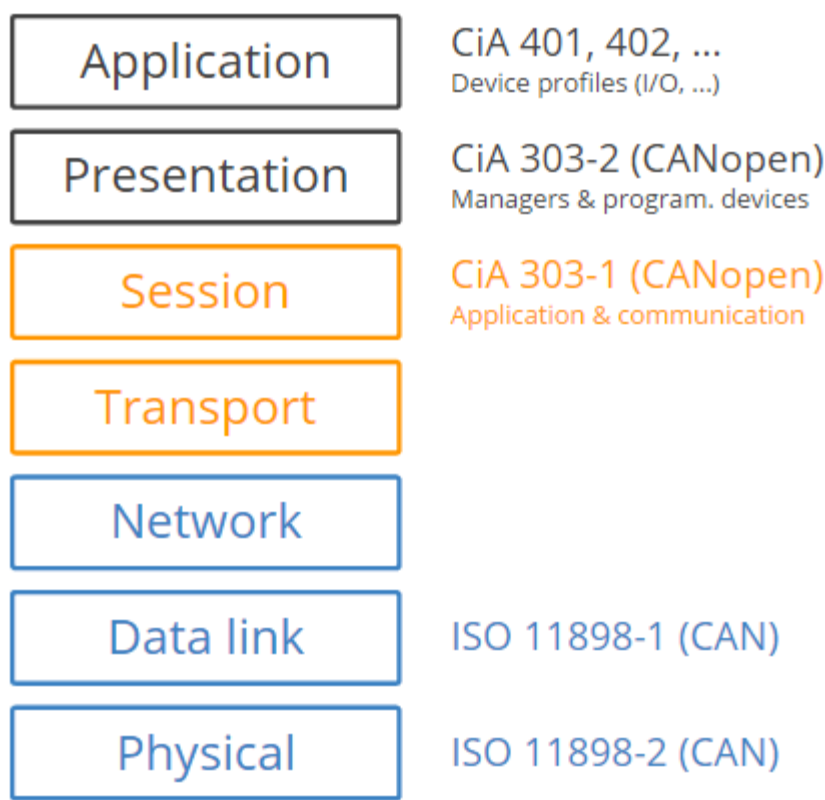
- CiA 430 series: Auxiliary equipment control system 辅助设备控制系统

- CiA 433 series: Interior light control system 内部灯控制系统

这些规范使得 CANopen 协议能够在各种工业和嵌入式系统中高效地运作，确保设备之间的互操作性和通信的一致性。不同的规范适用于不同类型的设备 and 应用，提供了灵活的解决方案来满足各种需求。

4. 各种协议之间的关系

7 layer OSI model



由图可知，我们开发的时，一般选择4系列的子协议，因为子协议在应用层，是最终的产品协议。

3.4 CANopen对象字典

更加详细的协议介绍请看 STM32F103开发板资料V9.0\03-芯片手册\CANopen_easy_begin.pdf。

1. 对象字典理解

CANopen对象字典（OD: Object Dictionary）是CANopen协议最为核心的概念。

对象字典就是一个有序的对象组，描述了对应CANopen节点的所有参数。

最简单的理解就是json字符串

```
{
  "name": "JSON中国",
  "url": "http://www.json.org.cn",
  "page": 88,
  "isNonProfit": true,
  "address": {
    "street": "浙大路38号.",
    "city": "浙江杭州",
    "country": "中国"
  },
  "links": [
    {
      "name": "Google",
      "url": "http://www.google.com"
    },
    {
      "name": "Baidu",
      "url": "http://www.baidu.com"
    }
  ]
}
```

对象字典就像体检表，具备这个人每个功能的参数，便于用人单位（主站）进行合理分配工作。如图所示。

姓 名		性 别		女		年 龄		30		部 门			
血 压		98/66mmHg（正常<140/90）				体重指数（体重÷身高 ² ）		19（正常值18-25）					
一般查体			正常			眼 科		轻度干眼症					
检查项目		测定值		正常值		口 腔		牙结石					
尿 液 分 析	PH 值(PH)		7		5--7		尿 液 分 析	蛋白质(PRO)		-		（-）	
	亚硝酸盐(NIT)		-		（-）			胆红素(BIL)		-		（-）	
	葡萄糖(GLU)		-		（-）			尿胆原(URO)		-		空白或±	
	尿比重(SG)		1.015		1.0-1.2			酮体(KET)		-		（-）	
	隐血(BLD)		-		（-）			白细胞(WBC)		-		（-）	
血 液 检 查	项 目		测定值		正常值		血 液 检 查	项 目		测定值		正常值	
	血 糖		4.44		3.89-6.10mmol/L			总蛋白		67.3		60-85g/L	
	甘油三脂		0.33		0.23-1.78mmol/L			白蛋白		43.2		35-55g/L	
	总胆固醇		2.83		2.8-5.85mmol/L			转氨酶 AST		23.7		0-40 U/L	
	尿素氮		3.11		1.81-7.14mmol/L			总胆红素		8.5		6.0-23umol/L	
	肌 酐		71.6		44-133umol/L			直接胆红素		2.1		1.2-6.0umol/L	
	尿 酸		211.1		160-400umol/L			甲胎蛋白		-		（-）	
参 数		结 果		正常值		参 数		结 果		正常值			
白细胞数目		6×10 ⁹ /L		4.0-10.0		平均红细胞体积		87.9 FL		80.0-97.0			
淋巴细胞数目		1.9×10 ⁹ /L		0.6-4.1		平均血红蛋白含量		29.3 Pg		26.5-33.5			

每个对象采用一个16位的索引值来寻址，这个索引值通常被称为索引，其范围在0x0000到0xFFFF之间。

为了避免数据大量时无索引可分配，在某些索引下定义了一个8 位的索引值，这个索引值通常被称为子索引，其范围是0x00到0xFF之间。

每个索引内具体的参数，最大用32位的变量来表示，即Unsigned32，四个字节。

Index range索引范围	Description描述
0000h	Reserved保留
0001h to 025Fh	Data types数据类型
0260h to 0FFFh	Reserved保留
1000h to 1FFFh	Communication profile area通讯对象子协议区
2000h to 5FFFh	Manufacturer-specific profile area制造商特定子协议区
6000h to 9FFFh	Standardized profile area标准化设备子协议区
A000h to AFFFh	Network variables网络变量（符合IEC61131-3）
B000h to BFFFh	System variables用于路由网关的系统变量

5. 制造商特定子协议

制造商特定子协议(Manufacturer-specific Profile)

Index range索引范围	Description描述
0000h	Reserved保留
0001h to 025Fh	Data types数据类型
0260h to 0FFFh	Reserved保留
1000h to 1FFFh	Communication profile area通讯对象子协议区
2000h to 5FFFh	Manufacturer-specific profile area制造商特定子协议区
6000h to 9FFFh	Standardized profile area标准化设备子协议区
A000h to AFFFh	Network variables网络变量（符合IEC61131-3）
B000h to BFFFh	System variables用于路由网关的系统变量
C000h to FFFFh	Reserved保留

对象字典索引2000h to 5FFFh为制造商特定子协议，通常是存放所应用子协议的应用数据。

而上文所描述的通讯对象子协议区(Communication profile area)是存放这些应用数据的通信参数。

6. 标准化设备子协议

标准化设备子协议(Standardized profile area)

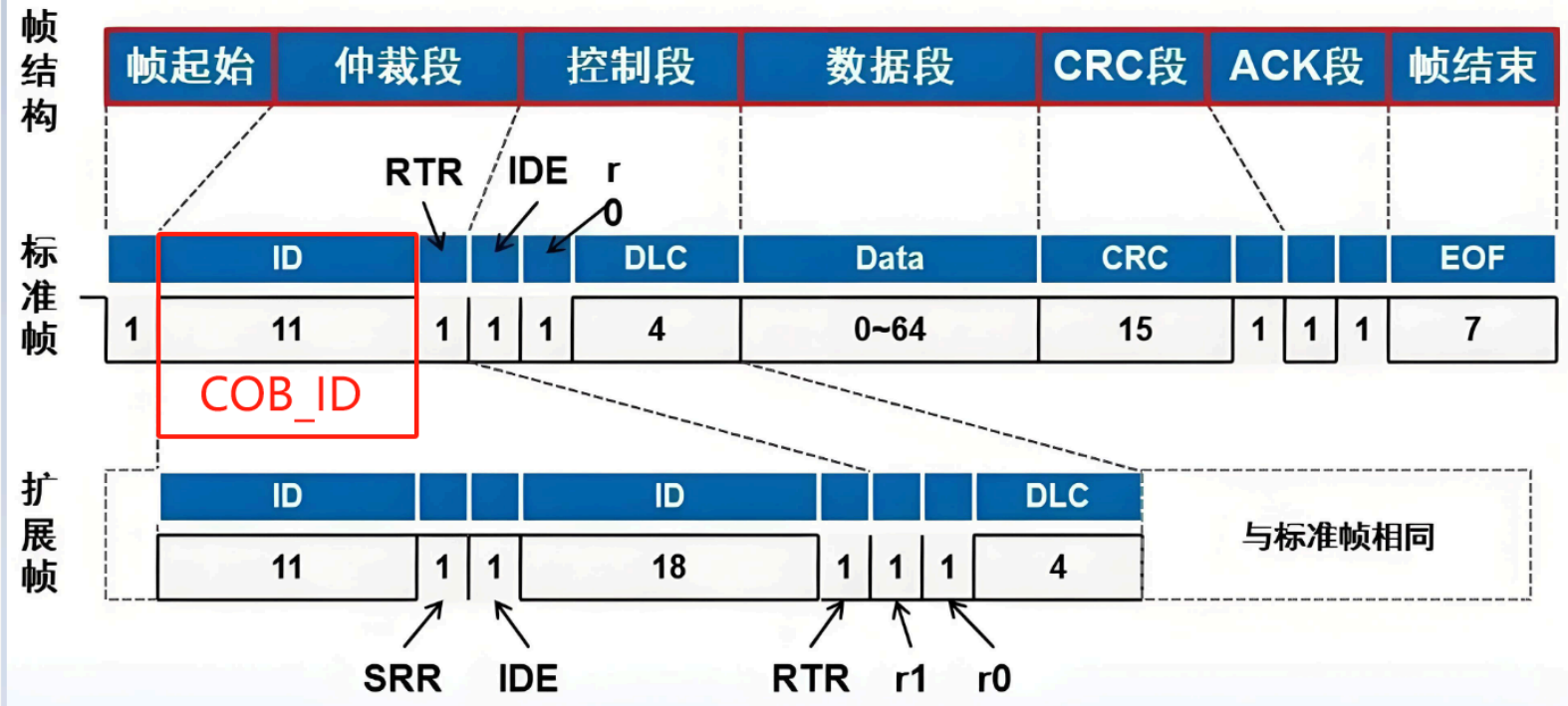
标准化设备子协议，为各种行业不同类型的标准设备定义对象字典中的对象。目前已有十几种为不同类型的设备定义的子协议，例如DS401、DS402、DS406等，其索引值范围为0x6000~0x9FFF。

同样，这个区域对于不同的标准化设备子协议来说，相同的对象字典项其定义不一定相同。

- 数字量和模拟量输入/输出模块(DS401)
- 电机(DS402)
- 控制设备(DS4P403)
- 闭环控制器(DSP404)
- PLC(DS405) (CANopen 和 IEC61131)的全部功能
- 编码器(DS406)

3.5 CANopen协议核心概念

1. CANopen通讯标识符



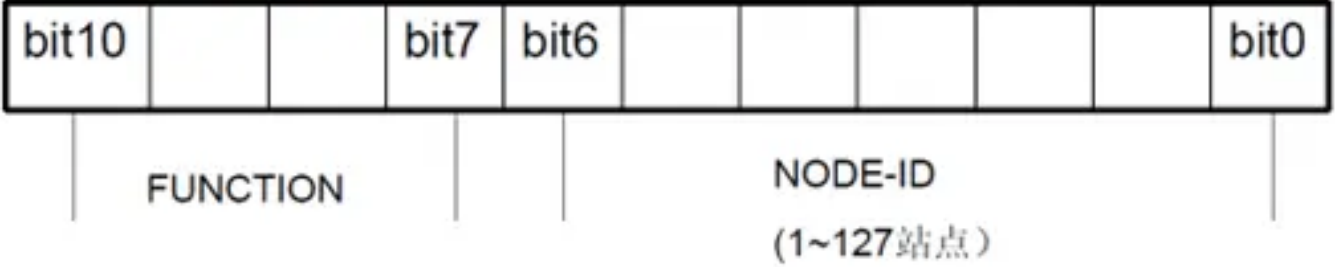
COB_ID位于上图的ID位置， 是11位的ID码。

Communication Object Identifier (COB_ID)即通讯标识符。每个CANopen帧都以COB-ID开头，COB-ID是数据帧的唯一标识符。

COB_ID包括功能段（FUNCTION）和地址段（NODE-ID）。

Node-ID由设备厂家定义，例如通过拨码开关设置。Node-ID范围是1~127（0不允许被使用）。

COB_ID越小报文优先级越高，CANopen的COB_ID范围从0-77F。



COB_ID		DLC	Data							
0	1		0	1	2	3	4	5	6	7
XX	XX	8	XX	XX	XX	XX	XX	XX	XX	XX

其中，DLC为数据包内的有效字节数，范围0~8。Data为数据包（0~8字节数据）。

2. 网络管理对象（NMT）

网络管理对象-Network Management Objects（NMT）

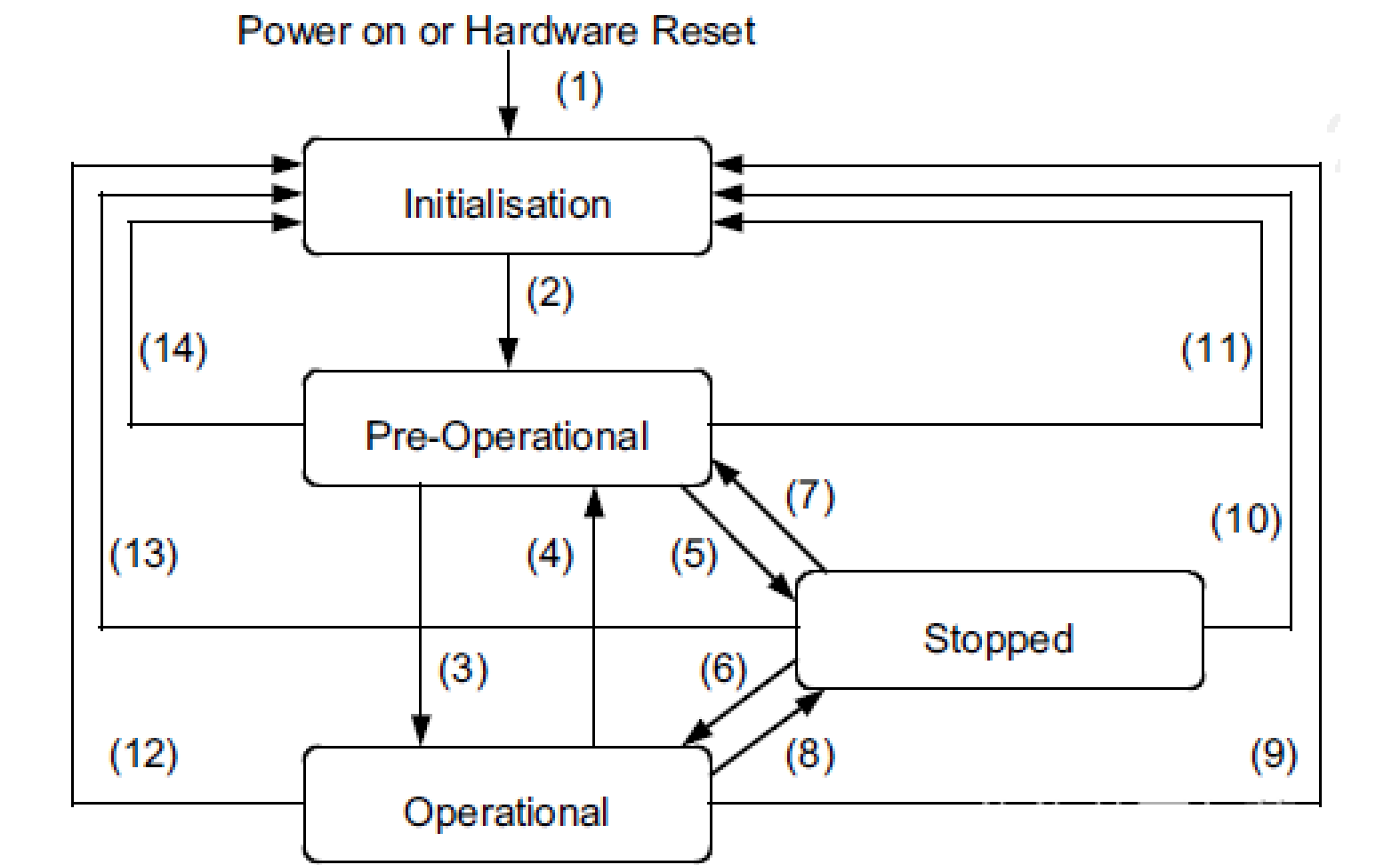
NMT到底是什么，简单理解就是由一台主机来管理从机，主机可以控制从机的状态。

通过NMT对象执行NMT服务，通过NMT服务，可以初始化、启动、监控或复位CAN设备节点。

NMT遵循主从结构。



NMT状态切换



序号	描述
1	上电后自动进入初始化模式
2	初始化后自动进入预操作状态
3	由NMT服务启动远程节点或者由设备本身控制启动节点
4、7	由NMT服务控制进入预操作状态

PDO 和 SDO 的通讯区别在于，**PDO 属于过程数据**，即单向传输，无需接收节点回应 **CAN 报文**来确认，从通讯术语上来说是属于“生产消费”模型。如图 4.4 所示。



图 4.4 生产消费模型

而 **SDO 属于服务数据**，有指定被接收节点的地址（**Node-ID**），并且需要指定的接收节点回应 **CAN 报文**来确认已经接收，如果超时没有确认，则发送节点将会重新发送原报文。这种通讯方式属于常见的“服务器客户端”的通信模型，即我们通常所说的轮询式。如图 4.5 所示。

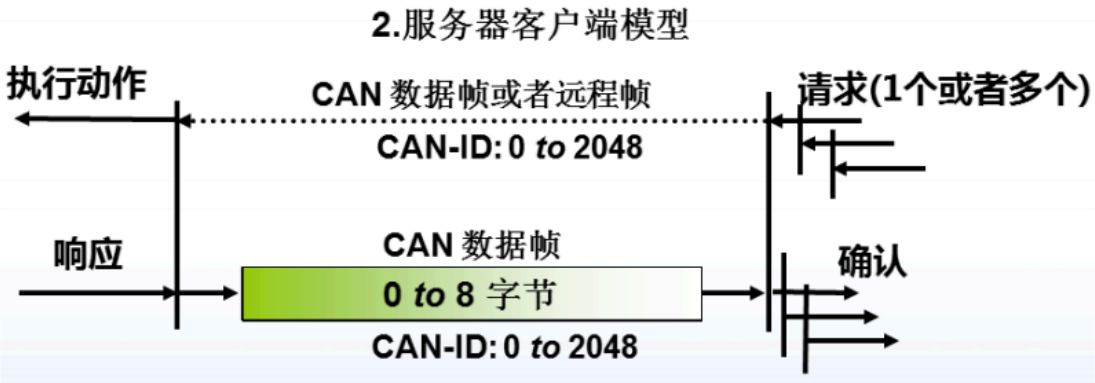


图 4.5 服务器客户端模型

总结对比

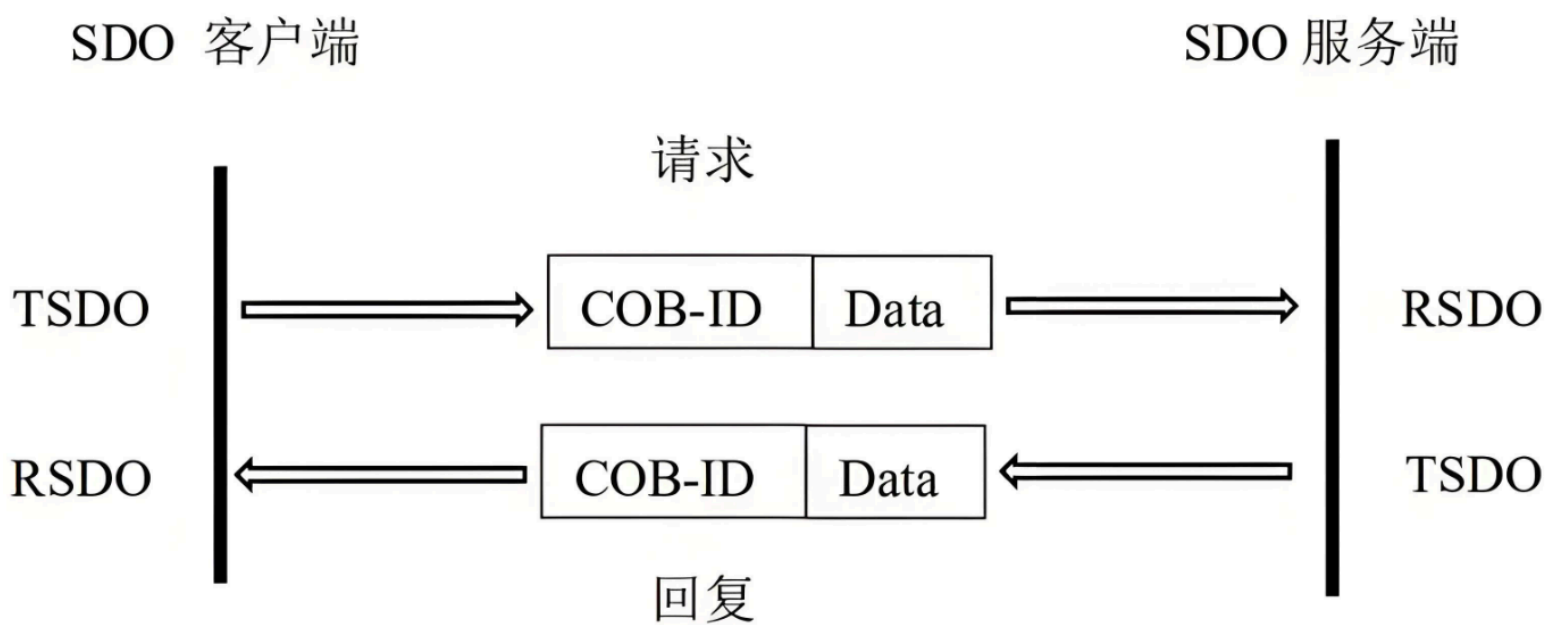
- **SDO**: 适用于配置和详细的数据访问，需要通过明确的请求和应答进行交互。就像在图书馆系统中逐一操作书籍的借阅和归还。
- **PDO**: 适用于实时数据传输，自动化且周期性地传输数据。就像在生产线上实时监控数据，以便即时处理和响应。

4. 服务数据对象（SDO）

SDO(Service Data Object)服务数据对象，用来传输非时间关键数据。服务确认是 SDO 的最大的特点，为每个消息都生成一个应答，确保数据传输的准确性。

SDO 属于服务数据，有指定被接收节点的地址（Node-ID），并且需要指定的接收节点回应 CAN 报文来确认已经接收，如果超时没有确认，则发送节点将会重新发送原报文。

这种通讯方式属于常见的“服务器客户端”的通信模型，即我们通常所说的轮询式或问答式。



快速SDO

快速SDO用于读写不大于四个字节的数​​据，一次交互就搞定。

快速SDO写报文格式

		COB-ID	0	1	2	3	4	5	6	7
客户端		600h+ nodeID	23h	索引		子索引	数据			
			27h				数据			
			2bh				数据			
			2fh				数据			
服务器	正常	580h+ nodeID	60h	索引		子索引				
	异常		80h				错误代码			

快速SDO读报文格式

		COB-ID	0	1	2	3	4	5	6	7
客户端		600h+ nodeID	40h	索引		子索引				
服务器	正常	580h+ nodeID	43h	索引		子索引	数据			
			47h				数据			
			4bh				数据			
			4fh				数据			
	异常		80h				错误代码			

普通SDO

当传输的数据超过四字节时，就不能使用快速SDO传输，必须使用普通SDO进行传输，这种应用在实际中很少，至少目前我还没有用过。

比较繁琐而且使用普通SDO读写数据实用性也不太强，使用较少。

SDO读取大于四字节对象起始帧格式

		COB-ID	0	1	2	3	4	5	6	7
客户端		600h+ nodeID	40h	索引		子索引	-	-	-	-
服务器	正常	580h+ nodeID	41h	索引		子索引	数据长度			
	异常		80h				终止代码			

SDO读取大于四字节对象传输过程报文格式

		COB-ID	0	1	2	3	4	5	6	7
客户端		600h+ nodeID	60h	-	-	-	-	-	-	-
服务器	正常	580h+ nodeID	00h	数据长度						
	异常		80h	索引		子索引	终止代码			
客户端		600h+ nodeID	70h	-	-	-	-	-	-	-
服务器	正常	580h+ nodeID	10h	数据长度						
	异常		80h	索引		子索引	终止代码			

5. 过程数据对象（PDO）

PDO(Process Data Object)过程数据对象，8字节全部用来传输实时数据，提供对设备应用对象的直接访问通道，它用来传输实时短帧数据，具有较高的优先权。

PDO 传输的数据必须少于或等于 8 个字节，PDO 的 CAN 报文数据域中每个字节都用作数据传输，因此，在应用层上不包含传输控制信息，报文利用率极高。

PDO 通信是基于生产者/消费者的通讯模式，如图所示，每个 PDO 有一个唯一的标识符且可以通过一个节点发送，但有多个节点可以接收。由生产者发送的 PDO称为发送 PDO(TPDO)，同样消费者接收的 PDO 称为接收 PDO(即 RPDO)，PDO 的接收不需要消费者的确认。

PDO 和 SDO 的通讯区别在于，PDO 属于过程数据，即单向传输，无需接收节点回应 CAN 报文来确认，从通讯术语上来说是属于“生产消费”模型。如图 4.4 所示。



图 4.4 生产消费模型

PDO对象

按照接收和发送的不同，PDO可分为RPDO和TPDO。PDO由通信参数和映射参数共同决定最终传输的方式及内容。

一般设备可能默认使用4个RPDO和4个TPDO来实现PDO的传输，对象列表如下。

名称		COB-ID	通信对象	映射对象
RPDO（以节点自身为参考）	1	200h+nodeID	1400h	1600h
	2	300h+nodeID	1401h	1601h
	3	400h+nodeID	1402h	1602h
	4	500h+nodeID	1403h	1603h
TPDO（以节点自身为参考）	1	180h+nodeID	1800h	1A00h
	2	280h+nodeID	1801h	1A01h
	3	380h+nodeID	1802h	1A02h
	4	480h+nodeID	1803h	1A03h

PDO通讯参数

PDO的通信参数，定义了该pdo使用的COB-ID，传输类型、定时周期等。

- RPDO通讯参数位于对象字典索引的1400h to 15FFh区间。
- TPDO通讯参数位于对象字典索引的1800h to 19FFh区间。

每条索引代表一个PDO的通信参数集，其中的子索引分别指向具体的各种参数。

子指标	名字	类型	值	存取
0x00	Highest SubIndex Supported	UNSIGNED8	6	只读
0x01	COB ID used by PDO	UNSIGNED32	\$NODEID+0x200	读 / 写
0x02	Transmission Type	UNSIGNED8	0x00	读 / 写
0x03	Inhibit Time	UNSIGNED16	0x0000	读 / 写
0x04	Compatibility Entry	UNSIGNED8	0x00	读 / 写
0x05	Event Timer	UNSIGNED16	0x0000	读 / 写
0x06	SYNC start value	UNSIGNED8	0x00	读 / 写

- 子索引0x0：表示该索引中有几条子索引；
- 子索引0x01: COB-ID即这个PDO发出或者接收的对应CAN帧 ID，确定了该PDO的总线优先级；
- 子索引0x02：定义了该条PDO使用哪种传输方式；PDO的传输共有两种传输方式，分别为同步传输和事件传输。
 - RPDO传输类型
 - 为254或255时，会将接收到的数据立即更新到应用程序；
 - 为0-240时，只有接收到下一个同步帧则才会将最新接收到的RPDO数据更新到应用程序。
 - TPDO传输类型
 - 为1-240时，其同时也是TPDO的传输速率基于同步帧的倍频，
 - 为0时，只有映射数据发生改变且收到一个同步帧才会发送。

通讯类型数值	同步		异步
	循环	非循环	
0		√	
1-240	√		
241-253	reserved		
254、255（event-driven）			√

- 子索引0x03：定义了禁止时间用于约束同一条PDO的发送间隔，避免总线被同一条优先级较高的PDO长时间占领。
- 子索引0x04：我看DS301里面没有规定，不说了。
- 子索引0x05：定义了事件计时器，单位为ms。值为0时表示禁止事件定时器。
- 子索引0x06：定义了同步帧起始值，这个只针对TPDO。

PDO映射参数

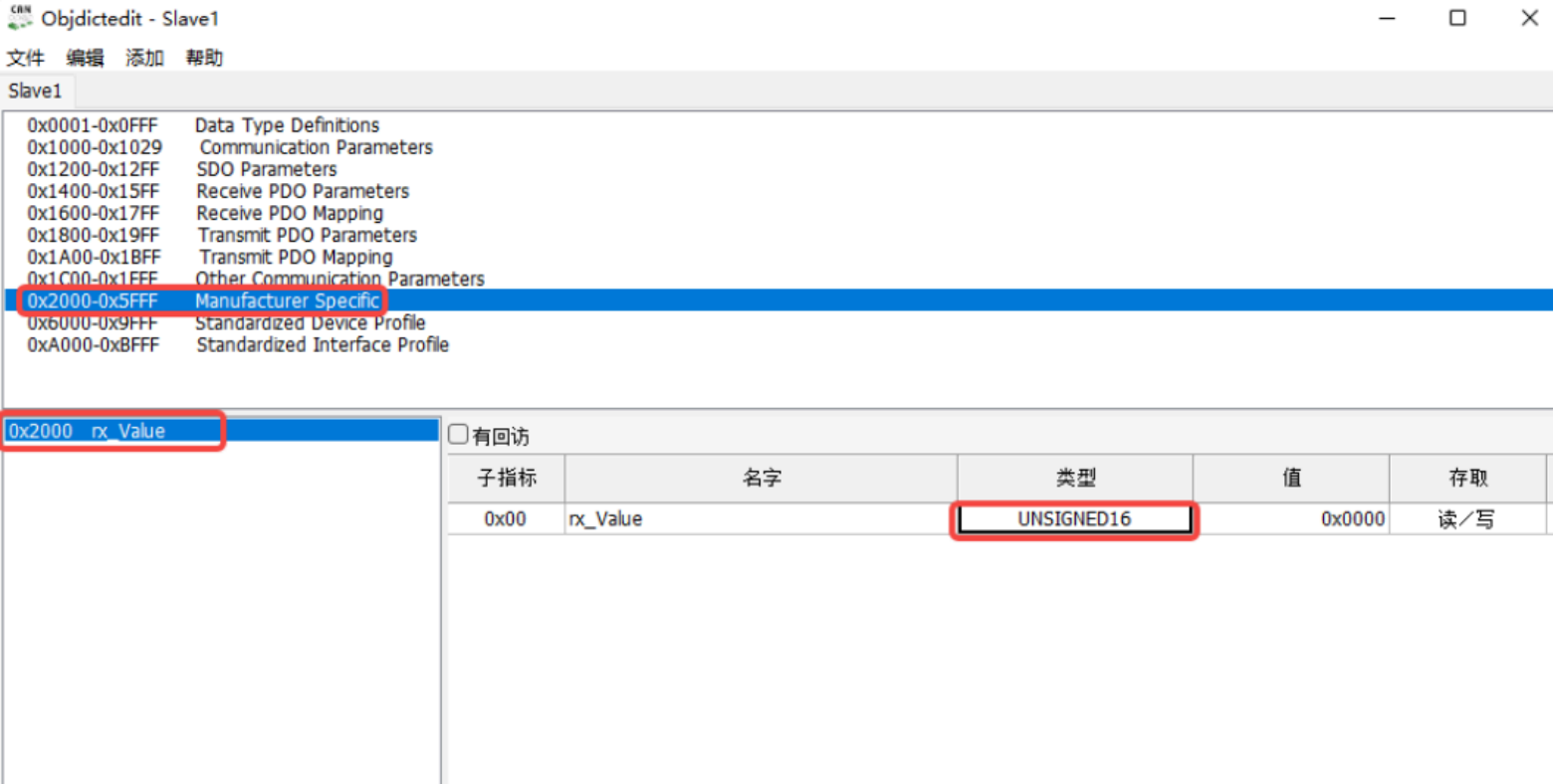
每个PDO数据长度最多可达8个字节，可同时映射一个或多个对象。PDO的映射参数就是指该条PDO（RPDO或TPDO）所包含的对象的个数及每个对象的索引、子索引及映射对象长度信息。

子指标	名字	类型	值	存取
0x00	Number of Entries	UNSIGNED8	8	读 / 写
0x01	PDO 1 Mapping for an application object 1	UNSIGNED32	毫无	读 / 写
0x02	PDO 1 Mapping for an application object 2	UNSIGNED32	毫无	读 / 写
0x03	PDO 1 Mapping for an application object 3	UNSIGNED32	毫无	读 / 写
0x04	PDO 1 Mapping for an application object 4	UNSIGNED32	毫无	读 / 写
0x05	PDO 1 Mapping for an application object 5	UNSIGNED32	毫无	读 / 写
0x06	PDO 1 Mapping for an application object 6	UNSIGNED32	毫无	读 / 写
0x07	PDO 1 Mapping for an application object 7	UNSIGNED32	毫无	读 / 写
0x08	PDO 1 Mapping for an application object 8	UNSIGNED32	毫无	读 / 写

- 子索引0x00：记录PDO具体映射的对象个数，是有写权限的的，无符号8位的值。
- 子索引0x01：四字节数据段包含的是映射对象的索引、子索引及数据长度信息，如下图所示。

PDO映射实例

创建一个厂商的自定义变量。



6. 同步对象（SYNC）

同步对象（SYNC）是控制多个节点发送与接收之间协调和同步的一种特殊机制，用于PDO的同步传输。

同步对象的传输遵循生产者-消费者模型，由同步生产者发出同步帧，网络中其它节点作为消费者接收该同步帧。

一般同一个can网络中只允许有一个激活的同步生产者。



7. 紧急对象服务（EMCY）

当CANOPEN节点出现错误时，节点会发送一帧紧急报文。紧急报文遵循生产者-消费者模型，节点故障发出后，CAN网络中其它节点可选择处理该故障。



3.6 实战项目6 - canfestival移植

1. 源码下载

<https://hg.beremiz.org/canfestival/archive/tip.zip>

2. 源码目录

解压 STM32F103开发板资料V9.0\08-移植所需文件\canfestival-de1fc3261f21

可以看到源码目录如下图：

STM32F103开发板资料V9.0 > 08-移植所需文件 > canfestival-de1fc3261f21				在 canfestival-de1fc3261f21... 中搜索	
名称	修改日期	类型	大小		
debian	2024/9/20 20:43	文件夹			
doc	2024/9/20 20:43	文件夹			
drivers	2024/9/20 20:43	文件夹			
examples	2024/9/20 20:43	文件夹			
include	2024/9/20 20:43	文件夹			
objdictgen	2024/9/20 20:43	文件夹			
src	2024/9/20 20:43	文件夹			
.hg_archival.txt	2019/1/24 20:53	文本文档	1 KB		
.hgignore	2019/1/24 20:53	HGIGNORE 文件	2 KB		
.hgtags	2019/1/24 20:53	HGTAGS 文件	1 KB		
.travis.yml	2019/1/24 20:53	YML 文件	1 KB		
AUTHORS	2019/1/24 20:53	文件	1 KB		
CanFestival-3.vc9.sln	2019/1/24 20:53	SLN 文件	7 KB		
CanFestival-3.vc9.vcproj	2019/1/24 20:53	VCPROJ 文件	7 KB		
CanFestival-3.vc10.sln	2019/1/24 20:53	SLN 文件	9 KB		
CanFestival-3.vc10.vcxproj	2019/1/24 20:53	VCXPROJ 文件	7 KB		
CanFestival-3.vc10.vcxproj.filters	2019/1/24 20:53	FILTERS 文件	5 KB		
configure	2019/1/24 20:53	文件	29 KB		
CONTRIBUTORS	2019/1/24 20:53	文件	1 KB		
COPYING	2019/1/24 20:53	文件	2 KB		
LICENCE	2019/1/24 20:53	文件	24 KB		
Makefile.in	2019/1/24 20:53	IN 文件	2 KB		

目录结构如下：

文件路径	说明
/src	与处理器无关的canopen协议栈标准源代码
/include	针对各处理器的头文件
/driver	针对各硬件的驱动
/examples	测试程序
/objdictgen	带有图形界面的对象字典编辑器
/doc	说明文档

3. 源码移植

在实战项目4的基础上完成移植工作。

在自己的工程目录下面创建Canfestival文件夹用来存放源码相关文件，文件目录参考如下。

名称	修改日期	类型	大小
CanFestival	2024/9/20 21:37	文件夹	
Core	2024/9/20 20:54	文件夹	
Drivers	2024/9/20 20:54	文件夹	
FreeModbus	2024/9/20 20:58	文件夹	
MDK-ARM	2024/9/20 21:19	文件夹	
.mxproject	2024/9/20 20:58	MXPROJECT 文件	9 KB
52-canopen.ioc	2024/9/20 20:58	STM32CubeMX	10 KB

在CanFestival目录下，分别创建Src和Inc目录，Src用来存放源码，Inc用来存放头文件。

emb2407 > 04-stm32 > 52-canopen > CanFestival				在 CanFestival 中搜索	
名称	修改日期	类型	大小		
Inc	2024/9/20 21:45	文件夹			
Src	2024/9/20 21:45	文件夹			

将canfestival源码目录src文件夹下面所有源文件复制到工程内的CanFestival/Src目录下，并删除win32目录、Makefile.in、symbols.c文件。

emb2407 > 04-stm32 > 52-canopen > CanFestival > Src				在 Src 中搜索	
名称	修改日期	类型	大小		
win32	2024/9/20 21:48	文件夹			
dcf.c	2019/1/24 20:53	C 文件	9 KB		
emcy.c	2019/1/24 20:53	C 文件	8 KB		
lifegrd.c	2019/1/24 20:53	C 文件	11 KB		
lss.c	2019/1/24 20:53	C 文件	37 KB		
Makefile.in	2019/1/24 20:53	IN 文件	5 KB		
nmtMaster.c	2019/1/24 20:53	C 文件	3 KB		
nmtSlave.c	2019/1/24 20:53	C 文件	4 KB		
objaccs.c	2019/1/24 20:53	C 文件	10 KB		
pdo.c	2019/1/24 20:53	C 文件	28 KB		
sdo.c	2019/1/24 20:53	C 文件	79 KB		
states.c	2019/1/24 20:53	C 文件	12 KB		
symbols.c	2019/1/24 20:53	C 文件	4 KB		
sync.c	2019/1/24 20:53	C 文件	8 KB		
timer.c	2019/1/24 20:53	C 文件	5 KB		

将源码目录中include文件夹下所有被红框圈中的头文件，复制到工程的CanFestival/Inc目录下。

STM32F103开发板资料V9.0 > 08-移植所需文件 > canfestival-de1fc3261f21 > include

名称	修改日期	类型	大小
AT91	2024/9/20 20:43	文件夹	
AVR	2024/9/20 20:43	文件夹	
hcs12	2024/9/20 20:43	文件夹	
none	2024/9/20 20:43	文件夹	
timers_kernel	2024/9/20 20:43	文件夹	
timers_kernel_xeno	2024/9/20 20:43	文件夹	
timers_rtai	2024/9/20 20:43	文件夹	
timers_unix	2024/9/20 20:43	文件夹	
timers_xeno	2024/9/20 20:43	文件夹	
unix	2024/9/20 20:43	文件夹	
win32	2024/9/20 20:43	文件夹	
can.h	2019/1/24 20:53	H 文件	2 KB
can_driver.h	2019/1/24 20:53	H 文件	4 KB
data.h	2019/1/24 20:53	H 文件	11 KB
dcf.h	2019/1/24 20:53	H 文件	2 KB
def.h	2019/1/24 20:53	H 文件	6 KB
emcy.h	2019/1/24 20:53	H 文件	3 KB
lifegrd.h	2019/1/24 20:53	H 文件	5 KB
lss.h	2019/1/24 20:53	H 文件	10 KB
nmtMaster.h	2019/1/24 20:53	H 文件	4 KB
nmtSlave.h	2019/1/24 20:53	H 文件	2 KB
objacces.h	2019/1/24 20:53	H 文件	13 KB
objdictdef.h	2019/1/24 20:53	H 文件	4 KB
pdo.h	2019/1/24 20:53	H 文件	5 KB
sdo.h	2019/1/24 20:53	H 文件	19 KB
states.h	2019/1/24 20:53	H 文件	4 KB
sync.h	2019/1/24 20:53	H 文件	3 KB
sysdep.h	2019/1/24 20:53	H 文件	1 KB
timers.h	2019/1/24 20:53	H 文件	3 KB
timers_driver.h	2019/1/24 20:53	H 文件	2 KB

复制后如下：

emb2407 > 04-stm32 > 52-canopen > CanFestival > Inc

名称	修改日期	类型	大小
can.h	2019/1/24 20:53	H 文件	2 KB
can_driver.h	2019/1/24 20:53	H 文件	4 KB
data.h	2019/1/24 20:53	H 文件	11 KB
dcf.h	2019/1/24 20:53	H 文件	2 KB
def.h	2019/1/24 20:53	H 文件	6 KB
emcy.h	2019/1/24 20:53	H 文件	3 KB
lifegrd.h	2019/1/24 20:53	H 文件	5 KB
lss.h	2019/1/24 20:53	H 文件	10 KB
nmtMaster.h	2019/1/24 20:53	H 文件	4 KB
nmtSlave.h	2019/1/24 20:53	H 文件	2 KB
objacces.h	2019/1/24 20:53	H 文件	13 KB
objdictdef.h	2019/1/24 20:53	H 文件	4 KB
pdo.h	2019/1/24 20:53	H 文件	5 KB
sdo.h	2019/1/24 20:53	H 文件	19 KB
states.h	2019/1/24 20:53	H 文件	4 KB
sync.h	2019/1/24 20:53	H 文件	3 KB
sysdep.h	2019/1/24 20:53	H 文件	1 KB
timers.h	2019/1/24 20:53	H 文件	3 KB
timers_driver.h	2019/1/24 20:53	H 文件	2 KB

将源码目录中include/AVR文件夹下 applicfg.h、canfestival.h、config.h、timerscfg.h 总共4个头文件，复制到工程的 CanFestival/Inc/stm32目录下。

STM32F103开发板资料V9.0 > 08-移植所需文件 > canfestival-de1fc3261f21 > include > AVR

名称	修改日期	类型	大小
applicfg.h	2019/1/24 20:53	H 文件	3 KB
can_AVR.h	2019/1/24 20:53	H 文件	2 KB
can_drv.h	2019/1/24 20:53	H 文件	10 KB
canfestival.h	2019/1/24 20:53	H 文件	2 KB
config.h	2019/1/24 20:53	H 文件	3 KB
iar.h	2019/1/24 20:53	H 文件	2 KB
timerscfg.h	2019/1/24 20:53	H 文件	2 KB

复制后如下：

emb2407 > 04-stm32 > 52-canopen > CanFestival > Inc > stm32

名称	修改日期	类型	大小
applicfg.h	2019/1/24 20:53	H 文件	3 KB
canfestival.h	2019/1/24 20:53	H 文件	2 KB
config.h	2019/1/24 20:53	H 文件	3 KB
timerscfg.h	2019/1/24 20:53	H 文件	2 KB

添加源文件到工程内

Manage Project Items

Project Items | Folders/Extensions | Books | Project Info/Layer

Project Targets:
52-canopen

Groups:
Application/MDK-ARM
Application/User/Core
Application/FreeModbus
Application/CanFestival
Drivers/STM32F1xx_HAL_Driver
Drivers/CMSIS

Files:
dcf.c
emcy.c
lifegrd.c
lss.c
nmtMaster.c
nmtSlave.c
objacces.c
pdo.c
sdo.c
states.c
sync.c
timer.c

Set as Current Target

Add Files...

OK | Cancel | Help

添加头文件路径。

Options for Target '52-canopen'

Device | Target | Output | Listing | User | C/C++ | Asm | Linker | Debug | Utilities

Preprocessor
Compiler
Library
Include Paths
Predefined Symbols
Compiler Options
Linker Options
Debug Options
Utilities

Folder Setup
Setup Compiler Include Paths:
../Core/Inc
../Drivers/STM32F1xx_HAL_Driver/Inc
../Drivers/STM32F1xx_HAL_Driver/Inc/Legacy
../Drivers/CMSIS/Device/ST/STM32F1xx/Include
../Drivers/CMSIS/Include
..\FreeModbus
..\CanFestival\Inc
..\CanFestival\Inc\stm32

OK | Cancel

OK | Cancel | Defaults | Help

至此，我们所有关于canfestival源码移植的文件操作已经完成了，接下来就是对源码文件中具体代码内容的修改和添加了。

4. 修改源码

在config.h中文件中将如下被红框圈的部分注释掉，消除编译报错。

```
23
24 #ifndef _CONFIG_H_
25 #define _CONFIG_H_
26
27
28 // #ifdef __IAR_SYSTEMS_ICC__
29 // #include <ioavr.h>
30 // #include <intrinsics.h>
31 // #include "iar.h"
32 // #else // GCC
33 // #include <inttypes.h>
34 // #include <avr/io.h>
35 // #include <avr/interrupt.h>
36 // #include <avr/pgmspace.h>
37 // #include <avr/sleep.h>
38 // #include <avr/wdt.h>
39 // #endif // GCC
40
41 // #define WD_SLEEP
42 // Needed defines by Atmel lib
43 // #define FOSC 8000 // 8 MHz External cristal
44 // #ifndef F_CPU
45 // #define F_CPU (1000UL*FOSC) // Need for AVR GCC
46 // #endif
47 // #define CAN_BAUDRATE 125
48
49 // Needed defines by Canfestival lib
50 #define MAX_CAN_BUS_ID 1
51 #define SDO_MAX_LENGTH_TRANSFER 32
52 #define SDO_BLOCK_SIZE 16
53 #define SDO_MAX_SIMULTANEOUS_TRANSFERS 1
54 #define NMT_MAX_NODE_ID 128
55 #define SDO_TIMEOUT_MS 3000U
56 #define MAX_NB_TIMER 8
```

编译工程，会有5个错误，我们先解决前2个错误。

```
Error: L6218E: Undefined symbol start_and_seek_node (referred from dcf.o).
Error: L6218E: Undefined symbol start_node (referred from dcf.o).
```

修改dcf.c文件，在这两个函数前加上static关键字，这个编译出错主要是编译的问题，修改如下：

```
static inline void start_node(CO_Data* d, UNS8 nodeId){
    /* Ask slave node to go in operational mode */
    masterSendNMTstateChange (d, nodeId, NMT_Start_Node);
    d->NMTable[nodeId] = Operational;
}

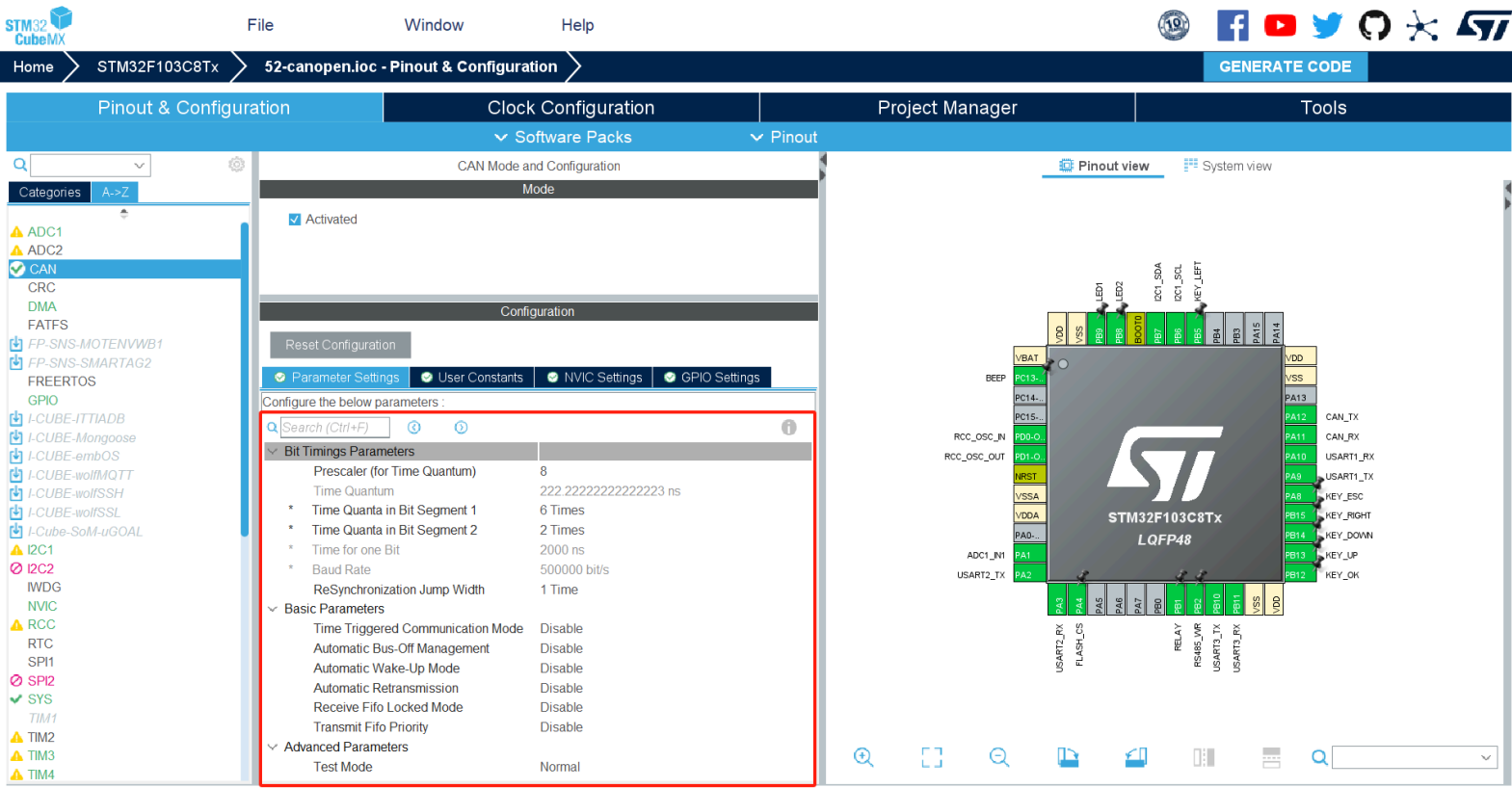
static inline void start_and_seek_node(CO_Data* d, UNS8 nodeId){
    UNS8 node;
    start_node(d,nodeId);
    /* Look for other nodes waiting to be started */
    for(node = 0 ; node<NMT_MAX_NODE_ID ; node++){
        if(d->NMTable[node] != Initialisation)
            continue;
        if(check_and_start_node(d, node) == 2)
            return;
    }
    /* No more node to start. Let's start our own node */
    setState(d, Operational);
}
```

再次编译还剩下3个错误

```
Build started: Project: 52-canopen
*** Using Compiler 'V5.06 update 7 (build 960)', folder: 'D:\Software\Keil_v5\ARM\ARMCC\Bin'
Build target '52-canopen'
compiling dcf.c...
..\CanFestival\Src\dcf.c(106): warning: #177-D: variable "node" was declared but never referenced
        UNS8 buf[4], match = 0, node;
..\CanFestival\Src\dcf.c: 1 warning, 0 errors
linking...
52-canopen\52-canopen.axf: Error: L6218E: Undefined symbol canSend (referred from emcy.o).
52-canopen\52-canopen.axf: Error: L6218E: Undefined symbol getElapsedTime (referred from timer.o).
52-canopen\52-canopen.axf: Error: L6218E: Undefined symbol setTimer (referred from timer.o).
Not enough information to list image symbols.
Not enough information to list load addresses in the image map.
Finished: 2 information, 0 warning and 3 error messages.
"52-canopen\52-canopen.axf" - 3 Error(s), 1 Warning(s).
Target not created.
Build Time Elapsed: 00:00:00
```

5. 添加CAN控制器设置

在CubeMX中配置CAN，设置波特率为500Kbps。



emb2407	>	04-stm32	>	52-canopen	>	Core	>	Inc	▼🔄	在 Inc 中搜
名称	^	修改日期	类型	大小						
 adc.h		2024/9/13 10:06	C Header 源文件	2 KB						
 aht20.h		2024/9/12 14:41	C Header 源文件	2 KB						
 cJSON.h		2024/8/30 11:29	C Header 源文件	16 KB						
 codetab.h		2024/8/30 15:29	C Header 源文件	20 KB						
 dma.h		2024/9/14 9:50	C Header 源文件	2 KB						
 f1can.h		2024/9/21 14:47	C Header 源文件	2 KB						
 fifo.h		2024/9/5 10:55	C Header 源文件	1 KB						
 gpio.h		2024/9/20 20:58	C Header 源文件	2 KB						
 i2c.h		2024/9/12 14:14	C Header 源文件	2 KB						

emb2407	>	04-stm32	>	52-canopen	>	Core	>	Src	▼🔄	在 Src 中搜
名称	^	修改日期	类型	大小						
 adc.c		2024/9/20 20:58	C 源文件	9 KB						
 aht20.c		2024/9/12 14:45	C 源文件	7 KB						
 cJSON.c		2024/8/30 11:29	C 源文件	78 KB						
 dma.c		2024/9/14 9:50	C 源文件	2 KB						
 f1can.c		2024/9/21 14:54	C 源文件	4 KB						
 fifo.c		2024/9/5 10:54	C 源文件	2 KB						
 gpio.c		2024/9/20 20:58	C 源文件	8 KB						
 i2c.c		2024/9/20 20:58	C 源文件	5 KB						
 ina226.c		2024/9/13 10:11	C 源文件	4 KB						

在CanFestival/Src目录下新建canfestival.c文件，在文件包含头文件canfestival.h，并定义如下函数。

canfestival.h

```
/*
This file is part of CanFestival, a library implementing CanOpen Stack.

Copyright (C): Edouard TISSERANT and Francis DUPIN
AVR Port: Andreas GLAUSER and Peter CHRISTEN

See COPYING file for copyrights details.

This library is free software; you can redistribute it and/or
modify it under the terms of the GNU Lesser General Public
License as published by the Free Software Foundation; either
version 2.1 of the License, or (at your option) any later version.

This library is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with this library; if not, write to the Free Software
Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA
*/

#ifdef __CAN_CANFESTIVAL__
#define __CAN_CANFESTIVAL__

#include "applicfg.h"
#include "data.h"
#include <stdint.h>

typedef struct
{
    uint32_t StdId; /*!< Specifies the standard identifier.
                    This parameter can be a value between 0 to 0x7FF. */
    uint32_t ExtId; /*!< Specifies the extended identifier.
                    This parameter can be a value between 0 to 0x1FFFFFFF. */
    uint8_t IDE; /*!< Specifies the type of identifier for the message that
                will be transmitted. This parameter can be a value
                of @ref CAN_identifier_type */
    uint8_t RTR; /*!< Specifies the type of frame for the message that will
                be transmitted. This parameter can be a value of
                @ref CAN_remote_transmission_request */
    uint8_t DLC; /*!< Specifies the length of the frame that will be
                transmitted. This parameter can be a value between
                0 to 8 */
    uint8_t Data[8]; /*!< Contains the data to be transmitted. It ranges from 0
                to 0xFF. */
} CanTxMsg;

// ----- to be called by user app -----
void initTimer(void);
UNS8 canSend(CAN_PORT notused, Message *m);
UNS8 canChangeBaudRate(CAN_PORT port, char* baud);
uint8_t CanFestival_Can_Init(void);

#endif
```

canfestival.c

```
#include "stm32f1xx_hal.h"
#include "canfestival.h"
#include "tim.h"
#include "testslave.h"
#include "f1can.h"

CAN_TxHeaderTypeDef TxHeader;
CAN_RxHeaderTypeDef RxHeader;

uint8_t CanFestival_Can_Init(void)
{
    CAN_FilterTypeDef sFilterConfig;

    /* Configure the CAN Filter */
    sFilterConfig.FilterBank = 0;
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;
    sFilterConfig.FilterIdHigh = 0x0000;
    sFilterConfig.FilterIdLow = 0x0000;
    sFilterConfig.FilterMaskIdHigh = 0x0000;
    sFilterConfig.FilterMaskIdLow = 0x0000;
    sFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0;
    sFilterConfig.FilterActivation = ENABLE;
    sFilterConfig.SlaveStartFilterBank = 14;

    if (HAL_CAN_ConfigFilter(&hcan, &sFilterConfig) != HAL_OK)
    {
        /* Filter configuration Error */
        Error_Handler();
    }

    /* Start the CAN peripheral */
    if (HAL_CAN_Start(&hcan) != HAL_OK)
    {
        /* Start Error */
        Error_Handler();
    }

    /* Activate CAN RX notification */
    if (HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK)
    {
        /* Notification Error */
        Error_Handler();
    }

    /* Configure Transmission process */

    return 0;
}

/**
 * @brief Rx Fifo 0 message pending callback in non blocking mode
 * @param CanHandle: pointer to a CAN_HandleTypeDef structure that contains
 * the configuration information for the specified CAN.
```

```
* @retval None
*/
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *CanHandle)
{
    Message RxMSG ;
    HAL_CAN_GetRxMessage(&hcan, CAN_RX_FIFO0, &RxHeader, RxMSG.data);
    RxMSG.cob_id = (uint16_t)(RxHeader.StdId);
    if( RxHeader.RTR == CAN_RTR_REMOTE )
    {
        RxMSG.rtr = 1;
    }
    else
    {
        RxMSG.rtr = 0;
    }
    RxMSG.len = RxHeader.DLC;
    canDispatch(&TestSlave_Data, &(RxMSG));
}

uint8_t canSend(CAN_PORT notused, Message *message)
{
    CanTxMsg TxMessage;
    uint32_t TxMailbox = 0;

    TxHeader.DLC = message->len;
    memcpy(TxMessage.Data, message->data, message->len);
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.StdId = message->cob_id;
    TxHeader.RTR = (message->rtr == CAN_RTR_DATA) ? 0 : 2;

    while(( HAL_CAN_AddTxMessage(&hcan,&TxHeader,TxMessage.Data,&TxMailbox)!=HAL_OK))
    {
        return 0;
    }
    return 1;
}

void setTimer(TIMEVAL value)
{
    TIM2->ARR = TIM2->CNT + value;
}

TIMEVAL getElapsedTime(void)
{
    return TIM2->CNT;
}
```

6. 添加TIM定时器的设置

在timerscfg.h中有三个宏的定义需要注意，分别是TIMEVAL_MAX、MS_TO_TIMEVAL和US_TO_TIMEVAL， 源码里面宏定义如下。

```
#ifndef __TIMERSCFG_H__
#define __TIMERSCFG_H__

// Whatever your microcontroller, the timer wont work if
// TIMEVAL is not at least on 32 bits
#define TIMEVAL UNS32

// The timer of the AVR counts from 0000 to 0xFFFF in CTC mode (it can be
// shortened setting OCR3A eg. to get 2ms instead of 2.048ms)
#define TIMEVAL_MAX 0xFFFF

// The timer is incrementing every 4 us.
// #define MS_TO_TIMEVAL(ms) ((ms) * 250)
// #define US_TO_TIMEVAL(us) (us>>2)

// The timer is incrementing every 8 us.
#define MS_TO_TIMEVAL(ms) ((ms) * 125)
#define US_TO_TIMEVAL(us) ((us)>>3)

#endif
```

MS_TO_TIMEVAL和US_TO_TIMEVAL我在这里把单位改成1us，如下图所示。

```
// The timer is incrementing every 1 us.
#define MS_TO_TIMEVAL(ms) ((ms) * 1000)
#define US_TO_TIMEVAL(us) ((us) * 1 )
```

这个需要和自己定时器频率对应，当然也可以根据这个宏去定义自己的定时器。

TIMEVAL_MAX不能超过最大计数周期，这里我设置为2000。

最后设置如下：

```
#ifndef __TIMERSCFG_H__
#define __TIMERSCFG_H__

// Whatever your microcontroller, the timer wont work if
// TIMEVAL is not at least on 32 bits
#define TIMEVAL UNS32

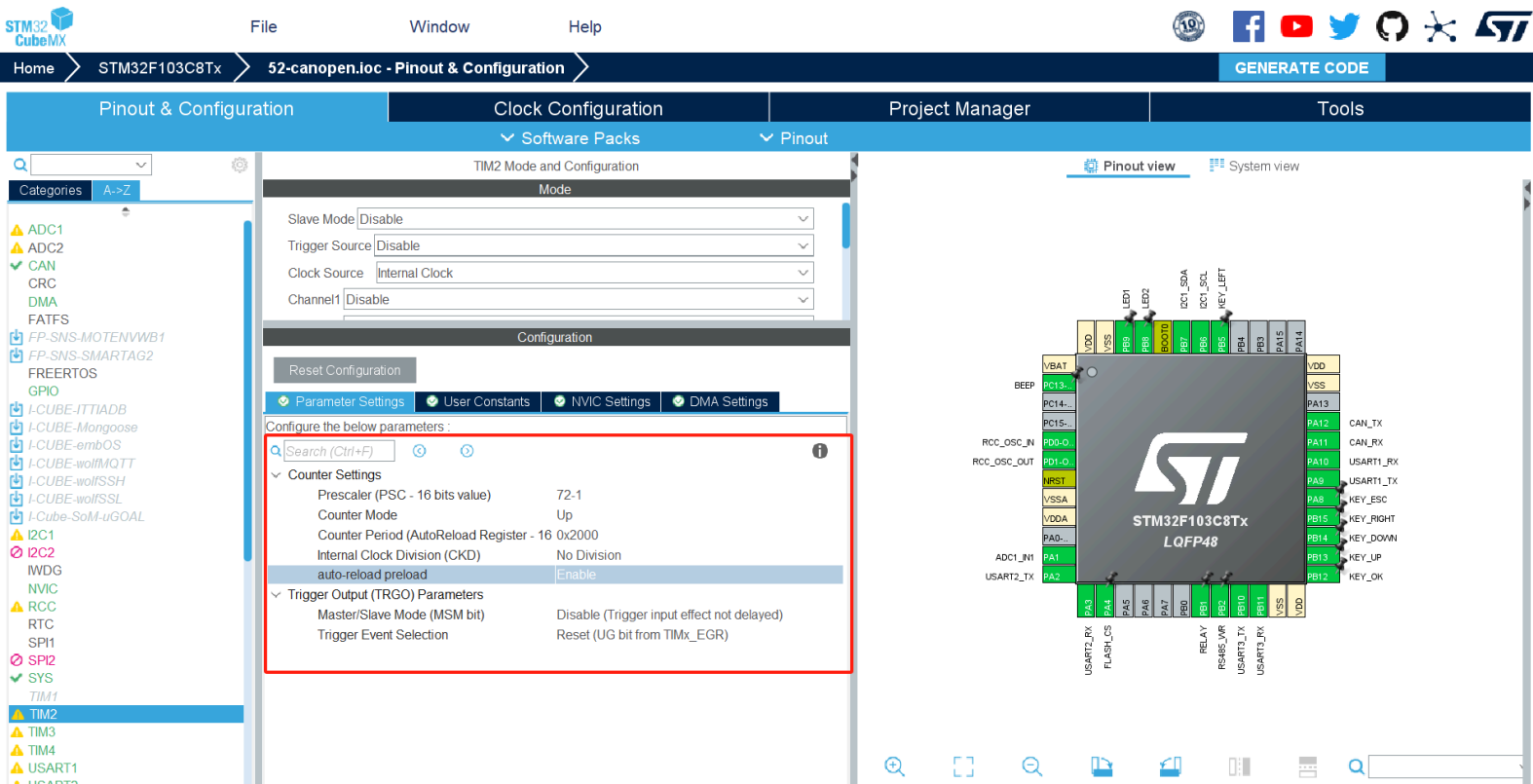
// The timer of the AVR counts from 0000 to 0xFFFF in CTC mode (it can be
// shortened setting OCR3A eg. to get 2ms instead of 2.048ms)
#define TIMEVAL_MAX 0x2000

// The timer is incrementing every 4 us.
// #define MS_TO_TIMEVAL(ms) (ms * 250)
// #define US_TO_TIMEVAL(us) (us>>2)

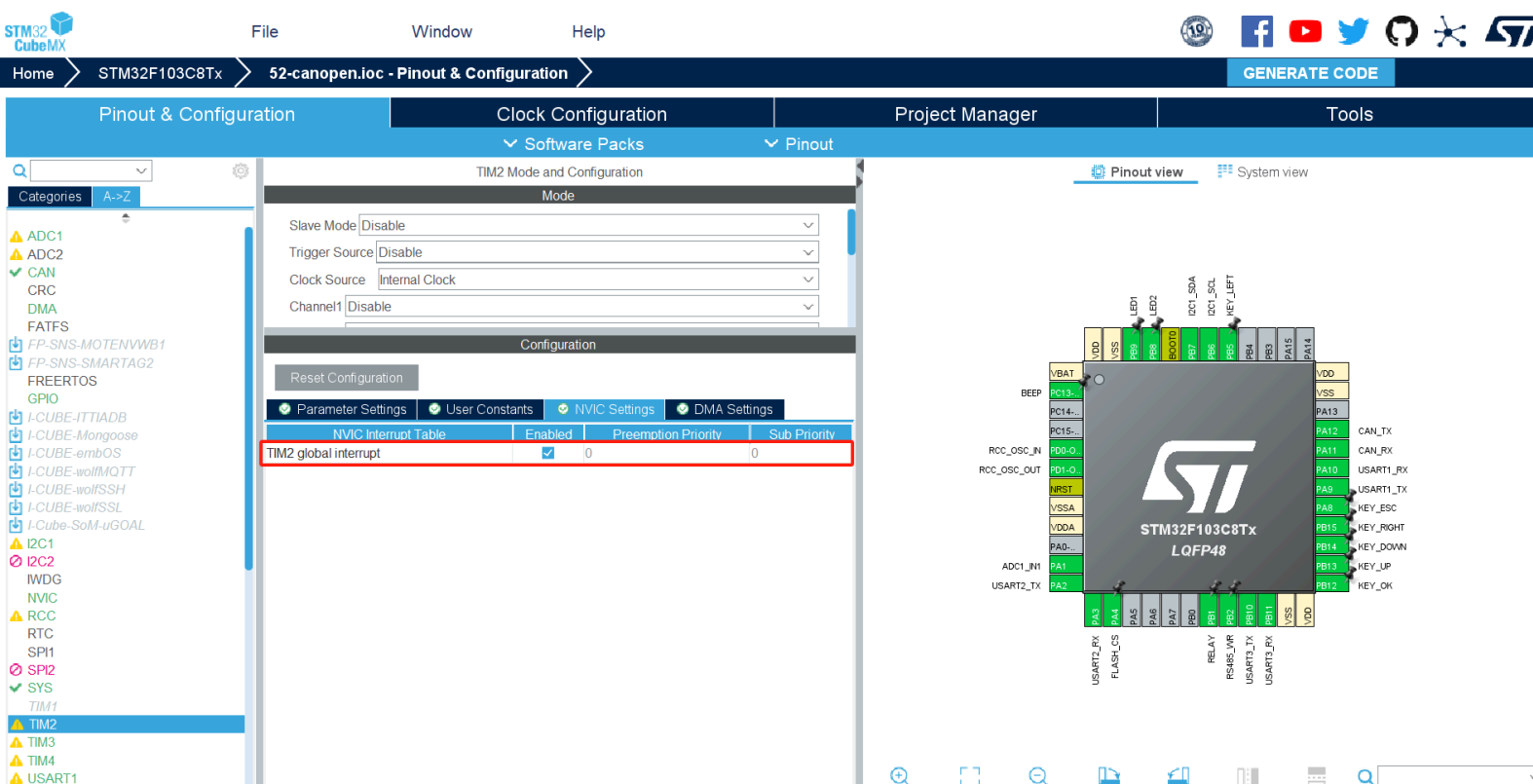
// The timer is incrementing every 1 us.
#define MS_TO_TIMEVAL(ms) ((ms) * 1000)
#define US_TO_TIMEVAL(us) ((us) * 1)

#endif
```

在CubeMX中设置定时器的计时周期为1us，这里我们使用定时器2。



开启TIM2中断。




```
{ RW, valueRange EMC, sizeof (UNS8), (void*)&TestSlave_highestSubIndex_obj1003 },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[0] },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[1] },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[2] },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[3] },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[4] },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[5] },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[6] },
{ RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1003[7] }
};

/* index 0x1005 : SYNC COB ID. */
UNS32 TestSlave_obj1005 = 0x80; /* 128 */
ODCallback_t TestSlave_Index1005_callbacks[] =
{
    NULL,
};
subindex TestSlave_Index1005[] =
{
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1005 }
};

/* index 0x1006 : Communication / Cycle Period. */
UNS32 TestSlave_obj1006 = 0x0; /* 0 */
ODCallback_t TestSlave_Index1006_callbacks[] =
{
    NULL,
};
subindex TestSlave_Index1006[] =
{
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1006 }
};

/* index 0x100C : Guard Time */
UNS16 TestSlave_obj100C = 0x0; /* 0 */

/* index 0x100D : Life Time Factor */
UNS8 TestSlave_obj100D = 0x0; /* 0 */

/* index 0x1010 : Store parameters. */
UNS8 TestSlave_highestSubIndex_obj1010 = 4; /* number of subindex - 1*/
UNS32 TestSlave_obj1010_Save_All_Parameters = 0x0; /* 0 */
UNS32 TestSlave_obj1010_Save_Communication_Parameters = 0x0; /* 0 */
UNS32 TestSlave_obj1010_Save_Application_Parameters = 0x0; /* 0 */
UNS32 TestSlave_obj1010_Save_Manufacturer_Parameters_1 = 0x0; /* 0 */
ODCallback_t TestSlave_Index1010_callbacks[] =
{
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
};
subindex TestSlave_Index1010[] =
{
    { RO, uint8, sizeof (UNS8), (void*)&TestSlave_highestSubIndex_obj1010 },
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1010_Save_All_Parameters },
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1010_Save_Communication_Parameters
},
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1010_Save_Application_Parameters
},
    { RW, uint32, sizeof (UNS32),
(void*)&TestSlave_obj1010_Save_Manufacturer_Parameters_1 }
};

/* index 0x1011 : Restore Default Parameters. */
UNS8 TestSlave_highestSubIndex_obj1011 = 4; /* number of subindex - 1*/
UNS32 TestSlave_obj1011_Restore_All_Default_Parameters = 0x0; /* 0 */
UNS32 TestSlave_obj1011_Restore_Communication_Default_Parameters = 0x0; /* 0 */
UNS32 TestSlave_obj1011_Restore_Application_Default_Parameters = 0x0; /* 0 */
UNS32 TestSlave_obj1011_Restore_Manufacturer_Defined_Default_Parameters_1 = 0x0; /* 0
*/
ODCallback_t TestSlave_Index1011_callbacks[] =
{
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
};
subindex TestSlave_Index1011[] =
{
    { RO, uint8, sizeof (UNS8), (void*)&TestSlave_highestSubIndex_obj1011 },
    { RW, uint32, sizeof (UNS32),
(void*)&TestSlave_obj1011_Restore_All_Default_Parameters },
    { RW, uint32, sizeof (UNS32),
(void*)&TestSlave_obj1011_Restore_Communication_Default_Parameters },
    { RW, uint32, sizeof (UNS32),
(void*)&TestSlave_obj1011_Restore_Application_Default_Parameters },
    { RW, uint32, sizeof (UNS32),
(void*)&TestSlave_obj1011_Restore_Manufacturer_Defined_Default_Parameters_1 }
};

/* index 0x1014 : Emergency COB ID. */
UNS32 TestSlave_obj1014 = 0x80; /* 128 */
subindex TestSlave_Index1014[] =
{
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1014 }
};

/* index 0x1016 : Consumer Heartbeat Time */
UNS8 TestSlave_highestSubIndex_obj1016 = 0;
UNS32 TestSlave_obj1016[]={0};

/* index 0x1017 : Producer Heartbeat Time. */
UNS16 TestSlave_obj1017 = 0x0; /* 0 */
ODCallback_t TestSlave_Index1017_callbacks[] =
{
    NULL,
};
subindex TestSlave_Index1017[] =
{
    { RW, uint16, sizeof (UNS16), (void*)&TestSlave_obj1017 }
};

/* index 0x1018 : Identity. */
UNS8 TestSlave_highestSubIndex_obj1018 = 4; /* number of subindex - 1*/
UNS32 TestSlave_obj1018_Vendor_ID = 0x12345678; /* 305419896 */
UNS32 TestSlave_obj1018_Product_Code = 0x90123456; /* 2417112150 */
UNS32 TestSlave_obj1018_Revision_Number = 0x78901234; /* 2022707764 */
UNS32 TestSlave_obj1018_Serial_Number = 0x56789012; /* 1450741778 */
subindex TestSlave_Index1018[] =
{
    { RO, uint8, sizeof (UNS8), (void*)&TestSlave_highestSubIndex_obj1018 },
    { RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1018_Vendor_ID },
    { RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1018_Product_Code },
    { RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1018_Revision_Number },
    { RO, uint32, sizeof (UNS32), (void*)&TestSlave_obj1018_Serial_Number }
};

/* index 0x1200 : Server SDO Parameter. */
UNS8 TestSlave_highestSubIndex_obj1200 = 2; /* number of subindex - 1*/
UNS32 TestSlave_obj1200_COB_ID_Client_to_Server_Receive_SDO = 0x600; /* 1536 */
UNS32 TestSlave_obj1200_COB_ID_Server_to_Client_Transmit_SDO = 0x580; /* 1408 */
subindex TestSlave_Index1200[] =
{
    { RO, uint8, sizeof (UNS8), (void*)&TestSlave_highestSubIndex_obj1200 },
    { RO, uint32, sizeof (UNS32),
(void*)&TestSlave_obj1200_COB_ID_Client_to_Server_Receive_SDO },
    { RO, uint32, sizeof (UNS32),
(void*)&TestSlave_obj1200_COB_ID_Server_to_Client_Transmit_SDO }
};

/* index 0x1800 : Transmit PDO 1 Parameter. */
UNS8 TestSlave_highestSubIndex_obj1800 = 5; /* number of subindex - 1*/
UNS32 TestSlave_obj1800_COB_ID_used_by_PDO = 0x180; /* 384 */
UNS8 TestSlave_obj1800_Transmission_Type = 0x1; /* 1 */
UNS16 TestSlave_obj1800_Inhibit_Time = 0x0; /* 0 */
UNS8 TestSlave_obj1800_Compatibility_Entry = 0x0; /* 0 */
UNS16 TestSlave_obj1800_Event_Timer = 0x0; /* 0 */
ODCallback_t TestSlave_Index1800_callbacks[] =
{
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
}
```

```
};
subindex TestSlave_Index1800[] =
{
    { RO, uint8, sizeof (UNS8), (void*)&TestSlave_highestSubIndex_obj1800 },
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1800_COB_ID_used_by_PDO },
    { RW|TO_BE_SAVE, uint8, sizeof (UNS8), (void*)&TestSlave_obj1800_Transmission_Type },
    { RW|TO_BE_SAVE, uint16, sizeof (UNS16), (void*)&TestSlave_obj1800_Inhibit_Time },
    { RW, uint8, sizeof (UNS8), (void*)&TestSlave_obj1800_Compatibility_Entry },
    { RW, uint16, sizeof (UNS16), (void*)&TestSlave_obj1800_Event_Timer }
};

/* index 0x1A00 : Transmit PDO 1 Mapping. */
UNS8 TestSlave_highestSubIndex_obj1A00 = 2; /* number of subindex - 1*/
UNS32 TestSlave_obj1A00[] =
{
    0x20000020, /* 536870944 */
    0x20010020 /* 536936480 */
};
subindex TestSlave_Index1A00[] =
{
    { RW, uint8, sizeof (UNS8), (void*)&TestSlave_highestSubIndex_obj1A00 },
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1A00[0] },
    { RW, uint32, sizeof (UNS32), (void*)&TestSlave_obj1A00[1] }
};

/* index 0x2000 : Mapped variable SlaveMap1 */
subindex TestSlave_Index2000[] =
{
    { RW, int32, sizeof (INTEGER32), (void*)&SlaveMap1 }
};

/* index 0x2001 : Mapped variable SlaveMap2 */
subindex TestSlave_Index2001[] =
{
    { RW, int32, sizeof (INTEGER32), (void*)&SlaveMap2 }
};

/*****
/* Declaration of pointed variables
*****/

const indextable TestSlave_objdict[] =
{
    { (subindex*)TestSlave_Index1000,sizeof(TestSlave_Index1000)/sizeof(TestSlave_Index1000[0]), 0x1000},
    { (subindex*)TestSlave_Index1001,sizeof(TestSlave_Index1001)/sizeof(TestSlave_Index1001[0]), 0x1001},
    { (subindex*)TestSlave_Index1003,sizeof(TestSlave_Index1003)/sizeof(TestSlave_Index1003[0]), 0x1003},
    { (subindex*)TestSlave_Index1005,sizeof(TestSlave_Index1005)/sizeof(TestSlave_Index1005[0]), 0x1005},
    { (subindex*)TestSlave_Index1006,sizeof(TestSlave_Index1006)/sizeof(TestSlave_Index1006[0]), 0x1006},
    { (subindex*)TestSlave_Index1010,sizeof(TestSlave_Index1010)/sizeof(TestSlave_Index1010[0]), 0x1010},
    { (subindex*)TestSlave_Index1011,sizeof(TestSlave_Index1011)/sizeof(TestSlave_Index1011[0]), 0x1011},
    { (subindex*)TestSlave_Index1014,sizeof(TestSlave_Index1014)/sizeof(TestSlave_Index1014[0]), 0x1014},
    { (subindex*)TestSlave_Index1017,sizeof(TestSlave_Index1017)/sizeof(TestSlave_Index1017[0]), 0x1017},
    { (subindex*)TestSlave_Index1018,sizeof(TestSlave_Index1018)/sizeof(TestSlave_Index1018[0]), 0x1018},
    { (subindex*)TestSlave_Index1200,sizeof(TestSlave_Index1200)/sizeof(TestSlave_Index1200[0]), 0x1200},
    { (subindex*)TestSlave_Index1800,sizeof(TestSlave_Index1800)/sizeof(TestSlave_Index1800[0]), 0x1800},
    { (subindex*)TestSlave_Index1A00,sizeof(TestSlave_Index1A00)/sizeof(TestSlave_Index1A00[0]), 0x1A00},
    { (subindex*)TestSlave_Index2000,sizeof(TestSlave_Index2000)/sizeof(TestSlave_Index2000[0]), 0x2000},
    { (subindex*)TestSlave_Index2001,sizeof(TestSlave_Index2001)/sizeof(TestSlave_Index2001[0]), 0x2001},
};

const indextable * TestSlave_scanIndexOD (UNS16 wIndex, UNS32 * errorCode, ODCallback_t **callbacks)
{
    int i;
    *callbacks = NULL;
    switch(wIndex){
        case 0x1000: i = 0;break;
        case 0x1001: i = 1;break;
        case 0x1003: i = 2;*callbacks = TestSlave_Index1003_callbacks; break;
        case 0x1005: i = 3;*callbacks = TestSlave_Index1005_callbacks; break;
        case 0x1006: i = 4;*callbacks = TestSlave_Index1006_callbacks; break;
        case 0x1010: i = 5;*callbacks = TestSlave_Index1010_callbacks; break;
        case 0x1011: i = 6;*callbacks = TestSlave_Index1011_callbacks; break;
        case 0x1014: i = 7;break;
        case 0x1017: i = 8;*callbacks = TestSlave_Index1017_callbacks; break;
        case 0x1018: i = 9;break;
        case 0x1200: i = 10;break;
        case 0x1800: i = 11;*callbacks = TestSlave_Index1800_callbacks; break;
        case 0x1A00: i = 12;break;
        case 0x2000: i = 13;break;
        case 0x2001: i = 14;break;
        default:
            *errorCode = OD_NO_SUCH_OBJECT;
            return NULL;
    }
    *errorCode = OD_SUCCESSFUL;
    return &TestSlave_objdict[i];
}

/*
 * To count at which received SYNC a PDO must be sent.
 * Even if no pdoTransmit are defined, at least one entry is computed
 * for compilations issues.
 */
s_PDO_status TestSlave_PDO_status[1] = {s_PDO_status_Initializer};

const quick_index TestSlave_firstIndex = {
    10, /* SDO_SVR */
    0, /* SDO_CLT */
    0, /* PDO_RCV */
    0, /* PDO_RCV_MAP */
    11, /* PDO_TRS */
    12 /* PDO_TRS_MAP */
};

const quick_index TestSlave_lastIndex = {
    10, /* SDO_SVR */
    0, /* SDO_CLT */
    0, /* PDO_RCV */
    0, /* PDO_RCV_MAP */
    11, /* PDO_TRS */
    12 /* PDO_TRS_MAP */
};

const UNS16 TestSlave_ObjdictSize = sizeof(TestSlave_objdict)/sizeof(TestSlave_objdict[0]);

CO_Data TestSlave_Data = CANOPEN_NODE_DATA_INITIALIZER(TestSlave);
```

再次编译后成功。

```
compiling stm32flxx_hal_cortex.c...
compiling stm32flxx_hal_flash_ex.c...
compiling stm32flxx_hal_flash.c...
compiling stm32flxx_hal_exti.c...
compiling stm32flxx_hal_pwr.c...
compiling stm32flxx_hal_can.c...
compiling stm32flxx_hal_i2c.c...
compiling system_stm32flxx.c...
compiling stm32flxx_hal_tim_ex.c...
compiling stm32flxx_hal_uart.c...
compiling stm32flxx_hal_tim.c...
linking...
Program Size: Code=43400 RO-data=3192 RW-data=1592 ZI-data=4208
FromELF: creating hex file...
After Build - User command #2: fromelf --bin -o "52-canopen\52-canopen.bin" "52-canopen\52-canopen.axf"
"52-canopen\52-canopen.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:20
```

main.c

```
#include "canfestival.h"
#include "testslave.h"

#pragma diag_suppress 870

int main(void)
{
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_I2C1_Init();
    MX_ADC1_Init();
    MX_USART1_UART_Init();
    MX_USART2_UART_Init();
    MX_USART3_UART_Init();
    MX_TIM4_Init();
    MX_CAN_Init();
    MX_TIM2_Init();
    /* USER CODE BEGIN 2 */
    printf("Uart1 init is ok\n");
    printf("CANopen Control\n");
    HAL_Delay(400); // 这里延时是给esp8266 打印日志使用的
```

// 经过测试，400ms时，日志输出就没有了，为了安全,我们设置400ms 以上的延时
// 等待esp8266 输出初始化日志，要不然一旦开启uart3就会触发dma中断,早期的esp8266初始化的数据波特率不是
// 115200，会造成uart3的错误,要在uart3中做错误恢复

```
OLED_Init();
OLED_CLS(); // 清除屏幕显示
OLED_ShowStr(0,0, (unsigned char*)"CANopen Control ", 2);
INA226_Init();

__HAL_UART_ENABLE_IT(&huart3, UART_IT_IDLE); //使能串UART3 IDLE(空闲)中断
HAL_UART_Receive_DMA(&huart3, RX3_Buf, DMA_BUF_SIZE); //设置DMA传输，将uart3的数据搬运到RX3_Buf中
Queue_Init(&UART3); // 队列初始化
Modbus_Init();

if(MQTT_Connect_Server() == SET) // mqtt服务器链接成功
{
    printf("MQTT服务器链接成功 \n");
    MQTT_Download_Flag = 1; // 链接服务器后，开启解析json数据标志位
}

HAL_TIM_Base_Start_IT(&htim2); //启动定时器2
CanFestival_Can_Init(); //初始化can过滤器
setNodeId(&TestSlave_Data, SlaveAddress);
setState(&TestSlave_Data, Initialisation);
setState(&TestSlave_Data, Operational);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    eMBPoll(); //轮询查询
    Modbus_Parse(); // 解析各种命令

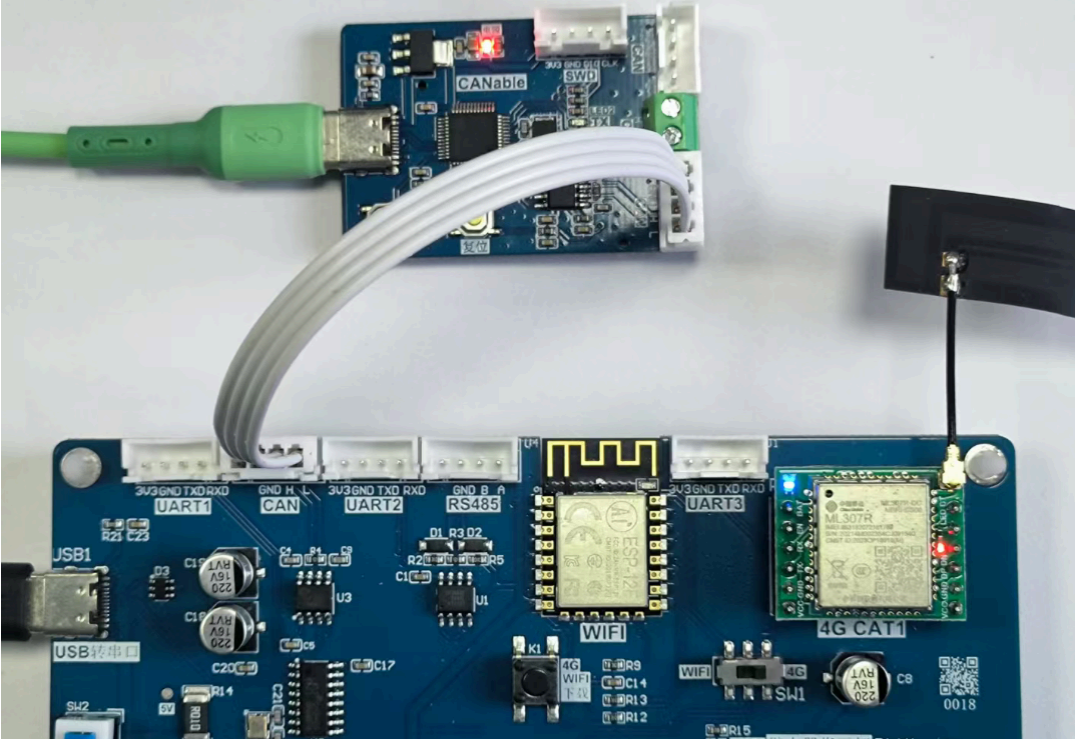
    if(MQTT_UPLOAD_Flag)
    {
        MQTT_UPLOAD_Flag = 0 ;
        MQTT_SendData(); // 每隔5秒发送一次数据
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

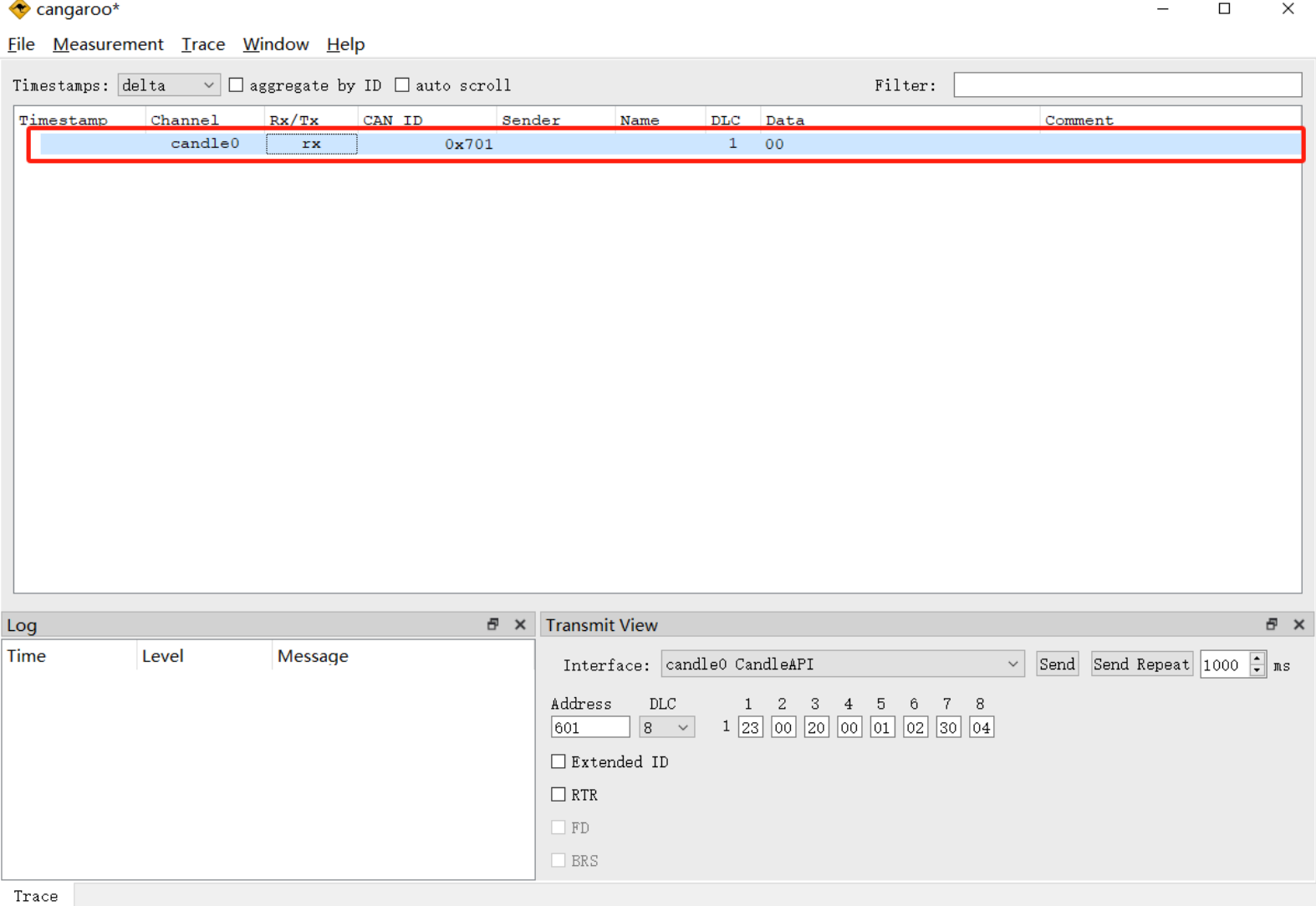
}
/* USER CODE END 3 */
}
```

7. 移植通信验证

下载程序到开发板内，接线如下图：



打开cangaroo软件，复位开发板，查看设备的通信数据。



这个就是从机上线后，要发送一个NMT boot-up 上线报文。

任何一个canopen从设备上线后，为了提示主站已经加入网络（便于热插拔）。这个从站必须发出节点上线报文（boot-up）。

报文格式如下，COB-ID为0x700+nodeID，数据长度为1个字节且数据为0。

至此CanFestival协议移植成功。

3.7 SDO upload与download协议

1. SDO协议介绍

SDO“服务数据对象”允许对对象字典进行读或写访问。

- SDO upload ： 上载是指从对象字典中读取对象的值。 读从机。
- SDO download ： 下载是指在对象字典中写入值。，写从机。

2. SDO upload 协议

当需要读取一个CANopen节点中对象字典的值时，使用SDO upload协议。

根据对象字典的数据类型，又分为SDO upload expedited和SDO upload normal 两种：

- 当数据字典的长度小于或等于4个字节时，使用SDO upload expedited（快速）。
- 当数据字典的长度超过4个字节，一帧数据传输不完时，使用SDO upload normal（正常）。

3. SDO upload expedited帧格式

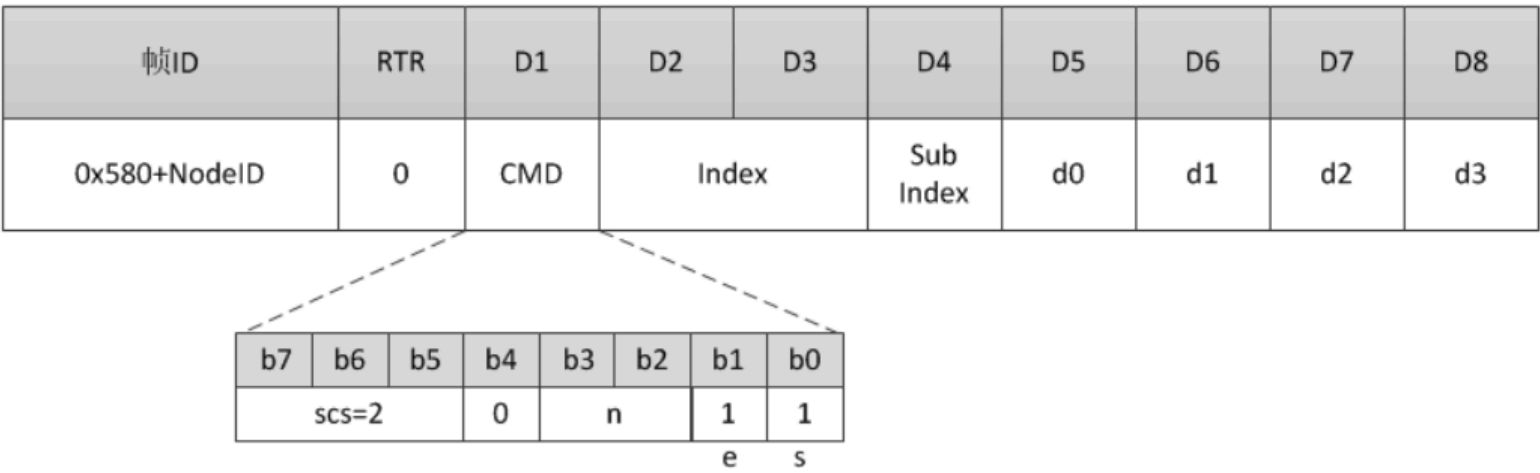
客户端请求：

帧ID	RTR	D1	D2	D3	D4	D5	D6	D7	D8
0x600+NodeID	0	CMD	Index		Sub Index	0	0	0	0

0x40

b7	b6	b5	b4	b3	b2	b1	b0
ccs=2			0	0	0	0	0

服务器正常响应：

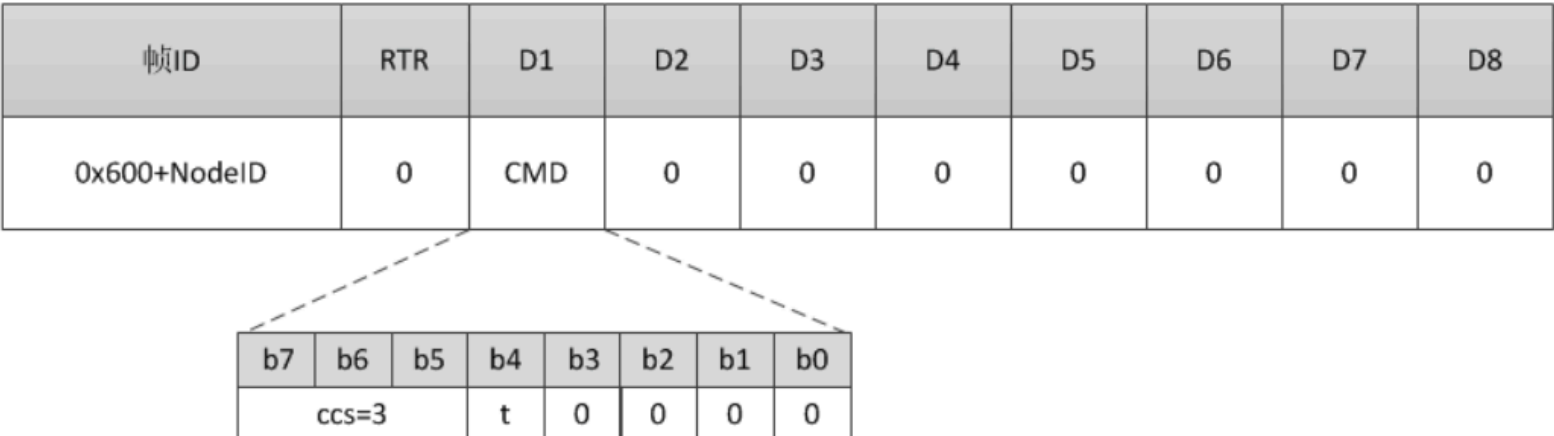


其中n与对象字典的数据长度有关：

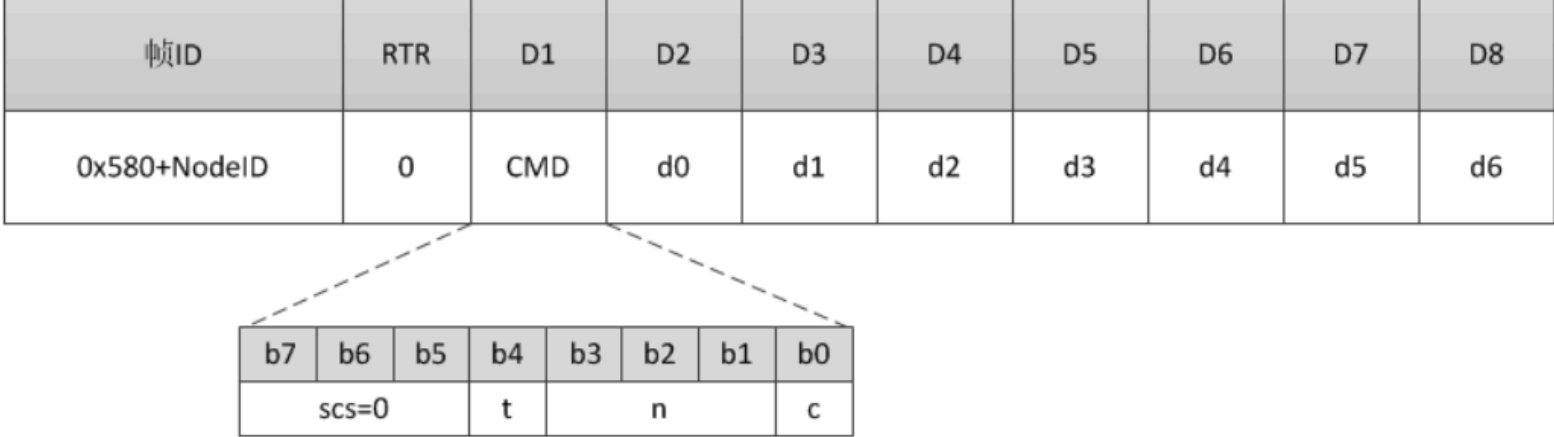
n	对象字典长度(字节)	CMD 值	有效的数据
0	4	0x43	d0,d1,d2,d3
1	3	0x47	d0,d1,d2
2	2	0x4B	d0,d1
3	1	0x4F	d0

4. SDO upload normal帧格式

客户端请求：



服务器响应：



- t:翻转位，每传输一次segment翻转一次，请求和响应中的t必须相等。
- n:代表数据d0~d6中无效数据的长度，n=0表示7个字节数据均有效。
- c:等于0表示还有更多数据等待传输，等于1表示传输完毕。

5. 错误响应

当发生错误时，服务器返回错误响应：

帧ID	RTR	D1	D2	D3	D4	D5	D6	D7	D8
0x580+NodeID	0	0x80	Index	Sub Index	错误码				

常见的错误码在def.h中定义：

```
/* definitions used for object dictionary access. ie SDO Abort codes . (See DS 301 v.4.02 p.48)
*/
#define OD_SUCCESSFUL 0x00000000
#define OD_READ_NOT_ALLOWED 0x06010001
#define OD_WRITE_NOT_ALLOWED 0x06010002
#define OD_NO_SUCH_OBJECT 0x06020000
#define OD_NOT_MAPPABLE 0x06040041
#define OD_LENGTH_DATA_INVALID 0x06070010
#define OD_NO_SUCH_SUBINDEX 0x06090011
#define OD_VALUE_TOO_LOW 0x06090031 /* Value range test result */
#define OD_VALUE_TOO_HIGH 0x06090032 /* Value range test result */
/* Others SDO abort codes
*/
#define SDOABT_TOGGLE_NOT_ALTERNED 0x05030000
#define SDOABT_TIMED_OUT 0x05040000
#define SDOABT_OUT_OF_MEMORY 0x05040005 /* Size data exceed SDO_MAX_LENGTH_TRANSFERT */
#define SDOABT_GENERAL_ERROR 0x08000000 /* Error size of SDO message */
#define SDOABT_LOCAL_CTRL_ERROR 0x08000021
```

6. SDO download

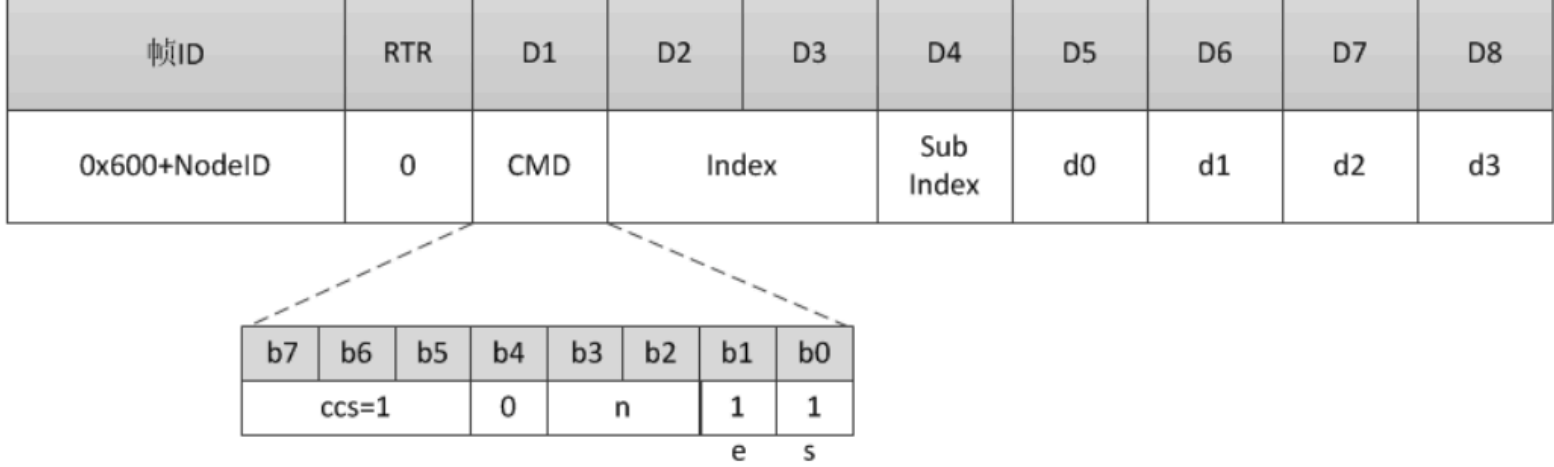
当需要写节点中的对象字典的值时，使用SDO download协议。

当对象字典的长度小于或等于4个字节时，使用SDO download expedited。

当对象字典的长度大于4个字节时，使用SDO download normal(segment)。

7. SDO download expedited 帧格式

客户端发送：



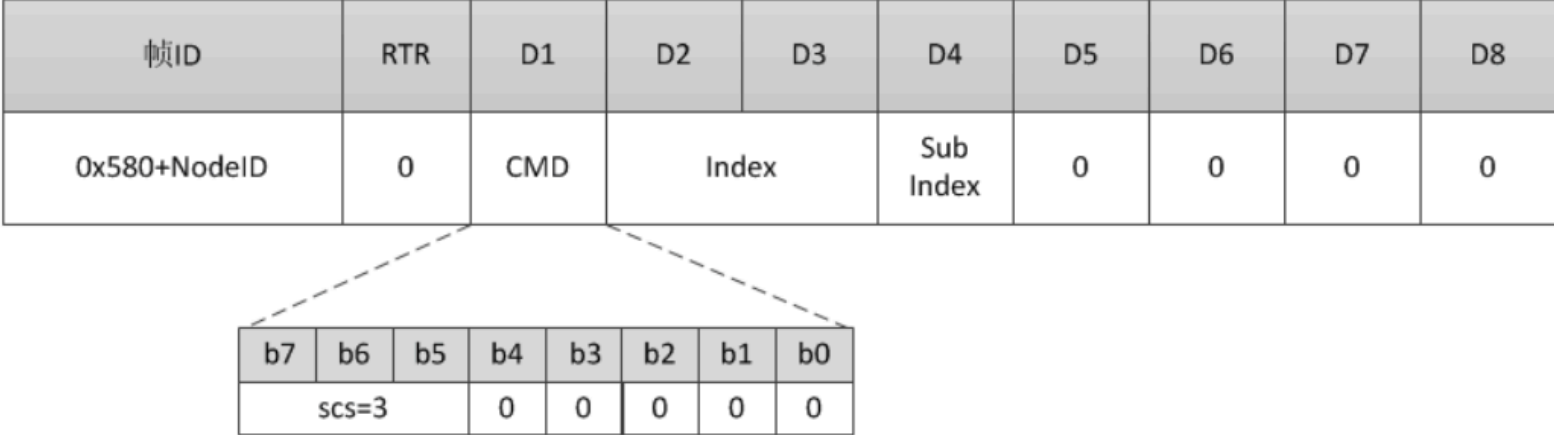
其中，n与对象字典的长度有关：

n	对象字典长度(字节)	CMD 值	有效的数据
0	4	0x23	d0,d1,d2,d3
1	3	0x27	d0,d1,d2
2	2	0x2B	d0,d1
3	1	0x2F	d0

服务器正常响应：

服务器正常相应的CMD 为： 0x60

服务器不正常相应的CMD 为： 0x80



服务器错误响应：

帧ID	RTR	D1	D2	D3	D4	D5	D6	D7	D8
0x580+NodeID	0	0x80	Index	Sub Index	错误码				

3.8 定义自己设备的通信协议

1. 地址索引范围

我们使用制造商特定子协议(Manufacturer-specific Profile)

Index range索引范围	Description描述
0000h	Reserved保留
0001h to 025Fh	Data types数据类型
0260h to 0FFFh	Reserved保留
1000h to 1FFFh	Communication profile area通讯对象子协议区
2000h to 5FFFh	Manufacturer-specific profile area制造商特定子协议区
6000h to 9FFFh	Standardized profile area标准化设备子协议区
A000h to AFFFh	Network variables网络变量（符合IEC61131-3）
B000h to BFFFh	System variables用于路由网关的系统变量
C000h to FFFFh	Reserved保留

对象字典索引2000h to 5FFFh为制造商特定子协议，通常是存放所应用子协议的应用数据。

我们使用地址2000h作为我们读写的地址。

2. 写设备协议

我们的设备控制使用1个8位的数据即可。

Index 索引地址	之索引	名称	控制的位	位说明
0x2000	0x1	LED1	0b0000 0001	为1开，为0关
0x2000	0x1	LED2	0b0000 0010	为1开，为0关
0x2000	0x1	BEEP	0b0000 0100	为1开，为0关
0x2000	0x1	RELAY	0b0000 1000	为1开，为0关

例如：

写设备的命令，我们写1个字节的数据，从机地址设置为1时，使用SDO的download协议通信。

发送打开设备命令：

```
// 发送数据如下：
// 地址： 0x601
// DLC : 8
// 数据 : 2f 00 20 01 0f 00 00 00

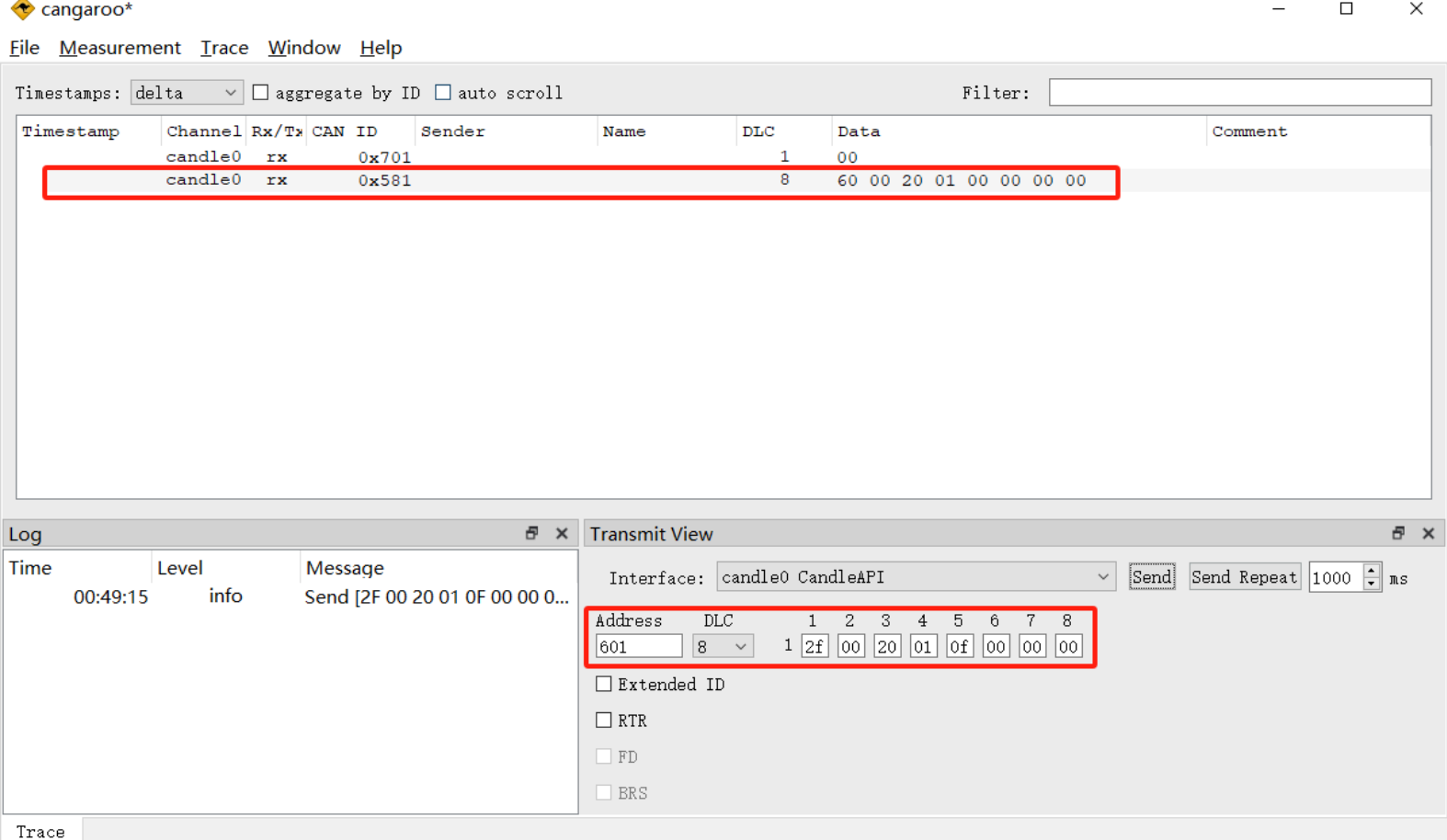
// 命令解析：
// 地址： 0x601 ，这个就是 0x600 + NodeID
// DLC : 8 ，发送字节数
// 2f : 表示1个字节有效
// 00 20 : 表示索引地址为2000
// 01 : 表示字索引为1
// 0f : 0b0000 1111 ，设备全部工作
```

设备返回数据：

```
// CANID : 0x581
// DLC : 8
// 数据 : 60 00 20 01 00 00 00 00

// 命令解析：
// CANID : 0x581 ，这个就是 0x580 + NodeID
// DLC : 8 ，接收字节数
// 60 : 数据正常相应
// 00 20 : 表示索引地址为2000
// 01 : 表示字索引为1
```

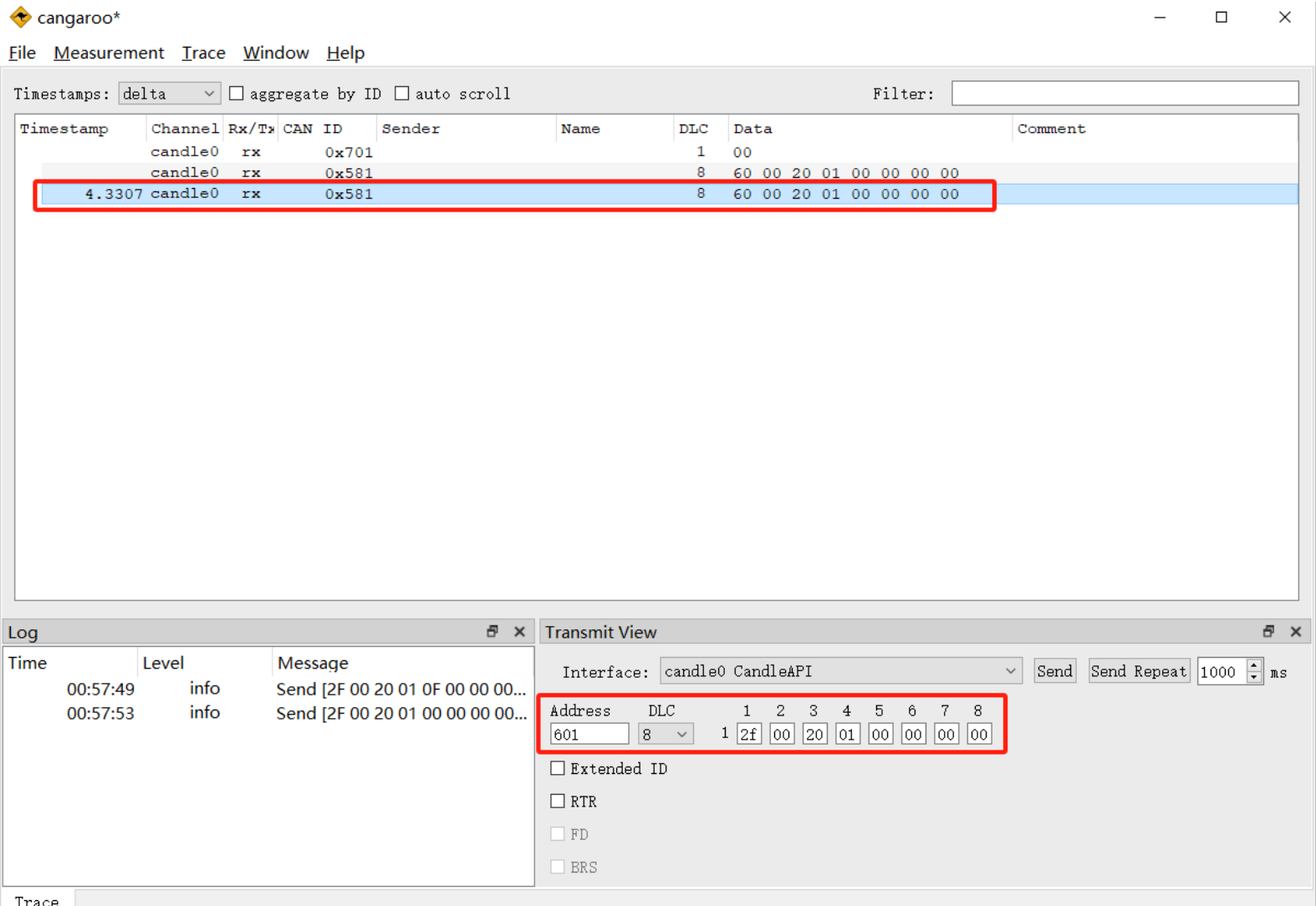
这部分是展示通信过程，这个实验需要完成对象字典后才可以。



关闭设备：

```
// 发送命令：
// 地址： 0x601
// DLC : 8
// 数据 : 2f 00 20 01 00 00 00 00

// 设备返回数据：
// 地址： 0x581
// DLC : 8
// 数据 : 2f 00 20 01 00 00 00 00
```



3. 读传感器协议

我们的传感器数据使用1个16位的数据即可。

读设备传感器数据：

Index 索引地址	之索引	名称	2字节
0x2000	0x2	温度	uint16
0x2000	0x3	湿度	uint16
0x2000	0x4	电压	uint16
0x2000	0x5	电流	uint16

Index 索引地址	之索引	名称	2字节
0x2000	0x6	功率	uint16
0x2000	0x7	电位器电压	uint16
0x2000	0x8	CPU温度	uint16

例如:

读温度传感器数据，从机地址设置为1时，使用SDO的upload协议通信。

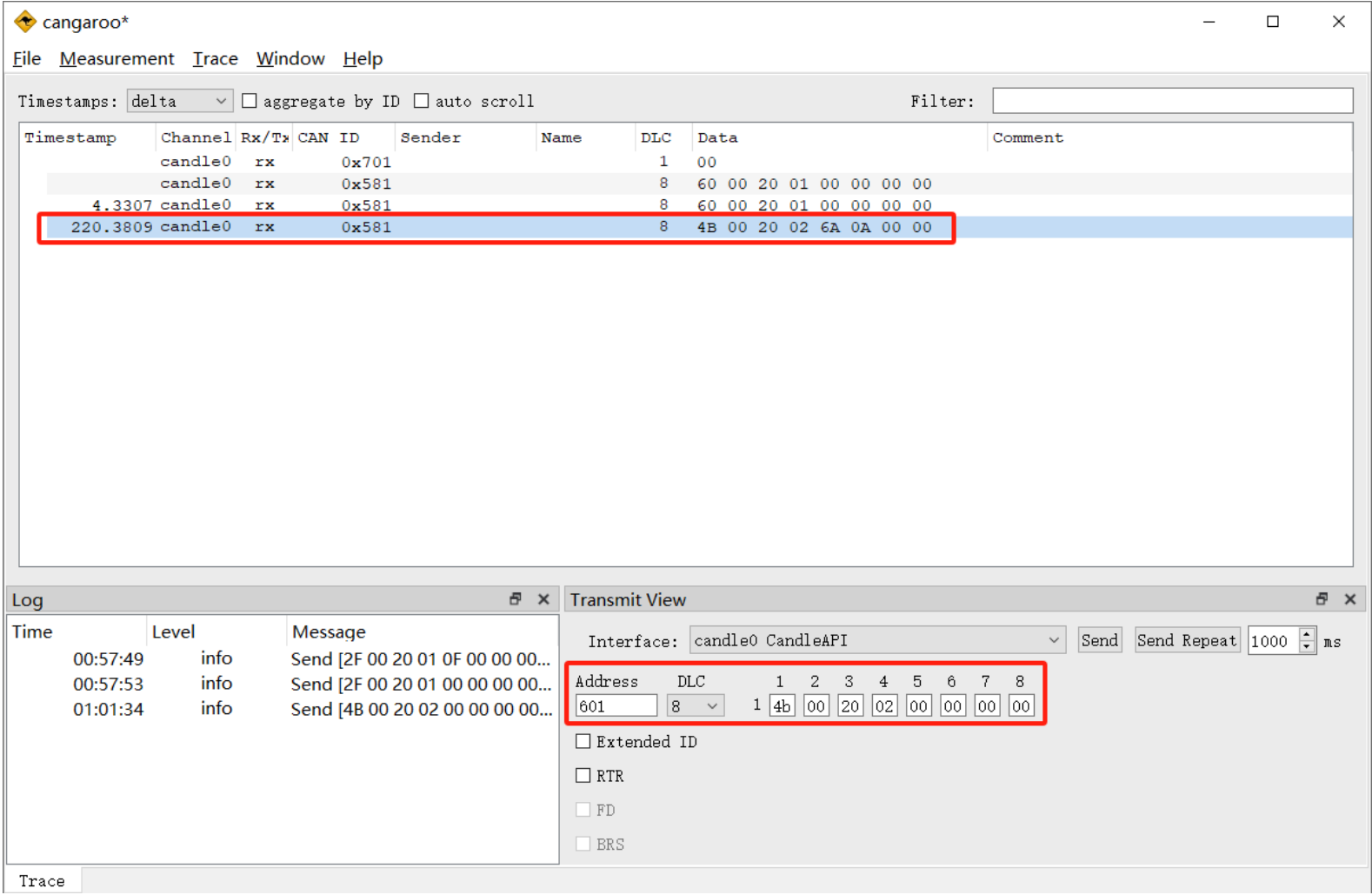
```
// 发送数据如下：
// 地址： 0x601
// DLC : 8
// 数据 : 4b 00 20 02 00 00 00 00

// 命令解析：
// 地址： 0x601 ， 这个就是 0x600 + NodeID
// DLC : 8 ， 发送字节数
// 4b : 表示2个字节有效
// 00 20 : 表示索引地址为2000
// 02 : 表示字索引为2
```

设备返回的数据

```
// CANID : 0x581
// DLC : 8
// 数据 : 4B 00 20 02 6A 0A 00 00

// 命令解析：
// CANID : 0x581 ， 这个就是 0x580 + NodeID
// DLC : 8 ， 接收字节数
// 4b : 表示2个字节有效
// 00 20 : 表示索引地址为 0x2000
// 02 : 表示字索引为2
// 6A 0A : 读回来的数据是 0xA6A
```

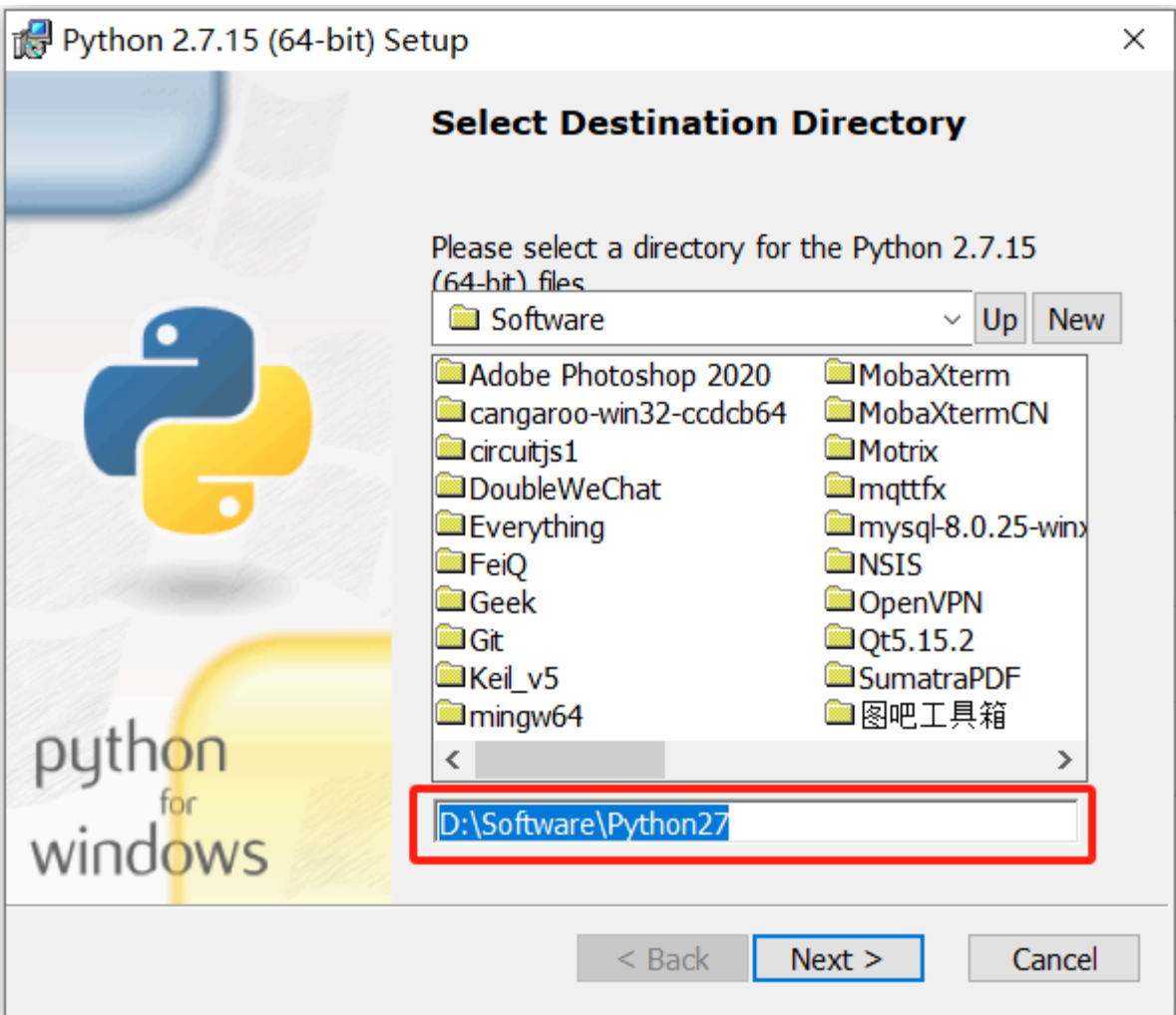


3.9 生成自己设备的对象字典

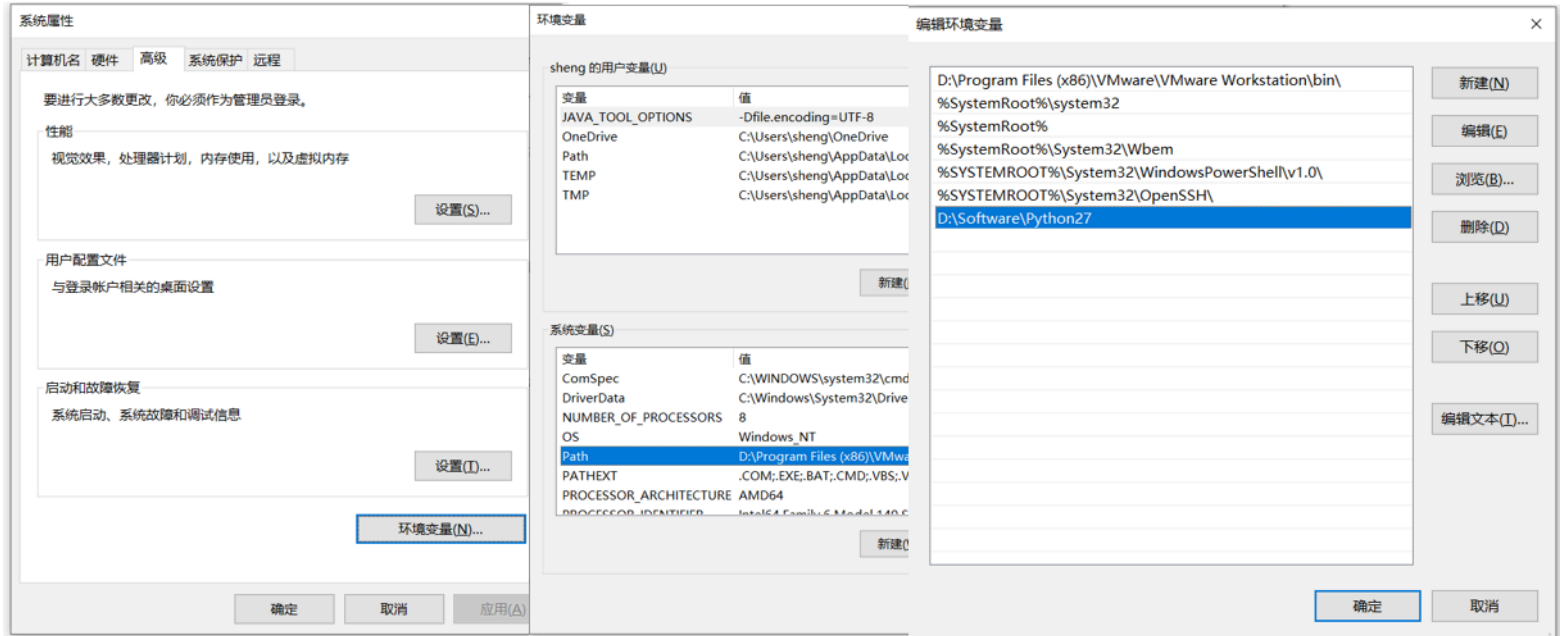
1. 对象字典生成工具环境搭建

双击运行\STM32F103开发板资料V9.0\01-工具软件\python-2.7.15.amd64.msi，选择默认路径安装即可。

安装路径可以选择默认，也可以选择自定义。

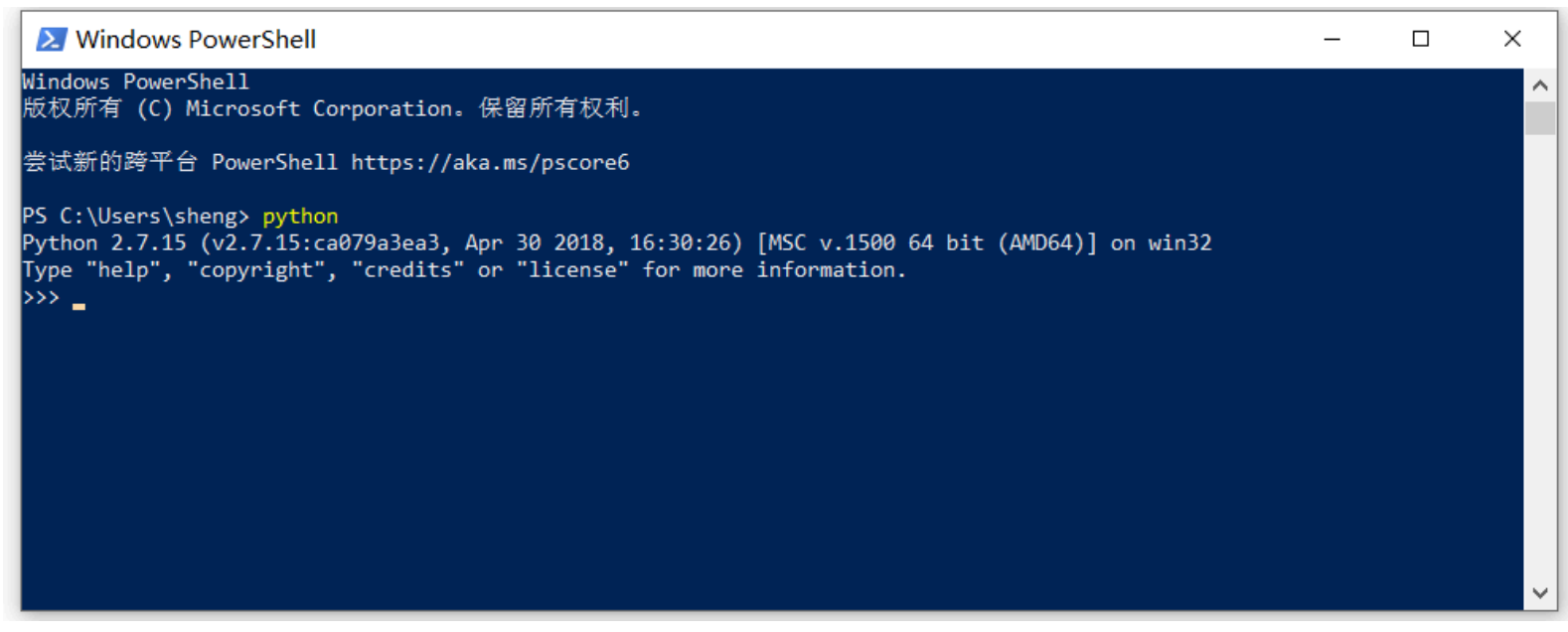


添加环境变量



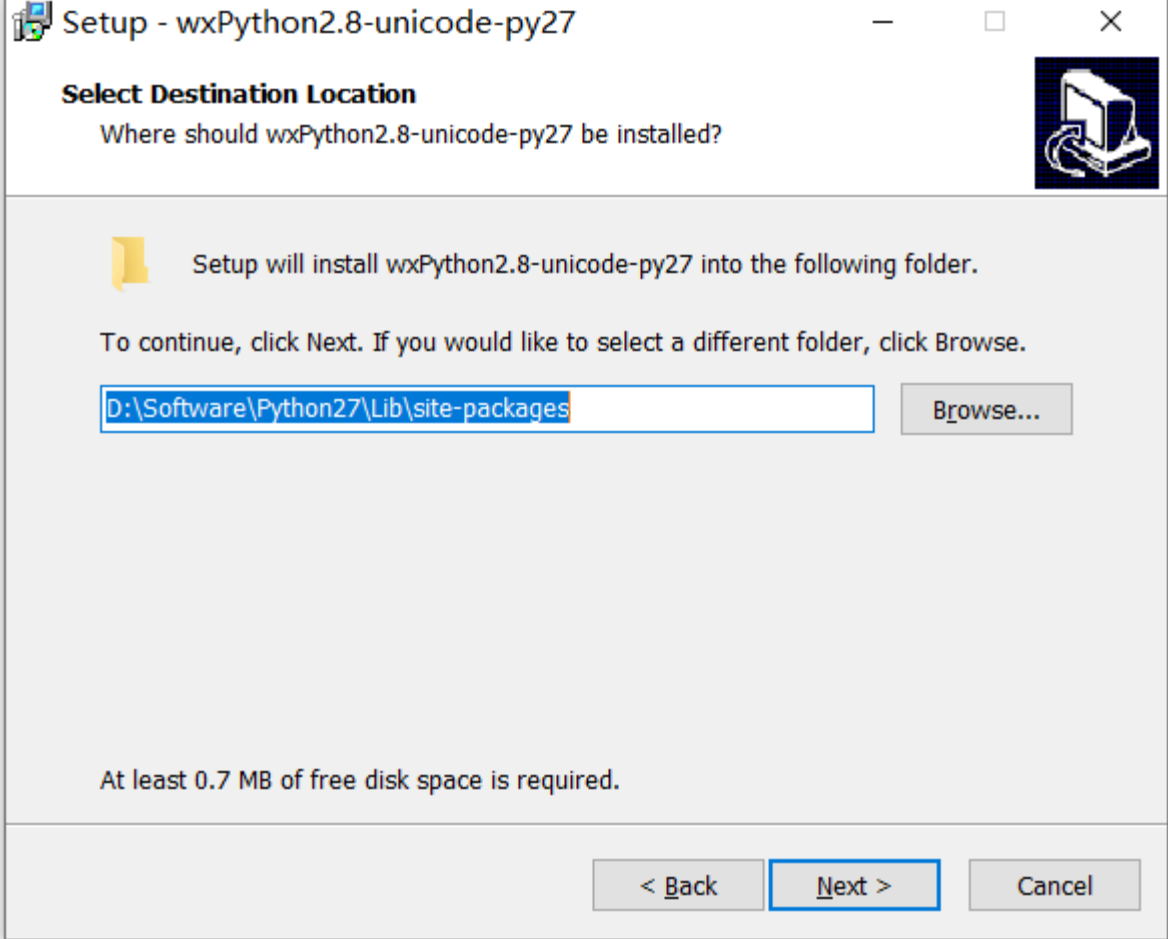
测试python环境

右击Windows图标，运行 Windows PowerShell，输入 python 测试环境安装是否成功。



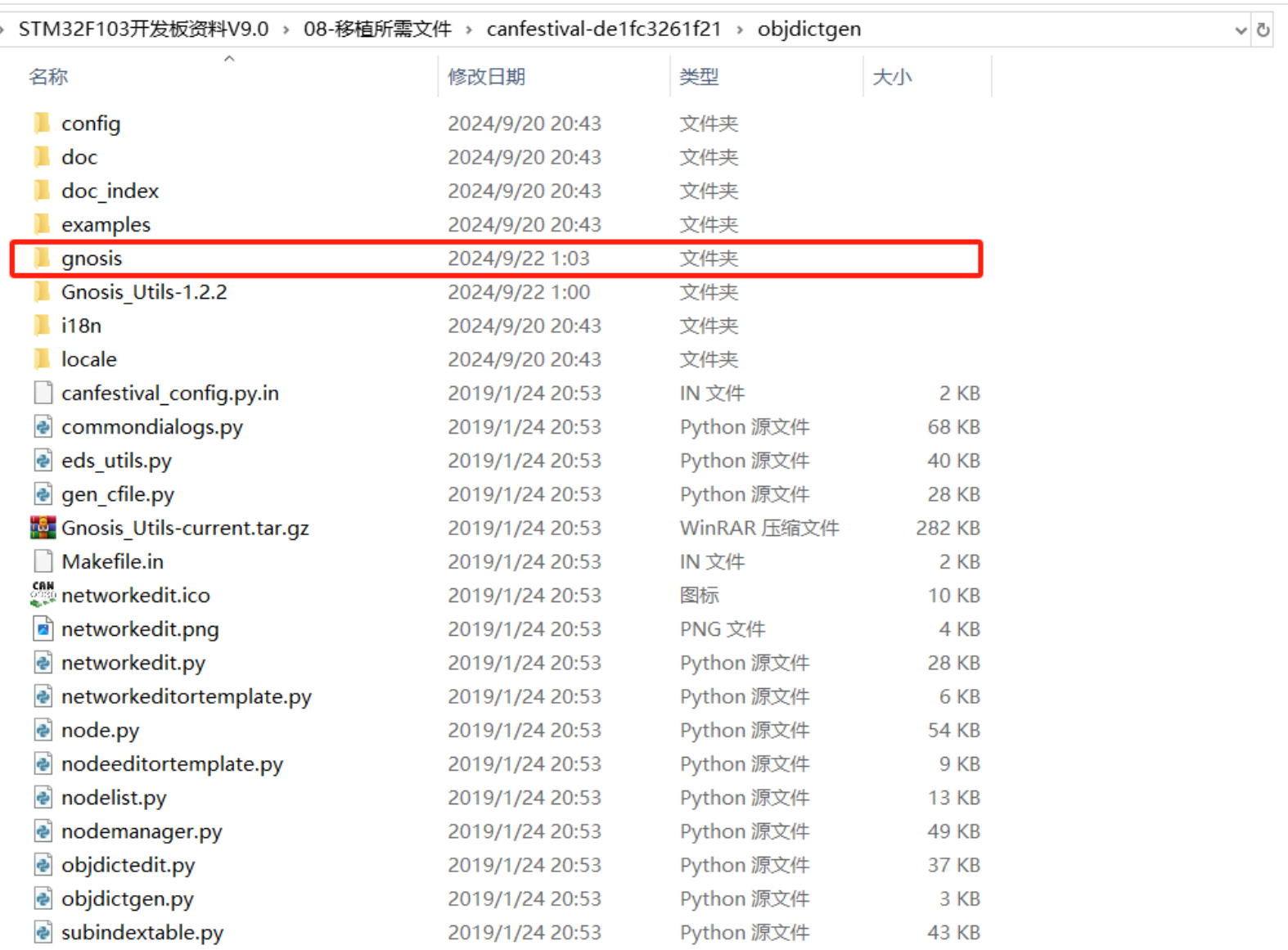
安装wxPhthon2.8工具，这个Python的图形界面工具。

双击运行\STM32F103开发板资料V9.0\01-工具软件\wxPython2.8-win64-unicode-2.8.12.1-py27.exe，会自动选择Python的安装路径进行安装，我们选默认即可。

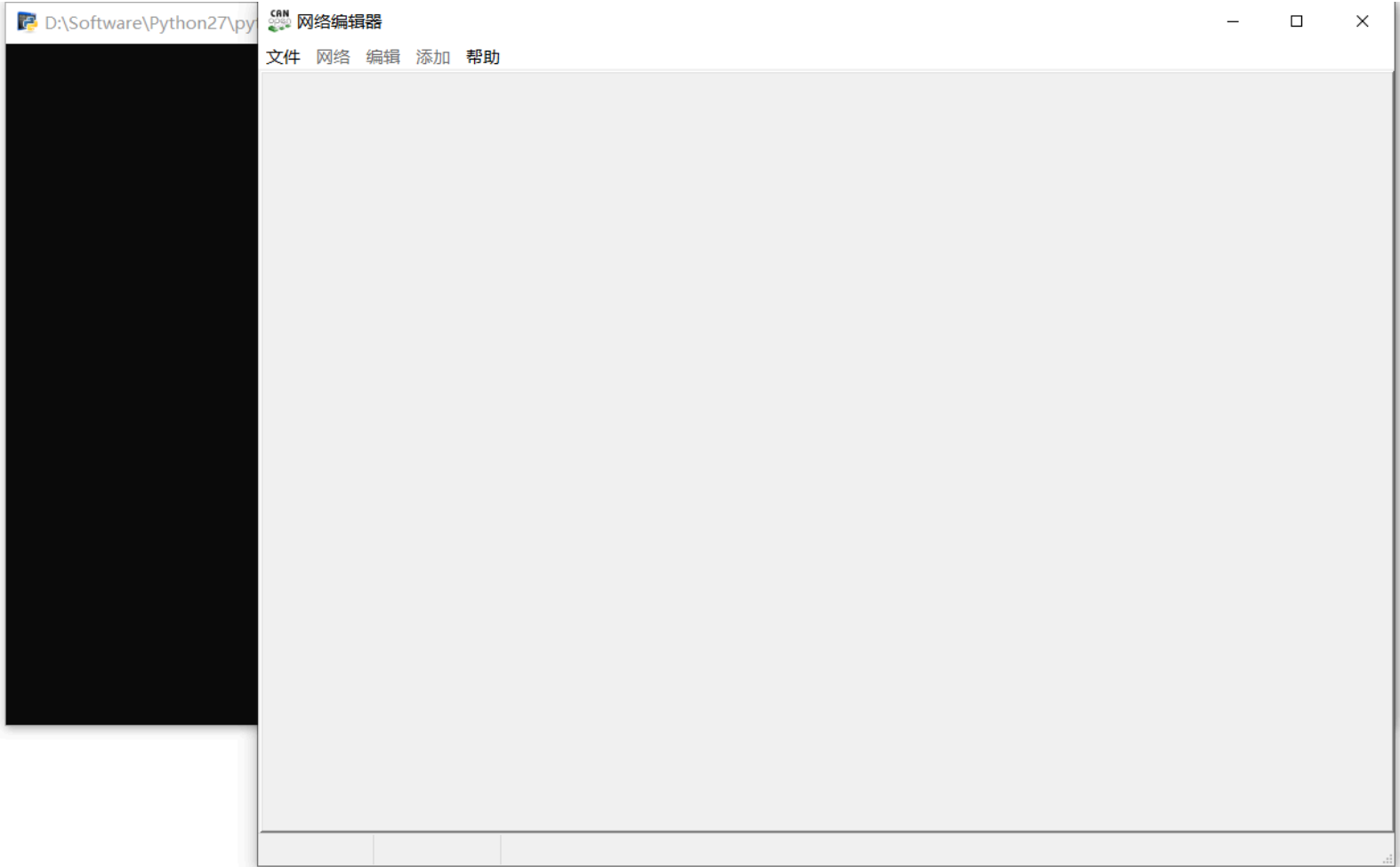


点击下一步，直到安装完成。

找到目录objdictgen目录下的Gnosis_Utills-current.tar.gz进行解压。复制 Gnosis_Utills-1.2.2 目录下的gnosis文件夹到objdictgen目录下。



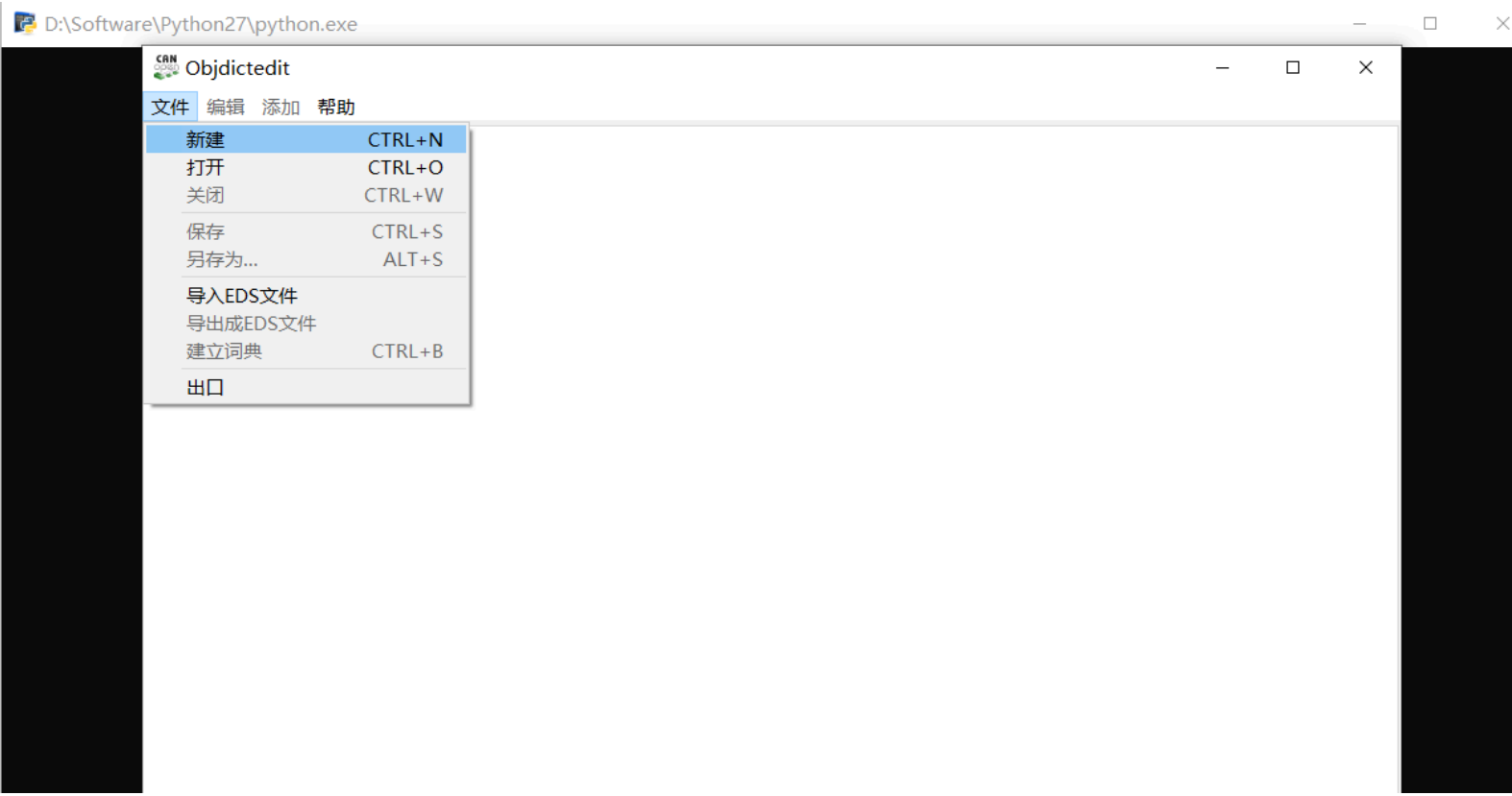
双击objdictedit.py，如果环境没有问题，就应该会出现如下界面。



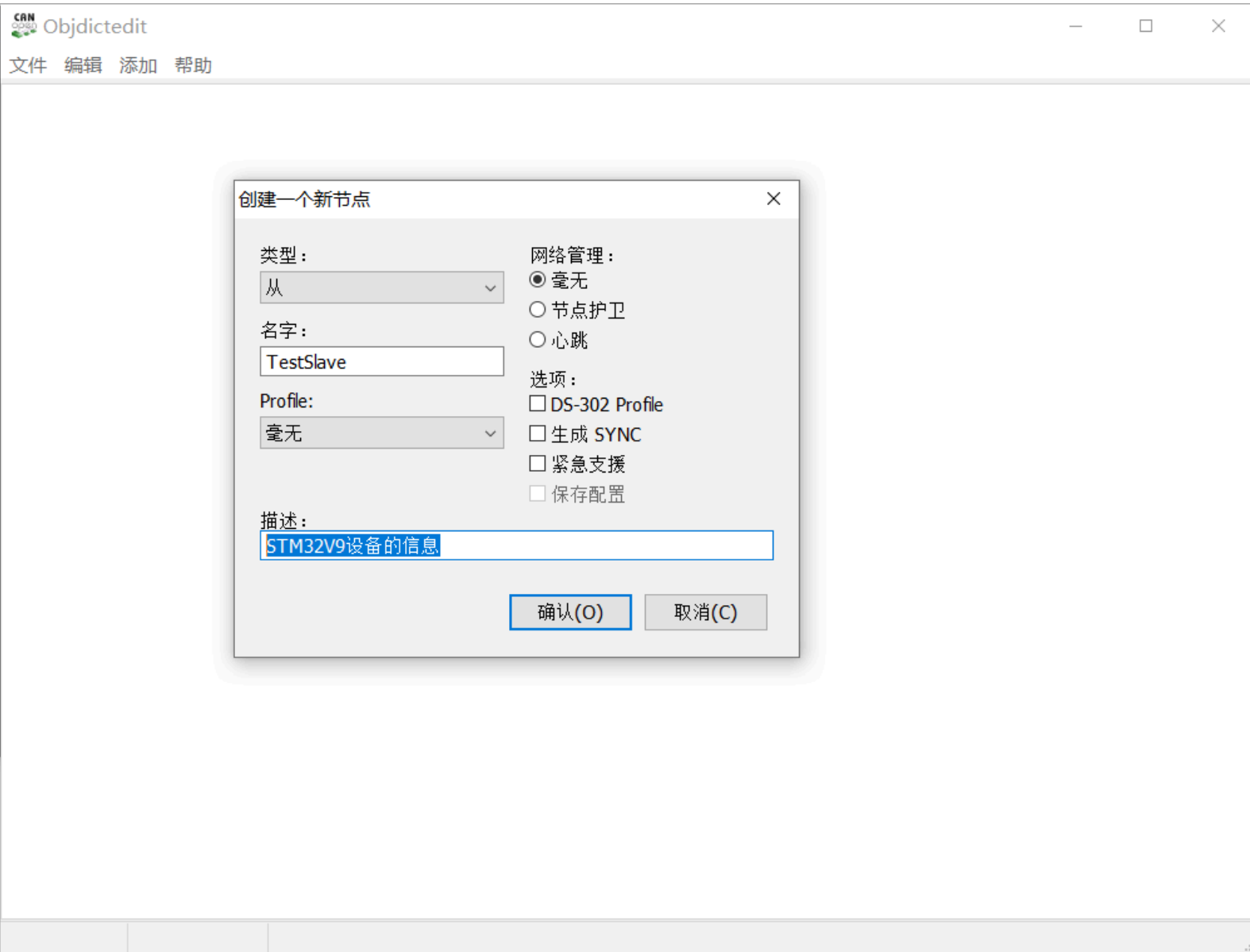
至此环境搭建完成。

2. 创建设备对象字典

双击运行objdictgen目录下的objdictedit.py，并点击文件 -> 新建，新建一个对象字典文件。



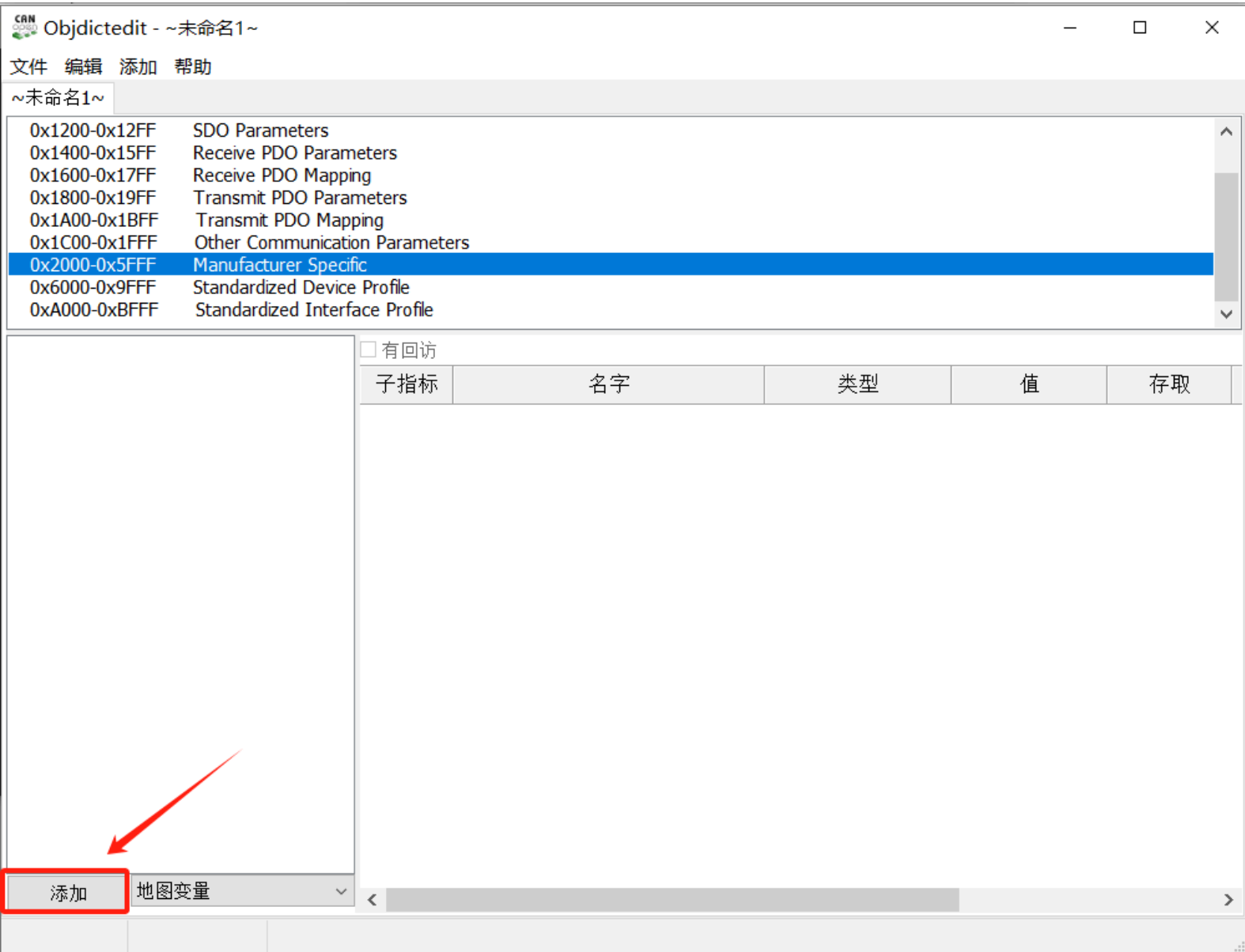
- 类型： 可选主、从节点， 我们这里选择从设备。
- 名字： 自己定义节点名称。
- Profile： 配置，如DS401等，我们这里不需要；
- 网络管理： 我们选择默认即可。



配置对象字典

创建节点确认后，会进入对象字典配置界面。

我们选择 0x2000 - 0x5fff 地址范围，这里设置厂商的参数。



这里选择数据类型：

- 指标： 0x2000 ， 表示的是地址
- 名字： DeviceInfo ， 可以随意命名即可。
- 类型：
 - VAR ： 单一的一个变量， 只能创建一个子索引
 - ARRAY ： 数组， 可以有很多个类型一样的数据
 - REC ： 结构体数据， 可以有多个类型不一样的成员数据。
- 数字： 表示数组或结构体成员的个数， 也就是子索引的个数

这里我们设置10个子索引，就是10个16位的寄存器，保持和Modbus与MQTT协议保持一致。

添加地图变量

指标：
0x2000

名字：
DeviceInfo

类型：
☐ VAR
☐ ARRAY
☒ REC

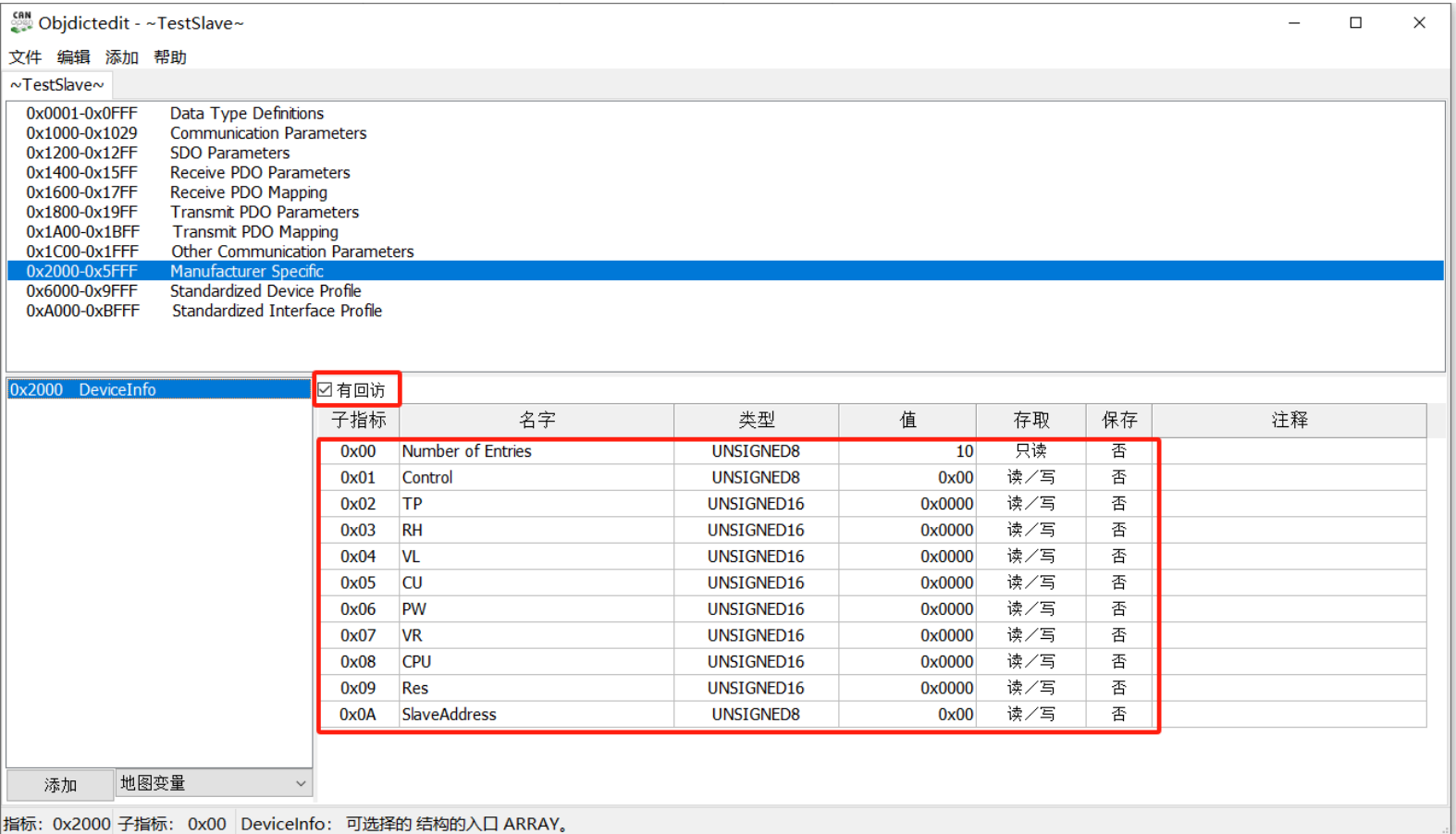
数字：
10

确认(O) 取消(C)

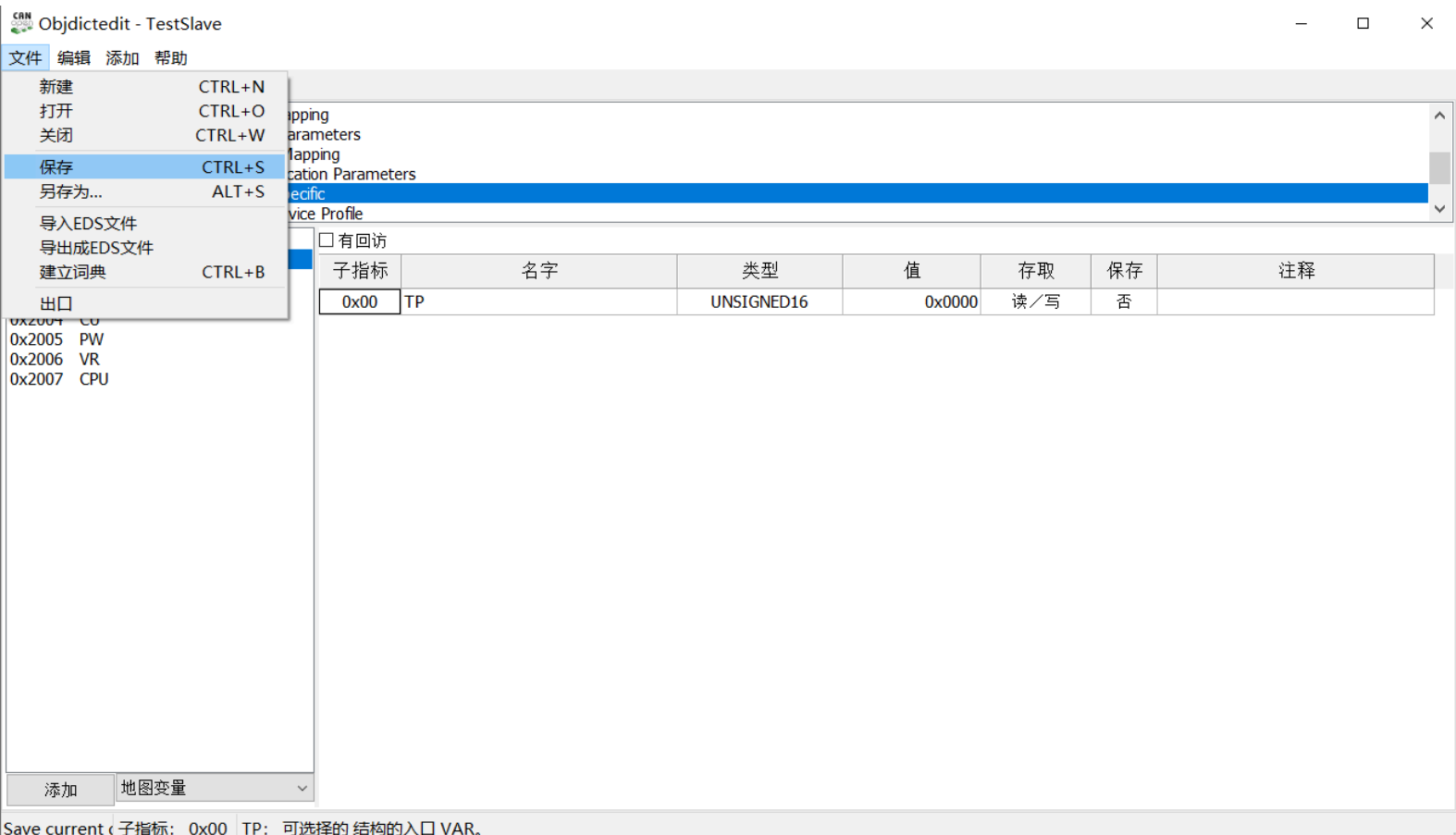
添加如下参数：

索引地址	子索引	名称	变量名称	数据大小
0x2000	0x1	设备控制	Control	1字节
0x2000	0x2	温度	TP	2字节
0x2000	0x3	湿度	RH	2字节
0x2000	0x4	电压	VL	2字节
0x2000	0x5	电流	CU	2字节
0x2000	0x6	功率	PW	2字节
0x2000	0x7	电位器电压	VR	2字节
0x2000	0x8	CPU温度	CPU	2字节
0x2000	0x9	保留	保留	2字节
0x2000	0xA	从机地址	SlaveAddress	1字节

添加完成后如下图：



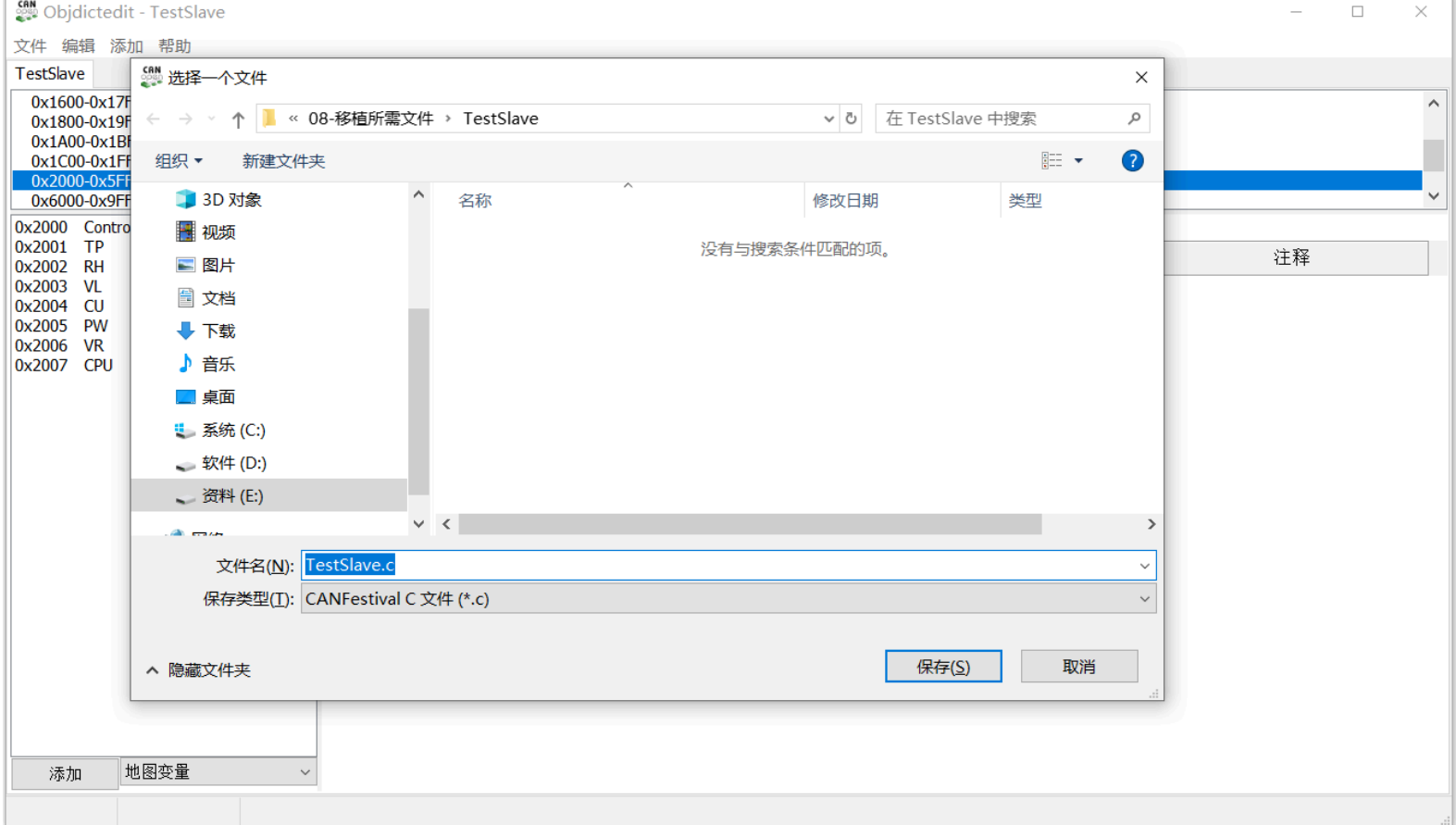
点击文件菜单，然后选择保存文件。



保存文件名选择默认即可。

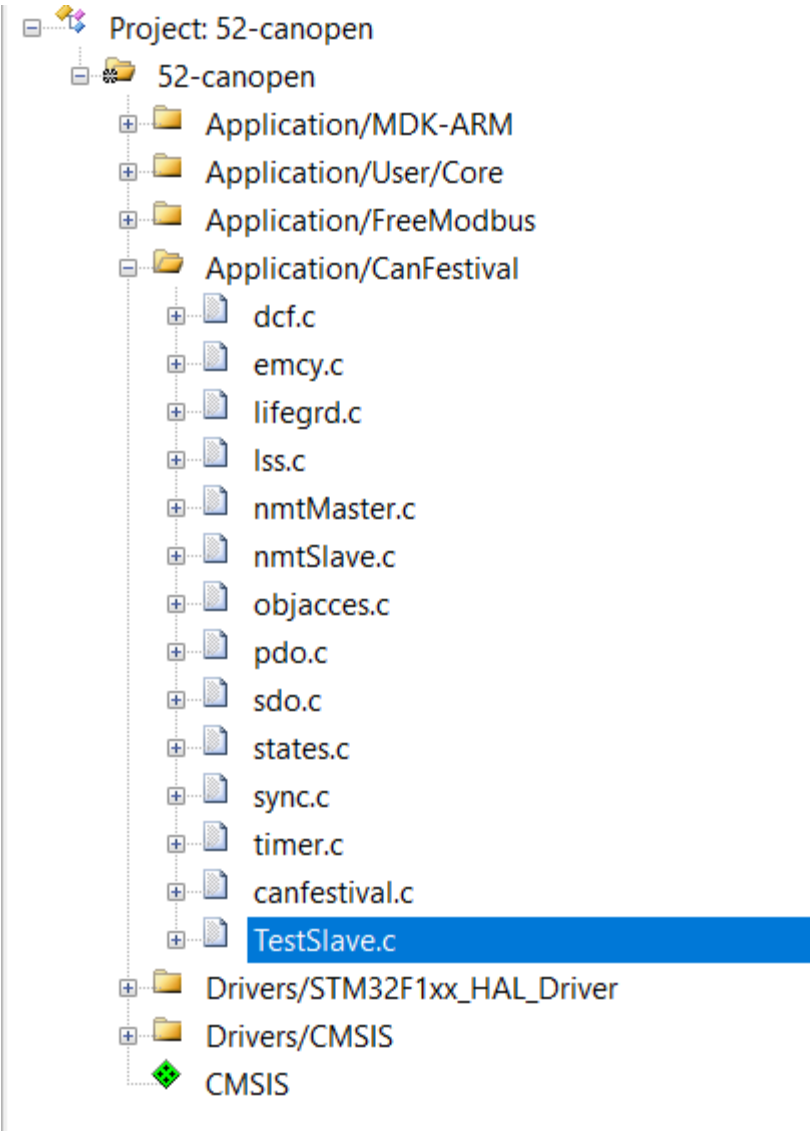
点击文件菜单，点击文件 -> 建立字典，即可生成字典文件，就是生成 .c和.h 文件。

使用默认生成的文件名即可。



点击保存，即可生成 .c 与 .h 文件。

复制 .c 到工程内的 CanFestival\Src 内，复制复制 .h 到工程内的 CanFestival\Inc 内，并把 TestSlave.c 文件添加到工程内。



修为文件里面的内容，保持和Modbus与MQTT协议兼容。

TestSlave.c

```
#include "modbus.h"

UNSI32 Set_SlaveAddress_Callback(CO_Data* d, const indextable *table, UNS8 bSubindex)
{
    //printf("value=%x\r\n",*(uint8_t*)table->pSubindex[bSubindex].pobject);
    Modify_SlaveAddress_Flag = 1;
    REG_HOLD_BUF[9] = *(uint8_t*)table->pSubindex[bSubindex].pobject ;
    return OD_SUCCESSFUL;
}

/*****
/* Declaration of value range types
*****/

/* index 0x2000 : Mapped variable DeviceInfo */
UNSI8 TestSlave_highestSubIndex_obj2000 = 10; /* number of subindex - 1*/
ODCallback_t DeviceInfo_callbacks[] =
{
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    Set_SlaveAddress_Callback, // 设置从机地址的回调函数
};
subindex TestSlave_Index2000[] =
{
    { RO, uint8, sizeof (UNSI8), (void*)&TestSlave_highestSubIndex_obj2000 },
    { RW, uint8, sizeof (UNSI8), (void*)&REG_HOLD_BUF[0] }, // 保持和Modbus协议保持一致
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[1] },
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[2] },
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[3] },
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[4] },
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[5] },
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[6] },
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[7] },
    { RW, uint16, sizeof (UNSI16), (void*)&REG_HOLD_BUF[8] },
    { RW, uint8, sizeof (UNSI8), (void*)&REG_HOLD_BUF[9] }
};
```

优化代码，让地址可以显示在OLED上，并支持CAN协议设置从机地址。

modbus.c

```
void Modbus_Init(void)
{
    uint8_t str[32];
    // 程序执行后，会自动从EEPROM中读取设备的地址
    ReadSlaveAddress(0,&SlaveAddress);
    printf("SlaveAddress:%x\n",SlaveAddress);
    sprintf((char *)str,"SlaveAddress:%2d ",SlaveAddress);
    OLED_ShowStr(0,6, (unsigned char*)str, 2);
    eMBInit(MB_RTU, SlaveAddress, 0, 115200, MB_PAR_NONE); // 初始化modbus为RTU方式，波特率115200，无校验
    eMBEnable();
}

void Modbus_Parse(void)
{
    if(Modify_SlaveAddress_Flag) //修改从机地址
    {
        Modify_SlaveAddress_Flag=0;
        //printf("SlaveAddress:%d\n",REG_HOLD_BUF[9]);
        WriteSlaveAddress(0,REG_HOLD_BUF[9]);
        HAL_Delay(10);
        eMBDisable();
        eMBClose();
        Modbus_Init();

        setNodeId(&TestSlave_Data, SlaveAddress);
    }
}
```

保存编译工程。

3.10 CANopen程序功能验证

1. 控制设备

写设备的命令，我们写1个字节的数据，从机地址设置为1时，使用SDO的download协议通信。

发送打开设备命令：

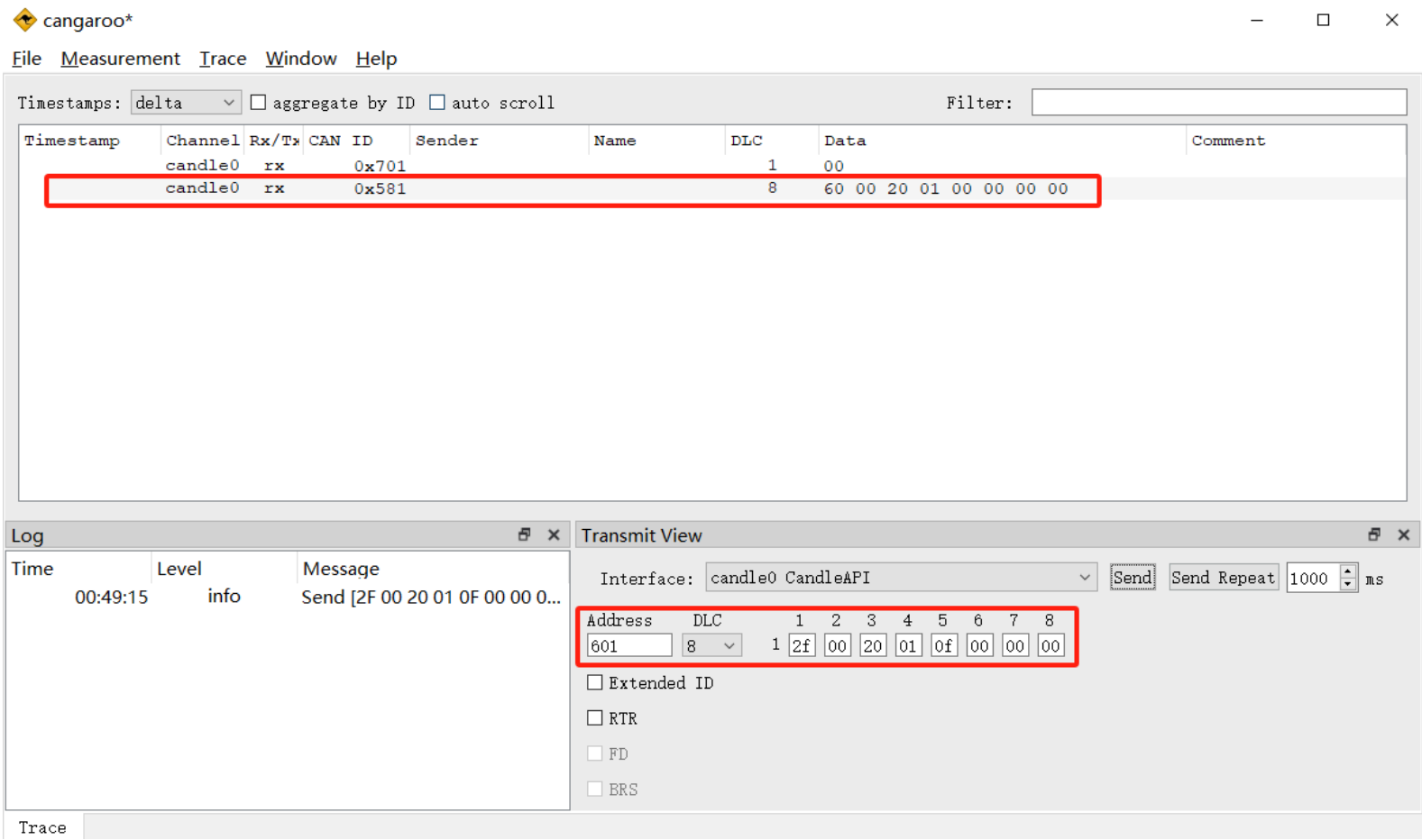
```
// 发送数据如下：
// 地址： 0x601
// DLC ： 8
// 数据 ： 2f 00 20 01 0f 00 00 00

// 命令解析：
// 地址： 0x601 ， 这个就是 0x600 + NodeID
// DLC ： 8 ， 发送字节数
// 2f ： 表示1个字节有效
// 00 20 ： 表示索引地址为2000
// 01 ： 表示字索引为1
// 0f ： 0b0000 1111 ， 设备全部工作
```

设备返回数据：

```
// CANID ： 0x581
// DLC ： 8
// 数据 ： 2f 00 20 01 00 00 00 00

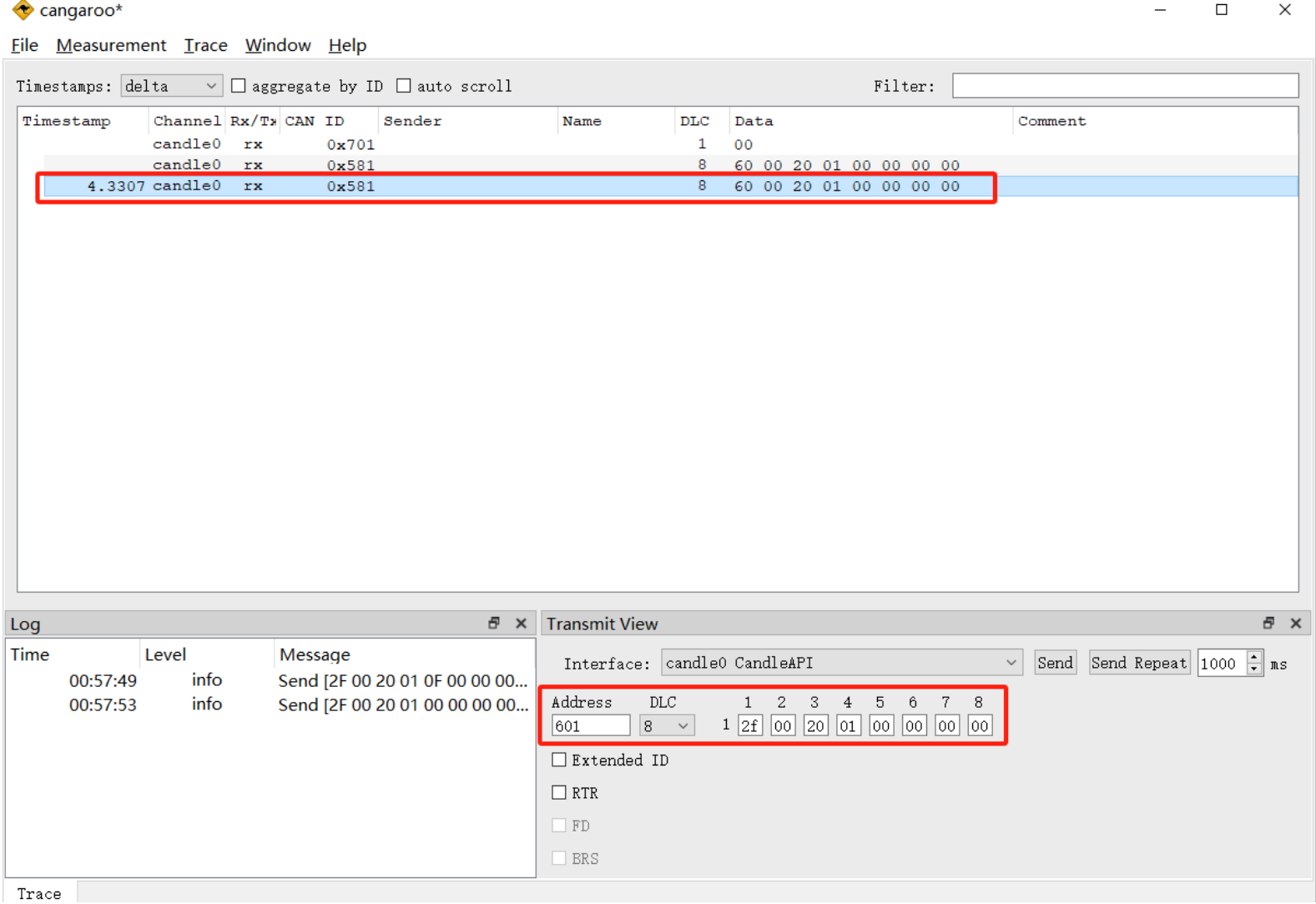
// 命令解析：
// CANID ： 0x581 ， 这个就是 0x580 + NodeID
// DLC ： 8 ， 接收字节数
// 2f ： 表示1个字节有效
// 00 20 ： 表示索引地址为2000
// 01 ： 表示字索引为1
```



关闭设备：

```
// 发送命令：
// 地址： 0x601
// DLC ： 8
// 数据 ： 2f 00 20 01 00 00 00 00

// 设备返回数据：
// 地址： 0x581
// DLC ： 8
// 数据 ： 2f 00 20 01 00 00 00 00
```



2. 读传感器协议

我们的传感器数据使用1个16位的数据即可。

读设备传感器数据：

Index 索引地址	之索引	名称	2字节
0x2000	0x2	温度	uint16
0x2000	0x3	湿度	uint16
0x2000	0x4	电压	uint16
0x2000	0x5	电流	uint16
0x2000	0x6	功率	uint16
0x2000	0x7	电位器电压	uint16
0x2000	0x8	CPU温度	uint16

例如:

读温度传感器数据，从机地址设置为1时，使用SDO的upload协议通信。

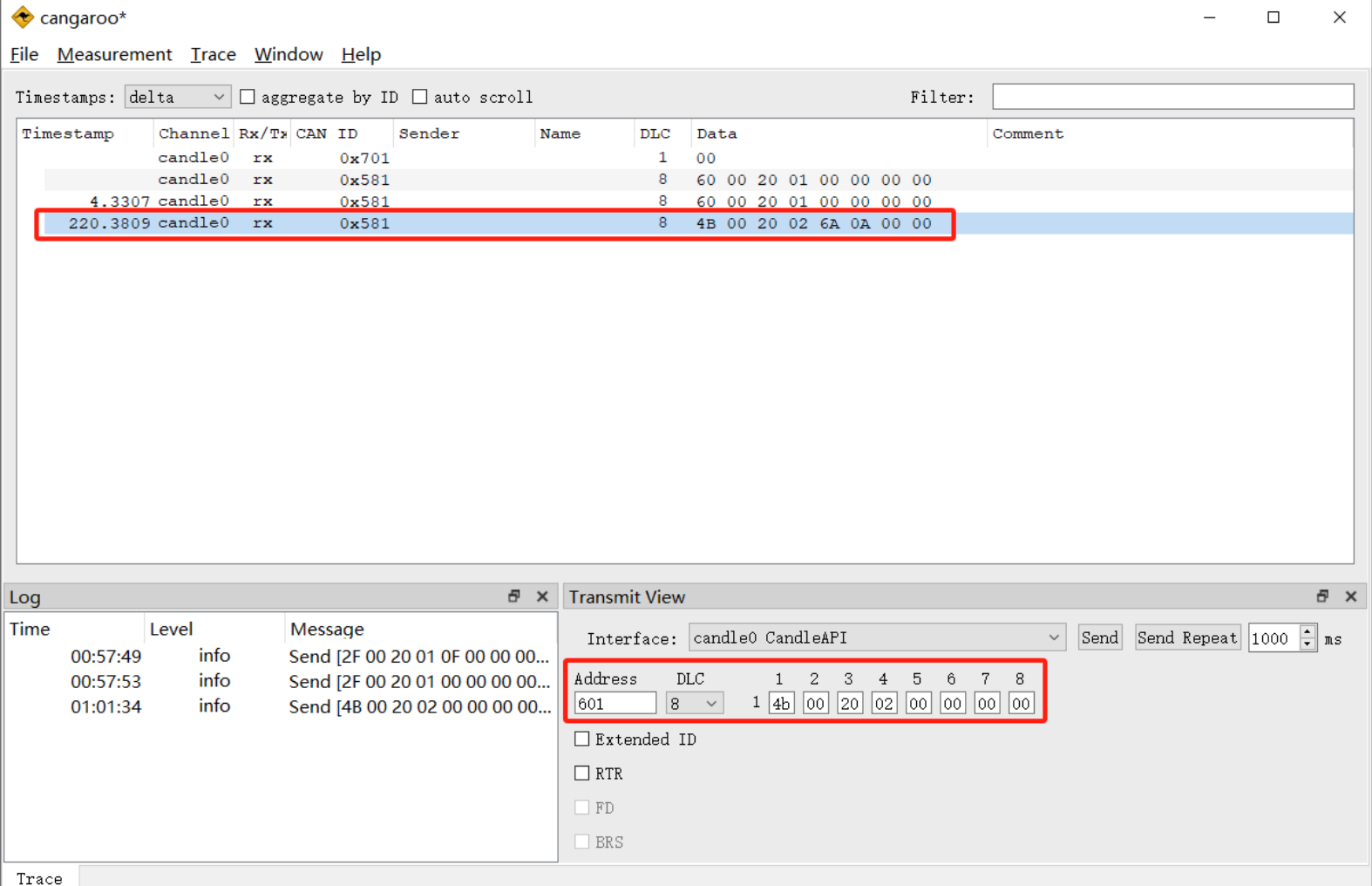
```
// 发送数据如下：
// 地址： 0x601
// DLC ： 8
// 数据 ： 4b 00 20 02 00 00 00 00

// 命令解析：
// 地址： 0x601 ， 这个就是 0x600 + NodeID
// DLC ： 8 ， 发送字节数
// 4b ： 表示2个字节有效
// 00 20 ： 表示索引地址为2000
// 02 ： 表示字索引为2
```

设备返回的数据

```
// CANID ： 0x581
// DLC ： 8
// 数据 ： 4B 00 20 02 6A 0A 00 00

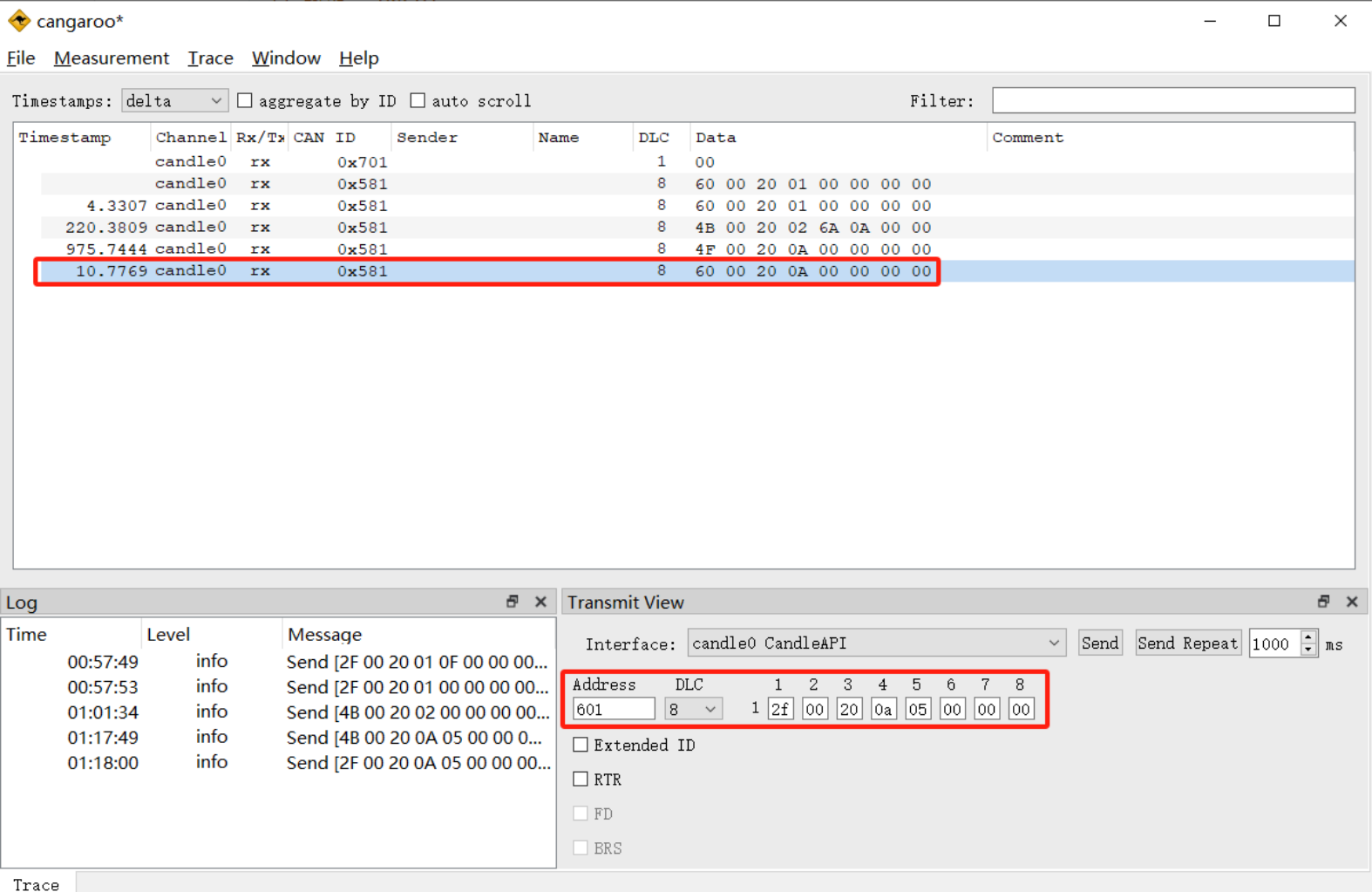
// 命令解析：
// CANID ： 0x581 ， 这个就是 0x580 + NodeID
// DLC ： 8 ， 接收字节数
// 4b ： 表示2个字节有效
// 00 20 ： 表示索引地址为 0x2000
// 02 ： 表示字索引为2
// 6A 0A ： 读回来的数据是 0xA6A
```



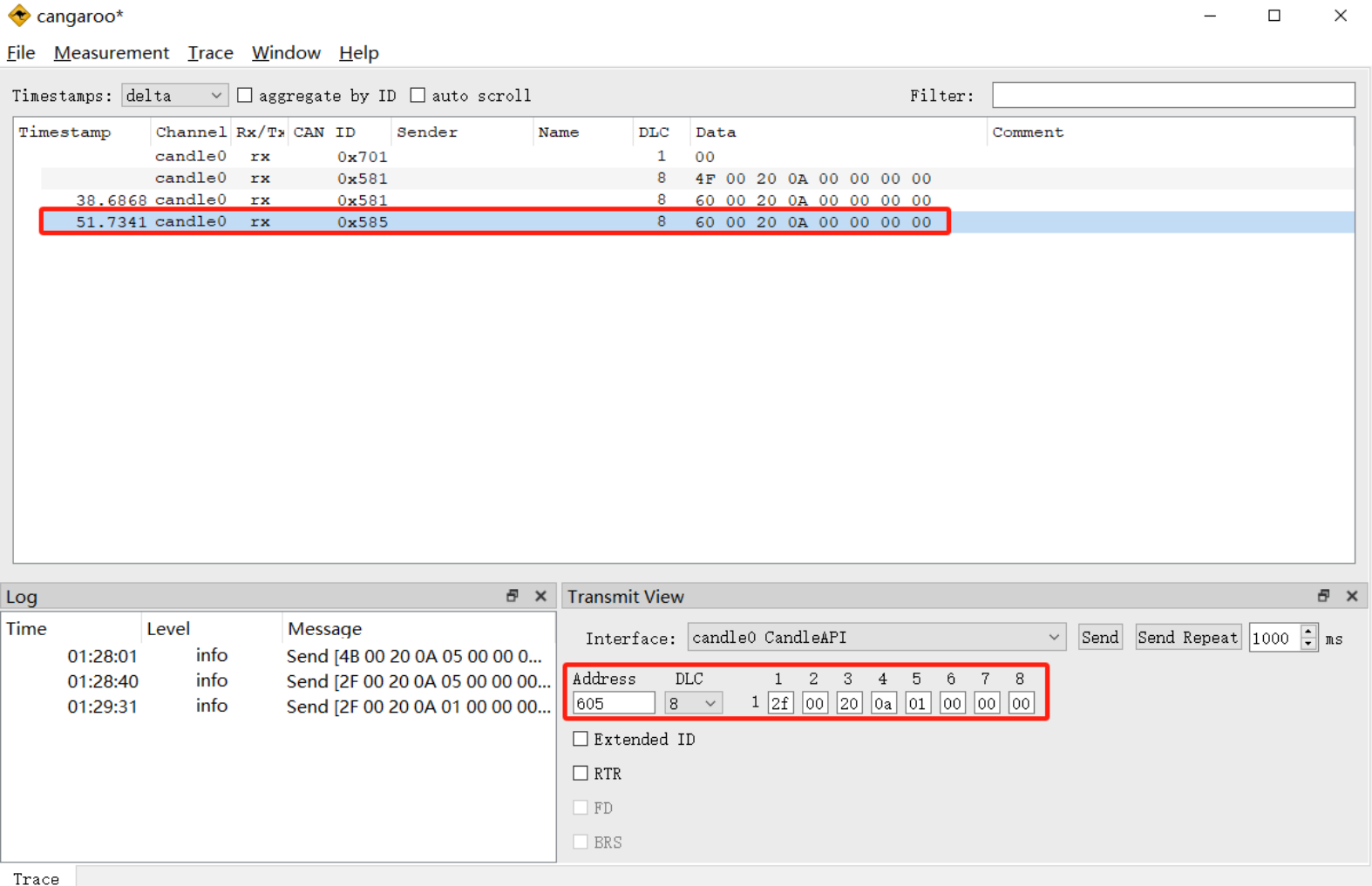
3. 设置地址

写设备地址的命令，我们写1个字节的数据，从机地址设置为1时，使用SDO的download协议通信。

设置从机地址为5。



把从机地址设置为1



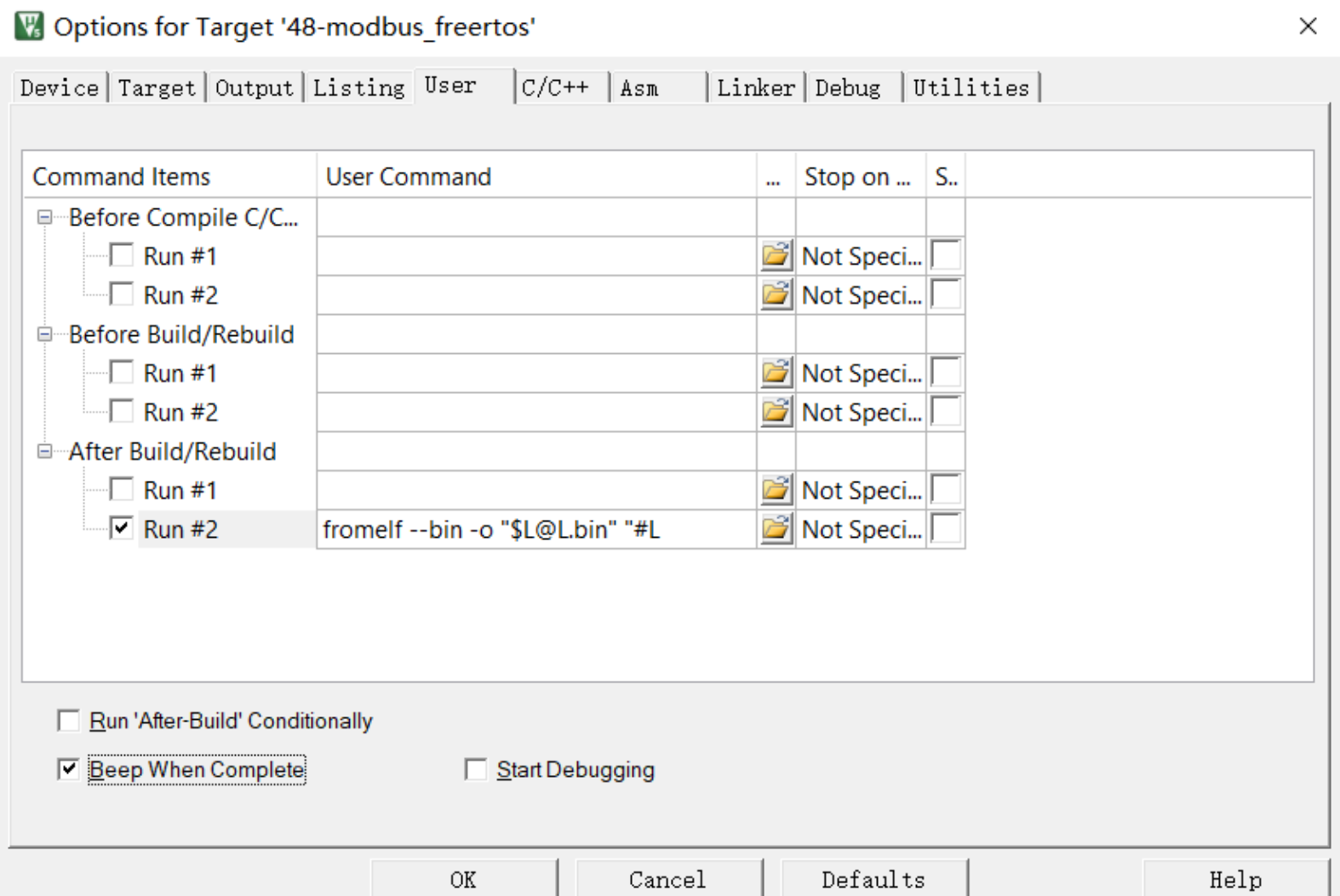
3.11 实战项目7 - CANOpen+Freertos

1. 程序设计要求：

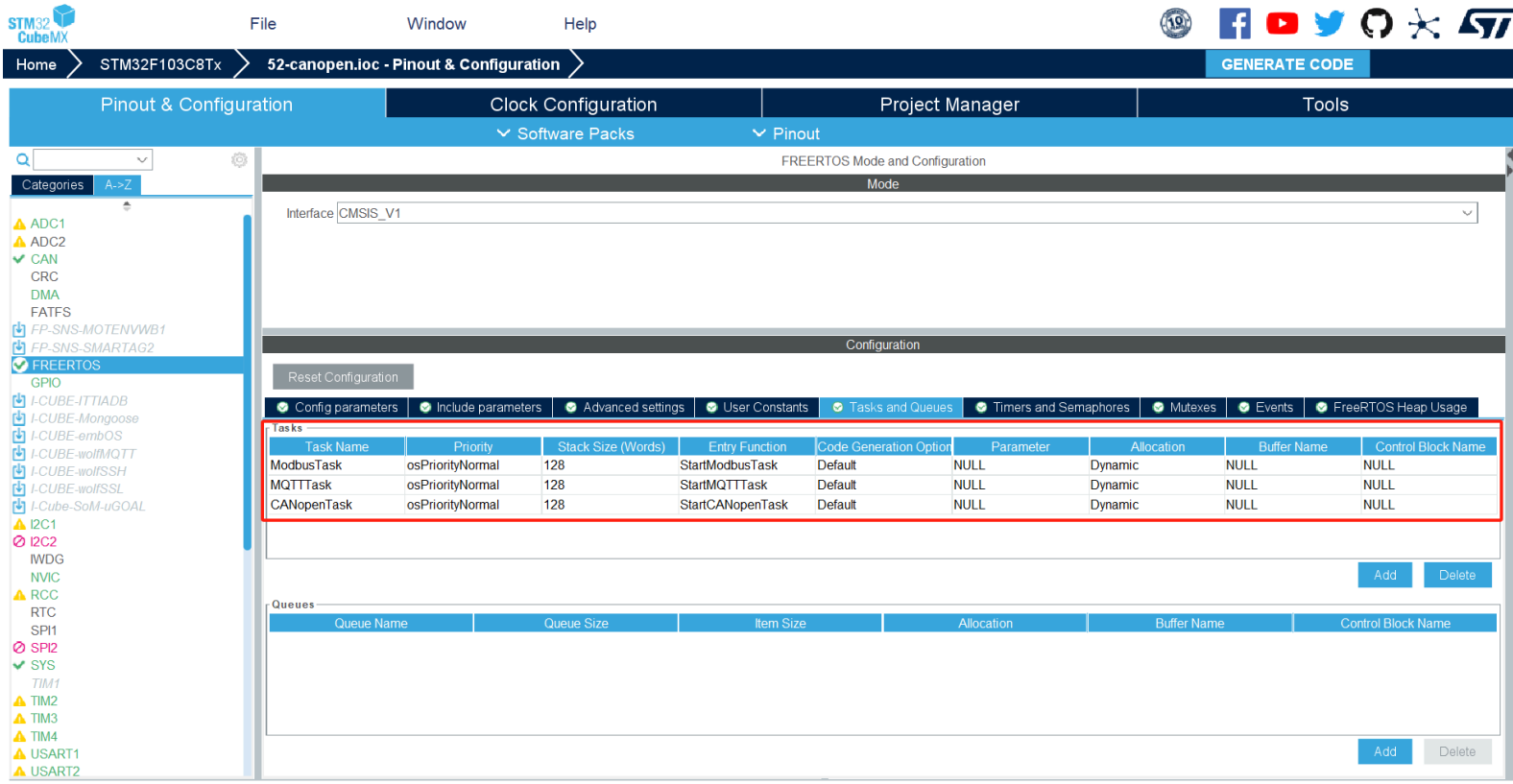
- 移植FreeRTOS操作系统。
- 程序支持CANopen协议通信，支持Modbus通信，同样也支持MQTT协议通信。
- 任务1用来处理modbus协议数据
- 任务2用来处理mqtt协议数据。
- 任务3用来处理canopen协议数据。
- TIM4用来处理MQTT返回数据。
- TIM3用来处理Modbus接收的数据。
- TIM2用来处理CANopen接收的数据。
- 其他需要需要使用定时器的地方使用软件定时器。
- 实时查看生成可执行程序的大小，防止程序超过64K。

2. 生成二进制的命令

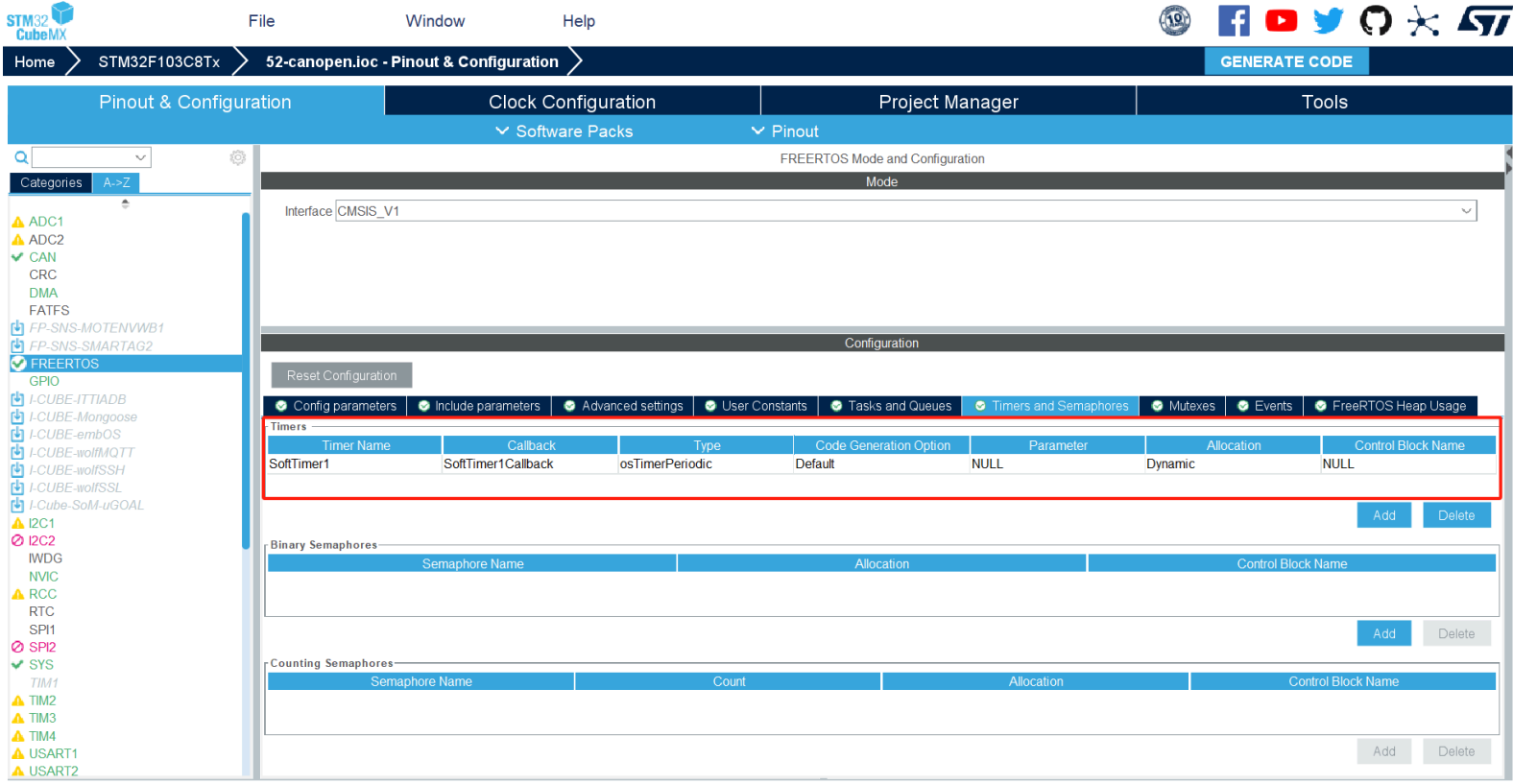
```
fromelf --bin -o "$L@L.bin" "#L
```



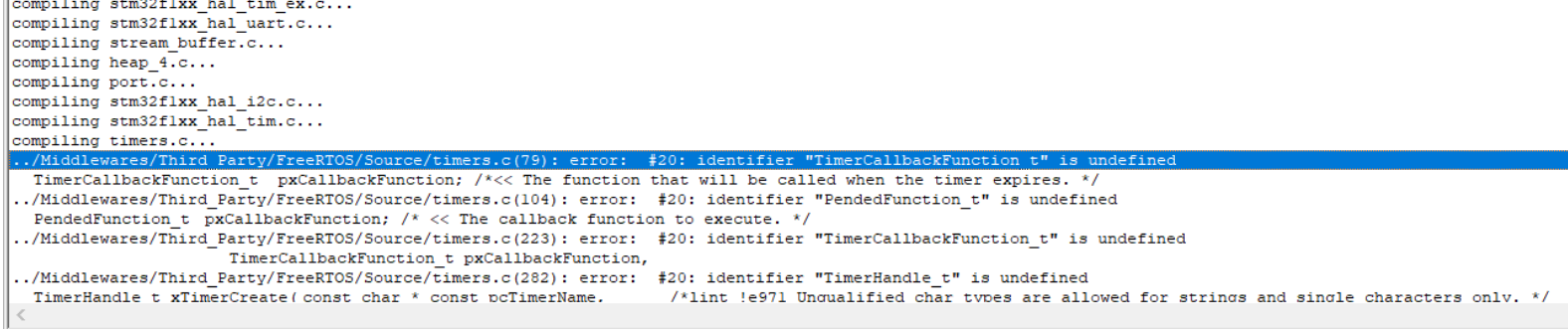
3. 创建任务



创建1个软件定时器



点击生成代码，移除工程生成的can.c 文件，重新编译工程，发现有错误。



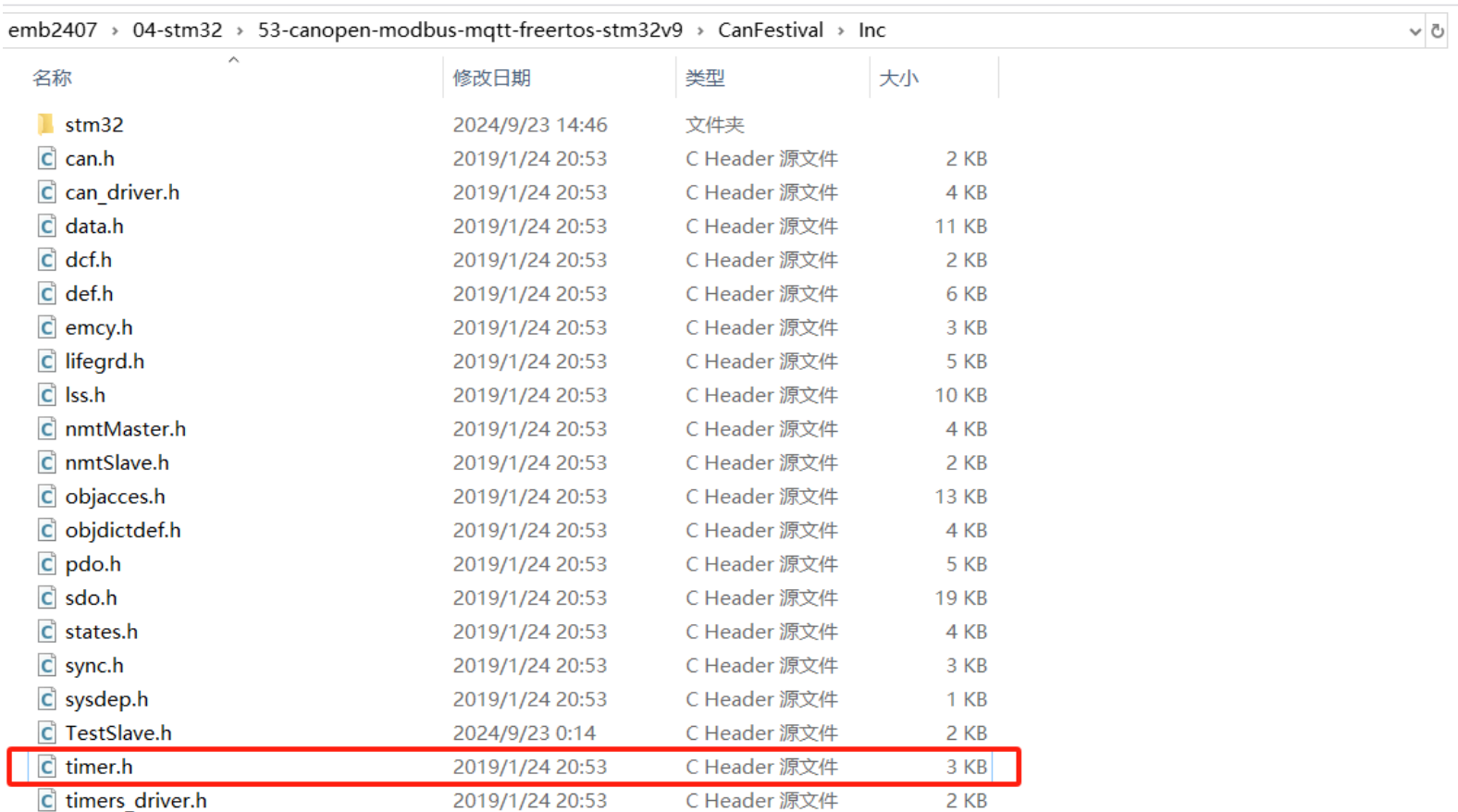
这是因为CanFestival中有一个头文件timers.h 和 FreeRTOS中也有一个timers.h， 两个头文件重名了，系统不知道如何解决。

解决办法两种思路：

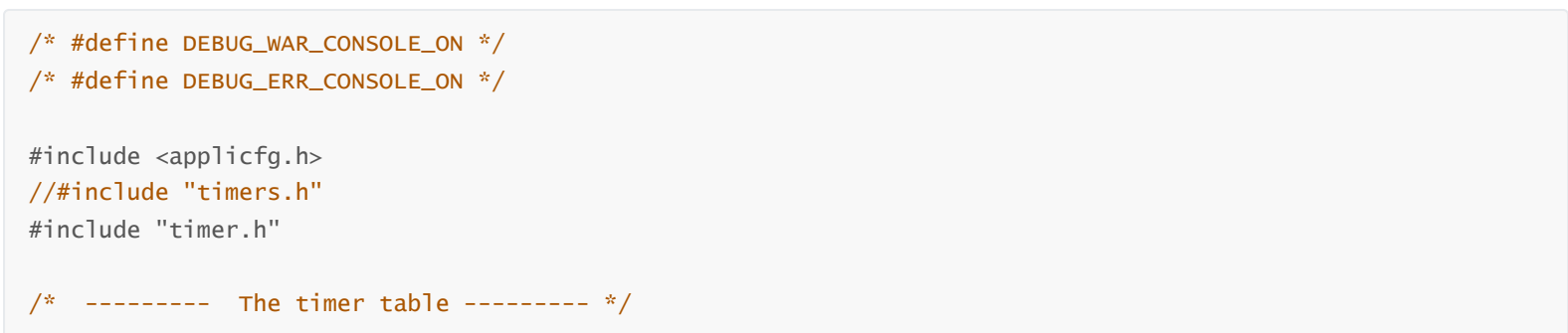
- 第1种是添加头文件的时候指定详细路径。
- 第2种是修改头文件名称。

我们使用第2种思路去解决问题，直接把两种头文件修改的不一样，主要是CanFestival中就使用了timers.h 几次，属于小改动，很容易修改。

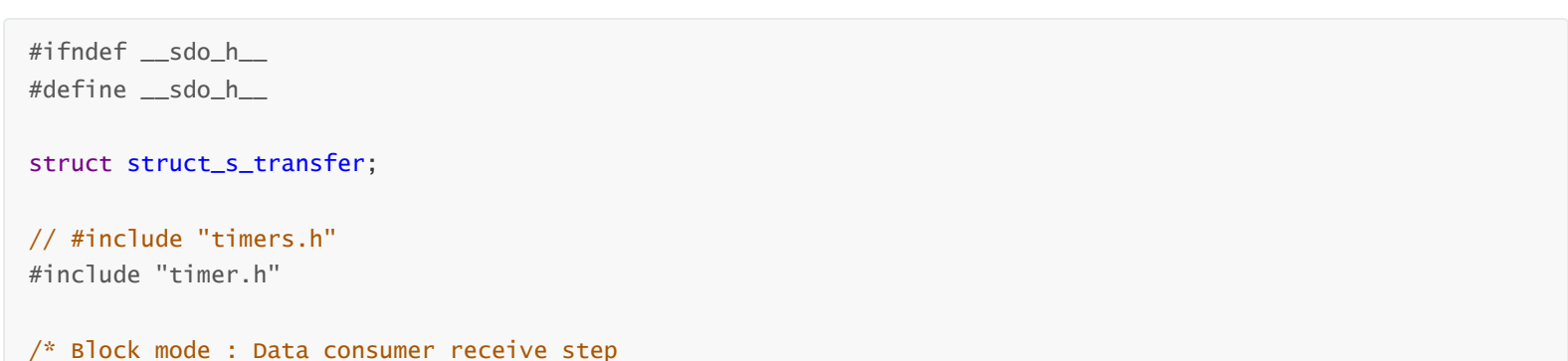
把工程CanFestival\Inc\timers.h 名称修改为：timer.h，并修改工程的几次包含信息。



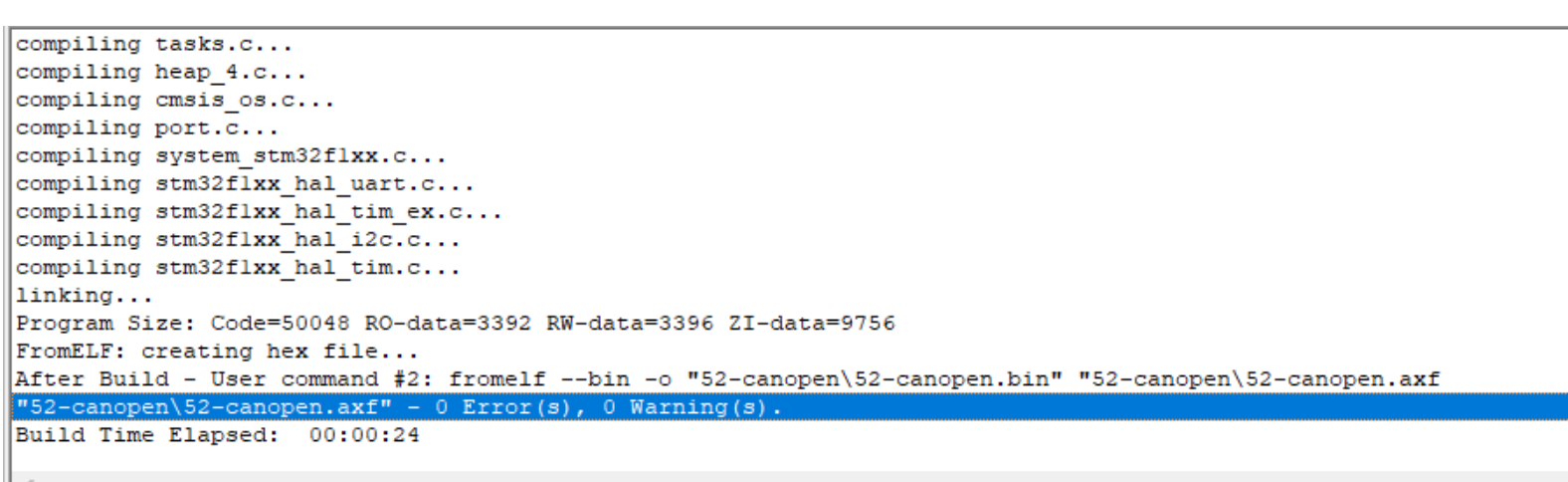
timer.c 中timers.h 更换为timer.h



sdo.c 把timers.h 更换为timer.h



再次编译工程，发现编译通过。



3. 优化程序

main.c



```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_I2C1_Init();
MX_ADC1_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();
MX_USART3_UART_Init();
MX_TIM4_Init();
MX_CAN_Init();
MX_TIM2_Init();
/* USER CODE BEGIN 2 */
    printf("Uart1 init is ok\n");
    printf("CANopen Control\n");
    HAL_Delay(400); // 这里延时是给esp8266 打印日志使用的
    // 经过测试 , 400ms时, 日志输出就没有了, 为了安全,我们设置400ms 以上的延时
    // 等待esp8266 输出初始化日志, 要不然一旦开启uart3就会触发dma中断,早期的esp8266初始化的数据波特率不是
    // 115200, 会造成uart3的错误,要在uart3中做错误恢复

    OLED_Init();
    OLED_CLS(); // 清除屏幕显示
    OLED_ShowStr(0,0, (unsigned char*)"CANopen Control ", 2);
    INA226_Init();

    __HAL_UART_ENABLE_IT(&huart3, UART_IT_IDLE); //使能串UART3 IDLE(空闲)中断
    HAL_UART_Receive_DMA(&huart3, RX3_Buf, DMA_BUF_SIZE); //设置DMA传输, 将uart3的数据搬运到RX3_Buf中
    Queue_Init(&UART3); // 队列初始化

    Modbus_Init();
/* USER CODE END 2 */

/* Call init function for freertos objects (in freertos.c) */
MX_FREERTOS_Init();

/* Start scheduler */
osKernelStart();
/* We should never get here as control is now taken by the scheduler */
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

}
/* USER CODE END 3 */
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    /* USER CODE BEGIN Callback 0 */

    /* USER CODE END Callback 0 */
    if (htim->Instance == TIM1) {
        HAL_IncTick();
    }
    /* USER CODE BEGIN Callback 1 */
    // 定时10ms, 产生中断
    else if(htim->Instance == TIM4)
    {
        // 关闭定时器 , 开始处理接收的数据
        HAL_TIM_Base_Stop(&htim4); // 关闭定时器4中断
        WIFI4G_CMD_Status = WIFI4G_Parse_Queue(&UART3) ;
    }
    /* USER CODE END Callback 1 */
}
```

freertos.c

```
/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include "modbus.h"
#include "mb.h"
#include "mqtt.h"
#include "wifi4g.h"
#include "canfestival.h"
#include "testslave.h"
#include "tim.h"

#pragma diag_suppress 870
/* USER CODE END Includes */

/* USER CODE END GET_TIMER_TASK_MEMORY */

/**
 * @brief FreeRTOS initialization
 * @param None
 * @retval None
 */
void MX_FREERTOS_Init(void) {

    /* USER CODE BEGIN RTOS_TIMERS */
    /* start timers, add new ones, ... */
    osTimerStart(SoftTimer1Handle,500); // 定时时间500ms
    /* USER CODE END RTOS_TIMERS */

}

/* USER CODE BEGIN Header_StartModbusTask */
/**
 * @brief Function implementing the ModbusTask thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_StartModbusTask */
void StartModbusTask(void const * argument)
{
    /* USER CODE BEGIN StartModbusTask */
    /* Infinite loop */
    for(;;)
    {
        eMBPoll(); //轮训查询
        Modbus_Parse(); // 解析各种命令
        osDelay(10);
    }
    /* USER CODE END StartModbusTask */
}

/* USER CODE BEGIN Header_StartMQTTTask */
/**
 * @brief Function implementing the MQTTTask thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_StartMQTTTask */
void StartMQTTTask(void const * argument)
{
    /* USER CODE BEGIN StartMQTTTask */
    /* Infinite loop */
    if(MQTT_Connect_Server() == SET) // mqtt服务器链接成功
    {
        printf("MQTT服务器链接成功 \n");
        MQTT_Download_Flag = 1; // 链接服务器后, 开启解析json数据标志位
    }
    for(;;)
    {
        MQTT_SendData(); // 每隔5秒发送一次数据
        osDelay(5000);
    }
    /* USER CODE END StartMQTTTask */
}

/* USER CODE BEGIN Header_StartCANopenTask */
/**
 * @brief Function implementing the CANopenTask thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_StartCANopenTask */
void StartCANopenTask(void const * argument)
{
    /* USER CODE BEGIN StartCANopenTask */
    /* Infinite loop */
    HAL_TIM_Base_Start_IT(&htim2); //启动定时器2
    CanFestival_Can_Init(); //初始化can过滤器
```

```
setNodeID(&TestSlave_Data, SlaveAddress);
setState(&TestSlave_Data, Initialisation);
setState(&TestSlave_Data, Operational);
for(;;)
{
    osDelay(1000);
}

/* USER CODE END StartCANOpenTask */
}

/* SoftTimer1callback function */
void SoftTimer1callback(void const * argument)
{
    /* USER CODE BEGIN SoftTimer1callback */
    Timeout_Flag = 1; // 500ms 置为1次
    /* USER CODE END SoftTimer1callback */
}
```

4. 验证通信

控制设备

写设备的命令，我们写1个字节的数据，从机地址设置为1时，使用SDO的download协议通信。

发送打开设备命令：

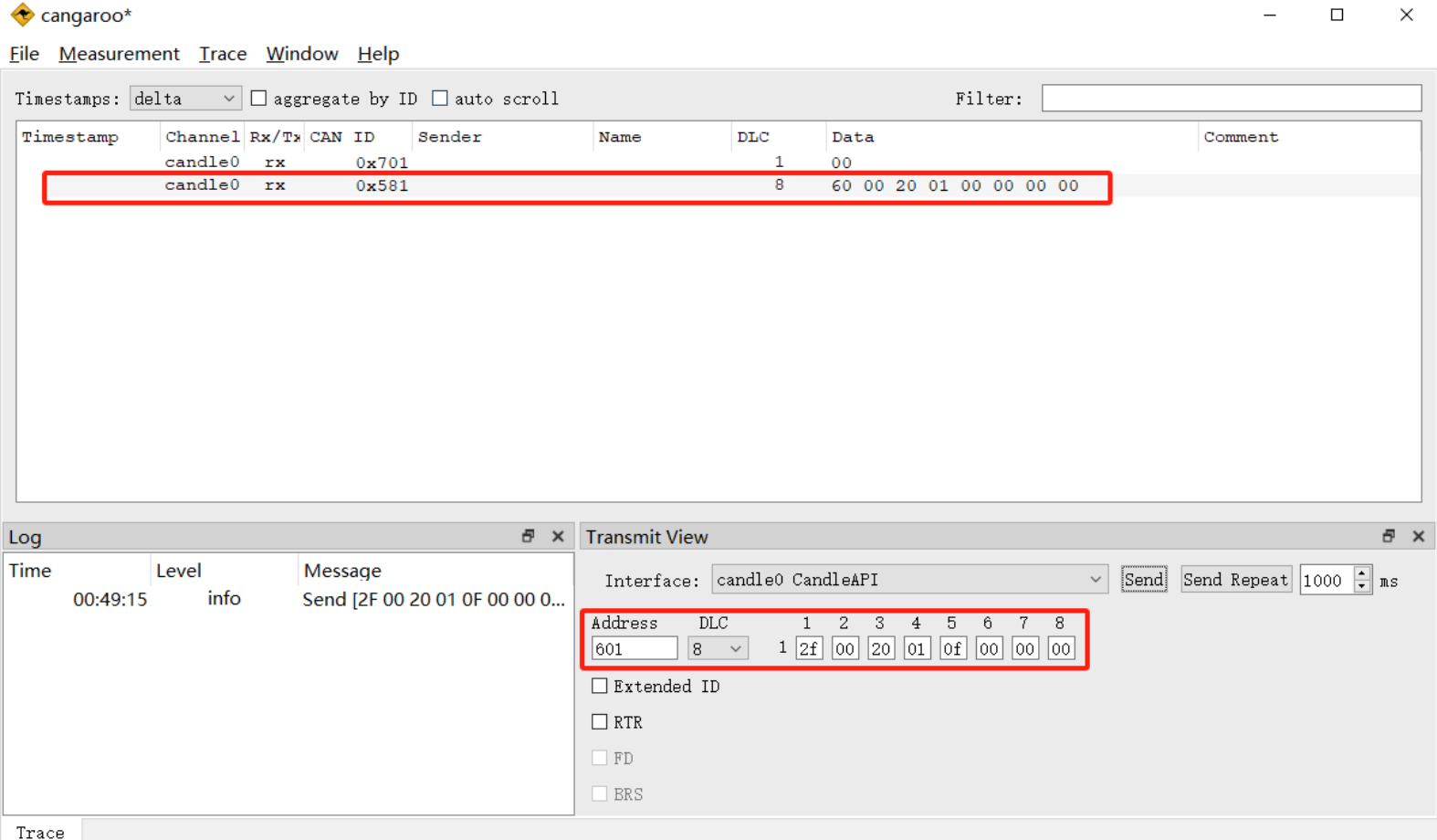
```
// 发送数据如下：
// 地址： 0x601
// DLC : 8
// 数据 : 2f 00 20 01 0f 00 00 00

// 命令解析：
// 地址： 0x601 ，这个就是 0x600 + NodeID
// DLC : 8 ，发送字节数
// 2f : 表示1个字节有效
// 00 20 : 表示索引地址为2000
// 01 : 表示字索引为1
// 0f : 0b0000 1111，设备全部工作
```

设备返回数据：

```
// CANID : 0x581
// DLC : 8
// 数据 : 2f 00 20 01 00 00 00 00

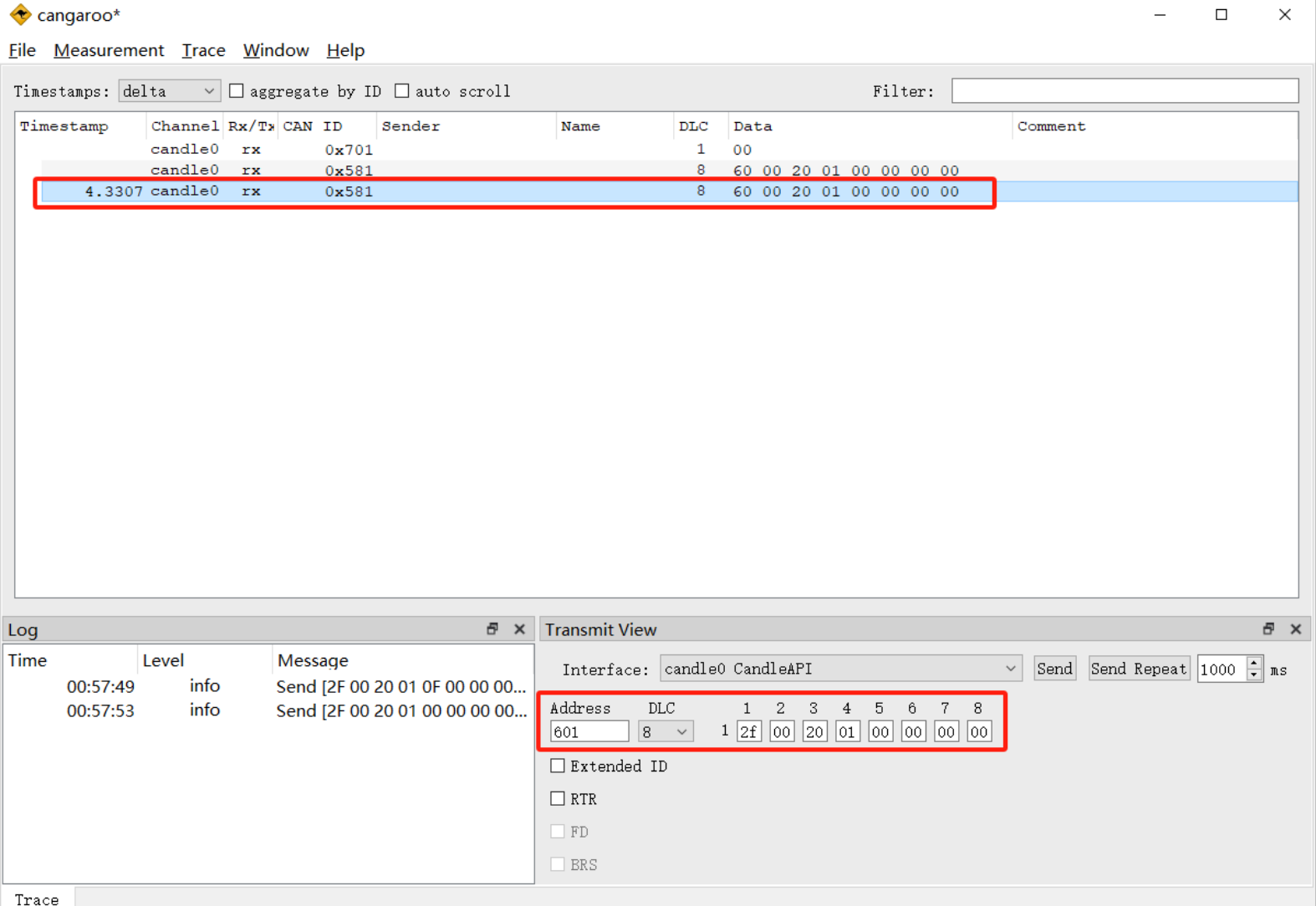
// 命令解析：
// CANID : 0x581 ，这个就是 0x580 + NodeID
// DLC : 8 ，接收字节数
// 2f : 表示1个字节有效
// 00 20 : 表示索引地址为2000
// 01 : 表示字索引为1
```



关闭设备：

```
// 发送命令：
// 地址： 0x601
// DLC : 8
// 数据 : 2f 00 20 01 00 00 00 00

// 设备返回数据：
// 地址： 0x581
// DLC : 8
// 数据 : 2f 00 20 01 00 00 00 00
```



读传感器协议

我们的传感器数据使用1个16位的数据即可。

读设备传感器数据：

Index 索引地址	之索引	名称	2字节
0x2000	0x2	温度	uint16
0x2000	0x3	湿度	uint16
0x2000	0x4	电压	uint16
0x2000	0x5	电流	uint16
0x2000	0x6	功率	uint16
0x2000	0x7	电位器电压	uint16
0x2000	0x8	CPU温度	uint16

例如:

读温度传感器数据，从机地址设置为1时，使用SDO的upload协议通信。

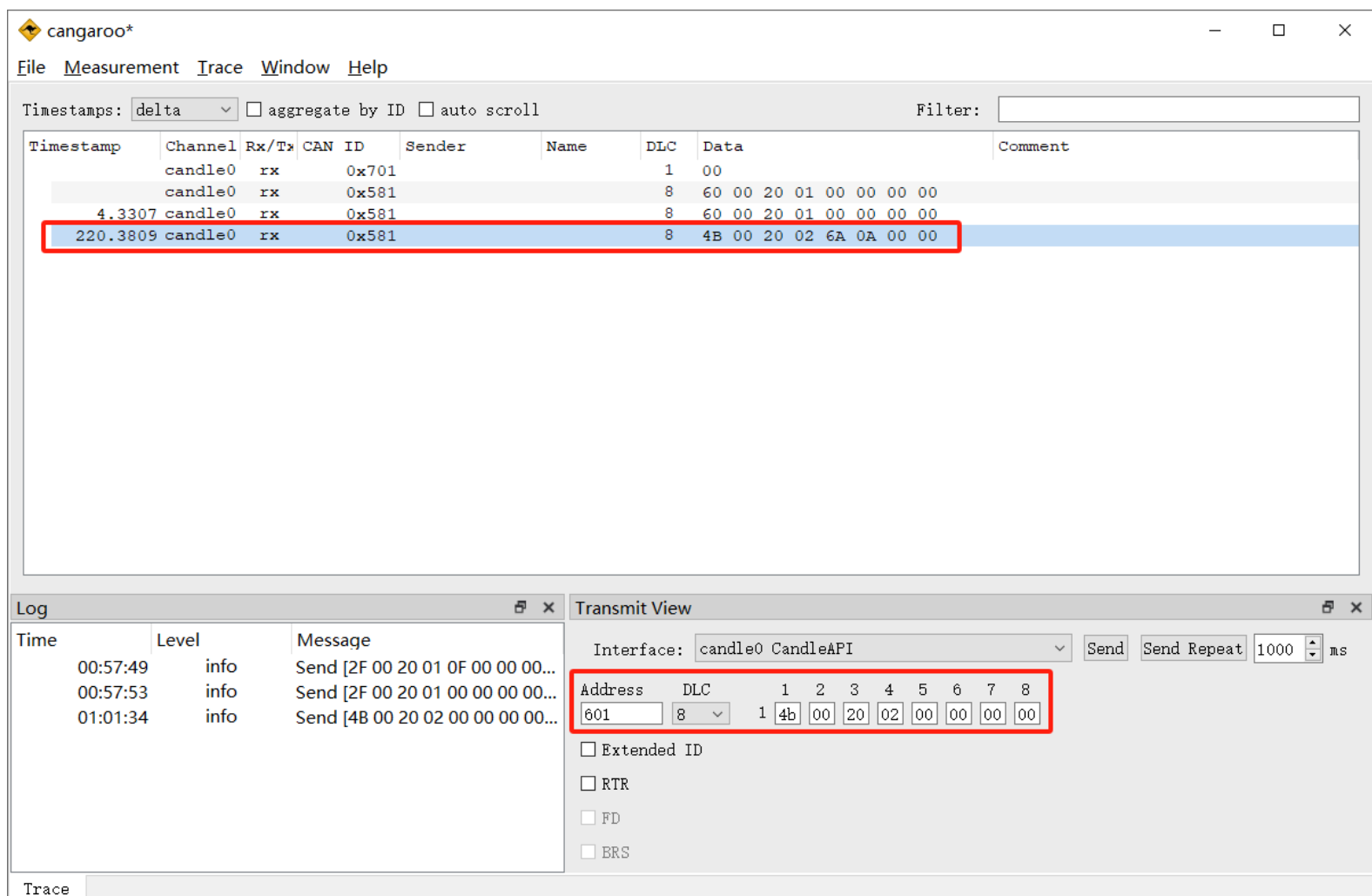
```
// 发送数据如下：
// 地址： 0x601
// DLC : 8
// 数据 : 4b 00 20 02 00 00 00 00

// 命令解析：
// 地址： 0x601 ，这个就是 0x600 + NodeID
// DLC : 8 ，发送字节数
// 4b : 表示2个字节有效
// 00 20 : 表示索引地址为2000
// 02 : 表示字索引为2
```

设备返回的数据

```
// CANID : 0x581
// DLC : 8
// 数据 : 4B 00 20 02 6A 0A 00 00

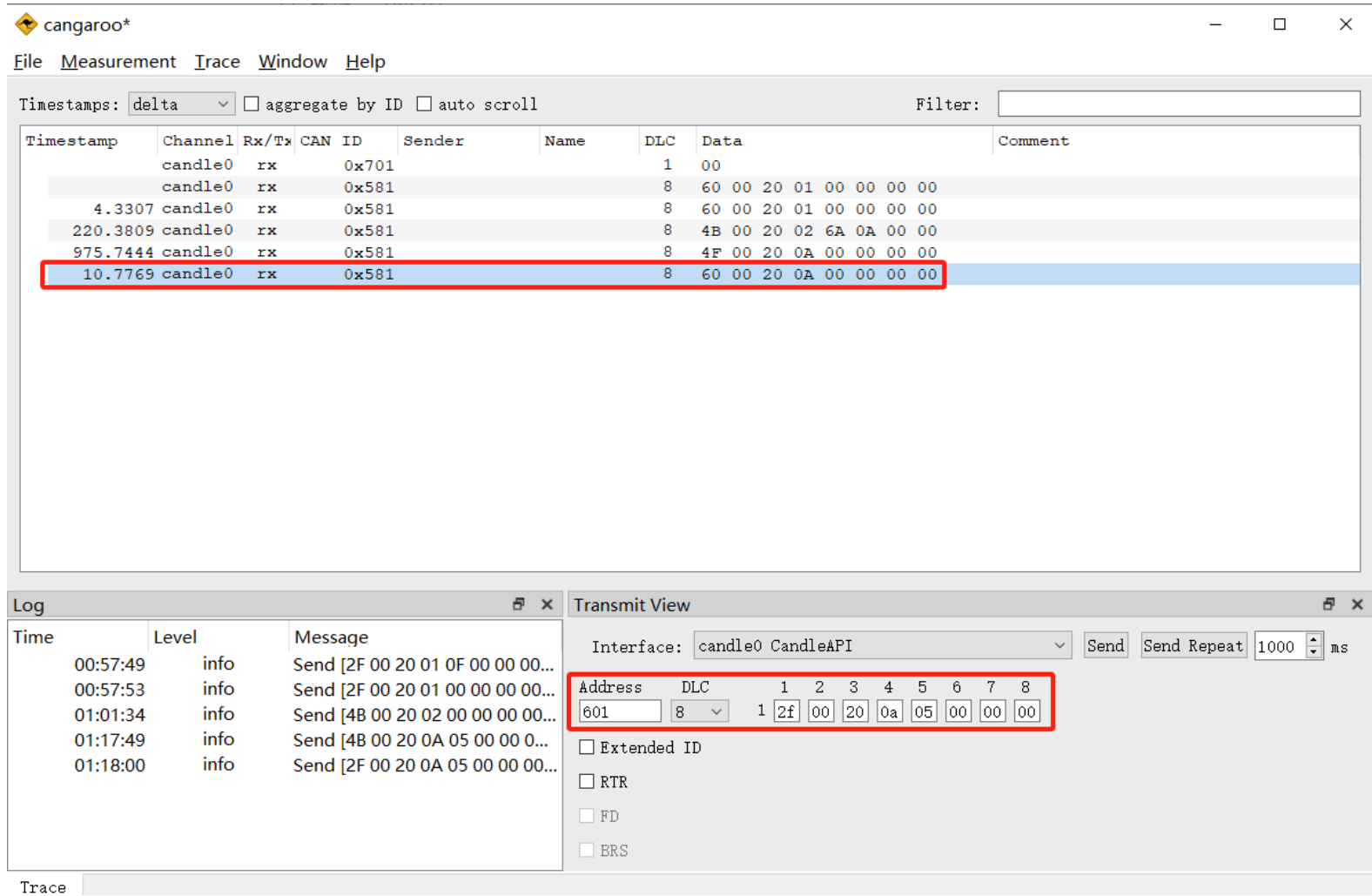
// 命令解析:
// CANID : 0x581 , 这个就是 0x580 + NodeID
// DLC : 8 , 接收字节数
// 4b : 表示2个字节有效
// 00 20 : 表示索引地址为 0x2000
// 02 : 表示字索引为2
// 6A 0A : 读回来的数据是 0xA6A
```



设置地址

写设备地址的命令，我们写1个字节的数据，从机地址设置为1时，使用SDO的download协议通信。

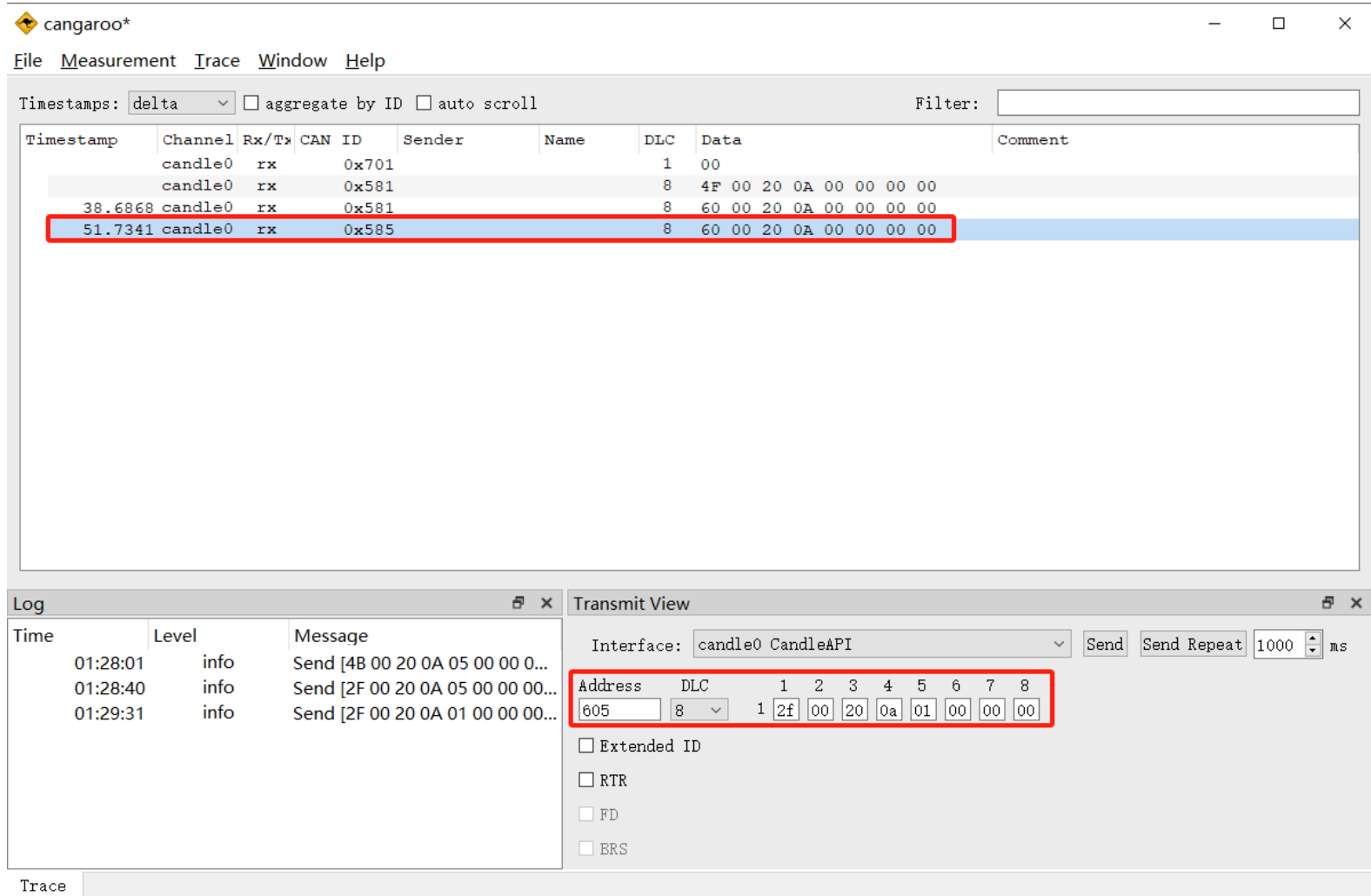
设置从机地址为5。



OLED会显示从机地址



把从机地址设置为1



QIFD显示如下:



第04章 OTA智能控制终端实战

4.1 OTA介绍

Over the Air

OTA 空中下载

OTA（Over-The-Air）是一项通过无线网络（如Wi-Fi或蜂窝移动网络）远程更新设备软件、固件或配置的技术，它让设备升级像手机系统更新一样方便，无需连接电脑或前往服务中心。

为了让你快速把握全局，下表汇总了OTA的核心信息。

特性维度	说明
中文名称	空中下载技术
核心价值	远程无线升级，无需物理接触设备
主要分类	FOTA （固件更新，涉及操作系统、底层驱动）和 SOTA （软件更新，涉及应用程序、地图等）
典型应用	智能手机系统更新、智能汽车功能升级、物联网设备维护
关键优势	快速修复漏洞、降低售后成本、持续为设备增加新功能

1. OTA的核心价值与分类

OTA技术从根本上改变了设备软件和维护的方式。对于厂商而言，它可以**快速修复系统缺陷**，避免大规模线下召回，显著**降低售后成本**；对于用户而言，设备可以**持续迭代更新**，获得新功能和**安全补丁**，体验“常开常新”。

在深入理解时，通常会遇到两个重要概念：

- SOTA**：主要指应用软件的在线升级，比如更新车载导航地图、娱乐系统里的App，或者为手机安装新版本的应用。它通常不涉及底层系统核心功能的改变。
- FOTA**：指固件的在线升级，是更深层次、更彻底的更新。它可以改变操作系统的核心部分，从而优化车辆的动力表现、刹车距离，甚至开放自动驾驶等高级功能。特斯拉通过FOTA为车辆提升加速性能和续航里程就是典型的例子。

简单来说，SOTA更像是“装软件”或“更新App”，而FOTA则更像是“升级电脑主板BIOS”或“重装整个操作系统”。

2. OTA在智能汽车中的应用

汽车是OTA技术大放异彩的领域。其发展大致经历了三个阶段：

- T-BOX阶段（2015年前）**：主要实现远程开车门、道路救援等基础服务。
- SOTA阶段（2015-2018年）**：开始支持车载信息娱乐系统的地图和应用程序更新。
- FOTA阶段（2018年后）**：实现整车级固件升级，能够同时更新控制车辆刹车、转向等关键功能的数十个电子控制单元（ECU），这依赖于集中式的电子电气架构。

这种“软件定义汽车”的趋势，使得汽车从一个交付即定型的机械产品，转变为一个可以持续进化的智能终端。

3. OTA如何工作及安全保障

一次完整的OTA升级通常包含三个步骤：

- 生成更新包**：厂商制作包含新功能或修复补丁的软件包，常采用“差分升级”技术，只生成新旧版本间的差异部分，大大减小下载体积。
- 传输更新包**：加密后的更新包被推送到云端服务器，设备在合适的时机（如连接Wi-Fi时）自动或经用户同意后下载。
- 安装与验证**：设备安装更新，并严格验证其完整性和正确性。成功后，新版本即开始运行。

安全是OTA的生命线。为确保升级过程可靠，会采用**端到端加密**（如AES-256）防止数据篡改，使用**数字签名**验证软件包来源的合法性，并设计**回滚机制**，以便在升级失败时能自动恢复至旧版本，保障设备基本功能不受影响。

4.2 STM32Bootloader原理

嵌入式的设备，或多或少都需要对设备进行更新已适配更多的需求，如果未出货的设备还可以使用下载线去下载，但是如果出货之后到了客户那里，客户没有下载线而且不是专业人员，无法对设备进行升级，这时候，bootLoader 的重要性就凸显出来了，bootLoader 的设计就是为了设备能够进行远程升级或者只用指令升级，极大简便了升级需要的步骤，做到傻瓜式升级，极大增强了产品的后续维护性。

1. Bootloader 是什么？

BootLoader 可以理解成是引导程序, 它的作用是启动正式的App应用程序。

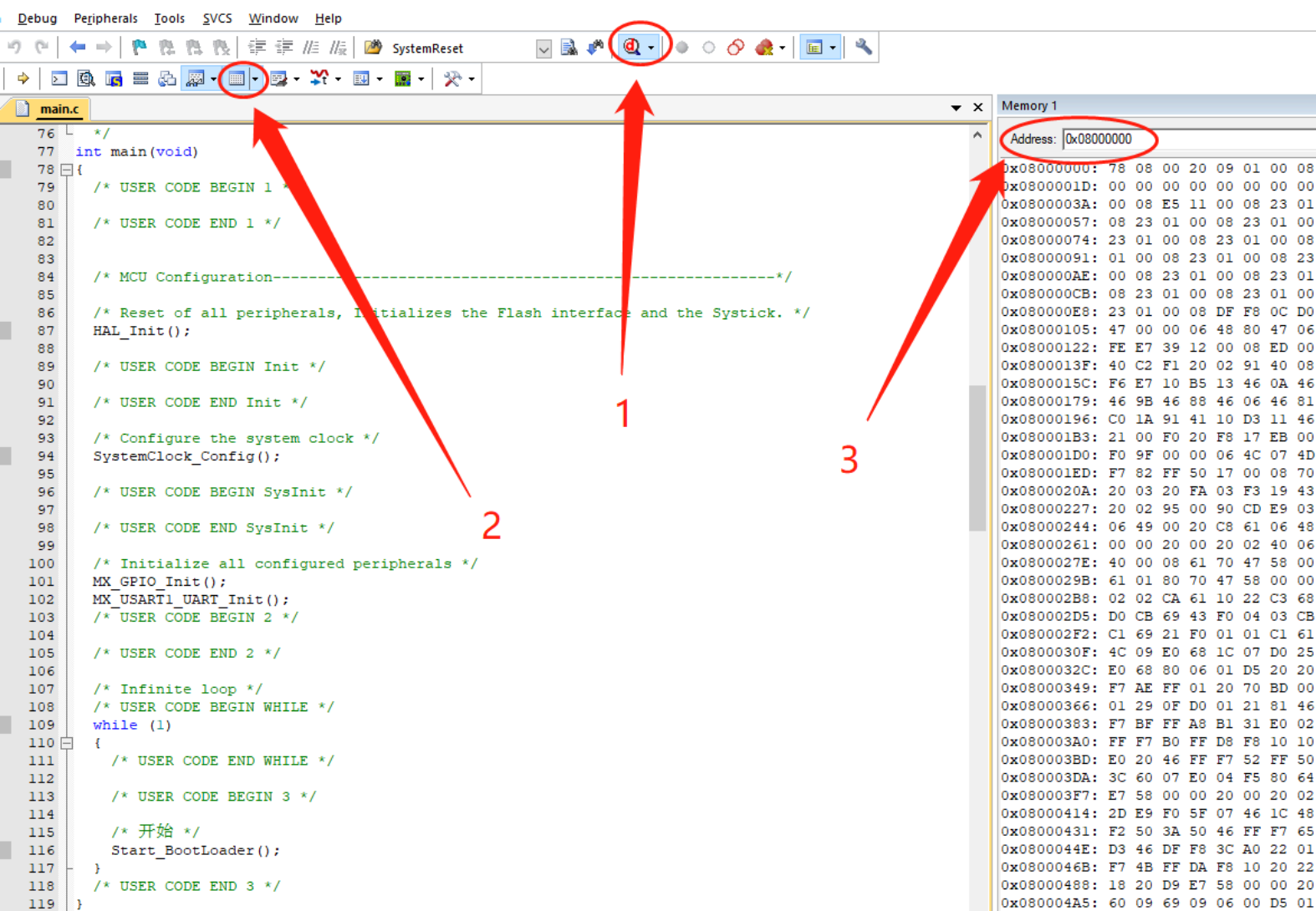
换言之， BootLoader 是一个程序，App也是一个程序， BootLoader程序是用于启动App程序的。

Bootloader是在应用程序开始前运行的一个小程序，里面可以进行一些初始化操作，升级引用程序等，在嵌入式设备中很常见。

2. BootLoader 的实现

正常情况下, 我们写的程序都是放在STM32片内Flash中(暂不考虑外扩Flash)，我们写的代码最终会变成二进制文件, 放进Flash中。

感兴趣的话可以在 Keil -> Debug -> Memory 中查看, 右边Memory窗口存储的就是代码进行分区



既然我们写的程序都会变成二进制文件存放到Flash中, 那么我们就可以进一步对我们程序进行分区。

以STM32F103CB为例，他的Flash一共128页，每页1K，见下图：

第0页	0x0800 0000 - 0x0800 03FF	1K
第1页	0x0800 0400 - 0x0800 07FF	1K
.	.	.
.	.	.
.	.	.
第127页	0x0801 FC00 - 0x0801 FFFF	1K

以它为例, 我将它分为三个区. BootLoader区、 App1区、 App2区(备份区) 具体划分如下图:

- BootLoader区 存放启动代码
- App1区 存放应用代码
- App2区 存放暂存的升级代码

BootLoader	第0页	0x0800 0000 - 0x0800 03FF	1K
	.	.	
	第19页	0x0800 4C00 - 0x0800 4FFF	1K
App1	第20页	0x0800 5000 - 0x0800 53FF	1K
	.	.	
	第59页	0x0800 EC00 - 0x0800 EFFF	1K
App2 (备份区)	第60页	0x0800 F000 - 0x0800 F3FF	1K
	.	.	
	第99页	0x0801 8C00 - 0x0801 8FFF	1K

3. 总体流程图

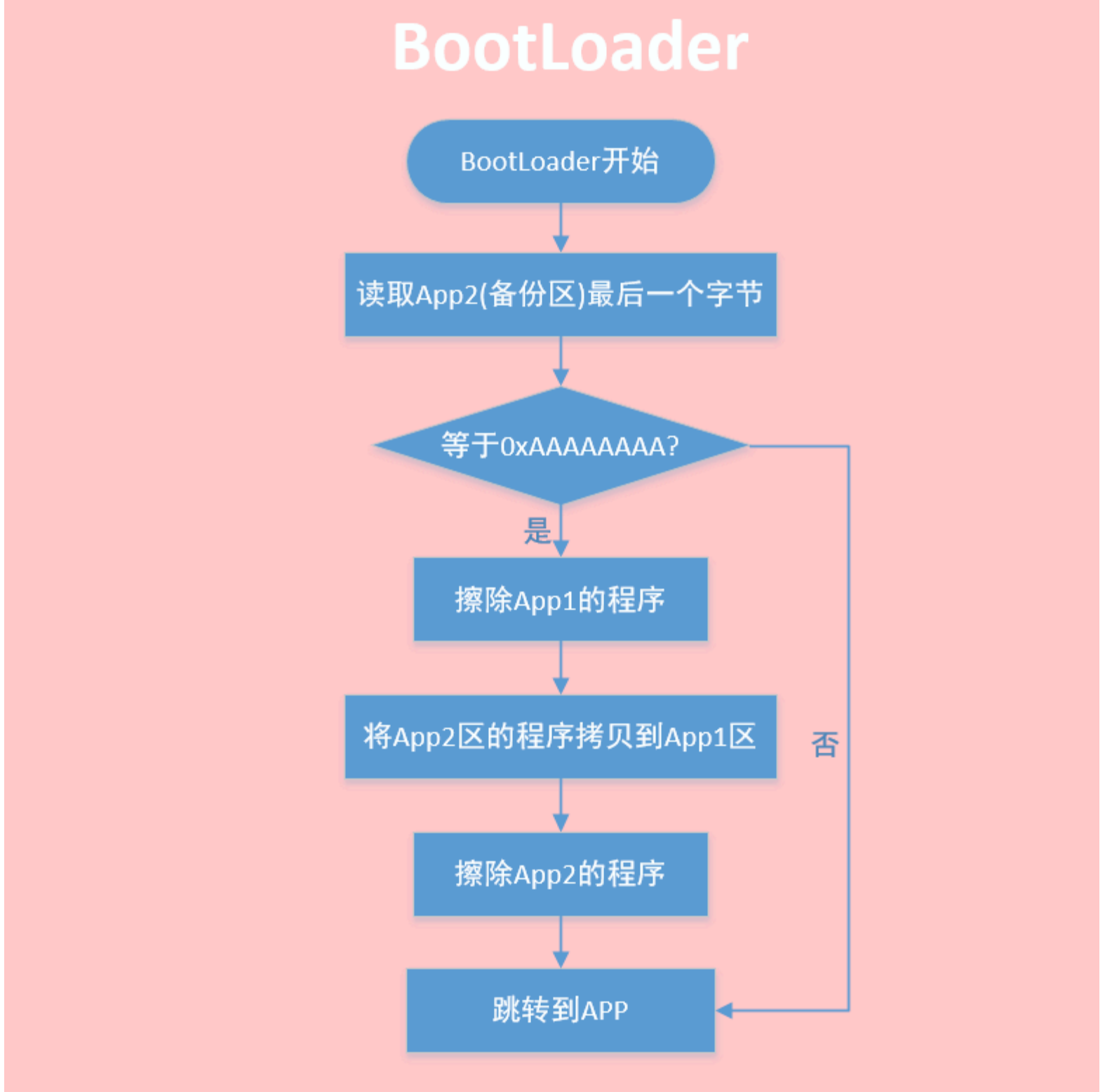
- 先执行 BootLoader 程序, 先去检查 APP2 区有没有程序, 如果有就将App2区(备份区)的程序拷贝到 App1区, 然后再跳转去执行 App1 的程序.
 - 然后执行 App1 程序, 因为 BootLoader 和 App1 这两个程序的向量表不一样, 所以跳转到 App1 之后第一步是先去更改程序的向量表. 然后再去执行其他的应用程序.
 - 在应用程序里面会加入程序升级的部分, 这部分主要工作是拿到升级程序, 然后将他们放到 App2区(备份区), 以便下次启动的时候通过 BootLoader 更新 App1 的程序.
- 流程图如下图所示:



4. 流程图分析

以历程的BootLoader为例:

将 App2区 的最后4个字节(0x0801FFFC)用来表示 App2区 是否有升级程序, STM32在擦除之后Flash的数据存放的都是 0xFFFFFFFF , 如果有, 我们将这个地址存放 0xAAAAAAAA , 具体的流程图见下图所示



4.3 实战项目8 - Bootloader

1. 程序关键分析

- 所需STM32的资源有:
- printf的使用
 - Flash的读写
 - 程序跳转指令, 可以参考如下代码:

```
/* 采用汇编设置栈的值 */
__asm void MSR_MSP (uint32_t u1Addr)
{
    MSR MSP, r0    //设置Main Stack的值
    BX r14
}

/* 程序跳转函数 */
typedef void (*Jump_Fun)(void);
void IAP_ExecuteApp (uint32_t App_Addr)
{
    Jump_Fun JumpToApp;

    if ( ( ( * ( __IO uint32_t * ) App_Addr ) & 0xFFE0000 ) == 0x20000000 ) //检查栈顶地址是否合法,
    {
```

```
JumpToApp = (Jump_Fun) * ( __IO uint32_t *) (App_Addr + 4); //用户代码区第二个字为程序开始地址(复位地址)
MSR_MSP( * ( __IO uint32_t * ) App_Addr ); //初始化APP堆栈指针(用户代码区的第一个字用于存放栈顶地址)
JumpToApp(); //跳转到APP.
}
}
```

- 在需要跳转的地方执行这个函数就可以了IAP_ExecuteApp(Application_1_Addr);

2. 程序源码

boot.h

```
#ifndef __BOOT_H_
#define __BOOT_H_

#define PageSize FLASH_PAGE_SIZE // 1K

/*====用户配置(根据自己的分区进行配置)====*/
// 0x400 1K
// 0x800 2K
// 0x1000 4K
// 0x2000 8K
// 0x7000 4K*7 = 28K
#define BootLoader_Size 0x2000U // BootLoader的大小 8K
#define Application_Size 0x7000U // APP 应用程序的大小 28K

#define Application_1_Addr 0x08002000U // 应用程序1的首地址 大小 29K 0x08002000 + 0x7000
#define Application_2_Addr 0x08009000U // 应用程序2的首地址 大小 29K 0x08009000 + 0x7000
/*=====*/

/* 启动的步骤 */
#define Startup_Normo1 0xFFFFFFFF // 正常启动
#define Startup_Update 0xAAAAAAA // 升级再启动

void Start_BootLoader(void);

#endif
```

boot.c

```
#include "main.h"
#include <stdio.h>
#include "boot.h"

/**
 * @bieaf 擦除页
 *
 * @param pageaddr 起始地址
 * @param num 擦除的页数
 * @return 1
 */
static int Erase_page(uint32_t pageaddr, uint32_t num)
{
    HAL_FLASH_Unlock();

    /* 擦除FLASH*/
    FLASH_EraseInitTypeDef FlashSet;
    FlashSet.TypeErase = FLASH_TYPEERASE_PAGES;
    FlashSet.PageAddress = pageaddr;
    FlashSet.NbPages = num;

    /*设置PageError，调用擦除函数*/
    uint32_t PageError = 0;
    HAL_FLASHEx_Erase(&FlashSet, &PageError);

    HAL_FLASH_Lock();
    return 1;
}

/**
 * @bieaf 写若干个数据
 *
 * @param addr 写入的地址
 * @param buff 写入数据的起始地址
 * @param word_size 长度
 * @return
 */
static void WriteFlash(uint32_t addr, uint32_t * buff, int word_size)
{
    /* 1/4解锁FLASH*/
    HAL_FLASH_Unlock();

    for(int i = 0; i < word_size; i++)
    {
        /* 3/4对FLASH烧写*/
        HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, addr + 4 * i, buff[i]);
    }

    /* 4/4锁住FLASH*/
    HAL_FLASH_Lock();
}

/**
 * @bieaf 读若干个数据
 *
 * @param addr 读数据的地址
 * @param buff 读出数据的数组指针
 * @param word_size 长度
 * @return
 */
static void ReadFlash(uint32_t addr, uint32_t * buff, uint16_t word_size)
{
    for(int i =0; i < word_size; i++)
    {
        buff[i] = *(__IO uint32_t*)(addr + 4 * i);
    }
    return;
}

/* 读取启动模式 */
unsigned int Read_Start_Mode(void)
{
    unsigned int mode = 0;
    ReadFlash((Application_2_Addr + Application_Size - 4), &mode, 1);
    return mode;
}

/**
 * @bieaf 进行程序的覆盖
 * @detail 1.擦除目的地址
 *          2.源地址的代码拷贝到目的地址
 *          3.擦除源地址
 *
 * @param 搬运的源地址
 * @param 搬运的目的地址
 * @return 搬运的程序大小
 */
void MoveCode(uint32_t src_addr, uint32_t des_addr, uint32_t byte_size)
{
    /*1.擦除目的地址*/
    printf("> Start erase des flash.....\r\n");
    Erase_page(des_addr, (byte_size/PageSize));
    printf("> Erase des flash down.....\r\n");

    /*2.开始拷贝*/
    unsigned int temp[256];

    printf("> Start copy.....\r\n");
    for(int i = 0; i < byte_size/1024; i++)
    {
        ReadFlash((src_addr + i*1024), temp, 256);
        writeFlash((des_addr + i*1024), temp, 256);
    }
    printf("> Copy down.....\r\n");

    /*3.擦除源地址*/
    printf("> Start erase src flash.....\r\n");
    Erase_page(src_addr, (byte_size/PageSize));
    printf("> Erase src flash down.....\r\n");
}
```

```
/* 采用汇编设置栈的值 */
__asm void MSR_MSP (uint32_t u1Addr)
{
    MSR MSP, r0                                //set Main Stack value
    BX r14
}

/* 程序跳转函数 */
typedef void (*Jump_Fun)(void);
void IAP_ExecuteApp (uint32_t App_Addr)
{
    Jump_Fun JumpToApp;

    if ( ( ( * ( __IO uint32_t * ) App_Addr ) & 0x2FFE0000 ) == 0x20000000 )    //检查栈顶地址是否合法。
    {
        JumpToApp = (Jump_Fun) * ( __IO uint32_t * )(App_Addr + 4);    //用户代码区第二个字为程序开始地址(复位地址)
        MSR_MSP( * ( __IO uint32_t * ) App_Addr );    //初始化APP堆栈指针(用户代码区的第一个字用于存放栈顶地址)
        JumpToApp();    //跳转到APP.
    }
}

/**
 * @bieaf 进行BootLoader的启动
 *
 * @param none
 * @return none
 */
void Start_BootLoader(void)
{
    /*=====打印消息=====*/
    printf("\r\n");
    printf("*****\r\n");
    printf("**      BootLoader      *\r\n");
    printf("*****\r\n");

    //printf("> Choose a startup method.....\r\n");
    switch(Read_Start_Mode())
    {
        case Startup_Normo1:
            // 正常启动
            {
                printf("> Normal start...\n");
                break;
            }
        case Startup_Update:
            // 升级再启动
            {
                printf("> Start update...\n");
                MoveCode(Application_2_Addr, Application_1_Addr, Application_Size);
                printf("> Update down.....\r\n");
                break;
            }
        default:
            // 启动失败
            {
                printf("> Error:%X!!!.....\r\n", Read_Start_Mode());
                return;
            }
    }

    /* 跳转到应用程序 */
    printf("> Start APP1...\r\n");
    IAP_ExecuteApp(Application_1_Addr);
}
```

因为我们在更新代码时使用了1K的栈空间， 在实际使用这种会造成printf函数出异常。

```
void MoveCode(uint32_t src_addr, uint32_t des_addr, uint32_t byte_size)
{
    /*1.擦除目的地址*/
    printf("> Start erase des flash.....\r\n");
    Erase_page(des_addr, (byte_size/PageSize));
    printf("> Erase des flash down.....\r\n");

    /*2.开始拷贝*/
    unsigned int temp[256]; // 栈空间被使用1K ， 需要扩大栈空间

    printf("> Start copy.....\r\n");
    for(int i = 0; i < byte_size/1024; i++)
    {
        ReadFlash((src_addr + i*1024), temp, 256);
        writeFlash((des_addr + i*1024), temp, 256);
    }
    printf("> Copy down.....\r\n");

    /*3.擦除源地址*/
    printf("> Start erase src flash.....\r\n");
    Erase_page(src_addr, (byte_size/PageSize));
    printf("> Erase src flash down.....\r\n");
}
```

因此我们需要调大栈的空间， 把栈空间放大一倍，如下图：

Project

Code Generator

Advanced Settings

Project Settings

Project Name

Project Location

Application Structure

Toolchain Folder Location

Toolchain / IDE

BOOT

E:\YShengli\02-Notes\embs\05-ota\08-boot-ota

Browse

Advanced

Do not generate the main()

E:\YShengli\02-Notes\embs\05-ota\08-boot-ota\BOOT\

MDK-ARM

Min Version

V5.32

Generate Under Root

Linker Settings

Minimum Heap Size

Minimum Stack Size

0x200

0x800

Thread-safe Settings

Cortex-M3NS

Enable multi-threaded support

Thread-safe Locking Strategy

Default - Mapping suitable strategy depending on RTOS selection.

Mcu and Firmware Package

Mcu Reference

Firmware Package Name and Version

Use Default Firmware Location

Firmware Relative Path

STM32F103C8Tx

STM32Cube_FW_F1 V1.8.6

Use latest available version

C:/Users/shengli/STM32Cube\Repository/STM32Cube_FW_F1_V1.8.6

Browse

下载程序的地址也需要调整，我们给BOOT设置的大小为8K， 如下图：

Device

Target

Output

Listing

User

C/C++

Asm

Linker

Debug

Utilities

STMicroelectronics STM32F103C8

Xtal (MHz):

8.0

Operating system:

None

System Viewer File:

STM32F103xx.svd

Use Custom File

Read/Only Memory Areas

default

off-chip

Start

Size

Startup

ROM1:

ROM2:

ROM3:

on-chip

IROM1:

0x8000000

0x2000

IROM2:

Code Generation

ARM Compiler:

Use default compiler version 5

Use Cross-Module Optimization

Use MicroLIB

Big Endian

Read/Write Memory Areas

default

off-chip

Start

Size

Nolnit

RAM1:

RAM2:

RAM3:

on-chip

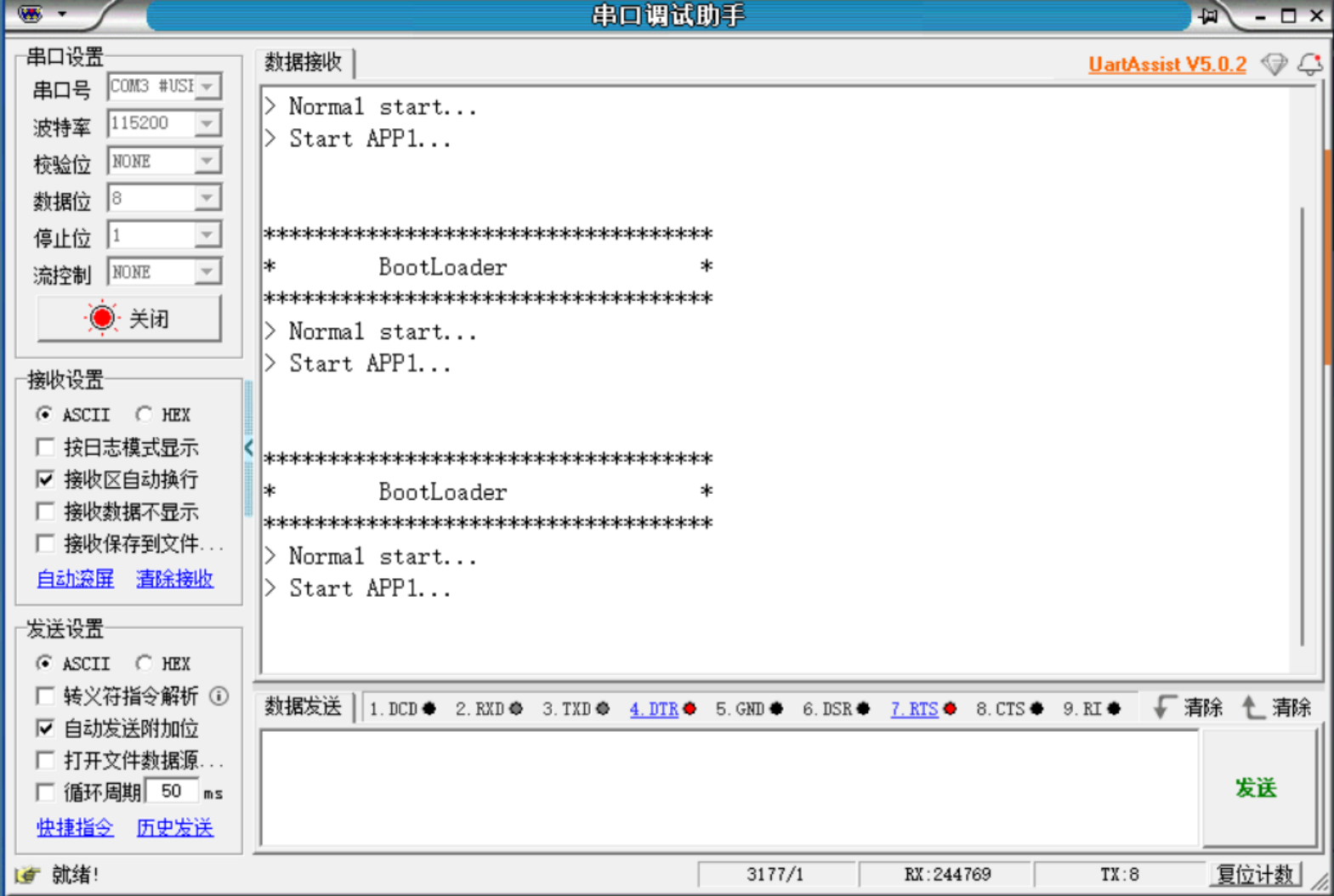
IRAM1:

0x20000000

0x5000

IRAM2:

下载程序运行后如下图：



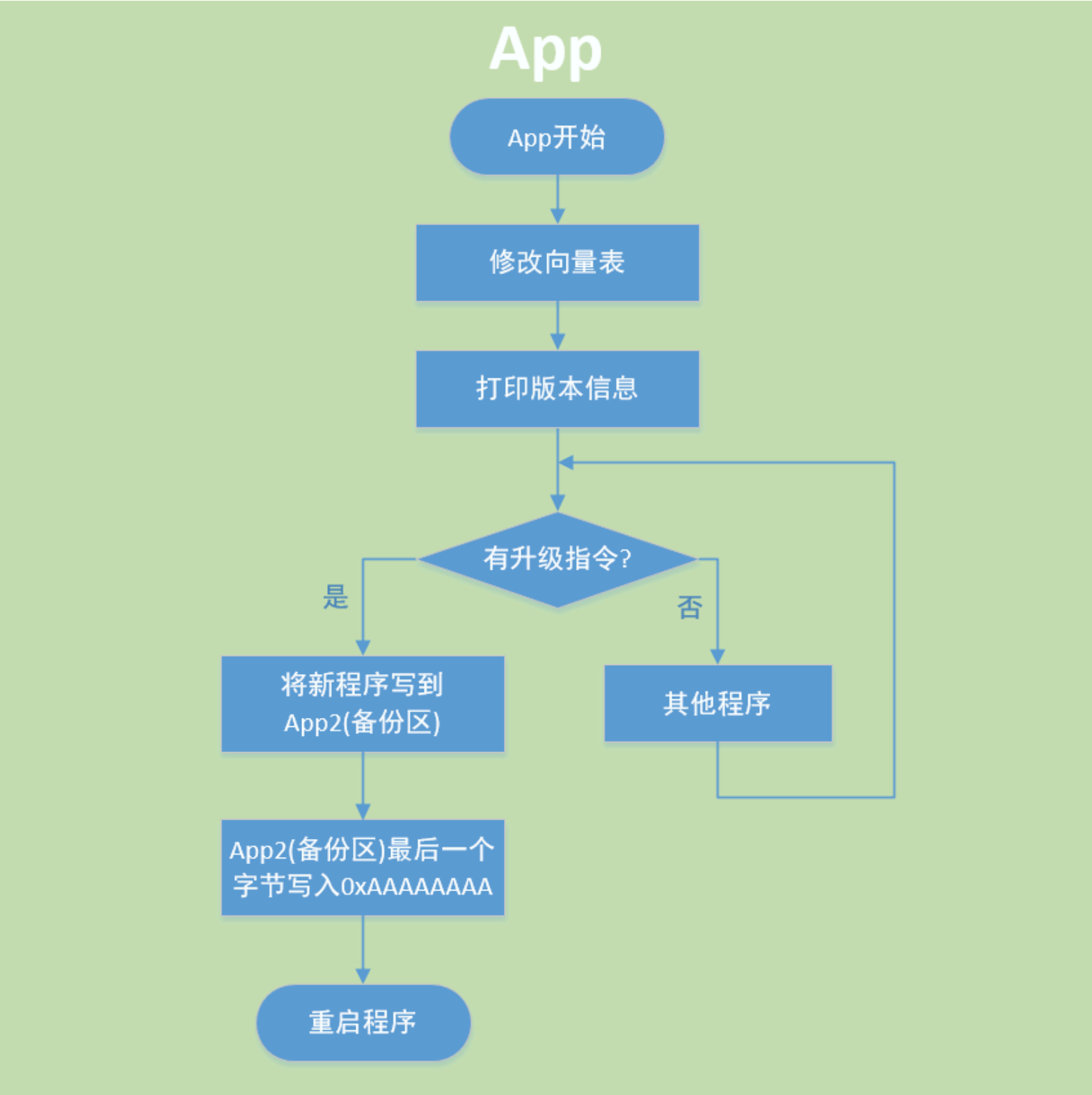
4.4 实战项目8 - APP

基于实战项目4来完成。

1. 流程图分析

以我历程的App为例:

- 先修改向量表, 因为本程序是由BootLoader跳转过来的, 不修改向量表后面会出现问题.
- 打印版本信息, 方便查看不同的App版本



2. 设置向量表

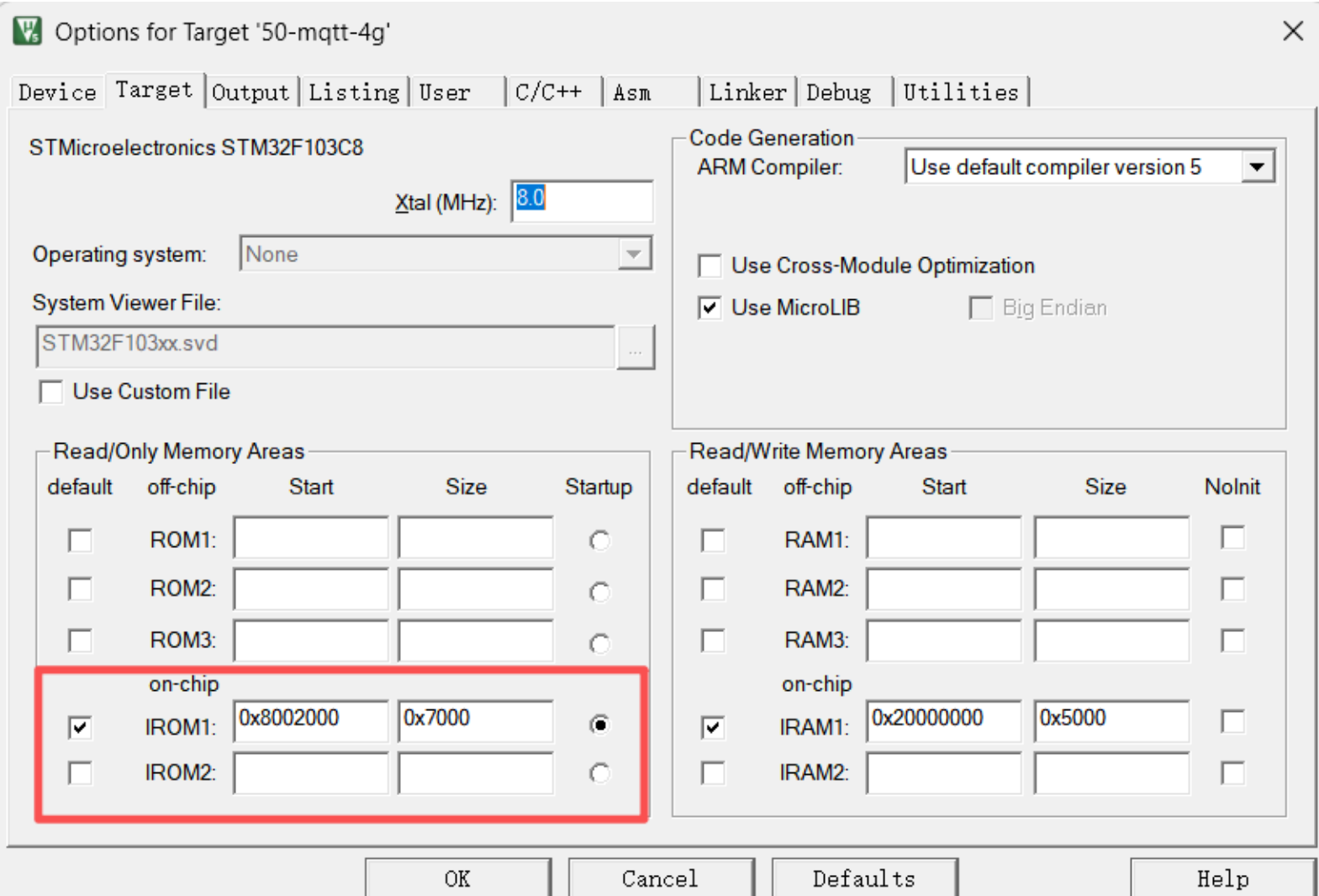
- 主函数添加修改向量表的指令

```
SCB->VTOR = FLASH_BASE | Application_1_Addr ; /* 更改中断向量表地址 偏移到APP1 地址 */

main.c
72 /* USER CODE BEGIN 0 */
73
74 /* USER CODE END 0 */
75
76 /**
77  * @brief The application entry point.
78  * @retval int
79  */
80 int main(void)
81 {
82
83     /* USER CODE BEGIN 1 */
84     SCB->VTOR = FLASH_BASE | Application_1_Addr ; /* 更改中断向量表地址 偏移到APP1 地址 */
85     /* USER CODE END 1 */
86
87     /* MCU Configuration-----*/
88
89     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
90     HAL_Init();
91
92     /* USER CODE BEGIN Init */
93
94     /* USER CODE END Init */
95
96     /* Configure the system clock */
97     SystemClock_Config();
98
99     /* USER CODE BEGIN SysInit */
100
101     /* USER CODE END SysInit */
102 }
```

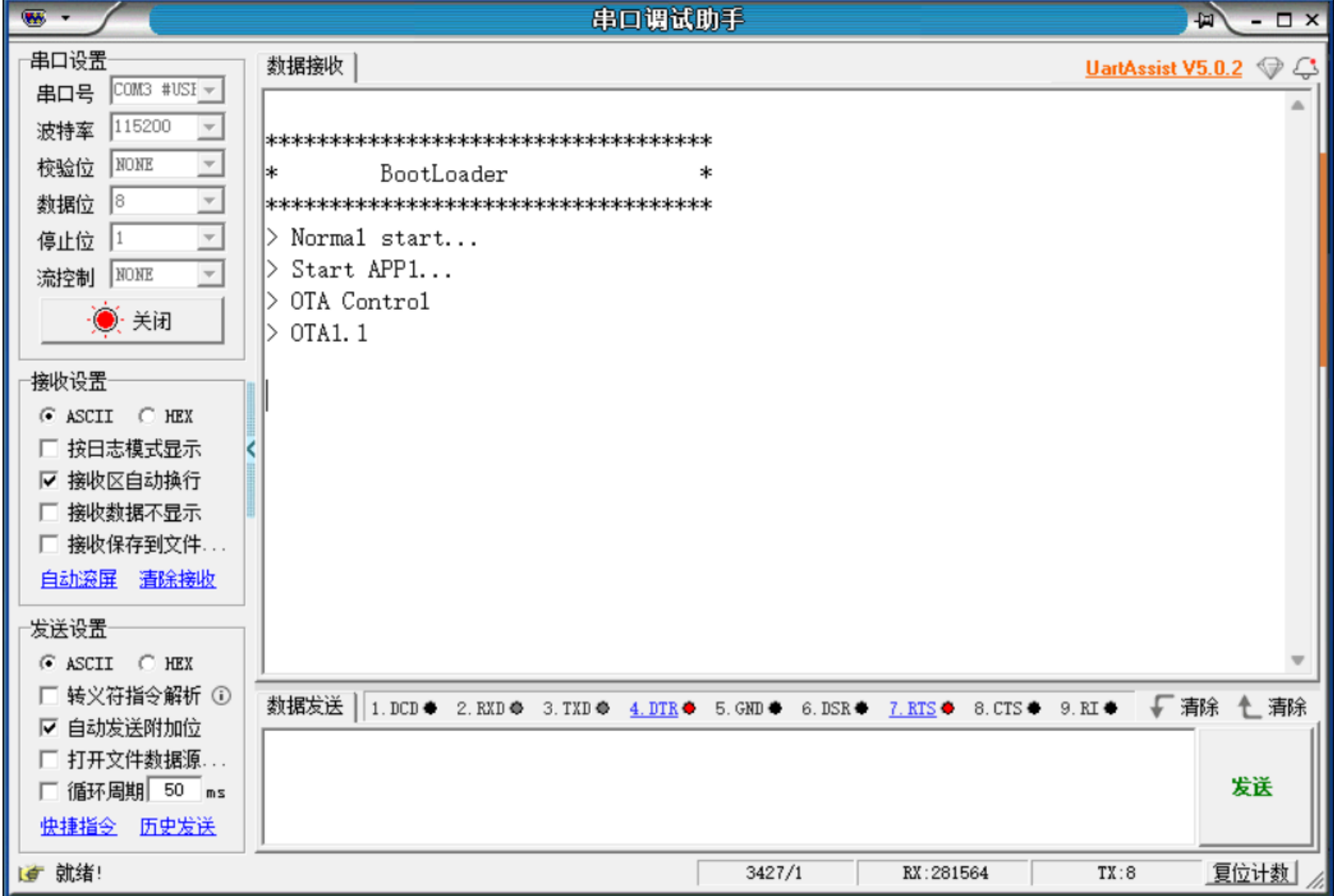
3. 代码下载配置

- 由下图可知两份代码的下载区域是不一样的
- 所以他们下载的区域也不一样



4. 运行程序

程序能够正常启动, 表示配置成功。



5. 开启串口1到3的调试功能

MQTT发送的命令集合

```
AT+MQMULTEN=1
AT+MQSUBM=0,1,0,4,"v1/devices/me/attributes"
AT+MQSUBM=1,1,0,4,"v2/fw/response+/chunk/+"
AT+MQPUBM=0,1,0,4,"v1/devices/me/telemetry"
AT+MQPUBM=1,1,0,4,"v2/fw/request/1/chunk/0"
AT+REST
```

查询订阅的主题情况

```
[2025-09-11 10:09:50.451]# SEND ASCII>
AT+MQSUBM=0?

[2025-09-11 10:09:50.627]# RECV ASCII>
+MQSUBM=0,1,0,4,"v1/devices/me/attributes"
OK

[2025-09-11 10:09:54.879]# SEND ASCII>
AT+MQSUBM=1?

[2025-09-11 10:09:55.056]# RECV ASCII>
+MQSUBM=1,1,0,4,"v2/fw/response+/chunk/+"
OK

[2025-09-11 10:09:59.617]# SEND ASCII>
AT+MQSUBM=2?

[2025-09-11 10:09:59.797]# RECV ASCII>
+MQSUBM=2,0,0,0
OK
```

查询发布的主题

```
[2025-09-11 10:18:33.979]# SEND ASCII>
AT+MQMULTEN?

[2025-09-11 10:18:34.116]# RECV ASCII>
+MQMULTEN=1
OK

[2025-09-11 10:12:40.616]# SEND ASCII>
AT+MQPUBM=0?

[2025-09-11 10:12:40.790]# RECV ASCII>
+MQPUBM=0,1,0,4,"v1/devices/me/telemetry"
OK

[2025-09-11 10:13:51.832]# SEND ASCII>
AT+MQPUBM=1?

[2025-09-11 10:13:52.010]# RECV ASCII>
+MQPUBM=1,1,0,4,"v2/fw/response+/chunk/+"
OK

[2025-09-11 10:14:09.801]# SEND ASCII>
AT+MQPUBM=2?

[2025-09-11 10:14:09.984]# RECV ASCII>
+MQPUBM=2,0,0,0
OK
```

可以一次发送一批命令，然后统一执行，这样效率更高

```
AT+MQMULTEN=1
AT+MQTTFILTER=0
AT+MQSUBM=0,1,0,4,"v1/devices/me/attributes"
AT+MQSUBM=1,1,0,4,"v2/fw/response+/chunk/#"
AT+MQPUBM=0,1,0,4,"v1/devices/me/telemetry"
AT+MQPUBM=1,1,0,4,"v2/fw/request+/chunk/#"
AT+REST
```

运行结果

```
[2025-09-11 11:19:09.011]# SEND ASCII>
AT+MQMULTEN=1
AT+MQTTFILTER=0
AT+MQSUBM=0,1,0,4,"v1/devices/me/attributes"
AT+MQSUBM=1,1,0,4,"v2/fw/response+/chunk/#"
AT+MQPUBM=0,1,0,4,"v1/devices/me/telemetry"
AT+MQPUBM=1,1,0,4,"v2/fw/request+/chunk/#"
AT+REST

[2025-09-11 11:19:09.347]# RECV ASCII>
+MQMULTEN=1
OK

[2025-09-11 11:19:09.549]# RECV ASCII>
+MQTTFILTER=0
OK

[2025-09-11 11:19:09.796]# RECV ASCII>
+MQSUBM=0,1,0,4,"v1/devices/me/attributes"
OK

[2025-09-11 11:19:10.030]# RECV ASCII>
+MQSUBM=1,1,0,4,"v2/fw/response+/chunk/#"
OK

[2025-09-11 11:19:10.278]# RECV ASCII>
+MQPUBM=0,1,0,4,"v1/devices/me/telemetry"
OK

[2025-09-11 11:19:10.529]# RECV ASCII>
+MQPUBM=1,1,0,4,"v2/fw/request+/chunk/#"
OK

+REST="1"
OK

[2025-09-11 11:19:12.741]# RECV ASCII>
ML307R-DC-RTU V4.34

[2025-09-11 11:19:15.395]# RECV ASCII>
SIM:READY

[2025-09-11 11:19:16.266]# RECV ASCII>
REGISTER:SUCCESS

[2025-09-11 11:19:16.905]# RECV ASCII>
MQTT_CONNECT:xthings.cloud:1883
```

一次查询设置状态

AT+MQSUBM=0?
AT+MQSUBM=1?
AT+MQPUBM=0?
AT+MQPUBM=1?

- 运行结果

```
[2025-09-11 11:21:55.180]# SEND ASCII>
AT+MQSUBM=0?
AT+MQSUBM=1?
AT+MQPUBM=0?
AT+MQPUBM=1?

[2025-09-11 11:21:55.368]# RECV ASCII>
+MQSUBM=0,1,0,4,"v1/devices/me/attributes"
OK

[2025-09-11 11:21:55.448]# RECV ASCII>
+MQSUBM=1,1,0,4,"v2/fw/response/+/chunk/#"
OK

[2025-09-11 11:21:55.512]# RECV ASCII>
+MQPUBM=0,1,0,4,"v1/devices/me/telemetry"
OK

[2025-09-11 11:21:55.606]# RECV ASCII>
+MQPUBM=1,1,0,4,"v2/fw/request/+/chunk/#"
OK
```

执行结果：

```
[2025-09-14 04:52:36.852]# SEND ASCII>
AT+MQSUBM=0,0,0,0
AT+MQSUBM=1,0,0,0
AT+MQSUBM=2,0,0,0
AT+MQSUBM=3,0,0,0
AT+MQSUBM=4,0,0,0

[2025-09-14 04:52:37.190]# RECV ASCII>
+MQSUBM=0,0,0,0
OK

[2025-09-14 04:52:37.437]# RECV ASCII>
+MQSUBM=1,0,0,0
OK

[2025-09-14 04:52:37.676]# RECV ASCII>
+MQSUBM=2,0,0,0
OK

[2025-09-14 04:52:37.920]# RECV ASCII>
+MQSUBM=3,0,0,0
OK

[2025-09-14 04:52:38.157]# RECV ASCII>
+MQSUBM=4,0,0,0
OK
```

查询订阅的主题状态

AT+MQSUBM=0?
AT+MQSUBM=1?
AT+MQSUBM=2?
AT+MQSUBM=3?
AT+MQSUBM=4?

运行结果：

```
[2025-09-14 04:54:09.020]# SEND ASCII>
AT+MQSUBM=0?
AT+MQSUBM=1?
AT+MQSUBM=2?
AT+MQSUBM=3?
AT+MQSUBM=4?

[2025-09-14 04:54:09.213]# RECV ASCII>
+MQSUBM=0,0,0,0
OK

[2025-09-14 04:54:09.292]# RECV ASCII>
+MQSUBM=1,0,0,0
OK

[2025-09-14 04:54:09.364]# RECV ASCII>
+MQSUBM=2,0,0,0
OK

[2025-09-14 04:54:09.452]# RECV ASCII>
+MQSUBM=3,0,0,0
OK

[2025-09-14 04:54:09.536]# RECV ASCII>
+MQSUBM=4,0,0,0
OK
```

4.5 ML307模组的后台服务器

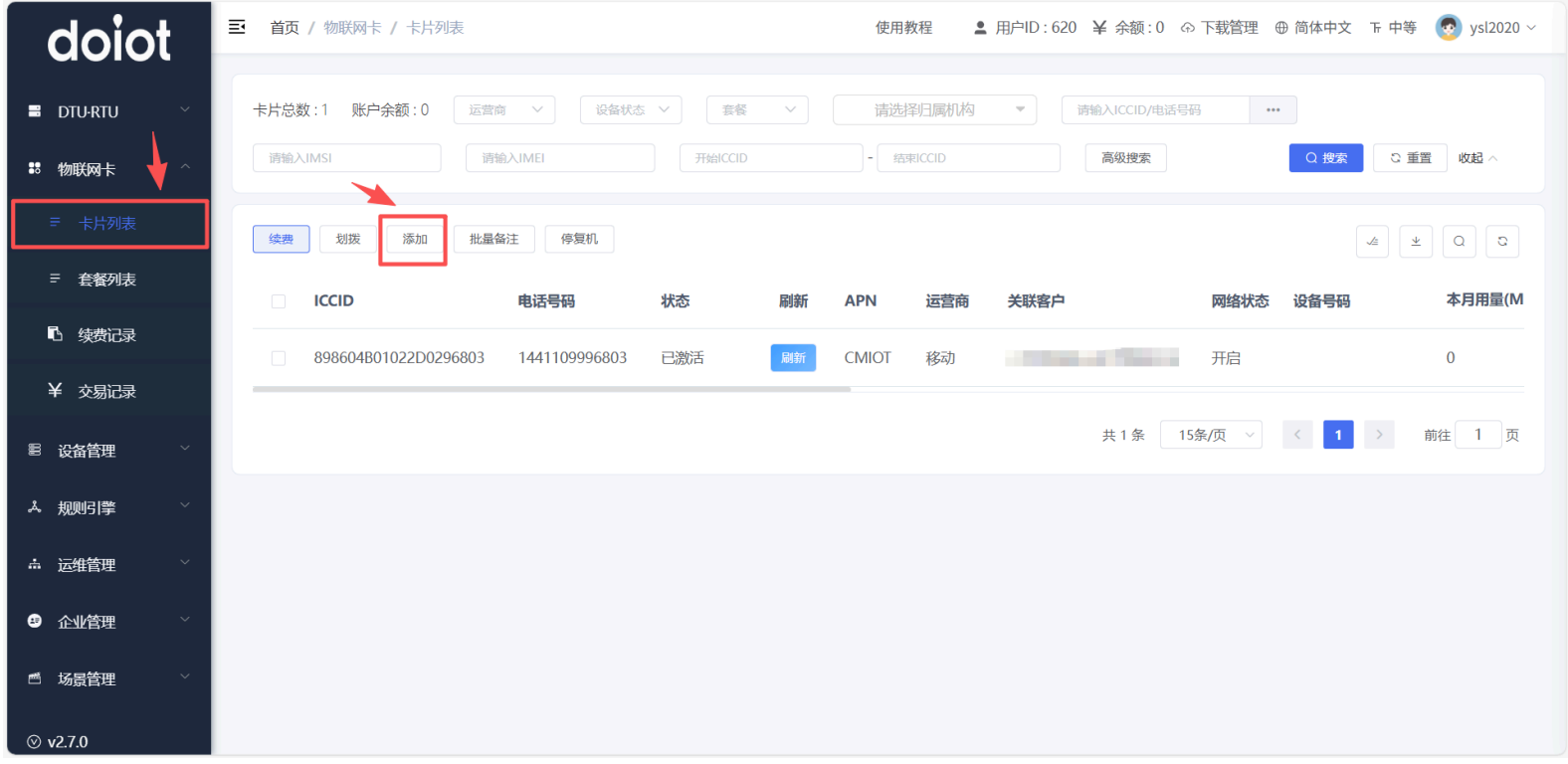
官网的地址：

https://5giot.cn

注册自己的账号并登录服务器，可以查看4G模块的套餐和给模块充值。



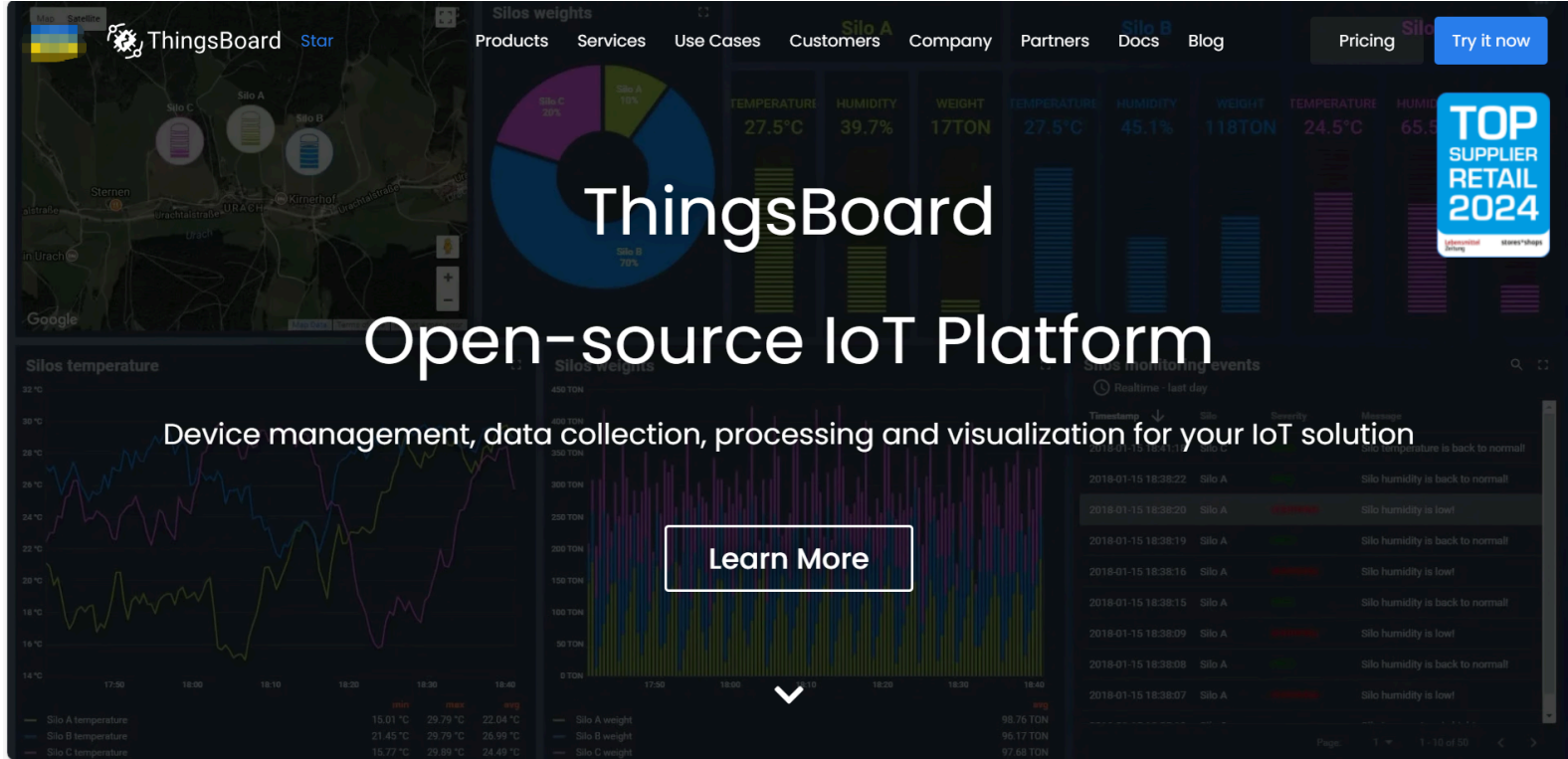
输入ICCID进行绑定物联网卡，ICCID可以使用AT命令进行查询可知。



绑定成功后，可以充值后给自己卡续费或者更改套餐，如有不懂可以找厂家寻求技术支持。

4.6 物联网服务器

1. ThingsBoard介绍



ThingsBoard 是一款功能强大的 **开源物联网（IoT）平台**，用于设备管理、数据收集、处理、可视化和设备管理。它旨在实现物联网项目的快速开发、管理和扩展。

下面是一个概括了 ThingsBoard 主要特性的表格，方便你快速了解：

特性类别	核心能力	说明
 核心功能	设备管理	支持通过 MQTT、CoAP、HTTP 等协议连接和设备管理。
	数据收集与存储	可靠地收集和存储遥测数据，支持数据转换和标准化。
	数据可视化	提供 30+ 可配置小部件和自定义仪表板，用于实时和历史数据可视化。
	规则引擎	使用灵活的规则链处理设备数据，支持自定义逻辑、触发警报、数据转发。
	多租户支持	支持多租户架构，单个租户可拥有多个管理员和数百万设备。
 高级功能	资产管理	管理物联网资产，并定义设备与资产之间的关系。
	警报管理	基于事件（如设备断开连接）触发告警。
	远程控制 (RPC)	支持服务器端和设备端的远程过程调用，用于设备控制。
 架构与部署	部署灵活性	支持单体架构（适合开发和小规模部署）和微服务架构（适合高可用和水平扩展）。
	可扩展性	支持构建集群，实现水平扩展和高可用性，能处理数百万设备。
	容错性	无单点故障，能自动检测节点故障并在不停机情况下更换。
 安全与兼容	安全机制	支持 MQTT/HTTP 传输加密、设备认证和凭证管理。
	协议支持	支持 MQTT、CoAP、HTTP 等物联网行业标准协议。
	数据库支持	支持多种数据库选项，用于存储实体和遥测数据。
 应用场景	智慧能源	电表数据收集、能源消耗可视化、远程控制和OTA更新。
	智能农业	传感器数据收集（如土壤条件）、远程控制执行器、资源优化。
	车队跟踪	车辆状态跟踪、实时路线绘制、地理围栏警报。
	智能计量、环境监测、智能办公、智能零售等	。
 其他信息	开源协议	基于 Apache License 2.0，可免费用于商业产品。
	版本	提供社区版（开源）和专业版（收费）。
	IoT 网关	提供开源网关，支持接入 Modbus、OPC-UA、MQTT 等协议的终端设备。

核心概念：

理解 ThingsBoard 中的一些核心实体有助于更好地使用它：

- **租户 (Tenant)**: 通常代表一个组织或企业，拥有或生产设备和资产。
- **客户 (Customer)**: 向租户购买或使用其设备和/或资产的组织或个人。
- **用户 (User)**: 平台的实际操作者，可分为系统管理员、租户管理员、客户用户等，拥有不同的权限。
- **设备 (Device)**: 直接产生遥测数据和处理RPC命令的基本物联网实体（如传感器）。
- **资产 (Asset)**: 可与其他设备和资产关联的抽象物联网实体（如工厂、车辆），常作为设备的容器或分组。
- **仪表板 (Dashboard)**: 用于可视化数据和控>制设备的界面。
- **规则链 (Rule Chain)**: 由多个规则节点组成的逻辑单元，用于自定义数据处理流程。

如何开始：

ThingsBoard 提供了多种部署方式，包括 Docker 部署、单机部署等，社区版可免费使用。

2. ThingsBoard部署

如果需要自己部署的话， 可以根据文档自己搭建， 我这里使用的是云服务器进行的搭建。

[Docker安装ThingsBoard](#) [thingsboard 云安装-CSDN博客](#)

核心配置文件

docker-compose.yml

```
services:
  mytb:
    restart: always
    image: "thingsboard/tb-postgres:3.6.4"
    ports:
      - "80:9090"
      - "1883:1883"
      - "7070:7070"
      - "5683-5688:5683-5688/udp"
    environment:
      TB_QUEUE_TYPE: in-memory
    volumes:
      - ~/.mytb-data:/data
      - ~/.mytb-logs:/var/log/thingsboard
```

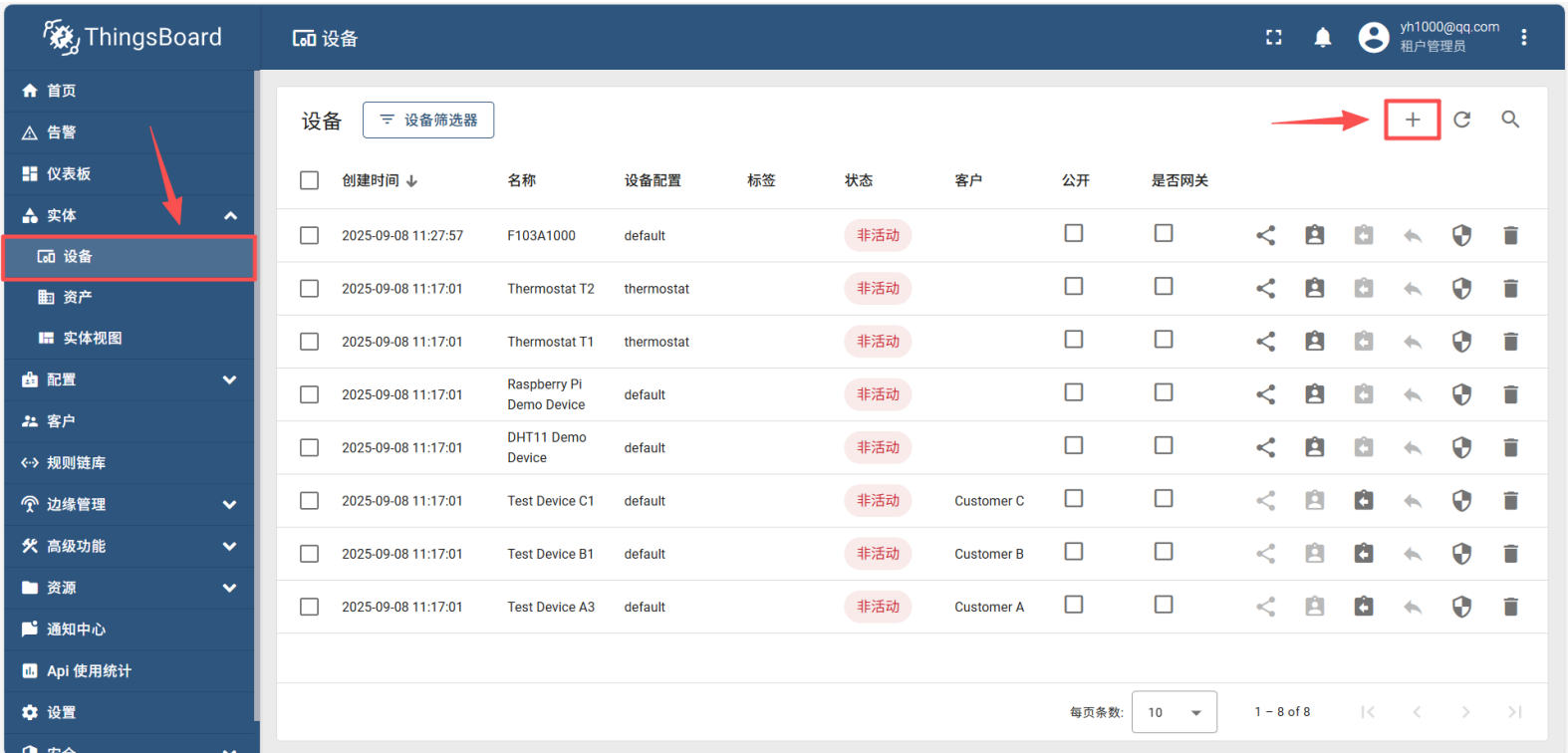
说明：

- docker run - 运行容器
- -it - 将终端会话与当前ThingsBoard进程输出连接
- 8080:9090 - 将本地端口8080转发至HTTP端口9090
- 1883:1883 - 将本地端口1883转发至MQTT端口1883
- 5683:5683 - 将本地端口5683转发至MQTT端口5683
- ~/.mytb-data:/data - 将主机的目录~/mytb-data挂载到ThingsBoard数据目录
- ~/.mytb-logs:/var/log/thingsboard - 将主机的目录~/mytb-logs挂载到ThingsBoard日志目录
- mytb - 计算机本地名称
- restart: always - 在系统重新启动的情况下自动启动ThingsBoard在出现故障的情况下自动重新启动ThingsBoard。
- image: thingsboard/tb-postgres - docker镜像也可以是thingsboard/tb-cassandra或thingsboard/tb

3. 创建物联网设备

每人给大家开通1个租户管理员账户， 请联系工作人员开通。

创建设备，选择 添加设备



添加设备如下图：

名称： F103A1001 ， 根据自己的需求写入设备的名称。

点击添加即可完成设备的添加。

添加设备

1 设备详细信息

2 凭据
Optional

名称*

F103A1001

标签

设备配置*

default

✕

✎

是否网关

分配给客户

说明

✎

下一个：凭据

取消

添加

点击添加后， 会弹出，设备已创建， 检查连通性，这里我们不做测试，后续使用工具测试。

设备已创建，检查连通性！

HTTP

MQTT

CoAP

请使用以下说明代表设备发送遥测数据

Windows

MacOS

Linux

安装必要的客户端工具

从 Windows 10 b17063 开始，cURL 已默认安装

执行以下命令

curl -v -X POST http://xthings.cloud:8080/api/v1/cn9xd6m331c91cwxj

复制

状态 非活动

最新遥测数据

时间

键名

数值

不再显示

关闭

3. 配置ML307连接Thingsboard服务器

设置ML307模块的属性， 配置MQTT服务器

登录服务器：

http://gzh.dosim.cn:10001/



- 添加设备



通过AT指令获取IMEI



输入IMEI，然后点击确定



创建分组



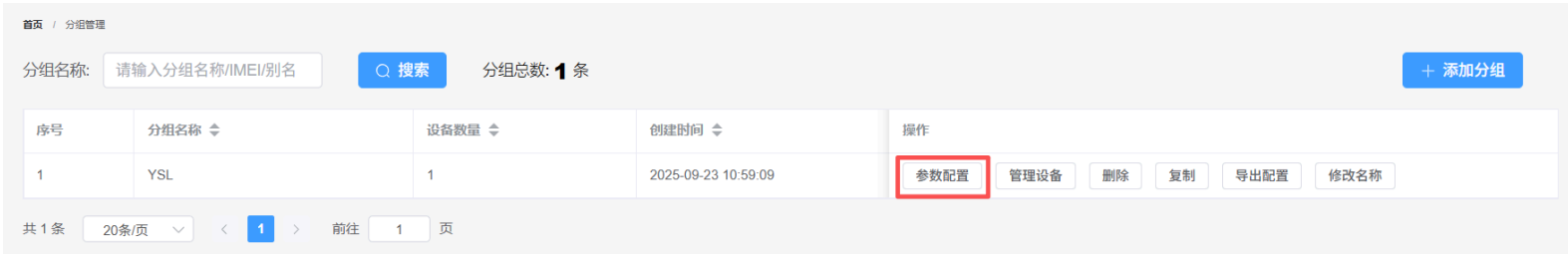
设置分组名，然后点击确定



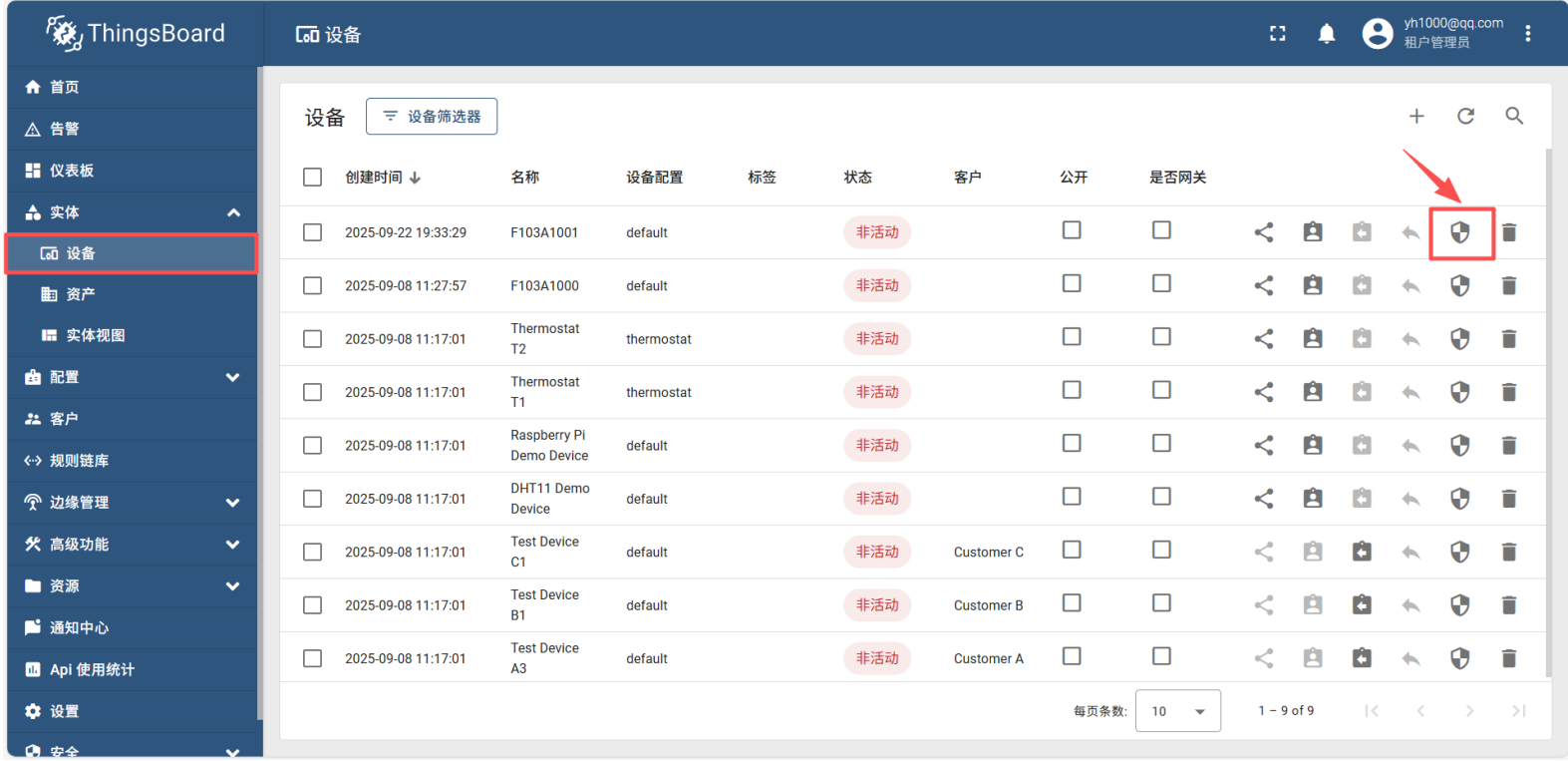
改分组后才可以设置设备的参数



参数配置



登录xthing.cloud，获取设备的密钥



复制设备的Token，就是这给的令牌或则说是密钥



ML307后台服务器配置上密钥，给每一个ML307模块设置一个密钥

- MQTT服务器：xthings.cloud
- 端口号：1883


```
        OLED_ShowStr(0,3,(unsigned char *)"MQTTServer... OK",2);    //测试8*16字符
    }
    else
    {
        OLED_ShowStr(0,3,(unsigned char *)"MQTTServer... --",2);    //测试8*16字符
        return  RESET;
    }
    return  SET;
}

}
```

4. 数据面板



5. 解析服务器下发的指令

mqtt.h

```
typedef struct
{
    char fw_title[32];
    char fw_version[32];
    uint32_t fw_size;
    char fw_checksum[32];
    uint8_t recv_buf[NSize];
    uint8_t recv_flag ; // OTA 接收到固件内容标识位
    uint32_t request_id ; // 请求的数据包id号
    uint32_t chunk_id ; // 请求的数据块id号
    uint32_t request_bytes ; // 请求下载的字节数
    uint32_t received_bytes ; // 接收到的字节数， 用于显示进度
}OTA_FW_Info_T ;
extern OTA_FW_Info_T OTA_Info; // 描述OTA固件信息的结构体
```

mqtt.c

```
OTA_FW_Info_T OTA_Info ; // 描述OTA固件信息的结构体
uint8_t CAT1_Parse_ATComander(uint8_t *recvbuf)
{
    char * leftp = (char *)recvbuf ;
    char * rightp = leftp ;
    char * startp = leftp ;
    uint32_t dataNum = 0 ;
    memset(OTA_Info.recv_buf,0,sizeof(OTA_Info.recv_buf)) ; // 清空数组

    // 解析服务器下发的命令
    char substr[64] = {"v1/devices/me/rpc/request/"};
    if( (startp = strstr((char *)startp,substr) ) != NULL )
    {
        dataNum = strtol(startp + strlen(substr), NULL, 10);
        leftp = strstr((char *)startp,"{"); // 找到左花括号
        rightp = strstr((char *)startp,"}"); // 找到右花括号
        if( (leftp != NULL) && (rightp != NULL))
        {
            strncpy((char *)OTA_Info.recv_buf,leftp,rightp-leftp+1); // 复制字符串
            //printf("jsonbuf=%s | dataNum=%d\n",OTA_Info.recv_buf,dataNum);
            MQTT_Parse_DeviceData((uint8_t*)OTA_Info.recv_buf,dataNum);// 解析json字符串并控制设备
            return SET;
        }
        //startp = startp + strlen(substr); // 移动位置
    }

    //*****//
    // 解析服务器下发的命令
    memset(substr,0,sizeof(substr));
    startp = (char *)recvbuf ;
    strcpy(substr,"v1/devices/me/attributes") ;
    if( (startp = strstr((char *)startp,substr) ) != NULL )
    {
        leftp = strstr((char *)startp,"{"); // 找到左花括号
        rightp = strstr((char *)startp,"}"); // 找到右花括号
        if( (leftp != NULL) && (rightp != NULL))
        {
            strncpy((char *)OTA_Info.recv_buf,leftp,rightp-leftp+1); // 复制字符串
            //printf("jsonbuf=%s\n",OTA_Info.recv_buf);
            MQTT_Parse_OTADData(OTA_Info.recv_buf);// 解析json字符串并控制设备
            return SET;
        }
        //startp = startp + strlen(substr); // 移动位置
    }

}

//*****//
// 解析服务器下发的命令
memset(substr,0,sizeof(substr));
startp = (char *)recvbuf ;

sprintf(substr,"v2/fw/response/%d/chunk/%d,%d,",
OTA_Info.request_id,OTA_Info.chunk_id,OTA_Info.request_bytes) ;
printf("ota:substr=%s\n",substr);
if( (startp = strstr((char *)startp,substr) ) != NULL )
{
    memcpy(OTA_Info.recv_buf,startp+strlen(substr),OTA_Info.request_bytes); // 复制字符串
    //printf("jsonbuf=%s\n",OTA_Info.recv_buf);
    OTA_Info.recv_flag = 1; // OTA 接收到固件的内容
    return SET;
    //MQTT_Parse_OTADData((uint8_t*)OTA_Info.recv_buf);// 解析json字符串并控制设备
    //startp = startp + strlen(substr); // 移动位置
}
return RESET;
}

uint8_t MQTT_Parse_DeviceData(uint8_t *json,uint32_t dataNum)
{
    cJSON *cjson_device = NULL;
    cJSON *cjson_method = NULL;
    cJSON *cjson_params = NULL;
    /* 解析整段JSON数据 */
    cjson_device = cJSON_Parse((char *)json);
    if (cjson_device == NULL)
    {
        //printf("parse fail.\n");
        return RESET;
    }
    else
    {
        // printf("json->%s\n",cJSON_Print(cjson_device));
    }
    /* 依次根据名称提取JSON数据（键值对） */
    cjson_method = cJSON_GetObjectItem(cjson_device, "method");
    cjson_params = cJSON_GetObjectItem(cjson_device, "params");
    //printf("method:%s\n",cjson_method->valuestring);
    //printf("params:%d\n",cjson_params->valueint);

    // 获取LED1的状态
    if(strcmp(cjson_method->valuestring,"led1Status") == 0 ) {
        // ESC 按键被按下后,引脚 变为低电平
        if (HAL_GPIO_ReadPin(LED1_GPIO_Port, LED1_Pin) == GPIO_PIN_RESET)
        {
            MQTT_SendData((uint8_t *)PUB2,dataNum,"true");
        }else {
            MQTT_SendData((uint8_t *)PUB2,dataNum,"false");
        }
    }
}
```

```
// 获取beep的状态
else if(strcmp(cjson_method->valuestring,"beepStatus") == 0 ) {
    // ESC 按键被按下后,引脚 变为低电平
    if (HAL_GPIO_ReadPin(BEEP_GPIO_Port, BEEP_Pin) == GPIO_PIN_SET)
    {
        MQTT_SendData((uint8_t *)PUB2,dataNum,"true");
    }else {
        MQTT_SendData((uint8_t *)PUB2,dataNum,"false");
    }
}

// 获取继电器的状态
else if(strcmp(cjson_method->valuestring,"relayStatus") == 0 ) {
    // ESC 按键被按下后,引脚 变为低电平
    if (HAL_GPIO_ReadPin(RELAY_GPIO_Port, RELAY_Pin) == GPIO_PIN_SET)
    {
        MQTT_SendData((uint8_t *)PUB2,dataNum,"true");
    }else {
        MQTT_SendData((uint8_t *)PUB2,dataNum,"false");
    }
}

}

// 控制LED1的状态
if(strcmp(cjson_method->valuestring,"led1Set") == 0 )
{
    if(cjson_params->valueint == 1 )
    {
        //LED_Control(LED1, ON);
        REG_HOLD_BUF[0] |= LED1_CMD ; // 第0位 写 1
        MQTT_SendData((uint8_t *)PUB2,dataNum,"true");
    }else if(cjson_params->valueint == 0 )
    {
        //LED_Control(LED1, OFF);
        REG_HOLD_BUF[0] = REG_HOLD_BUF[0] & (~LED1_CMD) ; // 第0位 写 0
        MQTT_SendData((uint8_t *)PUB2,dataNum,"false");
    }
}

}

// 控制BEEP的状态
else if(strcmp(cjson_method->valuestring,"beepSet") == 0 )
{
    if(cjson_params->valueint == 1 )
    {
        //LED_Control(LED1, ON);
        REG_HOLD_BUF[0] |= BEEP_CMD ; // 第0位 写 1
        MQTT_SendData((uint8_t *)PUB2,dataNum,"true");
    }else if(cjson_params->valueint == 0 )
    {
        //LED_Control(LED1, OFF);
        REG_HOLD_BUF[0] = REG_HOLD_BUF[0] & (~BEEP_CMD) ; // 第0位 写 0
        MQTT_SendData((uint8_t *)PUB2,dataNum,"false");
    }
}
}

// 控制relay的状态
if(strcmp(cjson_method->valuestring,"relaySet") == 0 )
{
    if(cjson_params->valueint == 1 )
    {
        //LED_Control(LED1, ON);
        REG_HOLD_BUF[0] |= RELAY_CMD ; // 第0位 写 1
        MQTT_SendData((uint8_t *)PUB2,dataNum,"true");
    }else if(cjson_params->valueint == 0 )
    {
        //LED_Control(LED1, OFF);
        REG_HOLD_BUF[0] = REG_HOLD_BUF[0] & (~RELAY_CMD) ; // 第0位 写 0
        MQTT_SendData((uint8_t *)PUB2,dataNum,"false");
    }
}
}

}

CJSON_Delete(cjson_device); // 一定要释放内存
return SET;

}

uint8_t MQTT_SendData(uint8_t *topic,uint32_t dataNum,char *status)
{
    uint8_t sendbuf[128] = {0};
    uint8_t buf[256] = {0};

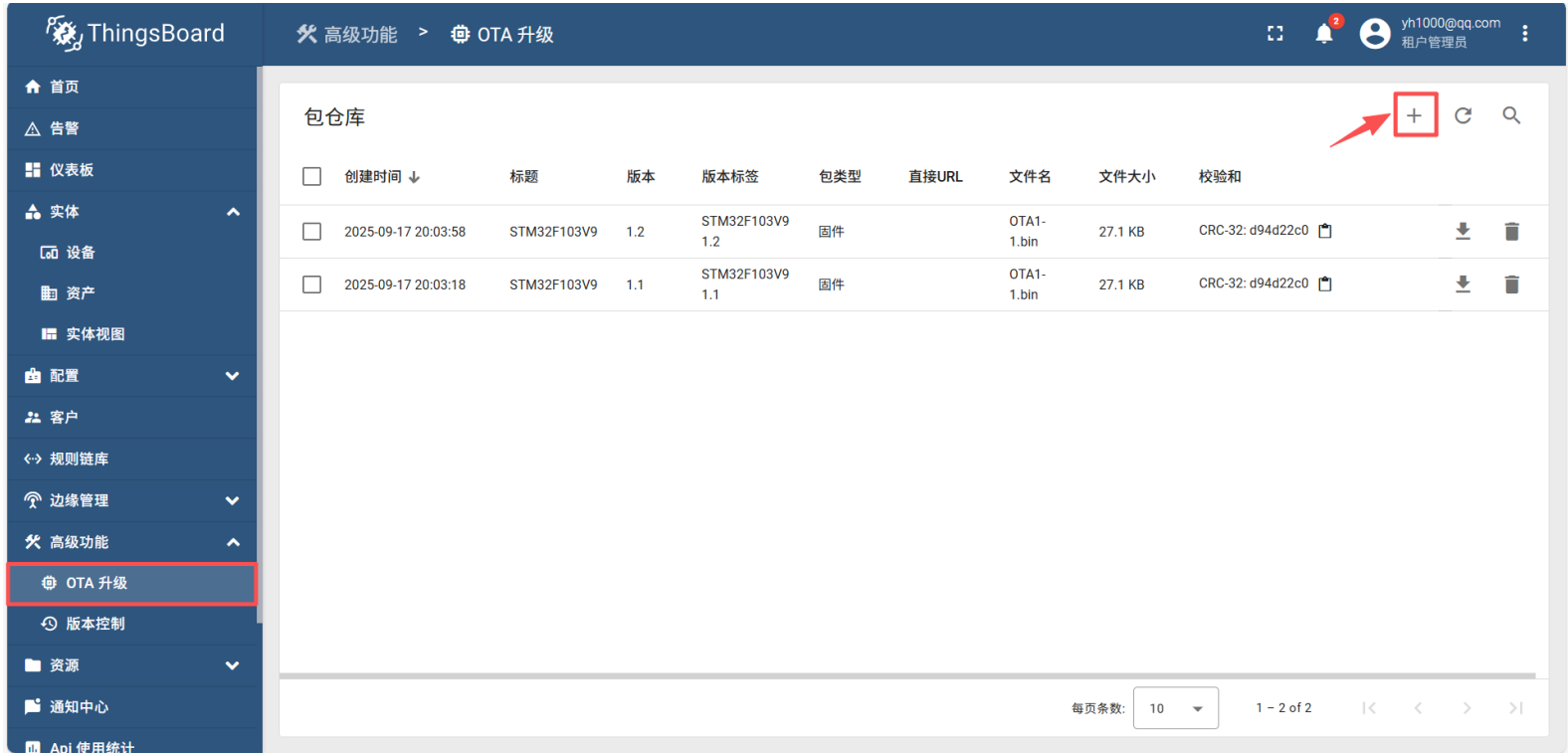
    if(strncmp((char *)topic,PUB1,strlen(PUB1)-1) == 0 )
    {
        if(dataNum == 1) // 上传传感器数据
        {
            // MQPUB,1,v1/devices/me/telemetry
            sprintf((char *)sendbuf, "{TP:%d,RH:%d,VO:%d,CU:%d,PW:%d,VR:%d,CPU:%d}",
            REG_HOLD_BUF[1],REG_HOLD_BUF[2], REG_HOLD_BUF[3],REG_HOLD_BUF[4],
            REG_HOLD_BUF[5],REG_HOLD_BUF[6],REG_HOLD_BUF[7]);
            // 上行主题1 上传服务器数据的 "v1/devices/me/telemetry"
            sprintf((char *)buf,"MQPUB,1,%s,%s",PUB1,sendbuf);
        }
        else if(dataNum == 2) // ota 状态更新
        {
            sprintf((char *)sendbuf, "{\\"fw_state\\":\\"UPDATED\\"}");
            // 上行主题1 上传服务器数据的 "v1/devices/me/telemetry"
            sprintf((char *)buf,"MQPUB,1,%s,%s",PUB1,sendbuf);
        }
        else if(dataNum == 3) // ota 状态更新
        {
            sprintf((char *)sendbuf, "{\\"fw_state\\":\\"FAILED\\"}");
            // 上行主题1 上传服务器数据的 "v1/devices/me/telemetry"
            sprintf((char *)buf,"MQPUB,1,%s,%s",PUB1,sendbuf);
        }
    }
    else if(strncmp((char *)topic,PUB2,strlen(PUB2)-1) == 0 )
    {
        // 上行主题2 回复服务器下发的命令 PUB2 "v1/devices/me/rpc/response/"
        sprintf((char *)buf,"MQPUB,1,%s%d,%s",PUB2,dataNum,status);
    }

    else if(strncmp((char *)topic,PUB3,strlen(PUB3)-1) == 0 )
    {
        // 上行主题2 发送下载固件命令 "v2/fw/request/+/chunk/#"
        sprintf((char *)buf,"MQPUB,1,v2/fw/request/%d/chunk/%d,%s",
        OTA_Info.request_id,OTA_Info.chunk_id,status);
    }
}

printf("MQTT Send Data:%s",buf);
HAL_UART_Transmit(&huart3,(uint8_t*)buf,strlen((char *)buf),1000);
return SET;

}
```

6. 服务器创建OTA升级包



mqtt.c

```
uint8_t MQTT_Parse_OTAData(uint8_t *json)
{

    CJSON *cjson_device = NULL;
    CJSON *cjson_title = NULL;
    CJSON *cjson_version = NULL;
```

```
cJSON *cjson_size = NULL;
cJSON *cjson_checksum = NULL;

/* 解析整段JSON数据 */
cjson_device = cJSON_Parse((char *)json);
if (cjson_device == NULL)
{
    printf("parse fail.\n");
    return RESET;
}
else
{
    // printf("json->%s\n",cJSON_Print(cjson_device));
}
/* 依次根据名称提取JSON数据（键值对） */
cjson_title = cJSON_GetObjectItem(cjson_device, "fw_title");
if(cjson_title == NULL )
{
    return RESET;
}
cjson_version = cJSON_GetObjectItem(cjson_device, "fw_version");
cjson_size = cJSON_GetObjectItem(cjson_device, "fw_size");
cjson_checksum = cJSON_GetObjectItem(cjson_device, "fw_checksum");
printf("fw_title:%s\n",cjson_title->valuelstring);
// printf("fw_version:%s\n",cjson_version->valuelstring);
// printf("fw_size:%d\n",cjson_size->valueint);
// printf("fw_checksum:%s\n",cjson_checksum->valuelstring);
strcpy(OTA_Info.fw_title,cjson_title->valuelstring);
strcpy(OTA_Info.fw_version,cjson_version->valuelstring);
strcpy(OTA_Info.fw_checksum,cjson_checksum->valuelstring);
OTA_Info.fw_size = cjson_size->valueint;

MQTT_OTA_FLAG = 1; // 可以处理OTA数据标志位
cJSON_Delete(cjson_device); // 一定要释放内存
return SET;
}

// 每收到数据包时调用（len=256或最后一包）
void Update_Progress(uint32_t len) {
    OTA_Info.received_bytes += len;
    // 核心代码：计算百分比并显示（无itoa）
    uint8_t percent = (OTA_Info.received_bytes * 100) / OTA_Info.fw_size;
    char progress[20] = { 0 }; // 固定格式" xx%"
    if(percent > 100 ) percent = 100;
    if(percent == 100)
    {
        sprintf(progress,"OTA%s      OK",OTA_Info.fw_version);
    }
    else
    {
        sprintf(progress,"OTA%s      %3d%%",OTA_Info.fw_version,percent);
    }
    OLED_ShowStr(0, 6, (uint8_t*)progress, 2); // 第2行显示（如" 75%"）
}

uint8_t MQTT_OTA_GetFW(void)
{
    uint32_t size = OTA_Info.fw_size/256 ;
    uint8_t msglen = OTA_Info.fw_size%256/4; // 计算包长度
    if(OTA_Info.fw_size%256 != 0 ) // 说明有余数， 需要再次传输1次
    {
        size++;
    }

    CRC32_Init(&crc_ctx); // 重置CRC
    // 先擦除芯片的存储空间， 计算需要擦除多少扇区
    printf("Erase APP2 ...|size=%d\r\n",size);
    /* 擦除App2 */
    Erase_page(Application_2_Addr, Application_Size/1024); //擦除 Application_Size 页

    OTA_Info.received_bytes = 0 ;
    // 或使用时间戳+随机数（更安全）
    OTA_Info.request_id = HAL_GetTick() + (rand() & 0xFFFF); ;
    OTA_Info.chunk_id = 0 ;
    OTA_Info.request_bytes = 256; // 默认请求数据为256字节
    for(uint32_t i =0;i < size ; i++,OTA_Info.chunk_id++)
    {
        char strNumber[10]={0};
        if( (i == size -1) && (OTA_Info.fw_size%256 != 0) ) // 最后1包数据
        {
            OTA_Info.request_bytes = OTA_Info.fw_size%256 ;
        }
        sprintf(strNumber,"%d",256);
        MQTT_SendData((uint8_t *)PUB3,OTA_Info.request_id,strNumber);
        /*****
        // 等待服务器返回OTA固件的数据
        uint32_t Timeout = 10*1000; // 超时等待10秒
        while (OTA_Info.recv_flag == 0) // 没有接收到 服务器下发的数据
        {
            if ((Timeout--) == 0)
                return RESET; // 超时时间到， 函数返回
            HAL_Delay(1);
        }
        printf("Timeout=%d\n",10*1000-Timeout);
        //HAL_Delay(100);
        OTA_Info.recv_flag = 0 ; // 清空标志位

        // 进度百分比实现
        // 每次收到数据包时调用
        Update_Progress(256); // 更新进度
        *****/

        printf("i=%d|msglen=%d\n",i,msglen);
        printf("\r\n*****\r\n");
        printf("otarecv_buf=%s",OTA_Info.recv_buf);
        printf("\r\n*****\r\n");

        // 把接收的数据写到Flash内， 每次写256字节
        if( (i == size -1) && (OTA_Info.fw_size%256 != 0) ) // 最后1包数据
        {
            if(OTA_Info.fw_size%256%4 != 0) // 有余数
            {
                msglen ++; // 多增加1包数据
            }
            //printf("zuihou:i=%d|msglen=%d\n",i,msglen);
            CRC32_Update(&crc_ctx, OTA_Info.recv_buf,OTA_Info.fw_size%256); // 增量更新CRC
            writeFlash(Application_2_Addr + i * 256,(uint32_t *)OTA_Info.recv_buf, msglen);
        }
        else
        {
            CRC32_Update(&crc_ctx, OTA_Info.recv_buf, 256); // 增量更新CRC
            writeFlash(Application_2_Addr + i * 256,(uint32_t *)OTA_Info.recv_buf, 256/4);
        }
    }
    // 进行crc32 校验
    uint32_t crc32 = CRC32_Final(&crc_ctx,1);
    printf("client_crc32=%x\n",crc32);
    char crcstr[10] = {0};
    sprintf(crcstr,"%x",crc32);
    printf("server_crc32=%s\n",OTA_Info.fw_checksum);
    if(strncmp(OTA_Info.fw_checksum,crcstr,strlen(OTA_Info.fw_checksum)) == 0 )
    {
        MQTT_SendData((uint8_t *)PUB1,2,NULL); // 成功
        uint32_t update_flag = 0xAAAAAAA; // 升级固件标志位
        writeFlash((Application_2_Addr + Application_Size - 4), &update_flag, 1);
        HAL_NVIC_SystemReset(); // 重启系统
    }
    else
    {
        MQTT_SendData((uint8_t *)PUB1,3,NULL); // 3 表示失败
        OLED_ShowStr(64+16, 6, (uint8_t*)" Fail", 2); // 第2行显示（如" 75%"）
        MQTT_OTA_FLAG = 1; // // 失败后继续重新下载固件， 可以处理OTA数据标志位。
        return RESET ;
    }

    return SET;
}

}
```

crc32.h

```
// crc32_ota.h
#ifndef __CRC32_H__
#define __CRC32_H__

#endif

#ifndef __cplusplus
```

```
extern "C" {
#ifdef

#include <stdint.h>

typedef struct {
    uint32_t crc;    // 中间CRC值
} CRC32_Context;
extern CRC32_Context  crc_ctx;    // 全局CRC上下文

void CRC32_Init(CRC32_Context *ctx);
void CRC32_Update(CRC32_Context *ctx, const uint8_t *data, uint32_t len);
uint32_t CRC32_Final(CRC32_Context *ctx, int swap_endian) ;

#ifdef __cplusplus
}
#endif
#endif /*__CRC32_H__ */
```

crc32.c

```
// crc32.c
#include "crc32.h"

CRC32_Context  crc_ctx;    // 全局CRC上下文

// 初始化CRC上下文（开始新校验前必须调用）
void CRC32_Init(CRC32_Context *ctx) {
    ctx->crc = 0xFFFFFFFF;    // Zlib初始值
}

// 追加数据包（可多次调用）
void CRC32_Update(CRC32_Context *ctx, const uint8_t *data, uint32_t len) {
    for (uint32_t i = 0; i < len; i++) {
        ctx->crc ^= data[i];
        for (uint8_t j = 0; j < 8; j++) {
            ctx->crc = (ctx->crc >> 1) ^ (0xEDB88320 & -(ctx->crc & 1));
        }
    }
}

// 通用字节交换（适用于任何平台）
uint32_t swap_uint32(uint32_t val) {
    return ((val >> 24) & 0xFF)      |    // 移动最高字节到最低位
           ((val >> 8)  & 0xFF00)    |
           ((val << 8)  & 0xFF0000)  |
           ((val << 24) & 0xFF000000);
}

uint32_t CRC32_Final(CRC32_Context *ctx, int swap_endian)
{
    uint32_t crc = ctx->crc ^ 0xFFFFFFFF;
    return swap_endian ? swap_uint32(crc) : crc;    // 按需交换字节序
}
```

4.8 OTA版本升级内容

1. APP1.2 升级功能

增加状态指示灯， 周期1秒， 亮的时间500ms， 熄灭时间500ms 。

4.9 生产环节设置

1. 提取完整的固件

4.10 改进思路

A区和B区轮换

A区和B区轮换启动，降低误操作的风险

启用外部Flash

使用外部SPI Flash 存放固件， 来降低片内Flash的占用，腾出更多的空间来存放代码。

